

Algoritmi e Laboratorio

Guida all'esame — scritto e orale

La teoria del corso e i metodi d'esame: analisi, strutture dati, programmazione

dinamica, greedy e grafi, con le prove scritte svolte

Corso di Laurea Triennale in Informatica — Università di Catania

Prof. Simone Faro, a.a. 2026–2027

Questa guida prepara l'esame di **Algoritmi e Laboratorio** (prof. Simone Faro): la **prova scritta**, con gli esercizi di analisi, ricorrenze e progettazione, e la **prova orale**. Segue il programma ufficiale del corso e il libro di testo, il Cormen.

Da dove vengono i contenuti. Dal libro di testo (*Introduzione agli algoritmi e strutture dati*, 3^a edizione italiana), con capitolo e pagina accanto a ogni definizione e algoritmo, e dalle **nove prove scritte** del 2025-2026, svolte per intero negli ultimi capitoli. Gli algoritmi sono nel pseudocodice del libro e dei temi d'esame; i disegni (heap, alberi rosso-neri, grafi) sono fatti apposta in LaTeX.

Come sono controllati i conti. Ogni esecuzione, tabella e soluzione di ricorrenza mostrata nella guida è riprodotta da un programma (cartella **verifiche/**): gli algoritmi sono implementati in Python e fatti girare sulle stesse istanze.

Indice

Come usare questa guida	1
1 Analisi degli algoritmi e notazioni asintotiche	3
1.1 Che cosa significa analizzare un algoritmo	3
1.2 L'analisi di Insertion-Sort	3
1.3 Gli invarianti di ciclo	4
1.4 Le notazioni asintotiche	5
1.5 Confronto di funzioni e ordini di crescita	6
2 Divide et impera e ricorrenze	8
2.1 Divide et impera: merge sort	8
2.1.1 La ricorrenza di merge sort	9
2.2 Risolvere una ricorrenza: il metodo di sostituzione	10
2.3 Il metodo dell'albero di ricorsione	11
2.4 Il teorema master, e la sua dimostrazione	12
2.4.1 La dimostrazione <i>da saper dimostrare</i>	12
2.5 Come si risolve all'esame	13
3 Heap, heapsort e code di priorità	16
3.1 Heap	16
3.2 Mantenere la proprietà: MAX-HEAPIFY	17
3.3 Costruire un heap: BUILD-MAX-HEAP	18
3.4 L'algoritmo HEAPSORT	19
3.5 Le code di priorità	20
4 Quicksort: partizionare per ordinare	23
4.1 Descrizione di quicksort	23
4.2 Partition	24
4.3 Prestazioni di quicksort	25
4.4 La versione randomizzata	26
4.5 Come si risolve all'esame	27
5 Ordinamento in tempo lineare e statistiche d'ordine	30
5.1 Il limite inferiore $\Omega(n \lg n)$ degli ordinamenti per confronti	30
5.2 Counting sort	31
5.3 Radix sort	32
5.4 Bucket sort	32
5.5 Minimo, massimo e statistiche d'ordine	33
5.6 Selezione in tempo atteso lineare: RANDOMIZED-SELECT	33
5.7 Selezione in tempo lineare nel caso peggiore: SELECT	34
6 Hashing	36
6.1 Tavole a indirizzamento diretto	36

6.2	Tavole hash e collisioni	36
6.2.1	Concatenamento	37
6.2.2	L'analisi del concatenamento	37
6.3	Le funzioni hash	38
6.3.1	Divisione e moltiplicazione	38
6.3.2	L'hashing universale	39
6.4	L'indirizzamento aperto	40
6.4.1	Lineare, quadratica, doppio hashing	40
6.4.2	L'analisi dell'indirizzamento aperto	41
6.5	Come si risolve all'esame	42
7	Alberi rosso-neri	44
7.1	Le cinque proprietà, e quanto è alto un albero rosso-nero	44
7.1.1	Alberi bilanciati e alberi rosso-neri	44
7.1.2	Perché funziona: un albero senza rossi è bilanciato, i rossi sono «non troppi»	46
7.1.3	L'altezza nera	47
7.1.4	Quanto è alto un albero rosso-nero	48
7.2	Le rotazioni	50
7.3	L'inserimento	52
7.4	Un esercizio svolto: inserire 41, 38, 31, 12, 19, 8	56
7.5	La cancellazione	57
7.5.1	Prima il successore: nodi con due figli e con al più un figlio	58
7.6	La riparazione: RB-DELETE-FIXUP	59
7.7	Un esempio svolto: cancellare 8, 12, 19, 31, 38	62
7.8	Come si risolve all'esame	63
8	Programmazione dinamica	65
8.1	Le catene di montaggio	66
8.2	La moltiplicazione di una sequenza di matrici	67
8.3	Sottostruttura ottima e sottoproblemi ripetuti	69
8.4	La più lunga sottosequenza comune (LCS)	69
8.5	Zaino 0/1 e problema del resto	70
8.6	Come si risolve all'esame	71
9	Algoritmi golosi	73
9.1	Un esempio: la selezione di attività	73
9.2	Gli elementi della strategia golosa	75
9.3	I codici di Huffman	77
9.4	Il problema del resto, e le fusioni ottimali	79
10	Visite dei grafi e ordinamento topologico	81
10.1	Rappresentare un grafo: liste e matrice di adiacenza	81
10.2	La visita in ampiezza (BFS)	81
10.3	La visita in profondità (DFS)	82
10.3.1	Classificare gli archi	83
10.4	L'ordinamento topologico	84
10.5	Le componenti fortemente connesse	85
10.6	Come si risolve all'esame	85
11	Cammini minimi da sorgente unica	87
11.1	Le proprietà dei cammini minimi	87
11.2	Il rilassamento	88

11.3	L'algoritmo di Bellman-Ford	89
11.4	Cammini minimi nei grafi orientati aciclici	91
11.5	L'algoritmo di Dijkstra	92
12	Cammini minimi fra tutte le coppie	94
12.1	Cammini con al più m archi, e il «prodotto» di matrici	95
12.2	Floyd-Warshall	96
12.3	Quale algoritmo, per quale input	97
12.4	Come si risolve all'esame	98
13	Le prove scritte del 2026, svolte	99
13.1	16 febbraio 2026	99
13.2	31 marzo 2026	102
13.3	27 aprile 2026	104
13.4	22 giugno 2026	106
13.5	13 luglio 2026	108
13.6	1 settembre 2026	109
13.7	21 settembre 2026	111
14	La prova del 2025 e la simulazione, svolte	114
14.1	27 ottobre 2025	114
14.2	22 gennaio 2026 (simulazione)	117
15	L'orale	123
15.1	Come funziona l'orale, e come si risponde	123
15.2	Le dimostrazioni da saper fare	124
15.3	Le domande tipiche, argomento per argomento	125
15.4	In due minuti	126

Come usare questa guida

Com'è fatto l'esame

Il corso è formato da due moduli, **Algoritmi** e **Laboratorio**, insegnati dallo stesso docente e con **un solo esame e un solo voto** [programma del corso]. L'esame ha una **prova scritta obbligatoria** (tre ore di tempo, esercizi da svolgere) e poi una **prova orale**, che può svolgersi lo stesso giorno o pochi giorni dopo.

- **Lo scritto** dà come esito *ammesso* (con un voto comunicato da 18 a 30), *ammesso con riserva* oppure *non ammesso*. Negli ultimi due casi l'orale è obbligatorio; chi è ammesso con voto ≥ 23 può invece chiedere l'**esonero** dall'orale, accettando una riduzione di **5 punti** [programma del corso].
- **L'orale** «integra» il voto dello scritto, non lo incrementa: valuta la preparazione, il ragionamento e la proprietà di linguaggio. Per questo la guida ha un capitolo apposta (capitolo 15) con le definizioni da saper esporre e tutte le dimostrazioni, con la pagina.
- **Il formato dello scritto**: i temi di febbraio-settembre 2026 hanno tre esercizi, sempre dello stesso tipo: (1) data una procedura in pseudocodice, scrivere l'equazione di ricorrenza del suo tempo di esecuzione e risolverla; (2) scrivere una procedura *ricorsiva* (su alberi, grafi o stringhe), spesso con il vincolo «senza parametri aggiuntivi»; (3) un problema di *progettazione* (programmazione dinamica o greedy) da svolgere per intero, fino all'applicazione a un'istanza. I temi di ottobre 2025 e della simulazione di gennaio 2026 hanno invece sei esercizi più brevi. **Tutte e nove le prove sono svolte per intero** nei capitoli 13 e 14.

Come è organizzata la guida

- **Capitoli 1 e 2**: le notazioni asintotiche e le ricorrenze. Sono il primo esercizio dello scritto e la base di tutto il resto.
- **Capitoli 3–6**: heap e ordinamenti, hashing: le strutture dati del programma.
- **Capitolo 7**: gli alberi rosso-neri, con tutti i disegni: rotazioni, inserimento e cancellazione nei casi del libro.
- **Capitoli 8 e 9**: le due strategie di progettazione, con le dimostrazioni del Laboratorio.
- **Capitoli 10–12**: visite dei grafi, ordinamento topologico e cammini minimi.
- **Capitoli 13 e 14**: le nove prove scritte, svolte come si scrivono sul foglio d'esame.
- **Capitolo 15**: come si risponde all'orale, le dimostrazioni da saper fare e le domande tipiche per argomento.

Dentro ogni capitolo, ogni argomento segue lo stesso ordine: prima **l'idea** a parole, poi le definizioni e gli algoritmi nel **pseudocodice del libro** (righe numerate, assegnazione $:=$), poi un **esempio svolto passo per passo**, poi il riquadro «**Come si risolve all'esame**» per i tipi di esercizio e «**Come rispondere**» per le domande d'orale.

Come si studia

1. **Gli algoritmi si eseguono a mano.** Una traccia di esecuzione (un array che cambia, una tabella che si riempie, un albero che ruota) vale più di dieci riletture: prendi un foglio e rifalla senza guardare.
2. **Le ricorrenze sono il primo esercizio dello scritto:** impara il metodo del capitolo 2 — contare le operazioni, scrivere $T(n)$, sciogliere con l'albero o con la sostituzione — e poi allenati sui testi dei temi.
3. **Le dimostrazioni del Laboratorio** (teorema master, altezza degli alberi rosso-neri, correttezza di Dijkstra, scelta golosa) sono le più chieste all'orale: il capitolo 15 le elenca tutte con la pagina.
4. **Prima di leggere lo svolgimento di un tema,** prova a farlo: tre ore, foglio, penna. Poi confronta con il capitolo 13.

Da dove viene ogni cosa

- La fonte principale è il libro di testo del corso: **Cormen, Leiserson, Rivest, Stein, *Introduzione agli algoritmi e strutture dati*, 3^a edizione italiana.** Ogni definizione, algoritmo e dimostrazione riporta accanto capitolo e pagina, per esempio [Cormen p. 283] o [Cormen cap. 6.2]. Gli algoritmi hanno i nomi del libro (MAX-HEAPIFY, RB-INSERT, DIJKSTRA).
- Il passo che sta nel libro ma *fuori dalle parti assegnate* dal programma (per esempio le visite dei grafi) è segnato [Cormen, cap. 22.4]. Ciò che è della guida e non del libro è segnato [non nelle fonti del corso].
- Le **prove scritte** (nove temi, dal 27 ottobre 2025 al 21 settembre 2026) sono riportate alla lettera e svolte per intero.
- **Ogni numero ed esecuzione è ricalcolato da un programma** (cartella **verifiche**): gli algoritmi sono implementati in Python e fatti girare sulle stesse istanze mostrate nel testo.

Convenzioni usate in tutta la guida

- **Pseudocodice** come il libro e come i temi d'esame: array 1-based, righe numerate, assegnazione $:=$ (il libro scrive $\leftarrow$: è la stessa cosa), nomi di procedura in maiuscolo.
- **Notazione asintotica** del capitolo 3 del libro: O , Ω , Θ , o , ω , con le costanti c e n_0 .
- I nomi delle procedure dei temi d'esame sono quelli strani del professore (MADCHAIN, REFINECOMBINE): restano come sono.

I riquadri

Indaco	l'idea, a parole, prima delle formule
Blu	definizioni, teoremi e algoritmi del libro
Verde acqua	come si risolve un tipo di esercizio d'esame
Viola	come rispondere a una domanda
Verde	che cosa chiede l'esame
Rosso	errori tipici
Arancio	regole e convenzioni da ricordare
Grigio	note e ripassi

Capitolo 1

Analisi degli algoritmi e notazioni asintotiche

Prima di chiedersi *quale* algoritmo usare, serve un modo per dire quanto costa un algoritmo senza dipendere dal computer: lo costruiamo su un esempio completo, INSERTION-SORT, e ne ricaviamo il linguaggio di tutta la guida, le notazioni Θ , O , Ω , o , ω del capitolo 3 del libro.

1.1 Che cosa significa analizzare un algoritmo

L'idea: prevedere il tempo, non cronometrarlo

Per sapere se un algoritmo è utilizzabile non serve farlo girare: basta *contare* quante operazioni esegue. Si immagina una macchina ideale con istruzioni elementari a costo costante (il modello RAM), si misura l'input con un numero n (per l'ordinamento, il numero di elementi) e si studia il numero di operazioni in funzione di n . Cambiando macchina cambiano i tempi reali, ma non *come cresce* il tempo al crescere di n : e per input grandi è la crescita che decide quale algoritmo vince.

Il modello RAM (random-access machine) [Cormen cap. 2.2]

Le istruzioni sono eseguite una dopo l'altra, ciascuna in tempo costante: aritmetica (addizione, sottrazione, moltiplicazione, divisione, resto, $\lfloor \cdot \rfloor$, $\lceil \cdot \rceil$), spostamento di dati (load, store, copy) e controllo (salti, chiamate). I dati sono interi e numeri in virgola mobile; una parola contiene $c \lg n$ bit, con $c \geq 1$ costante, così ogni valore dell'input sta in una parola e indicizzare un elemento costa $O(1)$. Il modello *non* ha istruzioni "furbe" (per esempio un ordinamento in un'istruzione) e non modella la memoria cache: è il modello dei computer reali.

Che cosa si conta. L'esecuzione dell' i -esima riga dello pseudocodice costa c_i ; il tempo $T(n)$ è la somma dei prodotti "costo $\times$ numero di volte" su tutte le righe [Cormen cap. 2.2]. Input di dimensione n possono costare in modo diverso: si distinguono il **caso migliore**, il **caso peggiore** e il **caso medio**, e si studia di norma il peggiore perché è un limite superiore valido per *ogni* input, capita spesso e il caso medio è quasi sempre dello stesso ordine.

1.2 L'analisi di Insertion-Sort

INSERTION-SORT(A)

```
1  for  $j := 2$  to  $A.length$ 
2       $key := A[j]$ 
3      // inserisce  $A[j]$  in  $A[1 \dots j-1]$ 
4       $i := j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i+1] := A[i]$ 
7           $i := i - 1$ 
8       $A[i+1] := key$ 
```

riga	costo	numero di volte
1	c_1	n
2	c_2	$n - 1$
3	0	$n - 1$
4	c_4	$n - 1$
5	c_5	$\sum_{j=2}^n t_j$
6	c_6	$\sum_{j=2}^n (t_j - 1)$
7	c_7	$\sum_{j=2}^n (t_j - 1)$
8	c_8	$n - 1$

t_j = esecuzioni del test della riga 5 nell'iterazione j (una in più del corpo, alla fine del ciclo). [Cormen cap. 2.1-2.2]

Il caso migliore è l'array già ordinato: ogni test della riga 5 fallisce subito, quindi $t_j = 1$ per ogni j . Il caso peggiore è l'array in ordine decrescente: la chiave va spostata fino in fondo, quindi $t_j = j$.

Sommando i prodotti della tabella si ottiene la formula generale, e poi i due casi [Cormen pp. 23–24]:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

migliore: $T(n) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) = an + b$

peggiore: $T(n) = \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n - (c_2 + c_4 + c_5 + c_8)$

$$= an^2 + bn + c \quad (1.1)$$

dove, nel caso peggiore, si sono usate le due somme $\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$ e $\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$.

Conclusione. I costi c_i dipendono dalla macchina, ma la *forma* di $T(n)$ no: nel caso migliore è lineare, nel caso peggiore quadratica. Ignorando i termini di ordine inferiore e il coefficiente di n^2 resta il solo tasso di crescita: il caso peggiore di INSERTION-SORT è $\Theta(n^2)$, quello migliore $\Theta(n)$ [Cormen cap. 2.2].

Esempio (la Figura 2.2 del libro). Su $A = (5, 2, 4, 6, 1, 3)$, per ogni iterazione l'array dopo l'inserimento e il suo t_j [Cormen fig. 2.2]; a fianco i conteggi totali per $n = 5$ e $n = 8$: i test del while valgono $n - 1$ nel caso migliore e $\frac{n(n+1)}{2} - 1$ nel peggiore, gli spostamenti 0 e $\frac{n(n-1)}{2}$.

j	chiave	array dopo l'inserimento	t_j	spost.
2	2	(2, 5, 4, 6, 1, 3)	2	1
3	4	(2, 4, 5, 6, 1, 3)	2	1
4	6	(2, 4, 5, 6, 1, 3)	1	0
5	1	(1, 2, 4, 5, 6, 3)	5	4
6	3	(1, 2, 3, 4, 5, 6)	4	3
totale			14	9

caso	n	test	spost.
migliore	5	4	0
migliore	8	7	0
peggiore	5	14	10
peggiore	8	35	28

“Test”: esecuzioni della riga 5 ($\sum t_j$), “spost.”: riga 6 ($\sum (t_j - 1)$). Le tabelle applicano le formule del libro; per $n = 6$ il caso peggiore dà 20 e 15: la traccia è in mezzo. [non nelle fonti del corso]

1.3 Gli invarianti di ciclo

Invariante di ciclo [Cormen cap. 2.1]

Un **invariante di ciclo** è una proprietà che lega le variabili del ciclo e che resta vera a ogni iterazione. Per dimostrare la correttezza di un algoritmo con un invariante di ciclo si provano tre cose.

- **Inizializzazione:** l'invariante è vera prima della prima iterazione.
- **Conservazione:** se è vera prima di un'iterazione, resta vera prima della successiva.
- **Conclusione:** quando il ciclo termina, l'invariante dà una proprietà utile a dimostrare che l'algoritmo è corretto.

Le prime due sono il caso base e il passo induttivo di un'induzione; la terza è ciò che l'induzione non ha: la fine del “processo induttivo”.

Per INSERTION-SORT l'invariante del ciclo esterno è: *all'inizio di ogni iterazione del ciclo for, il sottoarray $A[1 \dots j-1]$ è ordinato ed è formato dagli stessi elementi che erano originariamente in $A[1 \dots j-1]$* [Cormen cap. 2.1].

Inizializzazione. Prima della prima iterazione $j = 2$, quindi $A[1 \dots j-1] = A[1 \dots 1]$ è un solo elemento: ordinato, e uguale a sé stesso. **Conservazione.** Il corpo sposta a destra $A[j-1]$, $A[j-2]$, ... finché non trova il posto per $key = A[j]$ (righe 4–7) e ve lo inserisce (riga 8): $A[1 \dots j]$ è ordinato e

contiene gli stessi elementi di prima, quindi l'incremento di j conserva l'invariante. *Conclusione.* Alla fine del ciclo for (contatore del ciclo al valore che ha superato il limite) è $j = A.length + 1 = n + 1$: l'invariante dice che $A[1 \dots n]$ è ordinato ed è formato dagli elementi di partenza, cioè l'intero array è ordinato.

Invarianti di ciclo: come rispondere

1. **Definizione:** è una proprietà che resta vera all'inizio di ogni iterazione, con le tre proprietà: inizializzazione, conservazione, conclusione.
2. **Analogia con l'induzione:** le prime due sono caso base e passo; la conclusione dà la correttezza perché il ciclo finisce.
3. **Esempio pronto:** INSERTION-SORT con l'invariante " $A[1 \dots j - 1]$ è ordinato e contiene gli stessi elementi di partenza" e i tre passi come sopra; scritto *prima* del test del ciclo, e la conclusione usa l'uscita del ciclo.
4. **Perché conta:** è il modo standard di dimostrare la correttezza di un ciclo (lo useremo per gli heap, gli alberi rosso-neri e le visite); all'orale la domanda "dimostra che l'algoritmo è corretto" si apre così.

1.4 Le notazioni asintotiche

Il tasso di crescita è quello che conta per input grandi: per questo si descrivono i tempi di esecuzione a meno di costanti e di termini di ordine inferiore. Le notazioni si definiscono per *funzioni* su $\mathbb{N}$ che supporremo *asintoticamente non negative* (positive per ogni n sufficientemente grande); $\lg n$ indica $\log_2 n$ [Cormen cap. 3.1].

Θ , O , Ω [Cormen cap. 3.1]

$\Theta(g(n)) = \{ f(n) : \text{esistono } c_1, c_2, n_0 > 0 \text{ con } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ per ogni } n \geq n_0 \}$

$O(g(n)) = \{ f(n) : \text{esistono } c, n_0 > 0 \text{ con } 0 \leq f(n) \leq c g(n) \text{ per ogni } n \geq n_0 \}$

$\Omega(g(n)) = \{ f(n) : \text{esistono } c, n_0 > 0 \text{ con } 0 \leq c g(n) \leq f(n) \text{ per ogni } n \geq n_0 \}$

$f(n) = \Theta(g(n))$ significa che f sta fra $c_1 g$ e $c_2 g$ (limite *asintoticamente stretto*); $f(n) = O(g(n))$ che f sta sotto $c g$ (limite superiore); $f(n) = \Omega(g(n))$ che sta sopra (limite inferiore). Le costanti servono: $\frac{1}{2}n^2 - 3n = \Theta(n^2)$ con $c_1 = \frac{1}{14}$, $c_2 = \frac{1}{2}$ e $n_0 = 7$ (la disuguaglianza di sinistra è un'uguaglianza in $n = 7$).

Teorema 3.1 (dai due limiti a quello stretto) [Cormen cap. 3.1]

Per ogni coppia di funzioni $f(n)$ e $g(n)$

$$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ e } f(n) = \Omega(g(n)). \quad (1.2)$$

Dimostrazione. $(\Rightarrow)$ Da $c_1 g(n) \leq f(n) \leq c_2 g(n)$ per $n \geq n_0$: la disuguaglianza di sinistra dà $f = \Omega(g)$ con $c = c_1$, quella di destra dà $f = O(g)$ con $c = c_2$. $(\Leftarrow)$ Se $c_1 g \leq f$ per $n \geq n_1$ e $f \leq c_2 g$ per $n \geq n_2$, entrambe valgono per $n \geq \max(n_1, n_2)$: è la definizione di $f = \Theta(g)$. $\square$

Proprietà (tutte con f, g, h asintoticamente non negative): *transitività* (da $f = \Theta(g)$ e $g = \Theta(h)$ segue $f = \Theta(h)$, e così per O, Ω, o, ω); *riflessività* ($f = \Theta(f)$, $f = O(f)$, $f = \Omega(f)$); *simmetria* ($f = \Theta(g) \iff g = \Theta(f)$); *simmetria trasposta* ($f = O(g) \iff g = \Omega(f)$, e $f = o(g) \iff g = \omega(f)$) [Cormen cap. 3.1]. Le notazioni si comportano come i confronti fra numeri reali: $f = O(g)$ sta a $a \leq b$, $f = \Omega(g)$ a $a \geq b$, $f = \Theta(g)$ a $a = b$, $f = o(g)$ a $a < b$ e $f = \omega(g)$ a

$a > b$. Una proprietà dei reali *non* si trasferisce: la *tricotomia*. n e $n^{1+\sin n}$ non sono confrontabili (l'esponente oscilla fra 0 e 2), quindi “più piccolo” e “più grande” non esauriscono i casi.

o e ω : limiti non stretti [Cormen cap. 3.1]

$o(g(n))$ è l'insieme delle $f(n)$ tali che per ogni costante $c > 0$ esiste $n_0 > 0$ con $0 \leq f(n) < c g(n)$ per ogni $n \geq n_0$; la differenza con O è che lì basta *qualche* c . In $\omega(g(n))$ la disuguaglianza è $0 \leq c g(n) < f(n)$, e vale $f = \omega(g) \iff g = o(f)$. In termini di limiti, per funzioni asintoticamente non negative:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \iff f(n) = o(g(n)), \quad f(n) = \omega(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \quad (1.3)$$

(se il limite esiste). Esempi: $2n = o(n^2)$, ma $2n^2 \neq o(n^2)$; $n^2 = \omega(n)$ e per esempio $\frac{n^2}{\lg n} = \omega(n)$, perché $\frac{n}{\lg n} \rightarrow \infty$.

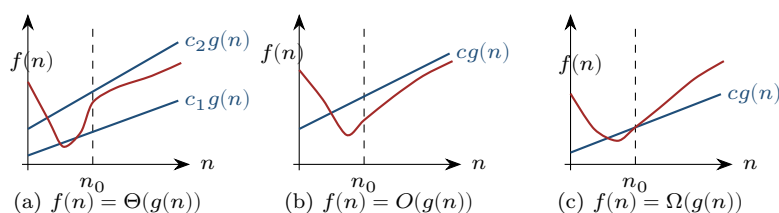


Figura 1.1: Le tre notazioni (ridisegno della Figura 3.1 del libro [Cormen fig. 3.1]). A destra di n_0 : Θ rinchiude f fra $c_1 g$ e $c_2 g$; O tiene f sotto $c g$; Ω la tiene sopra.

1.5 Confronto di funzioni e ordini di crescita

Molte funzioni standard compaiono nell'analisi; per confrontarle bastano due fatti: la *monotonia* (somma e composizione di funzioni monotone crescenti sono monotone crescenti [Cormen cap. 3.2]) e il *confronto col limite* della formula (1.3): se $L = \lim_{n \rightarrow \infty} f(n)/g(n)$ esiste, allora $L = 0 \Rightarrow f = o(g)$, $0 < L < \infty \Rightarrow f = \Theta(g)$ e $L = \infty \Rightarrow f = \omega(g)$. La tabella raccoglie gli ordini di crescita tipici [Cormen cap. 3.2]; i valori a $n = 1000$ sono valutazioni dirette [non nelle fonti del corso]. I fatti del libro usati: i polinomi sono $\Theta(n^d)$, $\lg^b n = o(n^a)$ per ogni $a > 0$, $n^b = o(a^n)$ per $a > 1$, $\lg(n!) = \Theta(n \lg n)$.

ordine di crescita	funzione tipica	valore per $n = 1000$
$\Theta(1)$	1	1
$\Theta(\lg n)$	$\lg n$	9.97
$\Theta(\sqrt{n})$	$\sqrt{n}$	31.6
$\Theta(n)$	n	1000
$\Theta(n \lg n)$	$n \lg n$	$9.97 \cdot 10^3$
$\Theta(n^2)$	n^2	$1.00 \cdot 10^6$
$\Theta(n^3)$	n^3	$1.00 \cdot 10^9$
$\Theta(2^n)$	2^n	$1.07 \cdot 10^{301}$
$\Theta(n!)$	$n!$	$4.02 \cdot 10^{2567}$

Esercizio tipico: ordinare le funzioni per ordine di crescita. Un sottoinsieme delle 30 funzioni del Problema 3-3 del libro [Cormen problema 3-3], in ordine di crescita ($\prec$ significa o : ogni funzione è o della successiva, e i rapporti tendono a 0 per i limiti (1.3)):

$$n^{1/\lg n} = 2 \prec \ln \ln n \prec \sqrt{\lg n} \prec \lg^2 n \prec \sqrt{n} \prec n \prec n \lg n \\ \prec 4^{\lg n} = n^2 \prec n^3 \prec (3/2)^n \prec 2^n \prec n!$$

Le trappole. $n^{1/\lg n}$ è costante: $2^{\lg n \cdot (1/\lg n)} = 2$. Poi $\ln \ln n = o(\sqrt{\lg n})$ (con $t = \lg n$ è $\ln t = o(t^{1/2})$) e $4^{\lg n} = 2^{2 \lg n} = n^2$, la stessa classe di equivalenza di n^2 ; infine $(3/2)^n = o(2^n)$ perché $(\frac{3}{4})^n \rightarrow 0$, e $2^n = o(n!)$. In mezzo: ogni potenza batte i polilogaritmi, ogni esponenziale i polinomi.

Dimostrare che $f(n) = O(g(n))$

Strada 1: con le costanti (la definizione). Devi esibire c e n_0 con $0 \leq f(n) \leq c g(n)$ per ogni $n \geq n_0$. Esempio, $f(n) = 3n^2 + 5n$ e $g(n) = n^2$: per $n \geq 5$ vale $5n \leq n^2$, quindi $3n^2 + 5n \leq 3n^2 + n^2 = 4n^2$: bastano $c = 4$ e $n_0 = 5$. Scaletta: (a) isola il termine dominante di f ; (b) maggiora i termini minori con g da un n_0 in poi; (c) dichiara c e n_0 .
Strada 2: col limite. Se $L = \lim_{n \rightarrow \infty} f(n)/g(n)$ esiste: L finito non nullo $\Rightarrow f = \Theta(g)$; $L = 0 \Rightarrow f = o(g)$; $L = \infty \Rightarrow f = \omega(g)$ (e $f = \Theta(g)$ implica O e Ω per il Teorema 3.1). Esempio: $\lim(3n^2 + 5n)/n^2 = 3$, quindi $3n^2 + 5n = \Theta(n^2)$, e va bene ogni $c > 3$. Se il limite non esiste (per esempio n contro $n^{1+\sin n}$) si torna alla definizione.

Notazione asintotica: come rispondere

- Definizioni:** Θ con c_1, c_2, n_0 ($c_1 g \leq f \leq c_2 g$ da n_0 in poi), O con c ($f \leq c g$), Ω con c ($c g \leq f$); o (f sotto $c g$ per ogni $c > 0$) e ω ($g = o(f)$).
- Relazione:** Teorema 3.1, $f = \Theta(g) \iff f = O(g)$ e $f = \Omega(g)$.
- Proprietà:** transitività, riflessività, simmetria di Θ , trasposizione ($O \leftrightarrow \Omega$, $o \leftrightarrow \omega$); analogia coi numeri reali ($O \leftrightarrow \leq$, $\Theta \leftrightarrow =$, $\Omega \leftrightarrow \geq$, $o \leftrightarrow <$, $\omega \leftrightarrow >$), ma la tricotomia non vale: n e $n^{1+\sin n}$ non sono confrontabili.
- Esempi:** $2n = o(n^2)$, $2n^2 = \Theta(n^2)$, $n^2 = \omega(n)$, $\frac{n^2}{\lg n} = \omega(n)$ (il rapporto $\frac{n}{\lg n} \rightarrow \infty$).
- Uso:** scrivere $T(n) = \Theta(n^2)$ per il caso peggiore di INSERTION-SORT significa che esistono costanti che lo rinchiudono; il caso migliore è $\Theta(n)$, quindi il tempo “senza aggettivi” è fra $\Omega(n)$ e $O(n^2)$.

Errori tipici

- Confondere O e Θ .** $n = O(n^2)$ è vero, ma $n = \Theta(n^2)$ è falso: O è solo un limite superiore. Dire “il tempo di INSERTION-SORT è $\Theta(n^2)$ ” senza dire *nel caso peggiore* è sbagliato (per un array ordinato è $\Theta(n)$).
- Dimenticare n_0 ,** o usare il limite quando non esiste (per esempio n contro $n^{1+\sin n}$); e da $\lim f/g = 0$ si deduce $f = o(g)$, non Θ (per Θ serve un limite finito e non nullo, o le due costanti).
- Confondere costo e conteggio:** il numero di esecuzioni di una riga (per esempio $\sum t_j$) si moltiplica per il suo costo c_i , non si sommano i c_i di righe diverse.

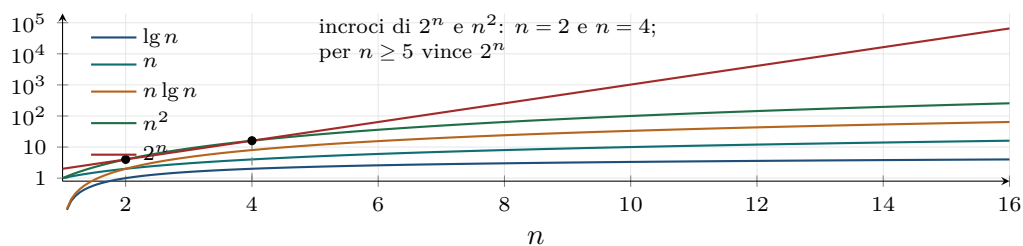


Figura 1.2: Gli ordini di crescita a confronto (y in scala logaritmica): 2^n incrocia n^2 in $n = 2$ e in $n = 4$, poi la supera. [non nelle fonti del corso]

Capitolo 2

Divide et impera e ricorrenze

Le due domande del primo esercizio dello scritto: *come si progetta* un algoritmo ricorsivo efficiente (divide et impera) e *come si risolve* la sua ricorrenza, con i tre strumenti (sostituzione, albero di ricorsione, teorema master) e la dimostrazione del master.

2.1 Divide et impera: merge sort

L'idea: dividere per regnare

Un problema grande si risolve dividendo l'input in parti più piccole dello stesso tipo, risolvendo *ricorsivamente* ciascuna parte e unendo i risultati. Per ordinare n numeri: si spezza l'array in due metà, si ordinano e si fondono. La fusione è quasi gratis: con due mazzi di carte *già ordinati* basta guardare le due carte in cima e prendere la più piccola, un passo per carta; e poiché si dimezza a ogni livello, il totale è $\Theta(n \lg n)$ invece di $\Theta(n^2)$.

Il paradigma divide et impera

Algoritmo *divide et impera*: risolve un problema con tre passi a ogni livello di ricorsione.

- **Divide**: il problema è diviso in sottoproblemi, istanze più piccole dello stesso problema.
- **Impera**: i sottoproblemi sono risolti ricorsivamente (o direttamente, se piccoli).
- **Combina**: le soluzioni dei sottoproblemi si uniscono in quella del problema originale.

Quando i sottoproblemi si risolvono ricorsivamente si ha il **caso ricorsivo**; quando non serve più ricorsione, la ricorsione ha **toccato il fondo**: è il **caso base**.

[Cormen cap. 2.3.1]

Merge sort divide l'array in due metà, le ordina ricorsivamente e le fonde con la procedura ausiliaria MERGE.

MERGE(A, p, q, r)

```
1   $n_1 := q - p + 1$ 
2   $n_2 := r - q$ 
3  crea due nuovi array  $L[1..n_1 + 1]$  e  $R[1..n_2 + 1]$ 
4  for  $i := 1$  to  $n_1$ 
5       $L[i] := A[p + i - 1]$ 
6  for  $j := 1$  to  $n_2$ 
7       $R[j] := A[q + j]$ 
8   $L[n_1 + 1] := \infty$ 
9   $R[n_2 + 1] := \infty$ 
10  $i := 1$ 
11  $j := 1$ 
12 for  $k := p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] := L[i]$ 
15          $i := i + 1$ 
16     else  $A[k] := R[j]$ 
17          $j := j + 1$ 
```

[Cormen p. 27]

La procedura assume che $A[p..q]$ e $A[q+1..r]$ siano già ordinati e li fonde in $A[p..r]$: le righe 4–7 copiano i due sottoarray in L e R , le righe 8–9 mettono in fondo a ciascuno una **sentinella** ∞ , così nelle righe 12–17 non serve controllare se un array è finito. Ogni riga 1–3 e 8–11 costa un tempo costante e i tre cicli costano $\Theta(n_1 + n_2) = \Theta(n)$: MERGE costa $\Theta(n)$, con $n = r - p + 1$.

Esempio (una fusione). Sia $A[9..16] = \langle 2, 4, 5, 7, 1, 2, 3, 6 \rangle$ e si chiami $\text{MERGE}(A, 9, 12, 16)$. Allora $L = \langle 2, 4, 5, 7, \infty \rangle$ e $R = \langle 1, 2, 3, 6, \infty \rangle$: le otto iterazioni del ciclo sono queste, con l'array $A[9..16]$ aggiornato dopo ogni passo.

k	9	10	11	12	13	14	15	16
$L[i]$	2	2	4	4	4	5	7	7
$R[j]$	1	2	2	3	6	6	6	∞
$A[k]$	1	2	2	3	4	5	6	7
(i, j) dopo	$(1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4), (4, 5), (5, 5)$							

Alla fine $A[9..16] = \langle 1, 2, 2, 3, 4, 5, 6, 7 \rangle$, e in L e R restano solo le due sentinelle: esattamente $r - p + 1 = 8$ elementi copiati, un confronto per elemento. [Cormen fig. 2.3]

$\text{MERGE-SORT}(A, p, r)$

```

1  if  $p < r$ 
2       $q := \lfloor (p + r)/2 \rfloor$ 
3       $\text{MERGE-SORT}(A, p, q)$ 
4       $\text{MERGE-SORT}(A, q + 1, r)$ 
5       $\text{MERGE}(A, p, q, r)$ 
```

[Cormen p. 29]

La chiamata iniziale è $\text{MERGE-SORT}(A, 1, A.length)$. Se $p \geq r$ il sottoarray ha al più un elemento ed è già ordinato; altrimenti il passo *divide* calcola l'indice di mezzo q (i due sottoarray hanno $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$ elementi), le righe 3–4 sono il passo *impera* e la riga 5 il passo *combina*.

Esempio (merge sort, Figura 2.4). Su $\langle 5, 2, 4, 7, 1, 3, 2, 6 \rangle$ l'algoritmo, letto dal basso, fonde prima le coppie ($\langle 2, 5, 4, 7, 1, 3, 2, 6 \rangle$), poi le sequenze di due ($\langle 2, 4, 5, 7, 1, 2, 3, 6 \rangle$), e infine le due metà: $\langle 1, 2, 2, 3, 4, 5, 6, 7 \rangle$. [Cormen fig. 2.4]

2.1.1 La ricorrenza di merge sort

Il tempo di esecuzione di un algoritmo ricorsivo si descrive con una **equazione di ricorrenza**: $T(n)$ in funzione del tempo per input più piccoli. Per un divide et impera che genera a sottoproblemi di dimensione $1/b$, con $D(n)$ tempo per dividere e $C(n)$ per combinare, si ha $T(n) = aT(n/b) + D(n) + C(n)$ per n grande e $T(n) = \Theta(1)$ per n piccolo. Per merge sort $a = b = 2$, $D(n) = \Theta(1)$ (il solo calcolo di q) e $C(n) = \Theta(n)$ (la fusione):

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1, \\ 2T(n/2) + \Theta(n) & \text{se } n > 1. \end{cases} \quad (2.1)$$

[Cormen cap. 2.3.2]

La ricorrenza “esatta”, per n dispari, è $T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n)$: *floor* e *ceiling* e le condizioni al contorno non influiscono sul tasso di crescita, quindi si trascurano e si assume $T(n)$ costante per n piccolo. Una ricorrenza con disuguaglianza, come $T(n) \leq 2T(n/2) + \Theta(n)$, dà un limite superiore (O), non Θ . [Cormen cap. 4, introduzione]

Per risolvere la (2.1) senza aspettare il teorema master, la si riscrive con una costante esplicita:

$$T(n) = 2T(n/2) + cn,$$

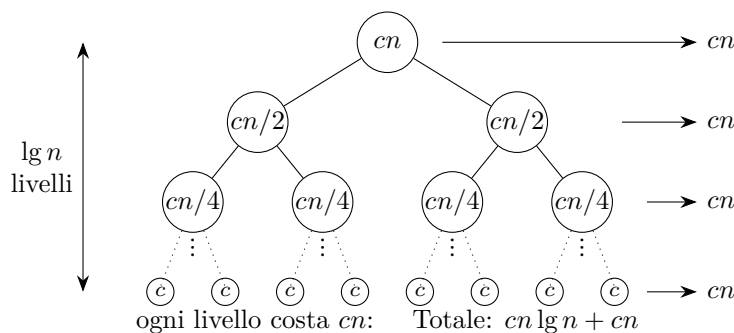


Figura 2.1: L'albero di ricorsione di merge sort. Ogni livello ha costo cn e i livelli sono $\lg n + 1$: il costo è $cn \lg n + cn$, cioè $\Theta(n \lg n)$.

[Cormen fig. 2.5]

per $n > 1$, con $T(1) = c$. L'albero di ricorsione della Figura 2.1 mostra come si sommano i costi livello per livello: la radice costa cn , e i due figli, di dimensione $n/2$, costano insieme $c(n/2) + c(n/2) = cn$; in generale il livello i ha 2^i nodi di costo $c(n/2^i)$ ciascuno, per un totale $2^i c(n/2^i) = cn$ per ogni livello. Si continua finché la dimensione non è 1: il numero di livelli è $\lg n + 1$ (il logaritmo in base 2, che nel libro si scrive $\lg$), e l'ultimo livello ha n nodi di costo c (cn). Il costo totale è $cn(\lg n + 1) = cn \lg n + cn$, cioè $\Theta(n \lg n)$.

Con le sentinelle i confronti di MERGE si contano esattamente: ogni iterazione del ciclo 12–17 fa *un* confronto, quindi una fusione di $r - p + 1$ elementi ne fa $r - p + 1$ e il totale non dipende dall'input (0, 2, 5, 8, 12, 16, 20, 24, 29, ... per $n = 1, 2, \dots, 9$; in formula $n \lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + n$). [non nelle fonti del corso]

2.2 Risolvere una ricorrenza: il metodo di sostituzione

L'idea: indovinare, poi verificare

Due passi: **1.** ipotizzare la forma della soluzione (per esempio $O(n \lg n)$); **2.** dimostrarla per induzione, scegliendo le costanti. Il nome viene dal fatto che la soluzione ipotizzata si *sostituisce* alla funzione quando l'ipotesi induttiva si applica ai valori più piccoli; funziona per i limiti superiori e inferiori. Serve saper indovinare, e per indovinare c'è l'albero di ricorsione.

[Cormen cap. 4.3]

Esempio 1: $T(n) = 2T(\lfloor n/2 \rfloor) + n$. Proviamo $T(n) = O(n \lg n)$, cioè $T(n) \leq cn \lg n$ con $c > 0$: se il limite vale per $\lfloor n/2 \rfloor$, cioè $T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$, sostituendo nella ricorrenza

$$\begin{aligned} T(n) &\leq 2(c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \leq cn \lg(n/2) + n \\ &= cn \lg n - cn \lg 2 + n = cn \lg n - cn + n \leq cn \lg n, \end{aligned}$$

e l'ultimo passo è vero quando $c \geq 1$. Restano le **condizioni al contorno**: con $T(1) = 1$ il limite a $n = 1$ diventa $T(1) \leq c \cdot 1 \cdot \lg 1 = 0$, falso. Ma la notazione asintotica chiede solo $n \geq n_0$: si escludono i valori piccoli e si prendono $T(2)$ e $T(3)$ come *casi base della dimostrazione*, con $n_0 = 2$. Dalla ricorrenza $T(2) = 4$ e $T(3) = 5$; per $n = 2$ serve $4 \leq 2c$, cioè $c \geq 2$, e per $n = 3$ serve $5 \leq 3c \lg 3$, che $c \geq 2$ soddisfa. Si distingue il *caso base della ricorrenza* ($n = 1$) dai *casi base della dimostrazione* ($n = 2, 3$).

Esempio 2: $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$. L'ipotesi " $T(n) \leq cn$ " dà $T(n) \leq cn + 1$, che non è $\leq cn$. Si *rafforza* allora l'ipotesi sottraendo un termine di ordine inferiore, $T(n) \leq cn - d$ con $d \geq 0$:

$$T(n) \leq (c \lfloor n/2 \rfloor - d) + (c \lceil n/2 \rceil - d) + 1 = cn - 2d + 1 \leq cn - d, \quad \text{per } d \geq 1,$$

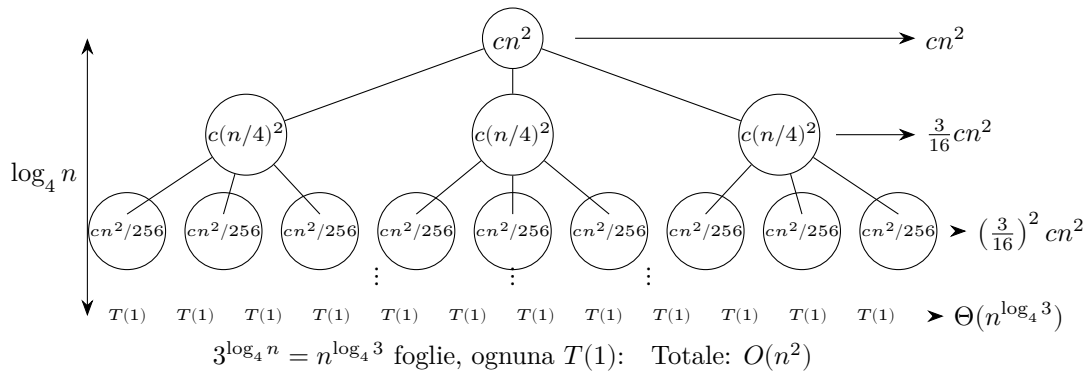


Figura 2.2: Albero di ricorsione per $T(n) = 3T(n/4) + cn^2$: al livello i ci sono 3^i nodi di costo $c(n/4^i)^2$, cioè $(3/16)^i cn^2$; i coefficienti formano una serie decrescente e il costo è dominato dalla radice.

[Cormen fig. 4.5]

con c grande abbastanza per i casi base. Il termine si sottrae una volta per ogni termine ricorsivo; e un'ipotesi più forte è anche più facile da dimostrare.

2.3 Il metodo dell'albero di ricorsione

L'idea: disegnare i costi, livello per livello

In un **albero di ricorsione** ogni nodo è il costo di un sottoproblema; i nodi di un livello si sommano in un *costo per livello*, e la somma dei livelli è il costo totale. L'albero serve a *fare l'ipotesi* (poi verificata con la sostituzione) e, disegnato con cura, è già una dimostrazione: l'albero di $T(n) = aT(n/b) + f(n)$ è a -ario, alto $\log_b n$, con $n^{\log_b a}$ foglie.

[Cormen cap. 4.4]

Esempio svolto: $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$. Trascurando floor e ceiling si studia $T(n) = 3T(n/4) + cn^2$ (Figura 2.2), con $c > 0$ costante e n potenza esatta di 4, in modo che tutte le dimensioni siano intere.

La dimensione alla profondità i è $n/4^i$, che diventa 1 a $i = \log_4 n$: l'albero ha $\log_4 n + 1$ livelli, e l'ultimo ha $3^{\log_4 n} = n^{\log_4 3}$ nodi di costo $T(1) = \Theta(1)$. Sommando:

$$T(n) = cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \dots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) = \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}).$$

I coefficienti formano una serie geometrica decrescente di ragione $3/16$; la somma infinita vale $1/(1 - 3/16) = 16/13$, quindi

$$T(n) < \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) = \frac{16}{13}cn^2 + \Theta(n^{\log_4 3}) = O(n^2) :$$

il costo totale è dominato dalla radice. Il limite è stretto (la radice da sola costa cn^2) e la sostituzione lo conferma, provando $T(n) \leq dn^2$:

$$T(n) \leq 3d(n/4)^2 + cn^2 = \frac{3}{16}dn^2 + cn^2 \leq dn^2 \quad \text{quando } d \geq \frac{16}{13}c.$$

Con $c = 1$: $T(4) = 19$, $T(16) = 313 = 256 + 48 + 9$, $T(64) = 5035 = 4096 + 768 + 144 + 27$ (livelli e foglie), e $5035 \leq (16/13) \cdot 64^2 = 5041.2$.

Un albero con rami disuguali. Per $T(n) = T(n/3) + T(2n/3) + cn$ ogni livello costa al più cn e il cammino più lungo è lungo $\log_{3/2} n$: la sostituzione conferma $O(n \lg n)$ con $d \geq c/(\lg 3 - 2/3) \approx 1.09c$.

2.4 Il teorema master, e la sua dimostrazione

L'idea: un ricettario per le ricorrenze $aT(n/b) + f(n)$

Per le ricorrenze $aT(n/b) + f(n)$ basta un “ricettario”: si confronta $f(n)$, il costo fuori dalle chiamate, con $n^{\log_b a}$, il costo delle $n^{\log_b a}$ foglie dell'albero (di qui il nome del libro, *metodo dell'esperto*). Se vince uno dei due la soluzione è il vincitore (casi 1 e 3); se sono pari si moltiplica per $\lg n$ (caso 2).

[Cormen cap. 4.5]

Teorema 4.1 — teorema master (nel libro: *teorema dell'esperto*) *da saper dimostrare*

Date le costanti $a \geq 1$ e $b > 1$ e una funzione $f(n)$, sia $T(n)$ una funzione definita sugli interi non negativi dalla ricorrenza

$$T(n) = aT(n/b) + f(n),$$

dove n/b sta per $\lfloor n/b \rfloor$ oppure $\lceil n/b \rceil$. Allora $T(n)$ si limita asintoticamente nei modi seguenti:

1. se $f(n) = O(n^{\log_b a - \varepsilon})$ per qualche costante $\varepsilon > 0$, allora $T(n) = \Theta(n^{\log_b a})$;
2. se $f(n) = \Theta(n^{\log_b a})$, allora $T(n) = \Theta(n^{\log_b a} \lg n)$;
3. se $f(n) = \Omega(n^{\log_b a + \varepsilon})$ per qualche costante $\varepsilon > 0$ e se $a f(n/b) \leq c f(n)$ per una costante $c < 1$ e per ogni n sufficientemente grande, allora $T(n) = \Theta(f(n))$.

[Cormen Teorema 4.1]

Nel caso 1 $f(n)$ deve essere *polinomialmente* più piccola di $n^{\log_b a}$ (serve il fattore n^ε); nel caso 3 polinomialmente più grande e **regolare** ($a f(n/b) \leq c f(n)$), com'è per le funzioni polinomiali che si incontrano. Per $2T(n/2) + n \lg n$ il rapporto $f(n)/n^{\log_b a} = \lg n$ non è n^ε per alcun $\varepsilon > 0$: il teorema *non si applica* (la soluzione è $\Theta(n \lg^2 n)$, Esercizio 4.6-2). E la forma deve essere esattamente $aT(n/b) + f(n)$, non $T(n-1)$ o $T(\lfloor n/2 \rfloor + 17)$.

ricorrenza	$n^{\log_b a}$	$f(n)$	caso	soluzione
$9T(n/3) + n$	n^2	n	1 ($\varepsilon = 1$)	$\Theta(n^2)$
$T(2n/3) + 1$	1	1	2	$\Theta(\lg n)$
$3T(n/4) + n \lg n$	$n^{0.792}$	$n \lg n$	3, regolare	$\Theta(n \lg n)$
$2T(n/2) + n \lg n$	n	$n \lg n$	nessuno	$\Theta(n \lg^2 n)$
$2T(n/2) + \Theta(n)$ (merge sort)	n	$\Theta(n)$	2	$\Theta(n \lg n)$
$8T(n/2) + \Theta(n^2)$ (matrici)	n^3	$\Theta(n^2)$	1	$\Theta(n^3)$
$7T(n/2) + \Theta(n^2)$ (Strassen)	$n^{2.807}$	$\Theta(n^2)$	1 ($\varepsilon = 0.8$)	$\Theta(n^{\lg 7})$

La regolarità del caso 3 si verifica subito: $3(n/4) \lg(n/4) \leq (3/4)n \lg n$ con $c = 3/4$; e $\lg 7$ sta fra 2.80 e 2.81, quindi $f(n) = O(n^{\lg 7 - \varepsilon})$ con $\varepsilon = 0.8$.

2.4.1 La dimostrazione

da saper dimostrare

[Cormen, cap. 4.6] Due parti: prima n potenza esatta di b , con tre lemmi (nell'albero, i tre casi sono il costo dominato dalle foglie, distribuito equamente fra i livelli o dominato dalla radice), poi ogni intero n , con un trucco per *floor* e *ceiling*.

Dimostrazione. Lemma 4.2. Siano $a \geq 1$ e $b > 1$ costanti e $f(n)$ una funzione non negativa definita sulle potenze esatte di b . Se $T(n)$ è definita sulle potenze esatte di b da $T(n) = \Theta(1)$ per $n = 1$ e $T(n) = aT(n/b) + f(n)$ per $n = b^i$, con i intero positivo, allora

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j). \quad (4.21)$$

Dimostrazione. Si usa l'albero di ricorsione: la radice ha costo $f(n)$ e a figli di costo $f(n/b)$; in generale, alla profondità j ci sono a^j nodi di costo $f(n/b^j)$. Ogni foglia è a profondità $\log_b n$, perché $n/b^{\log_b n} = 1$, e l'albero ha $a^{\log_b n} = n^{\log_b a}$ foglie, ciascuna di costo $T(1) = \Theta(1)$. Sommando i costi dei nodi interni, livello per livello, e delle foglie, si ottiene la (4.21). $\square$

Lemma 4.3. *Siano $a \geq 1$, $b > 1$ e $f(n) \geq 0$ come sopra, e sia $g(n) = \sum_{j=0}^{\log_b n-1} a^j f(n/b^j)$. Allora, sulle potenze esatte di b : (1) se $f(n) = O(n^{\log_b a - \varepsilon})$ per qualche $\varepsilon > 0$, $g(n) = O(n^{\log_b a})$; (2) se $f(n) = \Theta(n^{\log_b a})$, $g(n) = \Theta(n^{\log_b a} \lg n)$; (3) se $af(n/b) \leq cf(n)$ per una costante $c < 1$ e per ogni n grande, $g(n) = \Theta(f(n))$.*

Dimostrazione. Caso 1: da $f(n) = O(n^{\log_b a - \varepsilon})$ segue $f(n/b^j) = O((n/b^j)^{\log_b a - \varepsilon})$, e sostituendo in $g(n)$ si ottiene una serie geometrica crescente di ragione b^ε :

$$g(n) = O\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right) = O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n-1} (b^\varepsilon)^j\right) = O\left(n^{\log_b a - \varepsilon} \frac{b^\varepsilon - 1}{b^\varepsilon - 1}\right) = O(n^{\log_b a}),$$

perché $a^j b^{-j(\log_b a - \varepsilon)} = b^{\varepsilon j}$ e $b^{\varepsilon \log_b n} = n^\varepsilon$. Caso 2: i termini sono tutti uguali,

$$g(n) = \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} \left(\frac{a}{b^{\log_b a}}\right)^j\right) = \Theta(n^{\log_b a} \log_b n) = \Theta(n^{\log_b a} \lg n).$$

Caso 3: iterando $af(n/b) \leq cf(n)$ si ha $a^j f(n/b^j) \leq c^j f(n)$; la somma è una serie geometrica decrescente,

$$g(n) \leq f(n) \sum_{j=0}^{\infty} c^j + O(1) = f(n) \frac{1}{1-c} + O(1) = O(f(n)),$$

dove $O(1)$ copre i valori “piccoli”; e $g(n) = \Omega(f(n))$ perché il primo termine è $f(n)$. Dunque $g(n) = \Theta(f(n))$. $\square$

Lemma 4.4 (teorema master per le potenze esatte di b). *Con le ipotesi del Lemma 4.2, valgono le stesse tre conclusioni del teorema master. Dimostrazione.* Si sostituiscono i limiti del Lemma 4.3 nella (4.21): caso 1, $\Theta(n^{\log_b a}) + O(n^{\log_b a}) = \Theta(n^{\log_b a})$; caso 2, $\Theta(n^{\log_b a}) + \Theta(n^{\log_b a} \lg n) = \Theta(n^{\log_b a} \lg n)$; caso 3, $\Theta(n^{\log_b a}) + \Theta(f(n)) = \Theta(f(n))$, perché $f(n) = \Omega(n^{\log_b a + \varepsilon})$. $\square$

Floor e ceiling. Per ogni intero n , sia $T(n) = aT(\lceil n/b \rceil) + f(n)$ e $n_0 = n$, $n_j = \lceil n_{j-1}/b \rceil$. Da $\lceil x \rceil \leq x + 1$ segue $n_j \leq n/b^j + \sum_{i<j} 1/b^i < n/b^j + b/(b-1)$, quindi a profondità $\lfloor \log_b n \rfloor$ la dimensione è $O(1)$, e

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\lfloor \log_b n \rfloor - 1} a^j f(n_j). \quad (4.28)$$

La somma si stima come nel Lemma 4.3. Caso 3: $af(\lceil n/b \rceil) \leq cf(n)$ per n grande dà $a^j f(n_j) \leq c^j f(n)$. Caso 2: con $j \leq \lfloor \log_b n \rfloor$ (dunque $b^j/n \leq 1$), $f(n_j) = O((n/b^j + b/(b-1))^{\log_b a}) = O((n/b^j)^{\log_b a} (1 + \frac{b^j}{n} \cdot \frac{b}{b-1})^{\log_b a}) = O(n^{\log_b a}/a^j)$: il fattore fra parentesi è limitato da una costante, quindi vale il caso 2 del Lemma 4.3; il caso 1 è quasi identico. I limiti inferiori si ottengono con $\lceil n/b \rceil \geq n/b$ (e $\lfloor n/b \rfloor \leq n/b$). Questo completa la dimostrazione. $\square$

2.5 Come si risolve all'esame

Data una procedura in pseudocodice, scrivere $T(n)$ e risolverla

La scaletta, sempre la stessa:

1. **Le chiamate ricorsive:** quante sono e su che dimensione (una chiamata su (i, m) , $m = \lfloor (i+j)/2 \rfloor$, ha circa metà degli elementi; due righe di chiamata sono due chiamate).
2. **Il lavoro fuori:** le iterazioni dei cicli; le righe a costo costante spariscono nel Θ .
3. **Scrivere $T(n)$:** $aT(n/b) + f(n)$ col caso base; con dimensioni diverse si scrivono tutti i termini ($T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + f(n)$).

4. **Il metodo:** quello *chiesto dal tema*, controllando il risultato (albero e master devono concordare).

Esempio svolto: MADCHAIN, tema del 13 luglio 2026.

MADCHAIN(A, i, j)

```

1  if  $i \geq j$  return  $A[i]$ 
2   $m := \lfloor (i + j)/2 \rfloor$ 
3   $L := \text{MADCHAIN}(A, i, m)$ 
4   $R := \text{MADCHAIN}(A, m + 1, j)$ 
5  for  $c := 1$  to 10
6      for  $k := i$  to  $j$ 
7           $A[k] := A[k] + L + R$ 
8   $L_2 := \text{MADCHAIN}(A, i, m)$ 
9   $R_2 := \text{MADCHAIN}(A, m + 1, j)$ 
10 for  $b := 1$  to 5
11     for  $k := i$  to  $j$ 
12          $L_2 := L_2 + A[k]$ 
13 return  $L_2 + R_2$ 
```

Con $n = j - i + 1$: quattro chiamate (righe 3, 4, 8, 9) su circa $n/2$ elementi; il lavoro fuori è $10n + 5n = \Theta(n)$. Dunque $T(n) = 4T(n/2) + \Theta(n)$, $T(1) = \Theta(1)$, e l'albero dà $T(n) = \Theta(n^2)$: il livello i costa $2^i cn$, i livelli interni $cn(n-1)$, le n^2 foglie n^2 . Il master conferma (caso 1: $a = 4$, $b = 2$, $n^{\log_2 4} = n^2$, $f = O(n^{2-\varepsilon})$, $\varepsilon = 1$).

Il metodo lo chiede il tema

Quando il tema chiede un metodo preciso, la ricorrenza va risolta con quello, controllando il risultato con un secondo metodo; le procedure svolte nel capitolo 13 mostrano i tre metodi (albero di ricorsione, sostituzione, teorema master) al lavoro.

Teorema master: come rispondere

1. **La forma:** $T(n) = aT(n/b) + f(n)$, $a \geq 1$, $b > 1$; n/b sta per $\lfloor n/b \rfloor$ o $\lceil n/b \rceil$.
2. **Il confronto:** $f(n)$ contro $n^{\log_b a}$; vince la più grande, se sono pari si moltiplica per $\lg n$.
3. **I casi:** f polinomialmente più piccola $\rightarrow \Theta(n^{\log_b a})$; uguale $\rightarrow \Theta(n^{\log_b a} \lg n)$; più grande e regolare ($af(n/b) \leq cf(n)$, $c < 1$) $\rightarrow \Theta(f(n))$.
4. **Esempio e limiti:** $2T(n/2) + \Theta(n) \rightarrow \Theta(n \lg n)$ (caso 2); ma $2T(n/2) + n \lg n$ non si risolve, e la forma dev'essere esatta.

Metodo di sostituzione: come rispondere

1. **Due passi:** indovinare la forma della soluzione, poi dimostrarla per induzione.
2. **Il passo:** si sostituisce l'ipotesi induttiva sui valori piccoli e si fa tornare la disuguaglianza per n .
3. **Il caso base:** c grande abbastanza; il caso difficile ($T(1) = 1$) si esclude, con $n = 2, 3$.
4. **Se non torna:** si rafforza sottraendo un termine di ordine inferiore ($cn - d$), uno per termine ricorsivo; e va provata la forma esatta ($T(n) \leq cn$).

Errori tipici con le ricorrenze

- **Confondere O e Θ :** la ricorrenza con disuguaglianza dà solo un limite superiore.
- **Dimenticare il caso base:** $T(1) = 1$ non soddisfa $T(n) \leq cn \lg n$; si cambiano i casi base, non la ricorrenza.
- **Usare il master fuori dalla forma, o scambiare i casi:** non vale per $T(n) = T(n-1) + n$, e senza il fattore polinomiale (e la regolarità) il caso 3 non si applica; e a, b vengono dalle *chiamate*, non dai cicli.

Capitolo 3

Heap, heapsort e code di priorità

L'idea: un albero che vive in un array

Heapsort ordina n numeri in tempo $O(n \lg n)$ come merge sort, ma *sul posto* come insertion sort, con una struttura dati nuova: lo *heap*, un albero binario quasi completo i cui nodi stanno in un array senza puntatori, perché padre e figli del nodo i si calcolano con $\lfloor i/2 \rfloor$, $2i$, $2i + 1$. L'albero è quasi completo, quindi è sempre basso, altezza $\Theta(\lg n)$: in un *max-heap* il massimo sta in radice e “estrarre il massimo” costa una sola discesa, $O(\lg n)$. La stessa struttura realizza la *coda di priorità*. [Cormen cap. 6]

3.1 Heap

Heap binario

[Cormen cap. 6.1] Uno *heap (binario)* è una struttura dati composta da un array che possiamo considerare come un *albero binario quasi completo*: tutti i livelli sono pieni, tranne eventualmente l'ultimo, riempito da sinistra fino a un certo punto. Ogni nodo corrisponde a un elemento dell'array. Un array A che rappresenta un heap ha due attributi: $A.length$, il numero di elementi dell'array, e $A.heap-size$, quanti di essi sono elementi validi dell'heap ($0 \leq A.heap-size \leq A.length$). Gli elementi validi sono $A[1..A.heap-size]$; la radice è $A[1]$; il padre di i è $\lfloor i/2 \rfloor$, il figlio sinistro $2i$, il figlio destro $2i + 1$.

Proprietà del max-heap e del min-heap

[Cormen cap. 6.1] In un *max-heap*, per ogni nodo i diverso dalla radice, $A[PARENT(i)] \geq A[i]$: **il valore di un nodo è al più quello di suo padre**. Quindi il massimo è nella radice e ogni sottoalbero contiene valori non maggiori di quello della sua radice. In un *min-heap* vale $A[PARENT(i)] \leq A[i]$ e il minimo è nella radice. Heapsort usa i max-heap, le code di priorità l'uno o l'altro tipo.

Altezza e foglie di un heap

[Cormen esercizi 6.1-2 e 6.1-7] L'*altezza di un nodo* è il numero di archi del cammino semplice più lungo che da quel nodo scende a una foglia; l'*altezza di un heap* è l'altezza della radice. Poiché l'albero è quasi completo: (1) un heap di n elementi ha altezza $\lfloor \lg n \rfloor$; (2) le foglie sono esattamente i nodi di indice $\lfloor n/2 \rfloor + 1, \dots, n$. *Dimostrazione di (2)*. Il nodo i ha il figlio sinistro, $2i$, se e solo se $i \leq \lfloor n/2 \rfloor$: i nodi da $\lfloor n/2 \rfloor + 1$ a n sono senza figli, quindi foglie. (1) Un albero quasi completo di altezza h ha al più $2^{h+1} - 1$ nodi e più di $2^h - 1$: da $2^h - 1 < n \leq 2^{h+1} - 1$ segue $h = \lfloor \lg n \rfloor$.

La figura 3.1 mostra lo stesso heap come albero e come array, in tre momenti: l'input, dopo il build di un max-heap, durante heapsort.

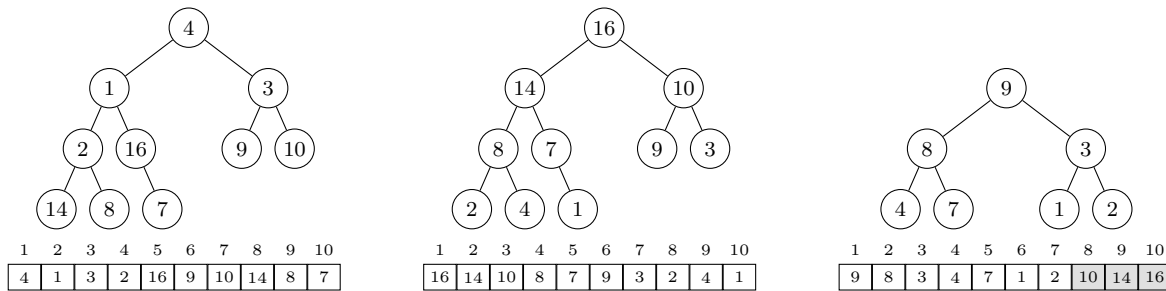


Figura 3.1: Un max-heap come albero quasi completo e come array. **(a)** L'array di input $A = \langle 4, 1, 3, 2, 16, 9, 10, 14, 8, 7 \rangle$: non è un max-heap. **(b)** Il max-heap costruito da BUILD-MAX-HEAP. **(c)** Un istante di heapsort: i nodi in grigio sono già fuori dall'heap. [Cormen figg. 6.1, 6.3a, 6.4d]

3.2 Mantenere la proprietà: MAX-HEAPIFY

Far scendere la chiave fuori posto

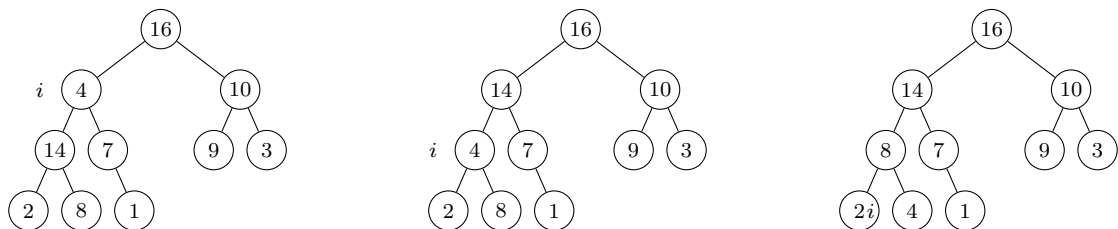
Supponiamo che i due sottoalberi di un nodo i siano già max-heap e che l'unico guaio sia in i : $A[i]$ è più piccolo di un figlio. Basta scambiare $A[i]$ con il *maggiore* dei due figli: in i arriva il maggiore dei tre e la proprietà è ripristinata lì. Il valore piccolo però è sceso nel figlio, dove può violare di nuovo la proprietà: si richiama lì la procedura, e la chiave scende finché non trova figli più piccoli o una foglia. Ogni livello costa $\Theta(1)$: una chiamata costa quanto l'altezza, $O(\lg n)$.

MAX-HEAPIFY(A, i)

```

1   $l := \text{LEFT}(i)$ 
2   $r := \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{massimo} := l$ 
5  else  $\text{massimo} := i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{massimo}]$ 
7       $\text{massimo} := r$ 
8  if  $\text{massimo} \neq i$ 
9      scambia  $A[i]$  con  $A[\text{massimo}]$ 
10     MAX-HEAPIFY( $A, \text{massimo}$ )
```

La figura 3.2 esegue MAX-HEAPIFY($A, 2$) sull'heap $A = \langle 16, 4, 10, 14, 7, 9, 3, 2, 8, 1 \rangle$ (fig. 6.2 del libro): la chiave 4 scende di livello in livello, con due scambi e quattro confronti.



(a) $A[2] = 4$: minore di entrambi i figli.

(b) scambio $A[2] \leftrightarrow A[4]$: ora viola il nodo 4.

(c) scambio $A[4] \leftrightarrow A[9]$: nodo 9 è una foglia.

Figura 3.2: L'azione di MAX-HEAPIFY($A, 2$) su $A = \langle 16, 4, 10, 14, 7, 9, 3, 2, 8, 1 \rangle$, con $A.\text{heap-size} = 10$. [Cormen fig. 6.2]

Quanto costa MAX-HEAPIFY. In un sottoalbero di dimensione n con radice in i , il costo è $\Theta(1)$ per sistemare i con i suoi figli, più il costo di MAX-HEAPIFY in un figlio; i sottoalberi dei figli hanno ciascuno dimensione che non supera $2n/3$, quindi

$$T(n) \leq T(2n/3) + \Theta(1).$$

Perché $2n/3$. Il caso peggiore è l'ultimo livello pieno a metà: con $2^h - 1$ nodi nei livelli pieni e 2^{h-1} nell'ultimo, il sottoalbero sinistro della radice è completo ($2^h - 1$ nodi) e $n = 3 \cdot 2^{h-1} - 1$: rapporto $(2^h - 1)/(3 \cdot 2^{h-1} - 1) < 2/3$. Il **caso 2 del teorema master** (enunciato e dimostrato per intero nel capitolo 2) con $a = 1$, $b = 3/2$ dà $n^{\log_b a} = n^0 = 1 = \Theta(1) = f(n)$, quindi $T(n) = O(\lg n)$; nel caso peggiore $\Omega(\lg n)$ (Esercizio 6.2-6), quindi $T(n) = \Theta(\lg n)$. Inoltre MAX-HEAPIFY su un nodo di altezza h costa $O(h)$: servirà per BUILD-MAX-HEAP. [Cormen cap. 6.2 e 4.5]

3.3 Costruire un heap: BUILD-MAX-HEAP

Sistemare i nodi dal basso verso l'alto

Le foglie $\lfloor n/2 \rfloor + 1, \dots, n$ sono già max-heap di un elemento. Percorro i nodi rimanenti *a rovescio*, da $\lfloor n/2 \rfloor$ a 1: quando tocca a i , i suoi figli hanno indice maggiore di i e sono già radici di max-heap, l'ipotesi di MAX-HEAPIFY. Il costo sembra $O(n \lg n)$, ma i nodi alti, quelli su cui MAX-HEAPIFY lavora di più, sono pochissimi (al più $n/4$ di altezza 1, $n/8$ di altezza 2, ...): la somma dei costi dà $O(n)$.

BUILD-MAX-HEAP(A)

```

1   $A.heap\text{-}size := A.length$ 
2  for  $i := \lfloor A.length/2 \rfloor$  downto 1
3      MAX-HEAPIFY( $A, i$ )
```

Correttezza [Cormen cap. 6.3]: vale l'invariante «a ogni iterazione, i nodi $i + 1, \dots, n$ sono radici di max-heap». *Inizializzazione*: per $i = \lfloor n/2 \rfloor$ i nodi $\lfloor n/2 \rfloor + 1, \dots, n$ sono foglie, radici di max-heap banali. *Conservazione*: i figli di i hanno indice più alto, quindi sono radici di max-heap, che è la condizione richiesta da MAX-HEAPIFY; la chiamata non tocca i nodi a indice più alto. *Conclusione*: alla fine $i = 0$ e ogni nodo è radice di un max-heap, in particolare la radice.

La figura 3.3 riproduce l'esempio del libro ($n = 10$, quindi i parte da 5 e le chiamate sono cinque); sull'istanza, BUILD-MAX-HEAP fa 14 confronti e 7 scambi.

Eseguire BUILD-MAX-HEAP su un array

1. Scrivi l'array e disegna l'albero quasi completo: l'indice di un nodo è il suo numero, non il suo valore.
2. Parti da $i = \lfloor n/2 \rfloor$: i nodi da $\lfloor n/2 \rfloor + 1$ a n sono foglie, già max-heap, e non si toccano.
3. Per ogni i esegui MAX-HEAPIFY: confronta $A[i]$ con i figli $2i$ e $2i + 1$ (esistono solo se $\leq A.heap\text{-}size$), scambia con il maggiore e ripeti la discesa dal figlio dove è scesa la chiave. Scrivi l'array dopo ogni chiamata (figura 3.3): allo scritto la traccia è questa sequenza.

L'analisi: perché $O(n)$ e non $O(n \lg n)$. Ogni chiamata costa $O(\lg n)$ e le chiamate sono $O(n)$, ma il limite $O(n \lg n)$ non è stretto: il tempo di MAX-HEAPIFY in un nodo di altezza h è $O(h)$, e i nodi alti sono pochi. In un heap di n elementi ci sono **al più** $\lceil n/2^{h+1} \rceil$ nodi di altezza h [Cormen esercizio 6.3-3], quindi il costo totale è limitato superiormente da

$$\sum_{h=0}^{\lceil \lg n \rceil} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lceil \lg n \rceil} \frac{h}{2^h}\right).$$

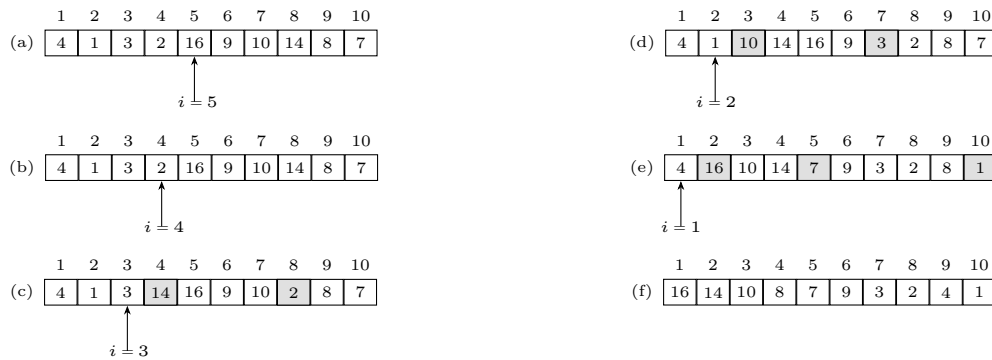


Figura 3.3: Il funzionamento di BUILD-MAX-HEAP sull'array $\langle 4, 1, 3, 2, 16, 9, 10, 14, 8, 7 \rangle$: prima di ogni chiamata di MAX-HEAPIFY, con l'indice i del ciclo e in grigio le posizioni cambiate al passo precedente. **(a)** $i = 5$: il nodo 5 ha il solo figlio 10, nulla da scambiare; **(b)** $i = 4$: scambio $A[4] \leftrightarrow A[8]$ (sale il 14); **(c)** $i = 3$: scambio $A[3] \leftrightarrow A[7]$ (sale il 10); **(d)** $i = 2$: scambia $A[2] \leftrightarrow A[5]$ (sale il 16), poi $A[5] \leftrightarrow A[10]$; **(e)** $i = 1$: la chiave 4 scende con tre scambi ($1 \leftrightarrow 2$, $2 \leftrightarrow 4$, $4 \leftrightarrow 8$); **(f)** il max-heap finale. [Cormen fig. 6.3]

L'ultima somma si calcola con la Formula (A.8), $\sum_{h=0}^{\infty} hx^h = x/(1-x)^2$, per $x = 1/2$: $\sum_{h \geq 0} h/2^h = 2$. Quindi il costo è $O(n \sum_{h \geq 0} h/2^h) = O(n)$: un max-heap si costruisce da un array non ordinato in tempo *lineare*. BUILD-MIN-HEAP è identica, con MIN-HEAPIFY. [Cormen cap. 6.3]

3.4 L'algoritmo HEAPSORT

Il massimo va in fondo, poi si ripete

Dopo BUILD-MAX-HEAP il massimo è in $A[1]$. Scambio $A[1]$ con l'ultimo elemento dell'heap: il massimo va nella sua posizione definitiva e lo “stacco” dall'heap (decremento $A.heap-size$). I sottoalberi della radice restano max-heap, ma la nuova radice — il valore che era in fondo, il più piccolo — può violare la proprietà: una MAX-HEAPIFY la rimette a posto. Ora $A[1]$ è il massimo dei rimanenti, e ripeto con $n-1$, $n-2$, ... finché l'heap ha un elemento. La parte destra dell'array è ordinata e cresce a ogni giro: alla fine è tutto ordinato, sul posto.

HEAPSORT(A)

```

1  BUILD-MAX-HEAP( $A$ )
2  for  $i := A.length$  downto 2
3      scambia  $A[1]$  con  $A[i]$ 
4       $A.heap-size := A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

La figura 3.4 traccia l'array a ogni iterazione (fig. 6.4 del libro): ogni riga è lo stato *dopo* la chiamata di MAX-HEAPIFY, e in grigio ci sono gli elementi già sistemati. Sull'istanza, heapsort fa 42 confronti e 30 scambi (build compreso).

Il ciclo è corretto per l'invariante [Cormen esercizio 6.4-2]: all'inizio di ogni iterazione $A[1..i]$ è un max-heap con gli i elementi più piccoli di $A[1..n]$, e $A[i+1..n]$ contiene gli $n-i$ più grandi, ordinati. Il tempo totale è $O(n \lg n)$: il build costa $O(n)$ e il ciclo fa $n-1$ iterazioni da $O(\lg n)$; con il caso peggiore $\Omega(n \lg n)$ [Cormen esercizio 6.4-4] si ha $\Theta(n \lg n)$. In pratica, però, una buona implementazione di quicksort batte di solito heapsort. [Cormen cap. 6.4 e 6.5]

(a)	16	14	10	8	7	9	3	2	4	1
(b)	14	8	10	4	7	9	3	2	1	16
(c)	10	8	9	4	7	1	3	2	14	16
(d)	9	8	3	4	7	1	2	10	14	16
(e)	8	7	3	4	2	1	9	10	14	16
(f)	7	4	3	1	2	8	9	10	14	16

(g)	4	2	3	1	7	8	9	10	14	16
(h)	3	2	1	4	7	8	9	10	14	16
(i)	2	1	3	4	7	8	9	10	14	16
(j)	1	2	3	4	7	8	9	10	14	16
(k)	1	2	3	4	7	8	9	10	14	16

Figura 3.4: Il funzionamento di HEAPSORT su $\langle 4, 1, 3, 2, 16, 9, 10, 14, 8, 7 \rangle$. **(a)** Dopo BUILD-MAX-HEAP. **(b)–(j)** L'array dopo l'iterazione i del ciclo, in grigio gli elementi fuori dall'heap, in posizione definitiva. **(k)** L'array ordinato. [Cormen fig. 6.4]

Eseguire HEAPSORT su un array

1. Prima BUILD-MAX-HEAP, come nel metodo qui sopra. Poi il ciclo: per $i = n$ giù fino a 2, *scambia* $A[1]$ con $A[i]$, *decrementa* $A.heap\text{-}size$, *fai scendere* la nuova radice con MAX-HEAPIFY($A, 1$).
2. A ogni passo scrivi l'array *intero*: le posizioni da i a n sono definitive, la parte $A[1..i-1]$ è il nuovo max-heap; alla fine l'array è ordinato crescente (sull'istanza, figura 3.4, 42 confronti in tutto).

3.5 Le code di priorità

Un insieme che restituisce sempre il massimo

Una *coda di priorità* mantiene un insieme S di elementi, ognuno con una *chiave*, e serve a fare due cose: “dammi l'elemento con la chiave più grande” e “aggiungine uno nuovo”. Pensate alla schedulazione dei lavori su un computer condiviso: il sistema tiene i lavori in attesa con le loro priorità e, appena libero, sceglie quello a priorità più alta. Un max-heap fa esattamente questo: il massimo è già in radice, e ogni operazione costa una discesa o una risalita, $O(\lg n)$. Nella simulazione a eventi serve invece il minimo (coda di min-priorità).

[Cormen cap. 6.5]

Le operazioni di una coda di max-priorità

[Cormen cap. 6.5] Una coda di max-priorità mantiene un insieme S e supporta: INSERT(S, x), che inserisce x ($S := S \cup \{x\}$); MAXIMUM(S), che restituisce l'elemento con la chiave più grande; EXTRACT-MAX(S), che lo rimuove e lo restituisce; INCREASE-KEY(S, x, k), che porta la chiave di x al valore k (non minore di quello corrente). Una coda di min-priorità ha le operazioni simmetriche MINIMUM, EXTRACT-MIN, DECREASE-KEY (Esercizio 6.5-3). Con un heap costano $O(\lg n)$ su un insieme di n elementi, tranne leggere il massimo ($\Theta(1)$).

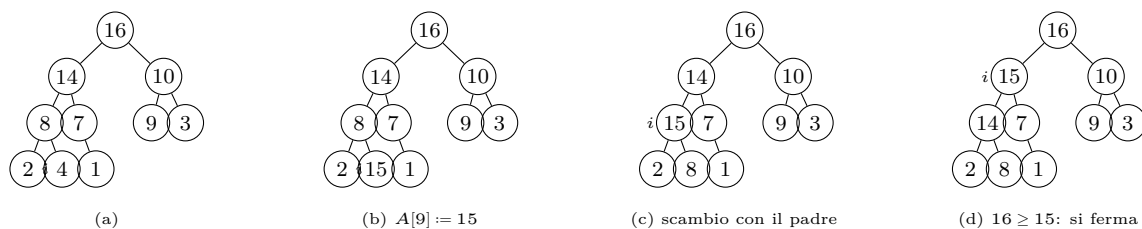


Figura 3.5: Il funzionamento di HEAP-INCREASE-KEY($A, 9, 15$) sul max-heap $A = \langle 16, 14, 10, 8, 7, 9, 3, 2, 4, 1 \rangle$ (fig. 6.5 del libro): la chiave 15 risale dal nodo 9 al 4 al 2, e si ferma perché il padre 16 è più grande. [Cormen fig. 6.5]

Le quattro procedure che le implementano su un max-heap:

HEAP-MAXIMUM(A)

1 **return** $A[1]$

HEAP-INCREASE-KEY(A, i, key)

```
1 if  $key < A[i]$ 
2     error "chiave più piccola della corrente"
3  $A[i] := key$ 
4 while  $i > 1$  and  $A[PARENT(i)] < A[i]$ 
5     scambia  $A[i]$  con  $A[PARENT(i)]$ 
6      $i := PARENT(i)$ 
```

HEAP-EXTRACT-MAX(A)

```
1 if  $A.heap-size < 1$ 
2     error "underflow dell'heap"
3  $max := A[1]$ 
4  $A[1] := A[A.heap-size]$ 
5  $A.heap-size := A.heap-size - 1$ 
6 MAX-HEAPIFY( $A, 1$ )
7 return  $max$ 
```

MAX-HEAP-INSERT(A, key)

```
1  $A.heap-size := A.heap-size + 1$ 
2  $A[A.heap-size] := -\infty$ 
3 HEAP-INCREASE-KEY( $A, A.heap-size, key$ )
```

HEAP-MAXIMUM costa $\Theta(1)$; HEAP-EXTRACT-MAX è come il corpo del ciclo di heapsort ($O(\lg n)$); HEAP-INCREASE-KEY, e MAX-HEAP-INSERT che la chiama, risale il cammino verso la radice scambiando con il padre (come il ciclo interno di insertion sort, $O(\lg n)$) e mette prima la chiave a $-\infty$ perché la proprietà deve valere [Cormen esercizio 6.5-4].

Eeguire le operazioni di una coda di priorità

1. HEAP-MAXIMUM: restituisce $A[1]$, l'array non cambia.
2. HEAP-EXTRACT-MAX: copia $A[1]$, porta l'ultimo elemento dell'heap in $A[1]$, decrementa $A.heap-size$, poi MAX-HEAPIFY($A, 1$).
3. HEAP-INCREASE-KEY(A, i, key): poni $A[i] := key$ e risali scambiando con il padre finché il padre è più piccolo.
4. MAX-HEAP-INSERT: cresci di uno, metti $-\infty$ nell'ultima foglia, poi HEAP-INCREASE-KEY.
5. Dopo ogni operazione controlla $A[PARENT(i)] \geq A[i]$ su tutti i nodi.

Un esempio di esecuzione. Dal max-heap della figura 3.5(d):

- HEAP-MAXIMUM(A) restituisce 16 in $\Theta(1)$, senza modificare l'array.
- HEAP-EXTRACT-MAX(A) restituisce 16: l'ultimo elemento (1) va in $A[1]$, $A.heap-size$ scende a 9, MAX-HEAPIFY($A, 1$) riporta in alto il 15: l'heap diventa $\langle 15, 14, 10, 8, 7, 9, 3, 2, 1 \rangle$.
- MAX-HEAP-INSERT($A, 12$): 12 è più grande del padre 7, sale al nodo 5 e qui il padre 14 lo ferma: $\langle 15, 14, 10, 8, 12, 9, 3, 2, 1, 7 \rangle$.

Figli, padri e array 1-based

- Gli array del libro sono **1-based**: i figli di i sono $2i$ e $2i + 1$, il padre è $\lfloor i/2 \rfloor$ (le formule 0-based con la radice in $A[1]$ non tornano).
- In heapsort gli elementi già ordinati **restano nell'array**, a destra: non confondere $A.length$ (fisso, n) con $A.heap-size$ (cala a ogni iterazione), o MAX-HEAPIFY rimette in gioco gli elementi già sistemati.
- Le foglie sono i nodi $\lfloor n/2 \rfloor + 1, \dots, n$ (per $n = 10$: 6–10): nel dubbio verifica $2i \leq n$; e la

proprietà del *max*-heap è $A[\text{PARENT}(i)] \geq A[i]$, il padre è più grande del figlio.

Come rispondere: «Che cos'è un heap? Il tempo di heapsort?»

1. **Definizione:** un heap binario è un array visto come albero binario quasi completo; $A.length$ è la lunghezza dell'array, $A.heap\text{-}size$ quanti elementi sono nell'heap.
2. **Proprietà** (max-heap): $A[\text{PARENT}(i)] \geq A[i]$, il massimo in radice; l'altezza è $\lfloor \lg n \rfloor$ perché l'albero è quasi completo.
3. **Le operazioni e i tempi:** MAX-HEAPIFY $O(\lg n)$ (ricorrenza $T(n) \leq T(2n/3) + \Theta(1)$, teorema master caso 2); BUILD-MAX-HEAP $O(n)$ con la sommatoria $\sum_h \lceil n/2^{h+1} \rceil O(h)$; HEAPSORT $O(n \lg n)$: build più $n - 1$ estrazioni da $O(\lg n)$.
4. **Un esempio:** l'array $\langle 4, 1, 3, 2, 16, 9, 10, 14, 8, 7 \rangle$ diventa il max-heap $\langle 16, 14, 10, 8, 7, 9, 3, 2, 4, 1 \rangle$, poi l'array ordinato $\langle 1, 2, 3, 4, 7, 8, 9, 10, 14, 16 \rangle$.
5. **Limiti:** heapsort è in loco e $\Theta(n \lg n)$ garantito, ma quicksort di solito lo batte; con la stessa struttura si fa la coda di priorità in $O(\lg n)$ per operazione.

Capitolo 4

Quicksort: partizionare per ordinare

Heapsort ordina n numeri in $\Theta(n \lg n)$ nel caso peggiore; quicksort ottiene lo stesso ordine di grandezza *in media*, con fattori costanti nascosti molto piccoli, ordinando sul posto e funzionando bene anche in ambienti con memoria virtuale: per questo, nonostante un caso peggiore $\Theta(n^2)$, è spesso la soluzione pratica migliore per ordinare. È ancora divide et impera, ma il passo *divide* fa quasi tutto il lavoro: il partizionamento manda un elemento, il *pivot*, nella sua posizione definitiva.

[Cormen cap. 7]

L'idea: mettere a posto un elemento per volta

Quicksort spezza il problema in due problemi più piccoli con una sola mossa. Si sceglie un elemento dell'array — il *pivot* — e si riarrangia l'array in modo che tutto ciò che è più piccolo del pivot stia a sinistra e tutto ciò che è più grande stia a destra. La posizione in cui il pivot si ritrova è quella *definitiva*: nessun elemento a sinistra dovrà mai più essere confrontato con uno a destra, quindi le due parti si possono ordinare separatamente e non serve ricomporle. La partizione costa un solo passaggio sull'array, $\Theta(n)$, e il pivot esce dal gioco: le due ricorsioni lavorano su $n - 1$ elementi in tutto. Se le due parti restano confrontabili in dimensione, i problemi si dimezzano a ogni livello dell'albero delle chiamate e la profondità è $\lg n$: da qui il tempo $O(n \lg n)$. Il pericolo è il contrario: se il pivot è ogni volta il più piccolo o il più grande, una parte è vuota e l'altra ha $n - 1$ elementi; l'albero degenera in una catena e il tempo diventa $\Theta(n^2)$.

4.1 Descrizione di quicksort

Quicksort, come merge sort, è basato sul paradigma divide et impera [Cormen cap. 2.3.1]. Per ordinare il sottoarray $A[p..r]$ i tre passi sono: [Cormen cap. 7.1]

- **Divide:** partizionare l'array $A[p..r]$ in due sottoarray $A[p..q - 1]$ e $A[q + 1..r]$ (eventualmente vuoti) tali che ogni elemento di $A[p..q - 1]$ sia minore o uguale ad $A[q]$ che, a sua volta, è minore o uguale a ogni elemento di $A[q + 1..r]$. L'indice q viene calcolato dalla procedura di partizionamento.
- **Impera:** ordinare i due sottoarray $A[p..q - 1]$ e $A[q + 1..r]$ chiamando ricorsivamente quicksort.
- **Combina:** poiché i sottoarray sono già ordinati e separati dal pivot, non occorre alcun lavoro per combinarli: l'intero array $A[p..r]$ è ordinato.

QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2       $q := \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )
```

Per ordinare un intero array A , la chiamata iniziale è QUICKSORT($A, 1, A.length$). Il lavoro sta tutto nel passo *divide*: le parti vuote non sono un problema, perché il test $p < r$ ferma subito la chiamata, e il pivot non compare in nessuna delle due ricorsioni.

4.2 Partition

L'elemento chiave di quicksort è **PARTITION**, che riarrangia il sottoarray $A[p..r]$ *sul posto*, senza array di appoggio: [Cormen cap. 7.1]

PARTITION(A, p, r)

```

1   $x := A[r]$ 
2   $i := p - 1$ 
3  for  $j := p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i := i + 1$ 
6          scambia  $A[i]$  con  $A[j]$ 
7  scambia  $A[i + 1]$  con  $A[r]$ 
8  return  $i + 1$ 

```

PARTITION sceglie sempre $x = A[r]$ come *pivot* intorno al quale partizionare $A[p..r]$. Durante l'esecuzione l'array si divide in quattro regioni (figura 4.1): a sinistra di i gli elementi minori o uguali a x , fra $i + 1$ e $j - 1$ quelli maggiori di x , fra j e $r - 1$ elementi di valore qualsiasi, in $A[r]$ il pivot x .

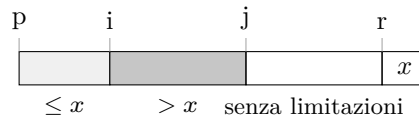


Figura 4.1: Le quattro regioni mantenute da **PARTITION** in un sottoarray $A[p..r]$. I valori in $A[p..i]$ sono tutti minori o uguali a x , i valori in $A[i + 1..j - 1]$ tutti maggiori di x , e $A[r] = x$. Gli elementi di $A[j..r - 1]$ possono avere valori qualsiasi.

Invariante di ciclo di **PARTITION**

[Cormen p. 142] All'inizio di ogni iterazione del ciclo **for** (righe 3–6), per qualsiasi indice k dell'array:

1. se $p \leq k \leq i$, allora $A[k] \leq x$;
2. se $i + 1 \leq k \leq j - 1$, allora $A[k] > x$;
3. se $k = r$, allora $A[k] = x$.

Gli indici fra j e $r - 1$ non rientrano in alcuno di questi tre casi e i valori in quelle posizioni non hanno una particolare relazione con il pivot x .

L'esempio del libro. La figura 7.1 esegue **PARTITION** su $A = \langle 2, 8, 7, 1, 3, 5, 6, 4 \rangle$: la tabella riporta lo stato dopo ogni giro del ciclo, la figura 4.2 i due momenti decisivi. [Cormen fig. 7.1]

giro	$A[j]$	$A[j] \leq 4?$	i	array
inizio	—	—	0	2 8 7 1 3 5 6 4
($j = 1$)	2	sì	1	2 8 7 1 3 5 6 4
($j = 2$)	8	no	1	2 8 7 1 3 5 6 4
($j = 3$)	7	no	1	2 8 7 1 3 5 6 4
($j = 4$)	1	sì	2	2 1 7 8 3 5 6 4
($j = 5$)	3	sì	3	2 1 3 8 7 5 6 4
($j = 6$)	5	no	3	2 1 3 8 7 5 6 4
($j = 7$)	6	no	3	2 1 3 8 7 5 6 4
righe 7–8	—	—	$q = 4$	2 1 3 4 7 5 6 8

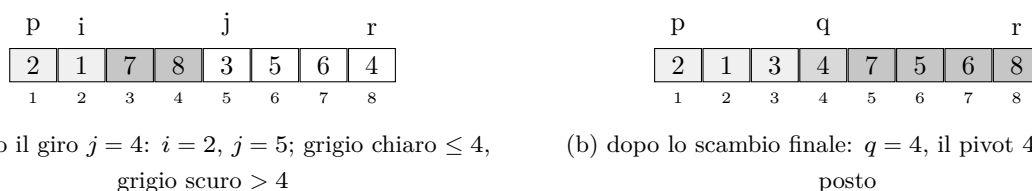


Figura 4.2: Due momenti della partizione dell'esempio: l'array con gli indici i e j che avanzano, e l'array finale con il pivot in posizione definitiva.

Correttezza di PARTITION

da saper dimostrare

[Cormen p. 143] L'invariante di ciclo è vera prima della prima iterazione, ogni iterazione la conserva, e quando il ciclo termina (con $j = r$) garantisce che le ultime due righe mettono il pivot al suo posto: l'output soddisfa le specifiche del passo *divide*.

Dimostrazione. Inizializzazione. Prima della prima iterazione, $i = p - 1$ e $j = p$. Non ci sono valori fra p e i né fra $i + 1$ e $j - 1$, quindi le prime due condizioni sono soddisfatte (vuote). L'assegnazione $x := A[r]$ della riga 1 soddisfa la terza.

Conservazione. Ci sono due casi, come nella figura 7.3 del libro. Se $A[j] > x$, l'unica azione del ciclo è incrementare j : la condizione 2 vale ora per $A[j - 1]$ e le altre posizioni non cambiano. Se $A[j] \leq x$, si incrementa i , si scambiano $A[i]$ e $A[j]$, poi j : dopo lo scambio $A[i] \leq x$ (condizione 1) e $A[j - 1] > x$, perché l'elemento spostato in $A[j - 1]$ era, per l'invariante, più grande di x .

Conclusione. Alla fine del ciclo $j = r$: ogni posizione dell'array sta in uno dei tre insiemi dell'invariante, cioè i valori sono ripartiti in quelli minori o uguali a x , quelli maggiori di x e il pivot. Le ultime due righe lo inseriscono fra le due partizioni, scambiandolo con l'elemento più a sinistra maggiore di x , e restituiscono il suo indice $q = i + 1$. $\square$

Il tempo di esecuzione di PARTITION su $A[p..r]$ è $\Theta(n)$, con $n = r - p + 1$: il ciclo for fa esattamente $r - p$ iterazioni a costo costante [Cormen esercizio 7.1-3].

4.3 Prestazioni di quicksort

Il tempo di esecuzione di quicksort dipende da quanto è bilanciata la partizione, e quindi da quali elementi vengono scelti come pivot. [Cormen cap. 7.2]

Caso peggiore. Si verifica quando la partizione produce un sottoproblema con $n - 1$ elementi e uno con 0 elementi, a ogni livello ricorsivo. Poiché partizionare costa $\Theta(n)$ e la chiamata su un array vuoto costa $\Theta(1)$, la ricorrenza è

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n),$$

che con il metodo di sostituzione dà $T(n) = \Theta(n^2)$: sommando i costi livello per livello si ottiene la serie aritmetica $\Theta(n) + \Theta(n - 1) + \dots + \Theta(1)$ [Cormen esercizio 7.2-1]. Lo sbilanciamento massimo si ha, per esempio, quando l'array è *già ordinato*: il pivot $x = A[r]$ è il massimo, ogni elemento finisce a sinistra e $q = r$ a ogni chiamata. Per $n = 8$ l'array $\langle 1, 2, \dots, 8 \rangle$ fa così $7 + 6 + \dots + 1 = 28$ confronti, il massimo possibile. Più in generale, la ricorrenza del caso peggiore

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n - q - 1)) + \Theta(n) \quad (7.1)$$

si risolve in $T(n) = \Theta(n^2)$: con il metodo di sostituzione, $T(n) \leq cn^2$ perché $q^2 + (n - q - 1)^2$ è massima agli estremi dell'intervallo, dove vale $(n - 1)^2 = n^2 - 2n + 1$, e il termine $-c(2n - 1)$ assorbe $\Theta(n)$ per c grande. [Cormen cap. 7.4.1]

Caso migliore. Nel caso di bilanciamento massimo PARTITION produce due sottoproblemi di dimensione al più $n/2$: uno ha $\lfloor n/2 \rfloor$ elementi e l'altro $\lceil n/2 \rceil - 1$. La ricorrenza

$$T(n) \leq 2T(n/2) + \Theta(n)$$

ha la soluzione $T(n) = O(n \lg n)$ per il caso 2 del teorema master (capitolo 2): questo è anche il limite inferiore per qualunque ordinamento per confronti (capitolo 5). Per $n = 8$ il minimo possibile è 13 confronti (per esempio su $\langle 1, 2, 4, 3, 7, 6, 8, 5 \rangle$), contro i 28 del caso peggiore: il divario è ampio.

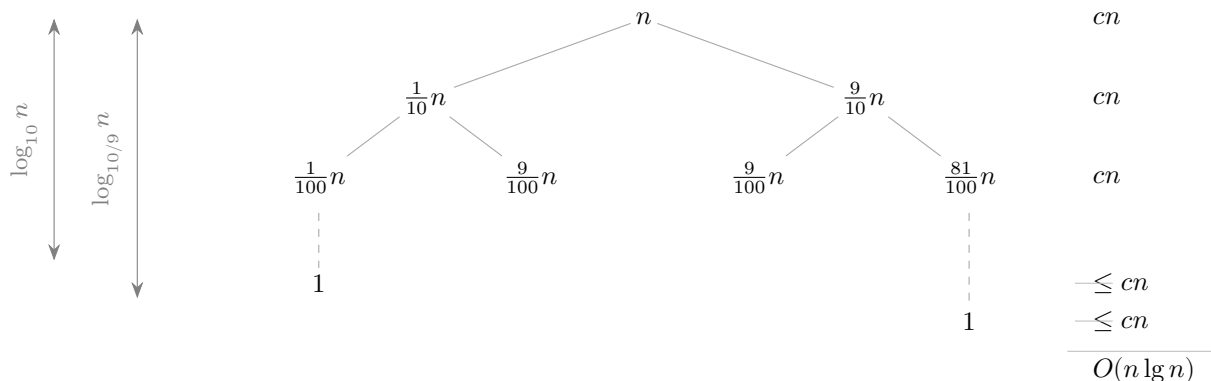


Figura 4.3: Albero di ricorsione di quicksort quando PARTITION genera sempre una ripartizione 9 a 1: ogni livello costa al più cn e la profondità è $\Theta(\lg n)$, quindi il tempo è $O(n \lg n)$. I nodi mostrano le dimensioni dei sottoproblemi, i costi per livello stanno a destra.

Partizionamento bilanciato. Anche una partizione 9 a 1, che sembra molto sbilanciata, va bene: con $T(n) \leq T(9n/10) + T(n/10) + cn$ (figura 4.3) ogni livello dell'albero costa al più cn finché non si raggiunge una condizione al contorno alla profondità $\log_{10} n = \Theta(\lg n)$, e la ricorsione termina alla profondità $\log_{10/9} n = \Theta(\lg n)$: il costo totale è $O(n \lg n)$. Vale in generale: qualsiasi ripartizione con proporzioni *costanti* — anche 99 a 1 — produce $\Theta(\lg n)$ livelli di costo $O(n)$ ciascuno, quindi $O(n \lg n)$. [Cormen cap. 7.2]

Caso medio. Le partizioni buone e quelle cattive si alternano irregolarmente nell'albero. Se a una partizione cattiva (costo $\Theta(n)$, parti di dimensioni 0 e $n - 1$) segue una buona, il costo complessivo dei due livelli è $\Theta(n) + \Theta(n - 1) = \Theta(n)$ e i tre sottoproblemi che restano non sono più grandi di quelli di una singola partizione ben bilanciata: la cattiva viene *assorbita* dalla buona, e il tempo resta $O(n \lg n)$ con una costante più grande. Nel caso medio quicksort è quindi molto più vicino al caso migliore che al caso peggiore: circa l'80% delle partizioni è più bilanciata di 9 a 1 [Cormen esercizio 7.2-6].

4.4 La versione randomizzata

La randomizzazione: nessun input è sempre cattivo

La versione appena vista ha un punto debole: il pivot è sempre $A[r]$, quindi un input costruito ad arte — per esempio già ordinato — la manda sempre nel caso peggiore. L'idea della versione randomizzata è togliere regolarità alla scelta del pivot: si sceglie a caso un elemento del sottoarray e lo si porta in posizione r . Così ogni elemento ha la stessa probabilità di fare da pivot: nessun input provoca *sistematicamente* il comportamento nel caso peggiore. Il tempo di esecuzione diventa una variabile casuale e si studia il suo *valore atteso*, che risulta $\Theta(n \lg n)$: le partizioni cattive capitano, ma non capitano sempre.

Le modifiche sono minime: RANDOMIZED-PARTITION scambia $A[r]$ con un elemento scelto a caso in $A[p..r]$ e poi chiama PARTITION; RANDOMIZED-QUICKSORT chiama la nuova procedura. Questo metodo di randomizzazione si chiama *campionamento casuale* e semplifica l'analisi. [Cormen cap. 7.3]

RANDOMIZED-PARTITION(A, p, r)

```

1   $i := \text{RANDOM}(p, r)$ 
2  scambia  $A[r]$  con  $A[i]$ 
3  return PARTITION( $A, p, r$ )

```

RANDOMIZED-QUICKSORT(A, p, r)

```

1  if  $p < r$ 
2       $q := \text{RANDOMIZED-PARTITION}(A, p, r)$ 
3      RANDOMIZED-QUICKSORT( $A, p, q - 1$ )
4      RANDOMIZED-QUICKSORT( $A, q + 1, r$ )

```

L'analisi, in breve. Ci sono al più n chiamate di PARTITION, ognuna con lavoro costante più una iterazione del ciclo per elemento, e ogni iterazione fa *un* confronto (la riga 4): detto X il numero totale di confronti della riga 4 nell'intera esecuzione, il tempo è $O(n + X)$ (Lemma 7.1). Per calcolare X si rinominano gli elementi in ordine crescente $z_1 < z_2 < \dots < z_n$ e si osserva che z_i e z_j vengono confrontati *se e solo se* il primo pivot scelto fra gli elementi $Z_{ij} = \{z_i, \dots, z_j\}$ è z_i oppure z_j : un pivot scelto in mezzo li separa per sempre. Ogni elemento di Z_{ij} ha la stessa probabilità di essere scelto per primo, quindi

$$\Pr\{z_i \text{ è confrontato con } z_j\} = \frac{2}{j - i + 1}. \quad (7.3)$$

Sommando su tutte le coppie e applicando il limite sulle serie armoniche:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k + 1} < \sum_{i=1}^{n-1} O(\lg n) = O(n \lg n). \quad (7.4)$$

Dunque, con elementi distinti, il tempo atteso di RANDOMIZED-QUICKSORT è $\Theta(n \lg n)$ (limite superiore $O(n \lg n)$ più il caso migliore $\Omega(n \lg n)$): quicksort randomizzato è, in media, asintoticamente ottimale. L'analisi completa è nel Paragrafo 7.4.2 del libro. Il caso peggiore resta $\Theta(n^2)$ [Cormen cap. 7.4]: la randomizzazione non lo elimina, ma lo rende improbabile.

n	$E[X]$ (7.4)	peggiore
8	16.9	28
100	647.9	4950
1000	10985.9	499500

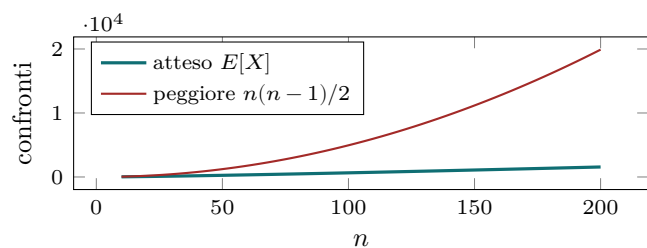


Figura 4.4: Confronti attesi (formula (7.4)) contro il caso peggiore $n(n-1)/2$: per $n = 200$ sono 1563 contro 19900, e il rapporto fra i due cresce con n (1.7, 7.6 e 45.5 per $n = 8, 100, 1000$).

4.5 Come si risolve all'esame

Eeguire PARTITION e QUICKSORT su un array

1. Scrivi l'array con gli indici $1..n$ (o $p..r$) e segna p e r .
2. PARTITION: poni $x := A[r]$ e $i := p - 1$; poi per $j = p, \dots, r - 1$ confronta $A[j]$ con x : se $A[j] \leq x$ incrementa i e scambia $A[i]$ con $A[j]$; se $A[j] > x$ non succede nulla. *Scrivi lo stato dell'array a ogni giro*: è quello che l'esercizio chiede di mostrare.
3. Chiusura: scambia $A[i + 1]$ con $A[r]$ e poni $q := i + 1$: il pivot è nella posizione definitiva.

Il numero di confronti fatti è $r - p$ (le iterazioni del ciclo).

4. **QUICKSORT:** elenca le chiamate come coppie (p, r) ; ognuna restituisce un q e ne genera due, $(p, q - 1)$ e $(q + 1, r)$; fermati quando $p \geq r$: la catena delle chiamate disegna l'albero della ricorsione e l'array è ordinato.

Esempio. Sull'array del libro $\langle 2, 8, 7, 1, 3, 5, 6, 4 \rangle$ (figura 7.1): la prima PARTITION fa 7 confronti e dà $q = 4$; poi $(1, 3) \rightarrow q = 3$ con 2 confronti, $(1, 2) \rightarrow q = 1$ con 1, $(5, 8) \rightarrow q = 8$ con 3, $(5, 7) \rightarrow q = 6$ con 2: in tutto 5 partizioni e 15 confronti (la media per $n = 8$ è 16.9).

Scrivere la ricorrenza di una procedura che partiziona

- **Costo di una chiamata:** conta le operazioni su n elementi (per quicksort, il ciclo di PARTITION): $\Theta(n)$.
- **Dimensioni delle parti:** caso peggiore $T(n) = T(n - 1) + \Theta(n)$ (sostituzione: $\Theta(n^2)$); caso migliore/bilanciato $T(n) \leq 2T(n/2) + \Theta(n)$ (caso 2 del teorema master: $O(n \lg n)$); proporzione costante qualsiasi: albero di profondità $\Theta(\lg n)$ con costo $O(n)$ per livello.
- **Se il pivot è casuale** si scrive il valore atteso della ricorrenza,

$$E[T(n)] = \frac{1}{n} \sum_{q=1}^n (E[T(q - 1)] + E[T(n - q)]) + \Theta(n),$$

e si dimostra $E[T(n)] = \Theta(n \lg n)$ per sostituzione [Cormen problema 7-3]: il valore atteso è $O(n \lg n)$ per *qualunque* input, perché la media è sulle scelte casuali del pivot.

- Nei temi il metodo (albero, sostituzione o master) è indicato dal testo: la ricorrenza va scritta contando le operazioni, non copiata da quicksort.

Indici e versione randomizzata

- i parte da $p - 1$, non da p : prima dell'array. E alla fine lo scambio è fra $A[i + 1]$ e $A[r]$, con $q = i + 1$: chi risponde $q = i$ sbaglia di uno.
- Il test è $A[j] \leq x$, non $A[j] < x$: con il minore stretto gli elementi uguali al pivot finirebbero tutti a destra. E attenzione comunque: se *tutti* gli elementi valgono x , il test $\leq$ manda tutto a sinistra e $q = r$, la partizione più sbilanciata possibile (Esercizio 7.1-2).
- Le ricorsioni sono su $q - 1$ e $q + 1$: il pivot non entra in nessuna delle due, è già al suo posto. E ogni chiamata si fa solo se $p < r$.
- Nella versione randomizzata lo scambio casuale avviene *a ogni* chiamata di RANDOMIZED-PARTITION (è lì che sta la randomizzazione), non una volta sola all'inizio. Il tempo atteso $O(n \lg n)$ è una media sulle scelte del generatore, non una garanzia: il caso peggiore resta $\Theta(n^2)$, semplicemente nessun input lo provoca *sistematicamente*.

Come rispondere: «Quicksort: come funziona? Tempi?»

1. **Definizione:** divide et impera *sul posto*; il passo divide è PARTITION, che dispone gli elementi intorno a un pivot e ne calcola la posizione definitiva.
2. **Meccanismo:** $x = A[r]$, $i = p - 1$; per $j = p, \dots, r - 1$ si porta $A[j] \leq x$ a sinistra; poi il pivot va fra le due parti con un ultimo scambio, e $q = i + 1$.
3. **Correttezza:** invariante delle tre regioni ($\leq x$, $> x$, x) prima, durante e dopo il ciclo.
4. **Tempi:** peggiore $\Theta(n^2)$ ($T(n) = T(n - 1) + \Theta(n)$; tipico dell'array già ordinato); migliore

$\Theta(n \lg n)$ ($T(n) \leq 2T(n/2) + \Theta(n)$); anche 9 a 1 resta $O(n \lg n)$; in media è vicino al caso migliore.

5. **Versione randomizzata:** pivot scelto a caso; $\Pr\{z_i \text{ confrontato con } z_j\} = 2/(j - i + 1)$ e $E[X] = \sum_{i < j} 2/(j - i + 1) = O(n \lg n)$, quindi tempo atteso $\Theta(n \lg n)$ (caso peggiore ancora $\Theta(n^2)$).
6. **Un esempio:** $A = \langle 2, 8, 7, 1, 3, 5, 6, 4 \rangle$: la partizione dà $q = 4$ e $A = \langle 2, 1, 3, 4, 7, 5, 6, 8 \rangle$; quicksort finisce in 5 partizioni e 15 confronti (media 16.9, minimo 13, massimo 28 per $n = 8$).
7. **Limiti:** non è stabile; il caso peggiore si evita con la randomizzazione del pivot; in pratica batte heapsort e merge sort.

Capitolo 5

Ordinamento in tempo lineare e statistiche d'ordine

Heapsort e merge sort ordinano n elementi in $O(n \lg n)$ nel caso peggiore (capitolo 3), quicksort in $\Theta(n \lg n)$ in media (capitolo 4). Qui: il limite inferiore $\Omega(n \lg n)$ che vale per *tutti* gli ordinamenti per confronti; i tre algoritmi lineari che lo pagano con un'ipotesi sull'input (counting, radix e bucket sort); la selezione con RANDOMIZED-SELECT. [Cormen cap. 8 e cap. 9]

5.1 Il limite inferiore $\Omega(n \lg n)$ degli ordinamenti per confronti

L'idea: ogni confronto è una domanda con due risposte

Un ordinamento per confronti non guarda i valori: chiede solo « a_i è minore o uguale di a_j ?». L'insieme delle esecuzioni possibili è un **albero binario**: le domande nei nodi, le due risposte nei rami, le permutazioni dell'input nelle foglie. Un algoritmo corretto vuole una foglia per *ognuna* delle $n!$ permutazioni, ma un albero di altezza h ha al più 2^h foglie, e l'altezza è il numero di confronti del caso peggiore: quindi $n! \leq 2^h$.

Ordinamento per confronti e albero di decisione

Un **ordinamento per confronti** usa soltanto confronti fra gli elementi ($a_i < a_j$, $a_i \leq a_j$, $a_i = a_j$, $a_i \geq a_j$, $a_i > a_j$) per determinare il loro ordine relativo: non esamina i valori e non ne ricava informazioni in altro modo.

L'**albero di decisione** è l'albero binario dei confronti effettuati su input di n elementi: ogni **nodo interno** è annotato con $i : j$ ($1 \leq i, j \leq n$) e rappresenta il confronto fra a_i e a_j ; il **sottoalbero sinistro** contiene i confronti successivi a $a_i \leq a_j$, il destro quelli successivi a $a_i > a_j$; ogni **foglia** porta la permutazione $\langle \pi(1), \dots, \pi(n) \rangle$ dei posti occupati, cioè $a_{\pi(1)} \leq \dots \leq a_{\pi(n)}$. Una esecuzione è un cammino dalla radice a una foglia, e l'**altezza** è il numero di confronti del caso peggiore. [Cormen cap. 8.1]

Teorema 8.1

da saper dimostrare

Qualsiasi algoritmo di ordinamento per confronti richiede $\Omega(n \lg n)$ confronti nel caso peggiore.

Dimostrazione. Basta un limite inferiore sull'altezza dell'albero di decisione, in cui ogni permutazione appaia in una foglia raggiungibile. Con h l'altezza e l il numero di foglie raggiungibili: le $n!$ permutazioni compaiono ciascuna in una foglia ($n! \leq l$) e un albero di altezza h ha al più 2^h foglie ($l \leq 2^h$). Quindi $n! \leq l \leq 2^h$ e, prendendo i logaritmi, $h \geq \lg(n!) = \Theta(n \lg n)$ per l'equazione (3.19). $\square$

Con $n = 3$: $3! = 6 \leq 8 = 2^3$ e $2^2 = 4 < 6$, quindi servono almeno 3 confronti; con $n = 10$, $\lceil \lg 10! \rceil = \lceil 21.79 \rceil = 22$. Corollario: heapsort e merge sort sono **asintoticamente ottimali** fra gli ordinamenti per confronti [Cormen Corollario 8.2].

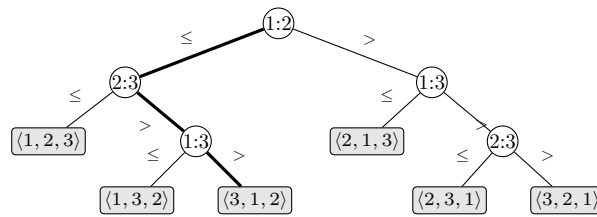


Figura 5.1: L'albero di decisione di insertion sort su tre elementi: 5 nodi interni ($i : j$), 6 foglie ($3! = 6$ permutazioni); in grassetto il cammino su $\langle 6, 8, 5 \rangle$ [Cormen fig. 8.1, p. 158].

5.2 Counting sort

L'idea: non confrontare, contare

Se di x so che ci sono esattamente 17 elementi minori o uguali a lui, allora al posto 17 dell'output c'è proprio x : la posizione non si scopre confrontando, si *calcola* contando. Counting sort conta quante volte compare ogni valore (array C), somma i conteggi in avanti (quanti elementi sono $\leq i$), poi scorre l'input *dall'ultimo al primo* collocando ogni elemento e decrementando il contatore: *stabile*, e lineare per interi piccoli, $k = O(n)$.

COUNTING-SORT(A, B, k)

```

1  sia  $C[0 \dots k]$  un nuovo array
2  for  $i := 0$  to  $k$ 
3       $C[i] := 0$ 
4  for  $j := 1$  to  $A.length$ 
5       $C[A[j]] := C[A[j]] + 1$ 
6  //  $C[i]$ : quanti elementi uguali a  $i$ .
7  for  $i := 1$  to  $k$ 
8       $C[i] := C[i] + C[i - 1]$ 
9  //  $C[i]$ : quanti elementi  $\leq i$ .
10 for  $j := A.length$  downto 1
11      $B[C[A[j]]] := A[j]$ 
12      $C[A[j]] := C[A[j]] - 1$ 

```

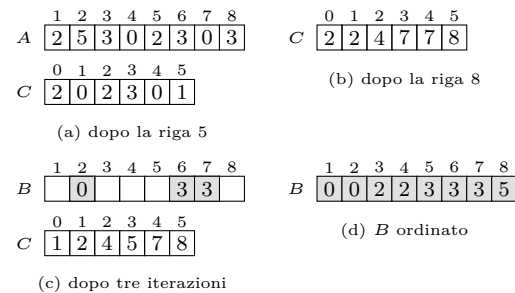


Figura 5.2: Counting sort su $A = \langle 2, 5, 3, 0, 2, 3, 0, 3 \rangle$, $k = 5$. (a) A e C dopo la riga 5. (b) C dopo la riga 8: $C[3] = 7$ elementi sono ≤ 3 . (c) B e C dopo tre iterazioni del ciclo finale: $A[j]$ va in $B[C[A[j]]]$, poi $C[A[j]]$ cala di uno. (d) B ordinato [Cormen fig. 8.2, p. 161].

[Cormen cap. 8.2]: A contiene interi in $[0 \dots k]$ (input), B è l'output, C la memoria di lavoro; il tempo è $\Theta(k + n)$, senza mai confronti fra elementi.

Eseguire COUNTING-SORT su un array

Con $A = \langle 2, 5, 3, 0, 2, 3, 0, 3 \rangle$ ($k = 5$, Figura 5.2):

- conteggi:** $C[0 \dots k]$ parte da zeri e si incrementa $C[A[j]]$ per ogni j : $C = \langle 2, 0, 2, 3, 0, 1 \rangle$;
- somma corrente:** per $i = 1, \dots, k$ fai $C[i] := C[i] + C[i - 1]$: $C = \langle 2, 2, 4, 7, 7, 8 \rangle$ — da qui $C[i]$ è il numero di elementi $\leq i$;
- collocazione:** scorri A **da destra a sinistra**: $A[8] = 3$ va in $B[C[3]] = B[7]$, $C[3] := 6$; $A[7] = 0$ in $B[C[0]] = B[2]$, $C[0] := 1$; e così via, finché B è ordinato.

Da sinistra a destra ordinerebbe ancora, ma perderebbe la **stabilità** (chiavi uguali nell'ordine di input), che serve a radix sort [Cormen p. 161].

5.3 Radix sort

L'idea: una cifra alla volta, dalla meno significativa

Ordinare dalla cifra *più* significativa richiederebbe pile intermedie da gestire; radix sort fa il contrario: ordina la cifra meno significativa, ricompatta, poi la seconda, fino alla d -esima. Funziona *solo* se ogni passaggio è stabile: l'ordine delle cifre già trattate non va rovinato.

$\text{RADIX-SORT}(A, d)$

```
1  for  $i := 1$  to  $d$ 
2      usa un ordinamento stabile per ordinare l'array  $A$  sulla cifra  $i$ 
```

La cifra 1 è quella di ordine più basso, la d quella di ordine più alto. Con n numeri di d cifre, ciascuna con al più k valori, e counting sort come ordinamento stabile, RADIX-SORT ordina in $\Theta(d(n+k))$ [Cormen Lemma 8.3]: $\Theta(n)$ se d è costante e $k = O(n)$. Con numeri di b bit a gruppi di r bit il tempo è $\Theta((b/r)(n+2^r))$ [Cormen Lemma 8.4].

input	cifra 1	cifra 2	cifra 3
329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839

Ogni colonna è la lista dopo l'ordinamento stabile su una cifra, dalla meno alla più significativa [Cormen fig. 8.3, p. 162]. Al primo passaggio 457 e 657 hanno la stessa cifra 7 e restano nel loro ordine; senza stabilità 657 scavalcherebbe 457 e l'array finale sarebbe sbagliato: $\langle 355, 329, 457, 436, 657, 720, 839 \rangle$ invece di $\langle 329, 355, 436, 457, 657, 720, 839 \rangle$.

5.4 Bucket sort

L'idea: distribuire in secchi, ordinarne pochi alla volta

Se gli n numeri sono estratti *uniformemente* da $[0, 1)$, dividerli in n intervalli uguali (i *bucket*) li separa bene: in media ogni bucket riceve pochi elementi, e basta ordinarli con insertion sort; il costo è la somma dei quadrati delle dimensioni, lineare in media.

$\text{BUCKET-SORT}(A)$

```
1  sia  $B[0..n-1]$  un nuovo array
2   $n := A.length$ 
3  for  $i := 0$  to  $n-1$ 
4      crea la lista vuota  $B[i]$ 
5  for  $i := 1$  to  $n$ 
6      inserisce  $A[i]$  nella lista  $B[\lfloor nA[i] \rfloor]$ 
7  for  $i := 0$  to  $n-1$ 
8      ordina la lista  $B[i]$  con insertion sort
9  concatena in ordine le liste  $B[0], \dots, B[n-1]$ 
```

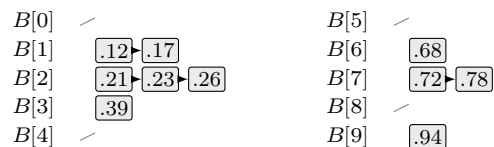


Figura 5.3: I bucket dopo la riga 8 con i numeri del libro: $B[1] = (.12, .17)$, $B[2] = (.21, .23, .26)$, $B[3] = (.39)$, $B[6] = (.68)$, $B[7] = (.72, .78)$, $B[9] = (.94)$ [Cormen fig. 8.4, p. 165].

[Cormen cap. 8.4]: input in $[0, 1)$ — il bucket i raccoglie $[i/n, (i+1)/n)$ — output = concatenazione dei bucket.

Tutte le righe tranne la 8 costano $O(n)$; le insertion sort costano $\sum_{i=0}^{n-1} O(n_i^2)$ con $E[n_i^2] = 2 - 1/n$ per ogni bucket (equazione 8.2, dimostrata nel libro con le variabili indicatrici), quindi $E[T(n)] =$

$\Theta(n) + n \cdot O(2 - 1/n) = \Theta(n)$: in media bucket sort è lineare. Nell'esempio la somma dei quadrati vale $4 + 9 + 1 + 1 + 4 + 1 = 20$ contro il valore atteso $n(2 - 1/n) = 2n - 1 = 19$. Il caso peggiore resta $\Theta(n^2)$, con tutti i valori in un solo bucket [Cormen cap. 8.4].

I tre algoritmi a confronto. Nessuno dei tre è un ordinamento per confronti: counting non confronta mai gli elementi (usa i valori come indici), bucket sort confronta solo dentro i bucket, radix delega a counting. Il limite $\Omega(n \lg n)$ del Teorema 8.1 non li riguarda; il prezzo è l'ipotesi sull'input e la memoria ausiliaria (non ordinano sul posto) [Cormen p. 164]. Costi: $\Theta(n + k) = \Theta(n)$ per counting, $\Theta(d(n + k))$ per radix, $\Theta(n)$ in media per bucket; counting e radix sono stabili (per radix *indispensabile*).

5.5 Minimo, massimo e statistiche d'ordine

Statistiche d'ordine

La *i*-esima statistica d'ordine di n elementi è l'*i*-esimo più piccolo: il **minimo** è la prima ($i = 1$), il **massimo** l'*n*-esima ($i = n$); la **mediana** è il punto di mezzo (con n dispari è unica, $i = (n + 1)/2$, con n pari c'è l'inferiore $\lfloor (n + 1)/2 \rfloor$ e la superiore $\lceil (n + 1)/2 \rceil$). Il **problema della selezione**: dati A di n numeri distinti e i con $1 \leq i \leq n$, restituire $x \in A$ maggiore esattamente di altri $i - 1$ elementi. [Cormen cap. 9]

MINIMUM(A)

```

1  min := A[1]
2  for i := 2 to A.length
3      if min > A[i]
4          min := A[i]
5  return min
```

MINIMUM fa $n - 1$ confronti e non si può fare di meglio: in un torneo in cui vince il più piccolo, ogni elemento tranne il vincitore deve perdere almeno una partita; il massimo si trova allo stesso modo. Separati costano $2n - 2 = 18$ confronti con $n = 10$; elaborandoli *in coppia* — i due fra loro, poi il più piccolo col minimo e il più grande col massimo, 3 confronti ogni 2 elementi — servono $1 + 3(n - 2)/2 = 3n/2 - 2$ confronti con n pari e $3\lfloor n/2 \rfloor$ con n dispari [Cormen cap. 9.1]. Verificato: su $\langle 3, 2, 9, 0, 7, 5, 4, 8, 6, 1 \rangle$ ($n = 10$) sono 13 confronti, su nove elementi 12.

5.6 Selezione in tempo atteso lineare: RANDOMIZED-SELECT

L'idea: la partizione di quicksort, buttandone via una metà

RANDOMIZED-SELECT è quicksort con una differenza decisiva: dopo la partizione si sa in che metà sta l'elemento cercato, quindi si ricorre *solo* su quella. Col pivot in mezzo ogni chiamata butta via metà dell'array ($\Theta(n)$), col pivot sempre massimo si butta via un elemento per volta e il tempo degenera in $\Theta(n^2)$; il pivot a caso rende improbabile il caso pessimo, e in media l'algoritmo è lineare.

RANDOMIZED-SELECT(A, p, r, i)

```

1  if p == r
2      return A[p]
3  q := RANDOMIZED-PARTITION(A, p, r)
4  k := q - p + 1
5  if i == k // il valore del pivot è la soluzione
6      return A[q]
7  elseif i < k
8      return RANDOMIZED-SELECT(A, p, q - 1, i)
9  else return RANDOMIZED-SELECT(A, q + 1, r, i - k)
```

[Cormen cap. 9.2]: RANDOMIZED-PARTITION è quella del capitolo 4; $A[p..q]$ ha k elementi: se $i < k$ (o $i > k$) l'elemento cercato sta nel lato basso (o alto), con indice $i - k$.

Esempio (istanza dell'esercizio 9.2-4, $A = \langle 3, 2, 9, 0, 7, 5, 4, 8, 6, 1 \rangle$, si cerca $i = 5$, la mediana inferiore):

chiamata	pivot	dopo la partizione	decisione
$(A, 1, 10, i=5)$	$A[6] = 5$	$A[1..5] = \langle 3, 2, 0, 1, 4 \rangle, q = 6$	$i < k = 6$: a sinistra
$(A, 1, 5, i=5)$	$A[4] = 1$	$A[1..5] = \langle 0, 1, 3, 4, 2 \rangle, q = 2$	$i > k = 2$: a destra, $i := 3$
$(A, 3, 5, i=3)$	$A[4] = 4$	$A[3..5] = \langle 3, 2, 4 \rangle, q = 5$	$i = k = 3$: è $A[5] = 4$

Questa sequenza di pivot (una delle possibili) costa $9 + 4 + 2 = 15$ confronti per $n = 10$. Il **caso peggiore** è $\Theta(n^2)$ — pivot sempre il massimo rimasto, cercando il minimo: $(n - 1) + \dots + 1 = n(n - 1)/2$ confronti, 45 con $n = 10$ — ma nessun input lo forza, perché il pivot è casuale [Cormen p. 178].

Perché in media è lineare. Con $T(n)$ il tempo su n elementi, il pivot è uniforme e il sottoarray $A[p..q]$ ha k elementi con probabilità $1/n$; supponendo che l'elemento cercato finisca sempre nel lato più grande si ottiene $E[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[T(k)] + O(n)$. Per sostituzione ($E[T(k)] \leq ck$ per $k < n$, termine non ricorsivo $\leq an$) si ha $E[T(n)] \leq \frac{2c}{n} \sum_{k \geq n/2} k + an \leq cn$ per $c > 4a$: $E[T(n)] = O(n)$ [Cormen cap. 9.2]. Misurato su array casuali, confronti per elemento: 3.02 con $n = 100$, 3.44 con 1000, 3.41 con 10^4 — sempre sotto $4n$. [non nelle fonti del corso]

5.7 Selezione in tempo lineare nel caso peggiore: SELECT

SELECT risolve la selezione in $O(n)$ nel caso peggiore, senza casualità: il pivot è la **mediana delle mediane** dei gruppi di 5. I passi del libro:

SELECT(A, p, r, i)

```

1   $n := r - p + 1$ 
2  dividi  $A[p..r]$  in  $\lfloor n/5 \rfloor$  gruppi di 5, e al più un resto
3  ordina ogni gruppo con insertion sort e fanne la mediana (il posto di mezzo)
4   $x :=$  la mediana (inferiore) delle  $\lfloor n/5 \rfloor$  mediane, con SELECT ricorsivo
5   $q :=$  PARTITION( $A, p, r, x$ ) // con il pivot  $x$  passato da fuori
6   $k := q - p + 1$  //  $x$  è il  $k$ -esimo più piccolo
7  if  $i == k$ 
8      return  $x$ 
9  elseif  $i < k$ 
10     return SELECT( $A, p, q - 1, i$ )
11 else return SELECT( $A, q + 1, r, i - k$ )
```

[Cormen cap. 9.3]: PARTITION è quella del capitolo 4, col pivot come parametro.

Perché il pivot è buono. Metà delle $\lfloor n/5 \rfloor$ mediane sono $\geq x$ e ogni gruppo completo trascina 3 elementi $> x$: tolti il gruppo di x e l'ultimo, il lato della ricorsione ha al più $7n/10 + 6$ elementi — almeno $3n/10 - 6$ restano fuori da ciascuna parte. La ricorrenza del caso peggiore è

$$T(n) \leq T(\lfloor n/5 \rfloor) + T(7n/10 + 6) + O(n) \quad (n \geq 140), \quad T(n) = O(1) \text{ per } n < 140,$$

e il punto è che $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$: i due sottoproblemi *insieme* sono più piccoli dell'input, quindi il lavoro decresce geometricamente e per sostituzione $T(n) \leq cn$ con $c \geq 20a$ (per $n \geq 140$ vale $n/(n - 70) \leq 2$: è da qui che viene la soglia 140 del libro): SELECT è $\Theta(n)$ nel caso peggiore. Rispetto a RANDOMIZED-SELECT è deterministico ma con costanti più alte; e, non ordinando, non tocca il limite $\Omega(n \lg n)$ del Teorema 8.1. [Cormen cap. 9.3]

Come rispondere: il limite inferiore $\Omega(n \lg n)$

1. **Definizione:** un ordinamento per confronti decide l'ordine solo con test del tipo $a_i \leq a_j$, senza guardare i valori.
2. **Modello:** l'albero di decisione — nodi interni $i : j$ (i confronti), foglie le permutazioni; l'altezza è il numero di confronti del caso peggiore.
3. **Proprietà:** ogni permutazione sta in una foglia ($n! \leq l$) e un albero di altezza h ha al più 2^h foglie ($l \leq 2^h$); quindi $h \geq \lg(n!) = \Theta(n \lg n)$ — per $n = 3$ servono 3 confronti ($6 \leq 8 = 2^3$), per $n = 10$ almeno 22.
4. **Limiti:** vale per heapsort, merge sort e quicksort (che sono ottimali), non per counting, radix e bucket sort, che non sono ordinamenti per confronti e pagano con le ipotesi sull'input.

Errori tipici da evitare

- Counting sort con k grande: il tempo è $\Theta(n + k)$, lineare solo se $k = O(n)$.
- Dimenticare $C[A[j]] := C[A[j]] - 1$ nel ciclo finale (i valori ripetuti finiscono nella stessa posizione), o scorrerlo da sinistra a destra: il primo errore colloca male i duplicati, il secondo perde la stabilità e rompe radix sort.
- Radix dalla cifra più significativa, o con un ordinamento intermedio non stabile: l'esempio del libro non verrebbe ordinato. E applicare $\Omega(n \lg n)$ a counting, radix e bucket sort, che sono fuori dal modello dell'albero di decisione.

Capitolo 6

Hashing

Un **dizionario** è un insieme dinamico con le sole operazioni INSERT, SEARCH e DELETE: è quello che serve, per esempio, al compilatore per la sua *tabella dei simboli*. Il modo più semplice per realizzarlo è l'**indirizzamento diretto**, una cella per ogni chiave possibile; se però l'universo delle chiavi è grande e le chiavi usate sono poche, quasi tutte quelle celle resterebbero vuote. Le **tavole hash** rimpiccioliscono la tavola e calcolano la cella con una **funzione hash**; il prezzo sono le *collisioni* e un'analisi che vale *in media*: sotto ipotesi ragionevoli ogni operazione costa $O(1)$ in media. [Cormen cap. 11]

6.1 Tavole a indirizzamento diretto

L'idea: una cella per ogni chiave

Se le chiavi sono interi di un intervallo piccolo, si usano come indirizzi: la chiave k va nella cella k , senza funzioni né collisioni.

Tavola a indirizzamento diretto

Sia $U = \{0, 1, \dots, m-1\}$ l'universo delle chiavi, piccolo, e si supponga che due elementi non possano avere la stessa chiave. Si usa un array $T[0..m-1]$, la **tavola a indirizzamento diretto**, in cui la cella k punta all'elemento con chiave k , e $T[k] = \text{NIL}$ se l'elemento non c'è. Le operazioni costano $O(1)$ *nel caso peggiore*: SEARCH è $T[k]$, INSERT è $T[x.\text{key}] := x$, DELETE è $T[x.\text{key}] := \text{NIL}$. [Cormen cap. 11.1]

Il difetto è evidente: se U è grande (per esempio tutte le stringhe di lunghezza al più 10) una tavola di $|U|$ celle può essere impraticabile, e se le chiavi effettive K sono poche rispetto a U lo spazio è quasi tutto sprecato. [Cormen cap. 11.2]

6.2 Tavole hash e collisioni

L'idea: rimpicciolire, accettando le collisioni

Un array di m celle proporzionale al numero di chiavi effettive, con la chiave k mandata nella cella $h(k)$ da una funzione hash. Poiché $|U| > m$ due chiavi finiranno prima o poi nella stessa cella, e non si può evitare: si può solo decidere che cosa fare (concatenamento: liste nelle celle; indirizzamento aperto: la tavola cerca la cella successiva).

Tavola hash, funzione hash, collisione

Una **funzione hash** h associa l'universo U delle chiavi alle celle di una **tavola hash** $T[0..m-1]$: $h : U \rightarrow \{0, 1, \dots, m-1\}$ con $m \ll |U|$; $h(k)$ è il *valore hash* di k . Due chiavi distinte con lo stesso valore hash sono in **collisione**: poiché $|U| > m$ ce ne sono per forza, occorre un metodo per risolverle. [Cormen cap. 11.2]

6.2.1 Concatenamento

Nel **concatenamento** gli elementi con lo stesso valore hash stanno in una lista concatenata: la cella j contiene il puntatore alla testa della lista (o NIL se è vuota). [Cormen p. 212]

CHAINED-HASH-INSERT(T, x)

1 inserisce x in testa alla lista $T[h(x.key)]$

CHAINED-HASH-SEARCH(T, k)

1 cerca un elemento con chiave k nella lista $T[h(k)]$

CHAINED-HASH-DELETE(T, x)

1 cancella x dalla lista $T[h(x.key)]$

L'inserimento costa $O(1)$ nel caso peggiore (si suppone che x non sia già nella tavola) e la cancellazione $O(1)$ se le liste sono *doppiamente* concatenate: CHAINED-HASH-DELETE riceve l'elemento x , non la chiave, quindi non deve cercare nulla.

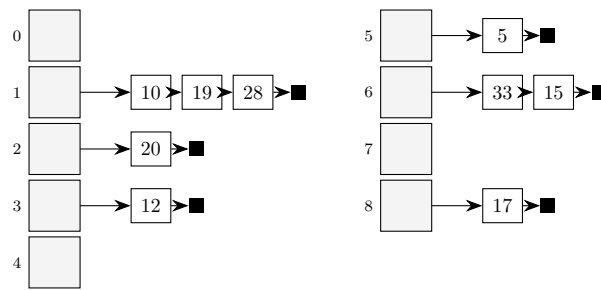


Figura 6.1: Esercizio 11.2-2: $m = 9$, $h(k) = k \bmod 9$, chiavi 5, 28, 19, 15, 20, 33, 12, 17, 10 inserite in testa (le celle 0, 4 e 7 restano vuote; ogni lista termina con NIL).

Esempio svolto (Esercizio 11.2-2 del libro). Le chiavi 5, 28, 19, 15, 20, 33, 12, 17, 10 entrano in una tavola $m = 9$ con $h(k) = k \bmod 9$ nelle celle 5, 1, 1, 6, 2, 6, 3, 8, 1, ciascuna in testa alla lista: la tavola della Figura 6.1, con le liste 1 : 10 → 19 → 28; 2 : 20; 3 : 12; 5 : 5; 6 : 33 → 15; 8 : 17. Le ricerche esaminano la lista di $h(k)$: cercare 28 ne esamina 3, cercare 15 ne esamina 2, una chiave assente come 6 ne esamina 2 e poi trova NIL. Le nove ricerche con successo esaminano $1 + 3 + 2 + 2 + 1 + 1 + 1 + 1 + 1 = 13$ elementi ($13/9 = 1.444$ in media), esattamente il valore della formula del Teorema 11.2 per $\alpha = 1$.

6.2.2 L'analisi del concatenamento

Fattore di carico e hashing uniforme semplice

Con n elementi in una tavola di m celle, il **fattore di carico** è $\alpha = n/m$, il numero medio di elementi per lista (può essere minore, uguale o maggiore di 1). Nell'ipotesi di **hashing uniforme semplice** qualsiasi elemento ha la stessa probabilità di essere mandato in una qualsiasi delle m celle, indipendentemente dagli altri: se n_j è la lunghezza della lista $T[j]$, allora $n = n_0 + \dots + n_{m-1}$ e $E[n_j] = \alpha$. [Cormen cap. 11.2]

Nel caso peggiore tutte le chiavi possono finire in una cella, ricerca $\Theta(n)$; nel caso medio valgono i due teoremi del Laboratorio. [Cormen pp. 213-215]

Teorema 11.1 (ricerca senza successo)*da saper dimostrare*

In una tavola hash le cui collisioni sono risolte con il concatenamento, una ricerca senza successo richiede un tempo $\Theta(1 + \alpha)$ nel caso medio, nell'ipotesi di hashing uniforme semplice.

Dimostrazione. Una chiave k non ancora memorizzata ha la stessa probabilità di essere mandata in ciascuna delle m celle: il tempo per cercarla senza successo è quello per scorrere fino in fondo alla lista $T[h(k)]$, di lunghezza attesa $E[n_{h(k)}] = \alpha$. Gli elementi esaminati sono α in media, e il tempo totale (compreso il calcolo di $h(k)$, che costa $O(1)$) è $\Theta(1 + \alpha)$. $\square$

Teorema 11.2 (ricerca con successo)*da saper dimostrare*

In una tavola hash le cui collisioni sono risolte con il concatenamento, una ricerca con successo richiede un tempo $\Theta(1 + \alpha)$ nel caso medio, nell'ipotesi di hashing uniforme semplice.

Dimostrazione. Sia x l'elemento cercato, con la stessa probabilità di essere uno qualsiasi degli n elementi. Gli elementi esaminati sono 1 più quelli che stanno *prima* di x nella sua lista, e sono tutti elementi inseriti *dopo* x (i nuovi vanno in testa). Con $x_1, \dots, x_n$ in ordine di inserimento, $k_i = x_i.key$ e $X_{ij} = I\{h(k_i) = h(k_j)\}$, si ha $\Pr\{h(k_i) = h(k_j)\} = 1/m$, quindi $E[X_{ij}] = 1/m$. Il numero atteso di elementi esaminati è

$$\begin{aligned} E\left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij}\right)\right] &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E[X_{ij}]\right) = \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m}\right) \\ &= 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2}\right) = 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}, \end{aligned}$$

per la linearità del valore atteso e per l'equazione (A.1). Il tempo totale è $\Theta(2 + \alpha/2 - \alpha/(2n)) = \Theta(1 + \alpha)$. $\square$

Il significato: se $n = O(m)$, cioè $\alpha = O(1)$, tutte le operazioni sono $O(1)$ in media. [Cormen p. 215]

6.3 Le funzioni hash

L'idea: una buona funzione hash sembra casuale

L'analisi precedente vale *se* la distribuzione delle chiavi è uniforme: una buona funzione hash rende $h(k)$ indipendente dalle regolarità dei dati (*pt* e *pts* non devono finire nella stessa cella). Le chiavi si interpretano come numeri naturali: *pt* è (112, 116) in ASCII, cioè $112 \cdot 128 + 116 = 14452$ in base 128. [Cormen cap. 11.3]

6.3.1 Divisione e moltiplicazione

Con il **metodo della divisione**, $h(k) = k \bmod m$: una sola operazione. Per esempio con $m = 12$ e $k = 100$ si ha $h(k) = 4$. Alcuni valori di m vanno evitati [Cormen cap. 11.3.1]: $m = 2^p$ no, perché $h(k)$ sarebbe fatto solo dai p bit meno significativi di k (il resto della chiave è ignorato); $m = 2^p - 1$ no, se k è una stringa di caratteri in base 2^p (permutare i caratteri non cambia il valore hash, Esercizio 11.3-3). La scelta tipica è un **numero primo non troppo vicino a una potenza esatta di 2**: per esempio, per $n \approx 2000$ stringhe con al più 3 elementi esaminati in media in una ricerca senza successo, il libro sceglie il primo $m = 701$, vicino a $2000/3 = 666.7$ ma non a una potenza di 2. [Cormen p. 218]

Con il **metodo della moltiplicazione** si prende la parte frazionaria di kA ($0 < A < 1$) e la si riporta in $\{0, \dots, m-1\}$:

$$h(k) = \lfloor m(kA \bmod 1) \rfloor, \quad kA \bmod 1 = kA - \lfloor kA \rfloor.$$

Qui m non è critico (di solito $m = 2^p$) e la funzione si implementa su una parola di w bit: con $s = A \cdot 2^w$ intero, il prodotto $k \cdot s = r_1 2^w + r_0$ ha $2w$ bit e $h(k)$ è fatto dai p bit più significativi di r_0 . Knuth propone $A \approx (\sqrt{5} - 1)/2 = 0.6180339887 \dots$ [Cormen cap. 11.3.2]

Esempio svolto (libro, p. 219). Con $k = 123456$, $p = 14$, $m = 2^{14} = 16384$, $w = 32$ e $A = 2654435769/2^{32}$ (la frazione $s/2^{32}$ più vicina a $(\sqrt{5} - 1)/2$): $k \cdot s = 327706022297664 = (76300 \cdot 2^{32}) + 17612864$, quindi $r_1 = 76300$, $r_0 = 17612864$. I 14 bit più significativi di r_0 si prendono scorrendo di $32 - 14 = 18$ posizioni: $17612864 \gg 18 = 67$. La formula esatta $\lfloor 16384(123456 A \bmod 1) \rfloor$ dà lo stesso valore 67.

6.3.2 L'hashing universale

L'idea: sorteggiare la funzione hash

Una funzione hash fissata ha un punto debole: un avversario che la conosce può scegliere n chiavi con lo stesso valore hash, e ogni ricerca costerà $\Theta(n)$ in media. La soluzione è sorteggiarla all'inizio dell'esecuzione da una famiglia progettata bene, indipendente dalle chiavi: come nel quicksort, nessun input provoca sistematicamente il caso peggiore.

Collezione universale di funzioni hash

Una collezione finita $\mathcal{H}$ di funzioni hash da U in $\{0, 1, \dots, m-1\}$ è **universale** se per ogni coppia di chiavi distinte $k, l \in U$ il numero di $h \in \mathcal{H}$ con $h(k) = h(l)$ è al massimo $|\mathcal{H}|/m$: con h scelta a caso da $\mathcal{H}$, due chiavi distinte collidono con probabilità al più $1/m$. [Cormen cap. 11.3.3]

Teorema 11.3 (e Corollario 11.4): le liste restano corte in media

Con h scelta a caso da una collezione universale e il concatenamento, la lista $T[h(k)]$ ha lunghezza attesa $\leq \alpha$ se k è assente e $\leq 1 + \alpha$ se c'è, per *qualsiasi* insieme di chiavi; così, con $O(m)$ inserimenti, ogni sequenza di n operazioni costa $\Theta(n)$ in media (dimostrazione nel libro, p. 220). [Cormen cap. 11.3.3]

Per costruirne una si sceglie un primo p più grande di ogni chiave (e $p > m$) e si definisce, per ogni $a \in \mathbb{Z}_p^* = \{1, \dots, p-1\}$ e $b \in \mathbb{Z}_p = \{0, \dots, p-1\}$,

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m, \quad \mathcal{H}_{pm} = \{h_{ab} : a \in \mathbb{Z}_p^*, b \in \mathbb{Z}_p\}, \quad (6.1)$$

$p(p-1)$ funzioni. La riduzione modulo p mescola le chiavi senza collisioni, quella modulo m le riporta nelle celle; m può essere qualsiasi, non serve che sia primo. [Cormen p. 221]

Esempio (libro, p. 221). Con $p = 17$ e $m = 6$: $h_{3,4}(8) = ((3 \cdot 8 + 4) \bmod 17) \bmod 6 = (28 \bmod 17) \bmod 6 = 11 \bmod 6 = 5$.

Teorema 11.5 ($\mathcal{H}_{pm}$ è universale)

da saper dimostrare

La classe $\mathcal{H}_{pm}$ definita da (6.1) è universale. [Cormen p. 221]

Dimostrazione. Siano $k \neq l$ due chiavi di $\mathbb{Z}_p$ e, per una data h_{ab} , $r = (ak + b) \bmod p$, $s = (al + b) \bmod p$. Anzitutto $r \neq s$: da $r - s \equiv a(k - l) \pmod{p}$ e dal fatto che p è primo segue che $a(k - l) \not\equiv 0 \pmod{p}$ (Teorema 31.6), quindi al «livello mod p » non ci sono collisioni. La corrispondenza fra le coppie (a, b) con $a \neq 0$ e le coppie (r, s) con $r \neq s$ è uno-a-uno, perché da r e s si ricostruiscono

$$a = ((r - s)((k - l)^{-1} \bmod p)) \bmod p, \quad b = (r - ak) \bmod p,$$

con $(k - l)^{-1}$ inverso moltiplicativo modulo p . Quindi scegliendo (a, b) uniforme la coppia (r, s) è uniforme fra le coppie di valori *distinti* modulo p , e k e l collidono esattamente quando $r \equiv s \pmod{m}$ con $r \neq s$:

per r fissato, dei $p - 1$ valori di s al più $\lceil p/m \rceil - 1 \leq (p - 1)/m$ sono $\equiv r \pmod{m}$, quindi $\Pr\{h_{ab}(k) = h_{ab}(l)\} \leq ((p - 1)/m)/(p - 1) = 1/m$. $\square$

6.4 L'indirizzamento aperto

L'idea: gli elementi stanno tutti nella tavola

Un'alternativa al concatenamento: una chiave per cella, senza puntatori né liste. Per inserire k si esaminano le celle in un ordine che dipende da k fino a trovarne una vuota; quando la tavola è piena non si inserisce più. Perdendo i puntatori si recupera spazio.

Indirizzamento aperto

Ogni cella contiene un elemento dell'insieme dinamico o la costante NIL, e $h : U \times \{0, 1, \dots, m - 1\} \rightarrow \{0, 1, \dots, m - 1\}$: per ogni chiave k la **sequenza di ispezione** $\langle h(k, 0), h(k, 1), \dots, h(k, m - 1) \rangle$ deve essere una permutazione di $\langle 0, 1, \dots, m - 1 \rangle$ (ogni cella deve poter essere usata). Il fattore di carico non supera mai 1. [Cormen cap. 11.4]

HASH-INSERT(T, k)

```

1   $i := 0$ 
2  repeat
3       $j := h(k, i)$ 
4      if  $T[j] == \text{NIL}$ 
5           $T[j] := k$ 
6          return  $j$ 
7      else  $i := i + 1$ 
8  until  $i == m$ 
9  error "overflow della tavola hash"
```

HASH-SEARCH(T, k)

```

1   $i := 0$ 
2  repeat
3       $j := h(k, i)$ 
4      if  $T[j] == k$ 
5          return  $j$ 
6       $i := i + 1$ 
7  until  $T[j] == \text{NIL}$  or  $i == m$ 
8  return NIL
```

HASH-SEARCH esamina la stessa sequenza dell'inserimento e può fermarsi alla prima cella vuota: la chiave sarebbe stata messa lì. Cancellare è invece difficile: rimettere NIL nella cella di una chiave cancellata fermerebbe troppo presto le ricerche di altre chiavi passate di lì; si marca la cella DELETED, che HASH-INSERT tratta come vuota e HASH-SEARCH ignora (ma con DELETED i tempi non dipendono più da α : se servono cancellazioni si preferisce il concatenamento). [Cormen pp. 224-225]

6.4.1 Lineare, quadratica, doppio hashing

Le tre sequenze di ispezione

Sia $h' : U \rightarrow \{0, \dots, m - 1\}$ una **funzione hash ausiliaria**. **Ispezione lineare**: $h(k, i) = (h'(k) + i) \bmod m$; le celle in ordine ciclico da $h'(k)$, solo m sequenze distinte; si formano lunghe file di celle occupate (**addensamento primario**: una cella vuota preceduta da i celle piene ha probabilità $(i + 1)/m$ di essere la prossima occupata). **Ispezione quadratica**: $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$ con $c_2 \neq 0$, posizioni distanziate in modo quadratico (c_1, c_2, m non si scelgono a caso, per usare tutta la tavola); due chiavi con la stessa posizione iniziale hanno la stessa sequenza, l'**addensamento secondario**, più lieve del primario. **Doppio hashing**: $h(k, i) = (h_1(k) + i h_2(k)) \bmod m$: dipendono dalla chiave sia la posizione iniziale sia il passo, con $\Theta(m^2)$ sequenze quasi casuali; serve $h_2(k)$ *relativamente primo* con m perché la tavola venga ispezionata tutta (Esercizio 11.4-4): m potenza di 2 e h_2 dispari, oppure m primo e $h_2(k) = 1 + (k \bmod m')$ con m' poco minore di m (come $m - 1$). [Cormen

cap. 11.4]

Per esempio, con $k = 123456$, $m = 701$ primo e $m' = 700$: $h_1(k) = 80$ e $h_2(k) = 1 + (123456 \bmod 700) = 257$: ispezione dalla cella 80, poi ogni 257 celle (modulo 701). [Cormen p. 226]

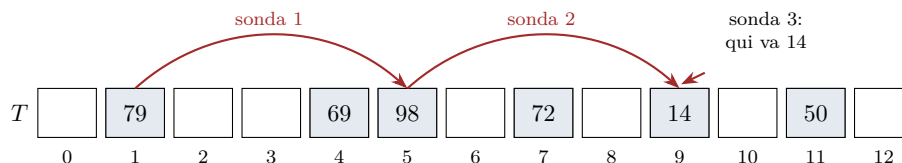


Figura 6.2: Figura 11.5 del libro: inserimento della chiave 14 con doppio hashing, $h_1(k) = k \bmod 13$ e $h_2(k) = 1 + (k \bmod 11)$. Poiché $14 \equiv 1 \pmod{13}$ e $14 \equiv 3 \pmod{11}$, $h_1(14) = 1$ e $h_2(14) = 4$: cella 1 (79), cella $1 + 4 = 5$ (98), cella $5 + 4 = 9$ libera: **14 va nella cella 9**, dopo due sonde fallite.

La stessa sequenza con le tre scansioni (Esercizio 11.4-1 del libro)

Si inseriscano le chiavi 10, 22, 31, 4, 15, 28, 17, 88, 59 in una tavola $m = 11$ con indirizzamento aperto e $h'(k) = k \bmod 11$, con l'ispezione lineare; con la quadratica $c_1 = 1$, $c_2 = 3$; con il doppio hashing $h_2(k) = 1 + (k \bmod 10)$.

Svolgimento. Celle iniziali: $10 \rightarrow 10$, $22 \rightarrow 0$, $31 \rightarrow 9$, $4 \rightarrow 4$, $15 \rightarrow 4$, $28 \rightarrow 6$, $17 \rightarrow 6$, $88 \rightarrow 0$, $59 \rightarrow 4$. Con le sonde: *lineare* $15 \rightarrow 5$, $17 \rightarrow 7$, $88 \rightarrow 1$, $59 \rightarrow 5, 6, 7, 8$; *quadratica* $15 \rightarrow 8$, $17 \rightarrow 10, 9, 3$, $88 \rightarrow 4, 3, 8, 8, 3, 4, 0, 2$ (la sequenza si ripete: c_1 , c_2 e m non si scelgono a caso), $59 \rightarrow 8, 7$; *doppio* $15 \rightarrow 10, 5$, $17 \rightarrow 3$, $88 \rightarrow 9, 7$, $59 \rightarrow 3, 2$:

	cella										
metodo	0	1	2	3	4	5	6	7	8	9	10
lineare	22	88			4	15	28	17	59	31	10
quadratica	22		88	17	4		28	59	15	31	10
doppio	22		59	17	4	15	28	88		31	10

6.4.2 L'analisi dell'indirizzamento aperto

Ipotesi di hashing uniforme

Ogni chiave ha la stessa probabilità di avere come sequenza di ispezione una qualsiasi delle $m!$ permutazioni di $\langle 0, 1, \dots, m-1 \rangle$ (l'estensione dell'hashing uniforme semplice a una sequenza intera). Nessuna delle tre tecniche la soddisfa (al più m^2 sequenze), ma il doppio hashing vi si avvicina molto. [Cormen cap. 11.4]

Teorema 11.6 (ricerca senza successo)

da saper dimostrare

Nell'ipotesi di hashing uniforme, in una tavola a indirizzamento aperto con fattore di carico $\alpha = n/m < 1$, il numero atteso di ispezioni in una ricerca senza successo è al massimo $1/(1 - \alpha)$.

Dimostrazione. In una ricerca senza successo ogni ispezione, tranne l'ultima, trova una cella occupata che non contiene la chiave; l'ultima trova una cella vuota. Sia X il numero di ispezioni e A_i l'evento «l' i -esima ispezione viene eseguita e trova una cella occupata». Allora $\{X \geq i\} = A_1 \cap \dots \cap A_{i-1}$ e, condizionando a catena (Esercizio C.2-5),

$$\Pr\{X \geq i\} = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \dots \cdot \frac{n-i+2}{m-i+2} \leq \left(\frac{n}{m}\right)^{i-1} = \alpha^{i-1},$$

perché la j -esima ispezione trova una delle $n - j + 1$ chiavi in una delle $m - j + 1$ celle non ancora esaminate, e $(n - j)/(m - j) \leq n/m$ per ogni j . Poiché $E[X] = \sum_{i \geq 1} \Pr\{X \geq i\}$ (equazione C.25),

$$E[X] \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \sum_{i=0}^{\infty} \alpha^i = \frac{1}{1-\alpha}.$$

□

Il limite $1/(1-\alpha) = 1 + \alpha + \alpha^2 + \dots$ ha un'interpretazione intuitiva: una ispezione si fa sempre; con probabilità circa α la cella è occupata e ne serve una seconda; con probabilità circa α^2 le prime due sono occupate, e così via. [Cormen p. 228]

Corollario 11.7 (inserimento)

L'inserimento in una tavola a indirizzamento aperto con fattore di carico α richiede in media non più di $1/(1-\alpha)$ ispezioni: è una ricerca senza successo seguita dalla sistemazione della chiave nella prima cella vuota trovata ($\alpha < 1$, altrimenti non c'è spazio).

Teorema 11.8 (ricerca con successo)

da saper dimostrare

In una tavola a indirizzamento aperto con fattore di carico $\alpha < 1$, il numero atteso di ispezioni in una ricerca con successo è al massimo $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$, supponendo l'hashing uniforme e che ogni chiave abbia la stessa probabilità di essere cercata.

Dimostrazione. La ricerca di una chiave segue la sequenza di ispezione del suo inserimento: se k era la $(i+1)$ -esima chiave inserita, la sua ricerca fa in media al più $1/(1-i/m) = m/(m-i)$ ispezioni (Corollario 11.7). Mediando sulle n chiavi:

$$\frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i} = \frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k} \leq \frac{1}{\alpha} \int_{m-n}^m \frac{1}{x} dx = \frac{1}{\alpha} \ln \frac{m}{m-n} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha},$$

dove la disuguaglianza limita la somma con l'integrale (A.12). □

Se la tavola è piena a metà ($\alpha = 0.5$) una ricerca senza successo fa al più 2 ispezioni e una con successo meno di 1.387; al 90% ($\alpha = 0.9$): 10 e 2.559; per $\alpha = 3/4$ (Esercizio 11.4-3): 4 e 1.848; per $\alpha = 7/8$: 8 e 2.377. [Cormen p. 229]

6.5 Come si risolve all'esame

Eeguire una sequenza di inserimenti e ricerche

Data la tavola (m celle), la funzione hash e le chiavi, calcola prima le celle ($h(k) = k \bmod m$, e $h_1(k)$, $h_2(k)$ nel doppio hashing), poi:

1. **Concatenamento:** aggiungi ogni chiave, in ordine, *in testa* alla lista di $h(k)$ (Figura 6.1); per cercare conta gli elementi esaminati fino alla chiave, o fino a NIL (fallimento: si esamina tutta la lista).
2. **Aperto:** prova $h(k, 0)$, poi $h(k, 1)$, $h(k, 2)$, ... $(+i; +c_1 i + c_2 i^2; +i h_2(k))$, scrivendo *le sonde di ogni chiave*, e inserisci nella prima cella vuota; la ricerca si ferma alla chiave o alla prima cella vuota.
3. **Controllo con l'analisi:** $\approx 1 + \alpha$ elementi al concatenamento, $\leq 1/(1-\alpha)$ e $\leq \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$ ispezioni all'aperto.

Come rispondere: tabelle hash

1. **Definizione:** una tavola hash realizza un dizionario con una funzione $h : U \rightarrow \{0, \dots, m-1\}$ e una tavola di m celle; collisioni inevitabili perché $|U| > m$.

2. **Le due famiglie:** concatenamento (liste nelle celle) e indirizzamento aperto (sequenza di ispezione, $\alpha \leq 1$); l'analisi si fa nell'ipotesi di hashing uniforme (semplice), con il fattore di carico $\alpha = n/m$.
3. **Concatenamento:** ricerca con e senza successo $\Theta(1 + \alpha)$, e $O(1)$ in media se $n = O(m)$.
4. **Aperto:** senza successo $\leq 1/(1 - \alpha)$, con successo $\leq \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$; l'inserimento costa come una ricerca senza successo; a tavola piena a metà: 2 ispezioni senza successo e 1.387 con successo.
5. **Limiti ed esempio:** il caso peggiore resta $\Theta(n)$; contro un avversario si usa l'hashing universale ($\mathcal{H}_{pm}$ è universale, $h_{ab}(k) = ((ak + b) \bmod p) \bmod m$).

Errore tipico da evitare

- $h_2(k)$ **non deve mai valere 0**. In $h(k, i) = (h_1(k) + i h_2(k)) \bmod m$ con $h_2(k) = 0$ la sequenza resta *ferma* su $h_1(k)$ e la tavola non viene mai percorsa: con $m = 11$ la scelta $h_2(k) = k \bmod 11$ è sbagliata ($h_2(22) = 0$). La formula $h_2(k) = 1 + (k \bmod m')$ dà sempre un valore fra 1 e m' , primo con m se m è primo e $m' = m - 1$.
- **Il fattore di carico α** . All'indirizzamento aperto $\alpha = n/m$ *non supera mai 1* (una chiave per cella: se la tavola è piena, HASH-INSERT va in overflow); nel concatenamento invece può superare 1. I limiti dei Teoremi 11.6 e 11.8 valgono solo per $\alpha < 1$: a tavola piena non c'è una cella vuota su cui fermare una ricerca senza successo.

Capitolo 7

Alberi rosso-neri

L'idea: alberi di ricerca che si tengono bassi da soli

Su un albero binario di ricerca ogni operazione elementare costa quanto l'altezza h : se l'albero è basso è veloce come la ricerca binaria, ma un albero costruito male è una catena di n nodi e allora ogni operazione costa n . Gli alberi rosso-neri lo impediscono aggiungendo a ogni nodo un solo bit, il *colore*, e imponendo delle regole su come i colori si alternano lungo i cammini che scendono dalla radice alle foglie: un nodo rosso, per esempio, non può avere figli rossi, quindi su ogni cammino i nodi neri sono almeno la metà. Le regole garantiscono che nessun cammino sia lungo più del doppio di un altro: l'albero è *approssimativamente bilanciato*, l'altezza è $O(\lg n)$ e le operazioni restano $O(\lg n)$ anche nel caso peggiore. Inserimento e cancellazione sono quelli di un albero binario di ricerca seguiti da una «riparazione» (RB-INSERT-FIXUP, RB-DELETE-FIXUP) che usa solo due mosse: cambiare qualche colore e fare una **rotazione**, cioè scambiare fra loro due nodi vicini senza cambiare l'ordine delle chiavi.

7.1 Le cinque proprietà, e quanto è alto un albero rosso-nero

7.1.1 Alberi bilanciati e alberi rosso-neri

Albero rosso-nero [slide RB pp. 2 e 4; Cormen cap. 13.1]

Un **albero rosso-nero** (*Red Black Tree*, *RB tree*) è un albero binario di ricerca in cui ad ogni nodo viene associato un colore, rosso (RED) o nero (BLACK): ogni nodo di un RB tree ha cinque campi, $x.key$, $x.left$, $x.right$ e $x.p$ (come nei BST ordinari) più $x.color$. Vincolando il modo in cui è possibile colorare i nodi lungo un qualsiasi percorso che va dalla radice a una foglia, riusciamo a garantire che l'albero sia **approssimativamente bilanciato**. Le regole sono cinque, le **RB-properties**: un RB tree è un albero binario di ricerca che soddisfa le proprietà

1. ogni nodo è rosso o nero;
2. la radice è nera;
3. ogni foglia è nera;
4. se un nodo è rosso, entrambi i suoi figli devono essere neri;
5. tutti i percorsi da un qualsiasi nodo n ad una qualsiasi delle sue foglie discendenti contengono lo stesso numero di nodi neri.

Come nel capitolo 12, un puntatore NIL si legge come un puntatore a una **foglia** (nodo esterno); i nodi con le chiavi sono i nodi **interni**, e tutte le foglie NIL di un albero si rappresentano con un'unica **sentinella** $T.nil$, nera, che fa anche da padre della radice. Nelle figure le foglie NIL sono dei *quadratini neri*, senza testo.

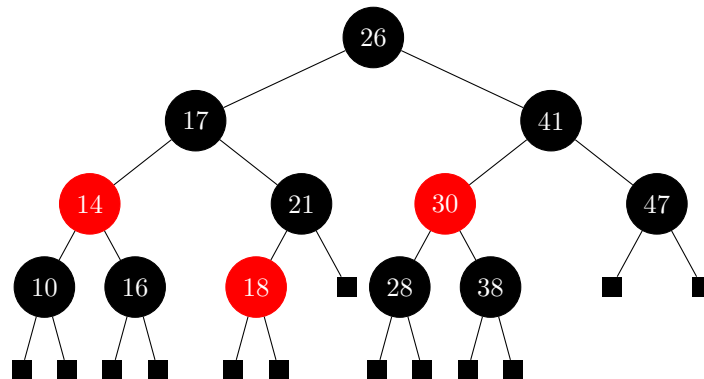
Alberi bilanciati [slide RB p. 3]

Sia T un albero binario e v un nodo di T . Il **fattore di bilanciamento** $\beta(v)$ di v è definito come la differenza tra l'altezza del suo sottoalbero sinistro e quella del suo sottoalbero destro:

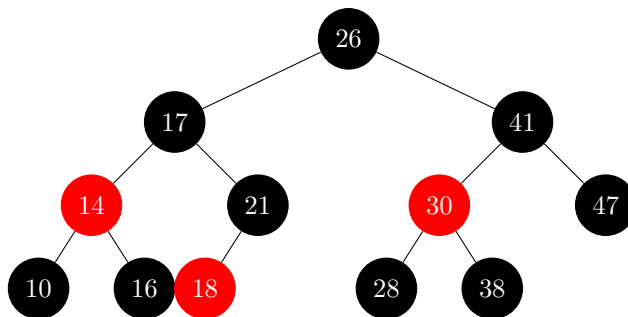
$$\beta(v) = \text{altezza}[\text{left}[v]] - \text{altezza}[\text{right}[v]].$$

Un albero è **bilanciato in altezza** se, per ogni suo nodo v , $|\beta(v)| \leq 1$.

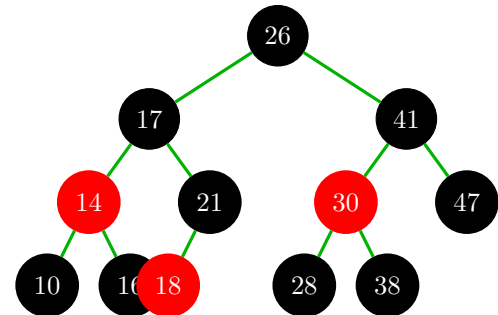
Il bilanciamento in altezza è la condizione più forte possibile: chiede che ogni nodo abbia i due sottoalberi alti quasi uguali. Un RB tree non chiede tanto: chiede solo che nessun cammino sia lungo più del doppio di un altro, cioè un bilanciamento «approssimativo»; le cinque proprietà sono il modo di ottenerlo pagando un solo bit per nodo.



(a) proprietà 1–3: ogni nodo è rosso o nero, la radice 26 è nera, ogni foglia (è il quadratino $T.nil$) è nera



(b) proprietà 4, nella versione semplificata *senza NIL fittizi*: i rossi 14, 18 e 30 hanno figli neri



(c) proprietà 5: da 26 a ogni foglia sempre lo stesso numero di nodi neri (3: senza contare 26, contando le foglie NIL)

Figura 7.1: Le cinque RB-properties sull'albero d'esempio del docente (chiavi 26, 17, 41, 14, 21, 30, 47, 10, 16, 18, 28, 38; i rossi sono 14, 18 e 30). Dalla proprietà 4 segue la conseguenza che serve nel Teorema 2: *almeno la metà* dei nodi lungo un qualsiasi cammino dalla radice alle foglie deve essere nera: dopo ogni rosso c'è almeno un nero, quindi un cammino tutto rosso non può esistere. [slide RB pp. 5–8]

Nelle due versioni senza NIL le foglie non si vedono, ma restano: ogni figlio mancante è una foglia $T.nil$, nera, con nessuna chiave. Contare i nodi neri *comprese le foglie NIL* è proprio ciò che ci serve nella prossima sottosezione.

Ripasso: alberi binari di ricerca (capitolo 12)

L'attraversamento simmetrico visita le chiavi in ordine crescente; TREE-SEARCH, TREE-MINIMUM, TREE-INSERT e TREE-DELETE costano $O(h)$. RB-INSERT è TREE-INSERT con la sentinella al posto di NIL, i due figli di z messi alla sentinella, z colorato di rosso e la chiamata alla riparazione; RB-DELETE è TREE-DELETE più la riparazione. [Cormen cap. 12.3, pp. 244 e 246]

7.1.2 Perché funziona: un albero senza rossi è bilanciato, i rossi sono «non troppi»

Per la proprietà 5, un RB tree *senza nodi rossi* deve essere bilanciato: tutti i suoi livelli sono completi, tranne, al più, l'ultimo, al quale può mancare qualche foglia. Non è però quasi completo (come gli heap del capitolo 3) perché le foglie mancanti non sono necessariamente quelle più a destra. A questo albero bilanciato possiamo aggiungere alcuni (*non troppi*) nodi rossi; per la proprietà 4, infatti, se un nodo è rosso i suoi figli devono essere neri: i rossi si inseriscono come foglie, o come nodi con due figli neri, mai due di seguito su un cammino. [slide RB pp. 9–11]

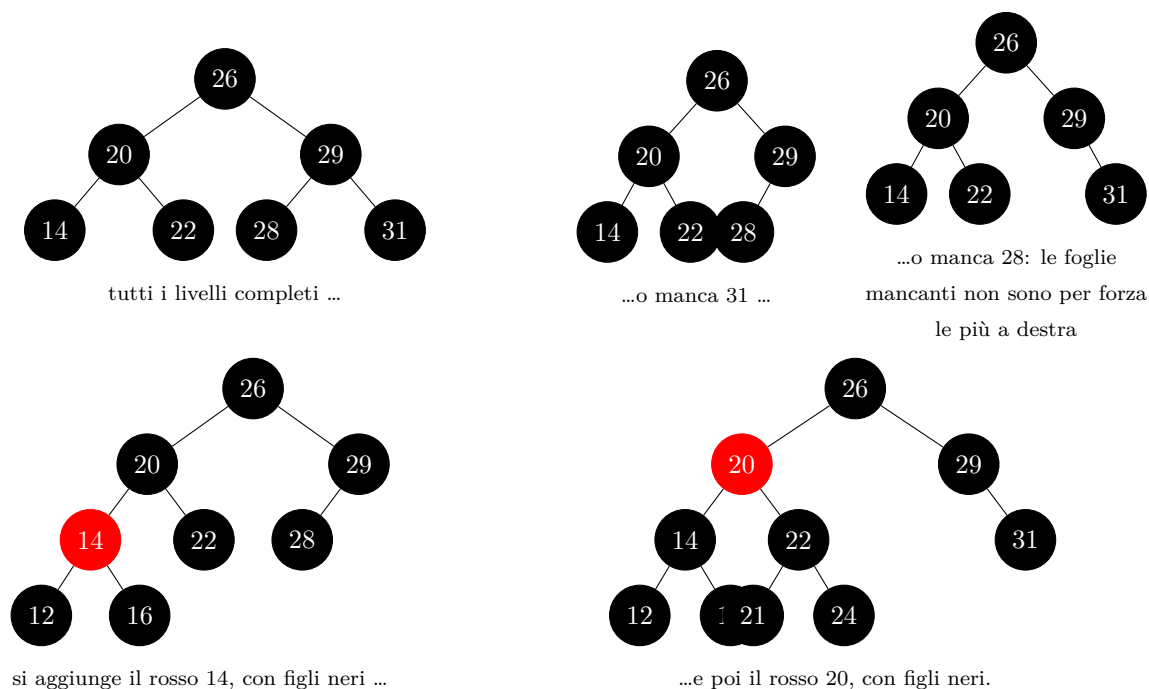


Figura 7.2: Il ponte tra «tutto nero» e «bilanciato». **In alto:** un RB tree senza nodi rossi è bilanciato — tutti i suoi livelli sono completi, tranne al più l'ultimo, al quale può mancare qualche foglia; le foglie mancanti non sono necessariamente le più a destra (29 con il solo figlio destro 31 è ancora valido). **In basso:** allo stesso albero si aggiungono «non troppi» nodi rossi, ciascuno con figli neri (i rossi sono foglie, o nodi con due figli neri). [slide RB pp. 9–11]

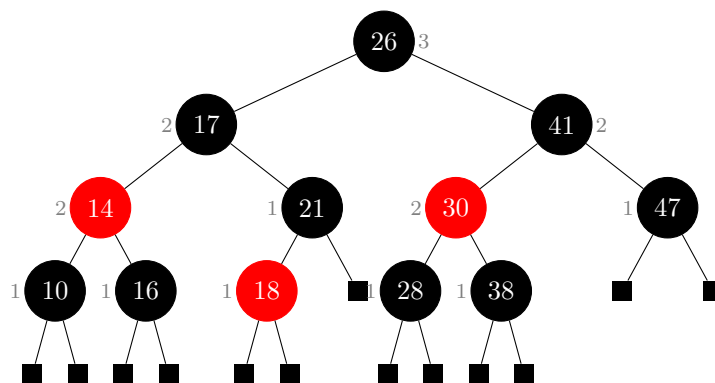
7.1.3 L'altezza nera

Altezza nera [slide RB pp. 12–14]

Sia x un nodo di un RB tree T . L'**altezza nera** $bh(x)$ di x è pari al numero di nodi neri (x escluso) lungo un cammino da x ad una delle sue foglie discendenti. L'altezza nera di T è l'altezza nera della radice r : $bh(T) = bh(r)$. Per la proprietà 5 il concetto di altezza nera è **ben definito** (tutti i cammini da x alle foglie hanno lo stesso numero di neri). Inoltre, se $p = x.p$:

1. x rosso implica $bh(x) = bh(p)$;
2. x nero implica $bh(x) = bh(p) - 1$;
3. in entrambi i casi, $bh(x) \geq bh(p) - 1$.

Una foglia NIL ha $bh = 0$, e un nodo nero i cui figli sono foglie ha $bh = 1$.



l'altezza nera $bh(x)$ annotata accanto a ogni nodo: dalla radice 26 a ogni foglia (NIL comprese) si incontrano sempre 3 nodi neri

Figura 7.3: L'albero d'esempio della figura 7.1 con l'altezza nera annotata. La radice ha $bh = 3$; i nodi rossi (14, 18, 30) hanno la stessa altezza nera del padre (x rosso $\Rightarrow bh(x) = bh(p)$). [slide RB p. 13]

Il calcolo di $bh(x)$ si fa per casi, guardando i figli L e R di x e le loro altezze nere; quando x è disegnato in grigio il suo colore è «qualunque» (il conto non lo usa):

- x è una foglia (NIL): $bh(x) = 0$;
- (a) x ha due figli neri, entrambi di altezza nera h : il cammino da x passa da un figlio (nero) e poi conta h neri, quindi $bh(x) = h + 1$;
- (b) x è nero con due figli rossi di altezza nera h : i figli non contano, $bh(x) = h$;
- (c) x è nero, il figlio sinistro è rosso di altezza nera h e il destro è nero di altezza nera $h - 1$: $bh(x) = h$ (a contare è il figlio nero);
- (d) x è nero, il figlio sinistro è nero di altezza nera $h - 1$ e il destro è rosso di altezza nera h : $bh(x) = h$.

In ogni caso vale la regola dei tre punti qui sopra: $bh(x)$ sta tra $bh(p) - 1$ e $bh(p)$. [slide RB p. 15]

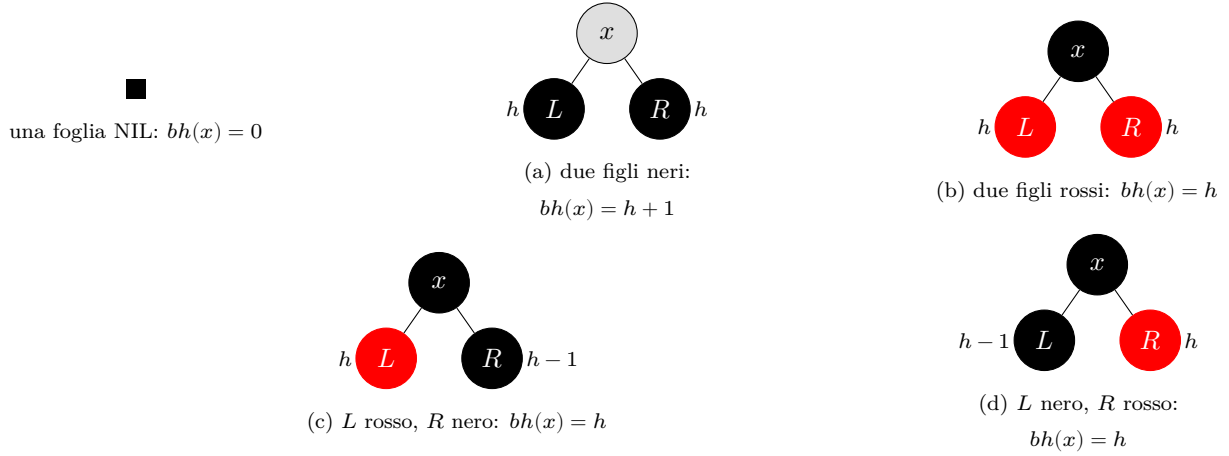


Figura 7.4: Il calcolo di $bh(x)$ per casi. Il quadratino in alto è la foglia $T.nil$ ($bh = 0$); il nodo grigio è di colore «qualunque», quello nero è nero e i rossi sono rossi. In (a) i due figli sono neri e il cammino più corto passa da uno di loro: $bh(x) = h + 1$. In (b) i figli rossi non contano. In (c) e (d) conta il figlio nero. Comunque sia, $bh(x)$ vale $bh(p)$ oppure $bh(p) - 1$. [slide RB pp. 14–15]

7.1.4 Quanto è alto un albero rosso-nero

Perché bh misura quanto è «grasso» l'albero

Il Lemma 1 risponde a una domanda da contabile: sotto un nodo x , per avere $bh(x)$ nodi neri su *ogni* cammino, quanti nodi ci vogliono? Risposta: almeno $2^{bh(x)} - 1$. È lo stesso conto dell'albero binario completo (per avere k livelli servono $2^k - 1$ nodi): tanti nodi neri obbligano ad avere tanti nodi. Il Teorema 2 ribalta la frase: tanti nodi $\Rightarrow$ l'altezza non può crescere troppo.

Lemma 1 (un sottoalbero nero-alto è grande)

da saper dimostrare

Il numero di nodi interni di un sottoalbero radicato in x è maggiore o uguale di $2^{bh(x)} - 1$.

[slide RB pp. 16–18; Cormen cap. 13.1, Lemma 13.1]

Dimostrazione. Indichiamo con $int(x)$ il numero dei nodi interni del sottoalbero radicato in x , e dimostriamo, per induzione sull'altezza (ordinaria) h di x , che $int(x) \geq 2^{bh(x)} - 1$.

Caso base: $h = 0$. In questo caso x è una foglia, $int(x) = 0$ e $bh(x) = 0$. Allora $int(x) = 0 \geq 2^{bh(x)} - 1 = 2^0 - 1 = 0$.

Passo induttivo: $h > 0$. In questo caso x ha due figli (non entrambi NIL) $l = x.left$ ed $r = x.right$; l'altezza di l e r è minore di quella di x , quindi vale l'ipotesi induttiva. Inoltre $bh(l), bh(r) \geq bh(x) - 1$ (proprietà dell'altezza nera). Per ipotesi induttiva:

$$int(l) \geq 2^{bh(l)} - 1 \geq 2^{bh(x)-1} - 1 \quad \text{e} \quad int(r) \geq 2^{bh(r)} - 1 \geq 2^{bh(x)-1} - 1.$$

Allora

$$int(x) \geq 1 + int(l) + int(r) \geq 1 + (2^{bh(x)-1} - 1) + (2^{bh(x)-1} - 1) = 2 \cdot 2^{bh(x)-1} - 1 = 2^{bh(x)} - 1.$$

□

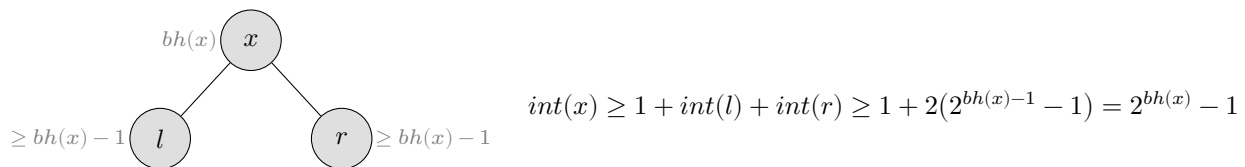


Figura 7.5: Il passo induttivo del Lemma 1: i due figli di x hanno altezza nera almeno $bh(x) - 1$, quindi per ipotesi induttiva almeno $2^{bh(x)-1} - 1$ nodi interni ciascuno. [non nelle fonti del corso]

Dal nero-alto al logaritmo

Ora mettiamo insieme i due ingredienti: la proprietà 4 garantisce che su ogni cammino dalla radice alle foglie i neri sono almeno la metà (mai due rossi di seguito), e il Lemma 1 trasforma i neri in nodi. Più l'albero è alto, più neri contiene ogni cammino, più nodi deve avere: catena di disuguaglianze e l'altezza finisce schiacciata contro $2\lg(n+1)$.

Teorema 2 (l'altezza è logaritmica)

da saper dimostrare

L'altezza di un RB tree con n nodi interni è minore o uguale di $2\lg(n+1)$, cioè $h = O(\lg n)$.

[slide RB pp. 16 e 19–21]

Dimostrazione. Sia h l'altezza dell'albero.

1. Per la proprietà 4, almeno la metà dei nodi in un qualsiasi cammino dalla radice ad una foglia (esclusa la radice) devono essere neri: dopo ogni nodo rosso c'è almeno un nodo nero. Di conseguenza $bh(T) = bh(root) \geq h/2$.
2. Per il Lemma 1, applicato alla radice: $n = int(T) \geq 2^{bh(T)} - 1 \geq 2^{h/2} - 1$, ossia $2^{h/2} \leq n + 1$.
3. Allora $h/2 \leq \lg(n+1)$ e quindi $h \leq 2\lg(n+1)$.

□

Sull'albero d'esempio della figura 7.3, $n = 12$ e $bh(26) = 3$: il Lemma 1 promette almeno $2^3 - 1 = 7$ nodi interni e ce ne sono 12; il Teorema 2 promette $h \leq 2\lg 13 = 7.40$ e l'altezza è 4. Le operazioni di ricerca su un BST costano $O(h)$, quindi SEARCH, MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR costano $O(\lg n)$ nel caso peggiore; per INSERT e DELETE non basta, perché gli algoritmi del capitolo 12 cambiano la struttura e possono distruggere le cinque proprietà. Le sezioni che seguono riparano il guaio, sempre in $O(\lg n)$. [Cormen cap. 13.1]

Che cos'è un albero rosso-nero e perché è bilanciato

1. È un **albero binario di ricerca** in cui ogni nodo ha un bit in più, il colore (rosso o nero), e valgono le cinque RB-properties: ogni nodo rosso o nero, radice nera, foglie NIL nere, nessun rosso con figli rossi, stesso numero di neri da un nodo a tutte le sue foglie.
2. La proprietà 4 dice che su un cammino dalla radice alle foglie i neri sono *almeno la metà*; la proprietà 5 dice che questo numero non dipende dal cammino: è l'**altezza nera** $bh(x)$, contata *senza* x ma *con* la foglia NIL, ben definita.
3. Lemma 1 (per induzione sull'altezza): un sottoalbero di altezza nera k contiene almeno $2^k - 1$ nodi interni.
4. Teorema 2: $bh(T) \geq h/2$ e $n \geq 2^{bh(T)} - 1 \geq 2^{h/2} - 1$, quindi $2^{h/2} \leq n + 1$ e $h \leq 2\lg(n+1) = O(\lg n)$.
5. Di conseguenza SEARCH, MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR costano $O(\lg n)$ nel *caso peggiore*; INSERT e DELETE costano $O(\lg n)$ con le riparazioni a rotazioni e ricolorazioni.

6. Esempio da citare: l'albero a 12 chiavi della figura 7.3 ha $n = 12$, $bh(26) = 3$ e altezza 4 (il teorema promette al più 7.40).

Errori tipici

- **Contare i nodi rossi nell'altezza nera:** $bh(x)$ conta solo i nodi *neri* da x a una foglia, e x stesso non si conta ($bh(\text{foglia NIL}) = 0$, un nodo nero con figli foglia ha $bh = 1$). Un nodo rosso ha la stessa altezza nera del padre.
- **Dimenticare che le foglie NIL contano come nere:** i cammini finiscono lì, quindi nell'altezza nera la NIL è un nero in più — e sulla proprietà 5 si confrontano i cammini fino alle foglie NIL, non fino all'ultimo nodo interno.
- **Ribaltare il Lemma 1:** dice $int(x) \geq 2^{bh(x)} - 1$ (tanti neri $\Rightarrow$ tanti nodi). Serve così per il Teorema 2; nel Teorema 2 non si parte da $h/2 \leq \lg(n+1)$ ma dalla catena $n \geq 2^{bh(T)} - 1 \geq 2^{h/2} - 1$.

7.2 Le rotazioni

La mossa che raddrizza

Cambiare i colori non cambia la forma dell'albero, ma per rimettere a posto le proprietà 4 e 5 qualche volta serve spostare dei nodi. La **rotazione** prende un nodo e un suo figlio e li scambia di posto, facendo «perno» sul collegamento fra i due: è un'operazione di *ristrutturazione locale*, un numero costante di puntatori. L'attraversamento simmetrico non se ne accorge — l'ordine delle chiavi resta lo stesso — e i colori non si toccano: è la mossa con cui si rimette in forma l'albero senza perdere il lavoro fatto. [slide RB p. 22]

LEFT-ROTATE(T, x)

```

1   $y := x.right$  // imposta  $y$ 
2   $x.right := y.left$  //  $\beta$  passa a  $x$ 
3  if  $y.left \neq T.nil$ 
4       $y.left.p := x$ 
5   $y.p := x.p$  // il padre di  $x$  a  $y$ 
6  if  $x.p == T.nil$ 
7       $T.root := y$ 
8  elseif  $x == x.p.left$ 
9       $x.p.left := y$ 
10 else  $x.p.right := y$ 
11  $y.left := x$  //  $x$  a sinistra di  $y$ 
12  $x.p := y$ 
```

RIGHT-ROTATE(T, y)

```

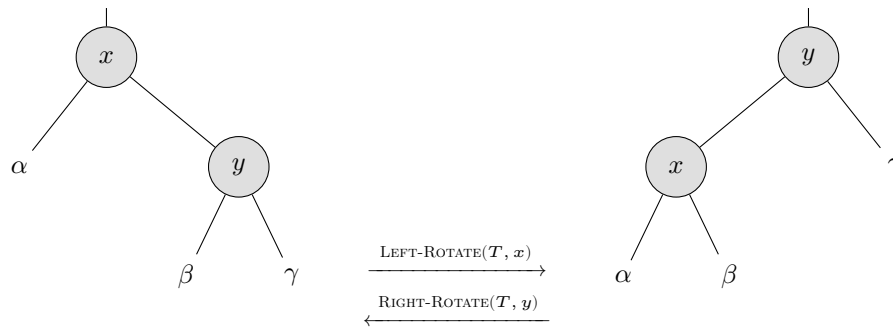
1   $x := y.left$  // imposta  $x$ 
2   $y.left := x.right$  //  $\beta$  passa a  $y$ 
3  if  $x.right \neq T.nil$ 
4       $x.right.p := y$ 
5   $x.p := y.p$  // il padre di  $y$  a  $x$ 
6  if  $y.p == T.nil$ 
7       $T.root := x$ 
8  elseif  $y == y.p.right$ 
9       $y.p.right := x$ 
10 else  $y.p.left := x$ 
11  $x.right := y$  //  $y$  a destra di  $x$ 
12  $y.p := x$ 
```

Nella rotazione sinistra il perno è il collegamento tra x e il figlio destro y : y sale al posto di x e il suo figlio sinistro (il sottoalbero β) passa a destra di x . L' α è il sottoalbero la cui radice è il figlio sinistro di x (può anche essere vuoto); β e γ sono i sottoalberi le cui radici sono il figlio sinistro e destro (risp.) di y (possono anch'essi essere vuoti). Vale l'ordine

chiavi in $\alpha < x.key < \text{chiavi in } \beta < y.key < \text{chiavi in } \gamma$,

ed è questa catena di disuguaglianze a garantire che l'attraversamento simmetrico non cambi. Entrambe le procedure costano $O(1)$ — poche righe, senza cicli — e toccano *solo* i puntatori: i colori non cambiano. [slide RB p. 22; Cormen cap. 13.2]

Le operazioni su un albero rosso-nero. Le operazioni di ricerca si implementano *esattamente* come sui BST ordinari, senza mai guardare i colori: SEARCH, MINIMUM, MAXIMUM, SUCCESSOR e



(a) le rotazioni: x e y sono grigi perché il colore è qualunque, la rotazione non lo tocca; β cambia padre

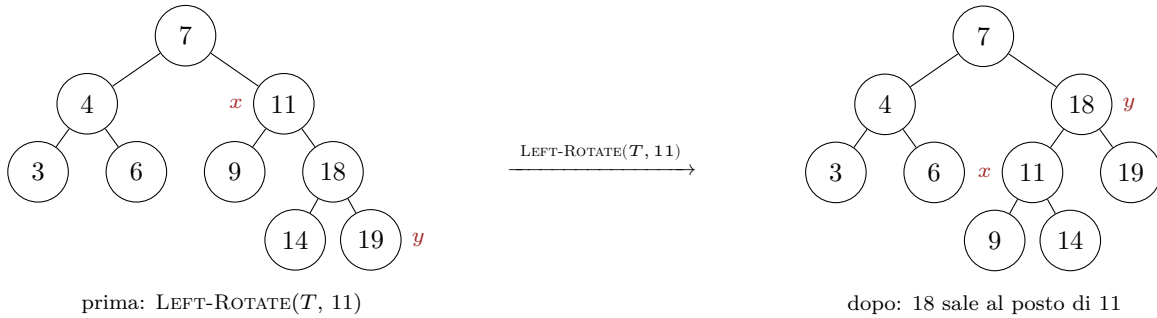


Figura 7.6: In alto le due rotazioni nella notazione delle slide (β cambia padre, i colori no). In basso un esempio su chiavi vere, prese dalla figura 13.3 del libro: $\text{LEFT-ROTATE}(T, 11)$ fa salire 18, i figli di 11 diventano 9 e 14, e l'attraversamento simmetrico 3, 4, 6, 7, 9, 11, 14, 18, 19 è identico prima e dopo. [slide RB p. 22; Cormen cap. 13.2, fig. 13.3]

PREDECESSOR sono quelli del capitolo 12. Siccome su un albero rosso-nero l'altezza è $O(\lg n)$ (il teorema dell'altezza, nella prima parte del capitolo), tutte costano $O(\lg n)$ *nel caso peggiore*. Anche inserimento e cancellazione, che invece devono riparare i colori, costano $O(\lg n)$. [slide RB p. 23]

Il costo delle operazioni: $O(\lg n)$ nel caso peggiore

Su un RB tree con n nodi interni, **SEARCH**, **MINIMUM**, **MAXIMUM**, **SUCCESSOR** e **PREDECESSOR** costano $O(\lg n)$: sono gli algoritmi del capitolo 12 eseguiti su un albero di altezza $h \leq 2 \lg(n + 1) = O(\lg n)$. Anche **RB-INSERT** e **RB-DELETE** costano $O(\lg n)$: la discesa è quella di un BST e la riparazione risale l'albero invece di scendere, quindi anche lei è $O(\lg n)$.

[slide RB p. 23; Cormen capp. 12.3 e 13.1]

7.3 L'inserimento

Inserire un nodo rosso, e poi riparare

Inserire una chiave in un albero rosso-nero è come inserirla in un BST: scendiamo dalla radice fino alla foglia $T.nil$ dove deve finire il nuovo nodo z . Di che colore coloriamo z ? Di nero no: il cammino di z avrebbe un nero in più degli altri e la proprietà 5 salterebbe in tutto l'albero. Colorandolo di rosso l'unica regola che può saltare è la 4, con il padre. Quindi z è **rosso**, e le uniche violazioni possibili sono la proprietà 2 (z radice rossa) o la 4 (z e $p[z]$ rossi). Se il padre di z è nero non c'è niente da riparare (a parte l'ultima riga del fixup, che ridipinga di nero la radice); altrimenti entriamo nel ciclo di RB-INSERT-FIXUP: tre casi più le varianti simmetriche dei casi 2 e 3, ricolorazioni e al più due rotazioni. [slide RB pp. 24–27]

Nel dettaglio, l'inserimento di z cerca un cammino dalla radice fino al nodo p che diventerà suo padre; una volta identificato p , z viene aggiunto come figlio sinistro se $z.key < p.key$, come figlio destro se $z.key \geq p.key$, e colorato di rosso, per evitare inconsistenze dell'altezza nera (la *black height*). Questo inserimento può causare una violazione della proprietà 2, se z viene inserito in un albero vuoto, o della proprietà 4, se z viene aggiunto come figlio di un nodo rosso: RB-INSERT-FIXUP(T, z) ripristina le proprietà. [slide RB pp. 24–27]

RB-INSERT(T, z)

```

1   $y := T.nil$ 
2   $x := T.root$ 
3  while  $x \neq T.nil$ 
4       $y := x$ 
5      if  $z.key < x.key$ 
6           $x := x.left$ 
7      else  $x := x.right$ 
8   $z.p := y$ 
9  if  $y == T.nil$ 
10      $T.root := z$  // l'albero era vuoto
11 elseif  $z.key < y.key$ 
12      $y.left := z$ 
13 else  $y.right := z$ 
14  $z.left := T.nil$ 
15  $z.right := T.nil$ 
16  $z.color := RED$ 
17 RB-INSERT-FIXUP( $T, z$ )
```

RB-INSERT-FIXUP(T, z)

```

1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y := z.p.p.right$  // lo zio
4          if  $y.color == RED$ 
5               $z.p.color := BLACK$  // Caso 1
6               $y.color := BLACK$  // Caso 1
7               $z.p.p.color := RED$  // Caso 1
8               $z := z.p.p$  // Caso 1
9          elseif  $z == z.p.right$ 
10              $z := z.p$  // Caso 3
11             LEFT-ROTATE( $T, z$ ) // Caso 3
12              $z.p.color := BLACK$  // Caso 2
13              $z.p.p.color := RED$  // Caso 2
14             RIGHT-ROTATE( $T, z.p.p$ ) // Caso 2
15          else // come sopra, con left/right scambiati
16
17   $T.root.color := BLACK$ 
```

Rispetto a TREE-INSERT: i NIL diventano $T.nil$, si inizializzano $z.left$ e $z.right$ alla sentinella, si colora z di rosso e si chiama RB-INSERT-FIXUP. Quest'ultima ripristina la proprietà 2 colorando di nero la radice (la riga finale, sempre eseguita) e la proprietà 4 con ricolorazioni e rotazioni; decide cosa fare confrontando il colore di z con quello di suo **zio** y , l'altro figlio di $p[p[z]]$. Nel ciclo while ci sono tre casi (più le varianti simmetriche della riga **else**); in tutti e tre il nonno $p[p[z]]$ è nero, perché il padre è rosso e la proprietà 4 può essere violata soltanto fra z e $p[z]$. [slide RB pp. 28–30; Cormen cap. 13.3]

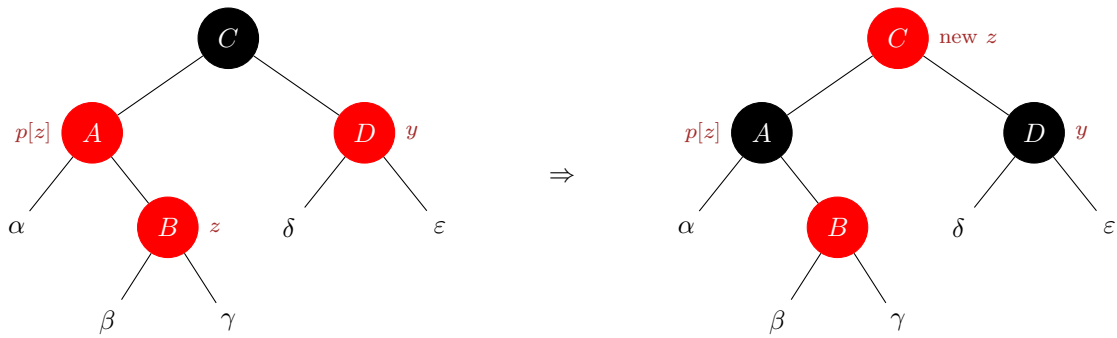


Figura 7.7: **Caso 1:** lo zio y di z è rosso. $p[z]$ e y diventano neri e il nonno $p[p[z]]$ rosso: la violazione è tolta, ma il nonno è il nuovo z (le slide lo chiamano *new z*) e il ciclo riparte da lì. Nessuna rotazione. [slide RB p. 31]

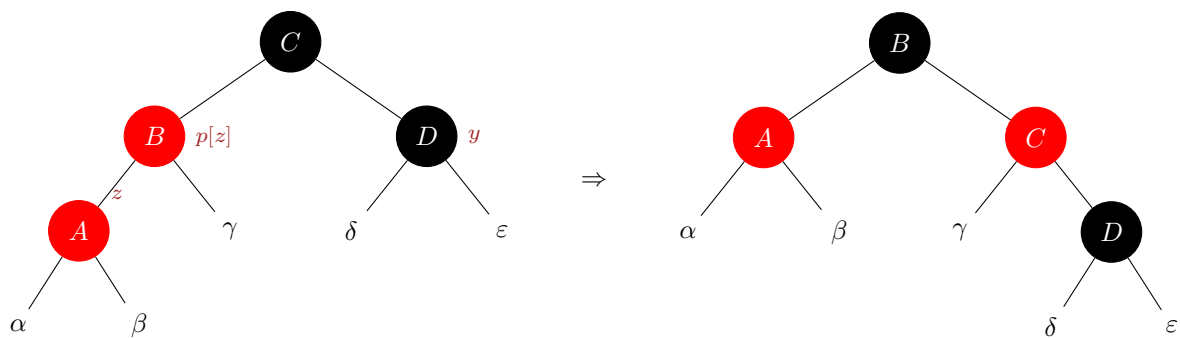


Figura 7.8: **Caso 2:** zio nero e z figlio *sinistro*. $p[z]$ diventa nero e $p[p[z]]$ rosso; per ripristinare le altezze nere si ruota $p[p[z]]$ a *destra*: il padre di z prende il posto del nonno, sopra z non ci sono più due rossi e il ciclo termina. [slide RB p. 32]

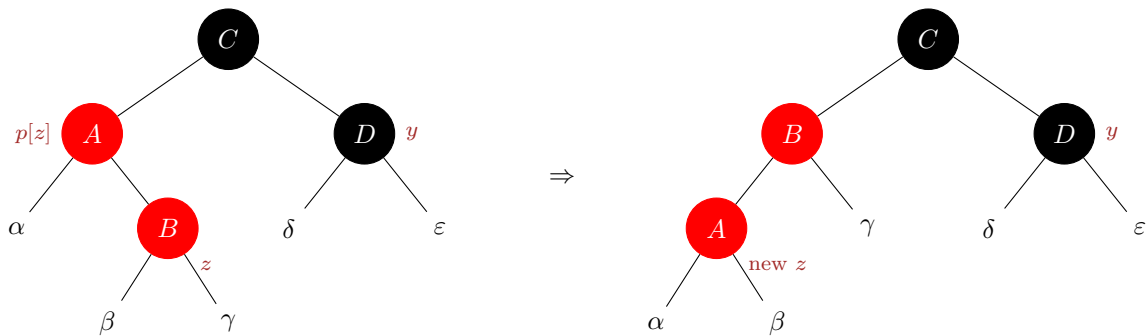


Figura 7.9: **Caso 3:** zio nero e z figlio *destro*. La rotazione sinistra su $p[z]$ riconduce al caso 2: z sale, l'ex $p[z]$ scende come suo figlio sinistro ed è il nuovo z (*new z*); poi si chiude come il caso 2 — nell'ultimo disegno, con B al posto di $p[z]$ e C al posto del nonno. [slide RB p. 33]

Se $p[z]$ è il figlio *destro* di $p[p[z]]$ — cioè lo zio è a sinistra — tutto si specchia, e i ruoli dei casi 2 e 3 si scambiano: chiude il *caso 3* (zio nero e z figlio destro: $p[z]$ nero, $p[p[z]]$ rosso, rotazione *sinistra* del nonno), mentre il *caso 2* (zio nero e z figlio sinistro) riduce al caso 3 con una rotazione *destra* di $p[z]$, che diventa il nuovo z . Attenzione alle simmetrie: è l'unico punto del capitolo in cui il riflesso cambia le mosse. [slide RB pp. 34–35]

Ogni iterazione del ciclo risale l'albero di due livelli (caso 1) oppure termina (casi 2 e 3), quindi le iterazioni sono al più $O(\lg n)$; dentro un'iterazione si fanno $O(1)$ ricolorazioni e al più una rotazione, che costa $O(1)$. Quindi RB-INSERT-FIXUP(T, z) richiede un tempo $O(\lg n)$ e di conseguenza anche

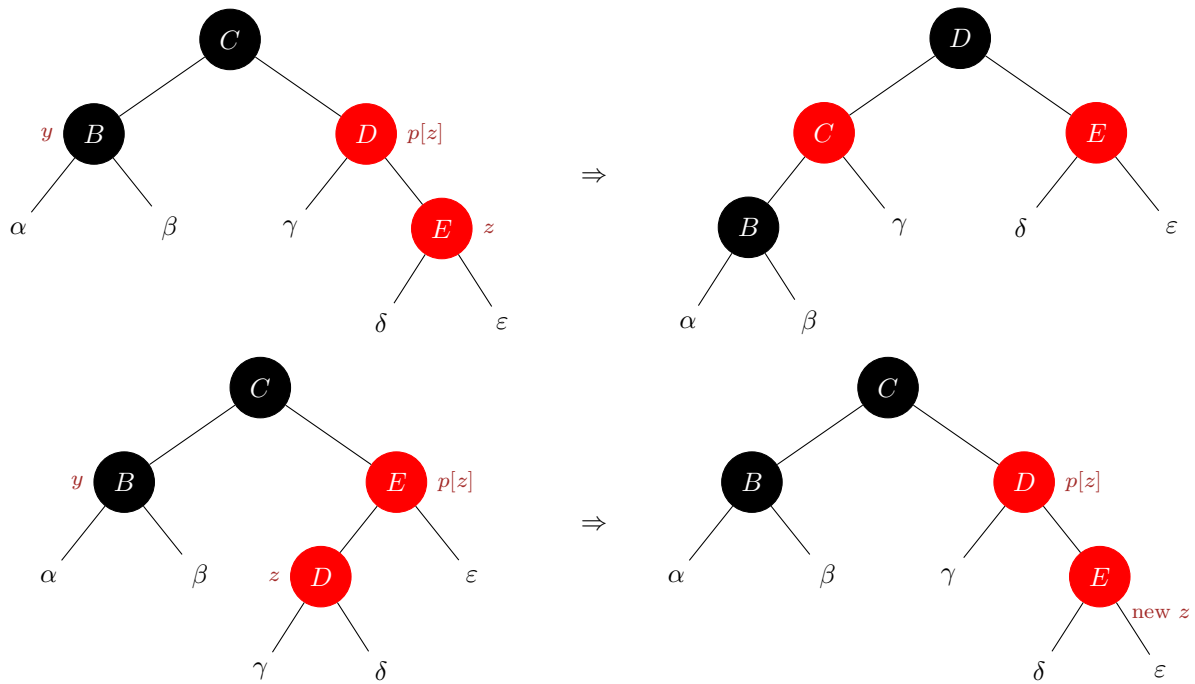
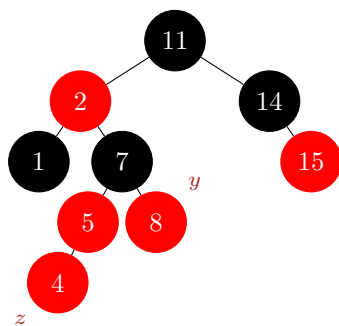
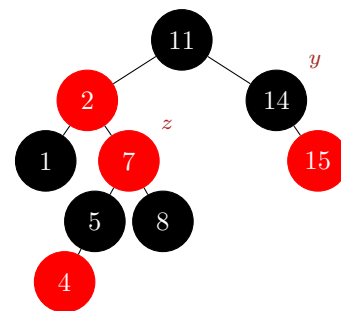
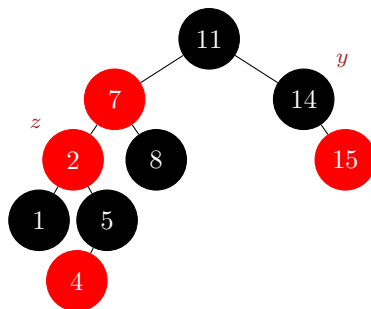
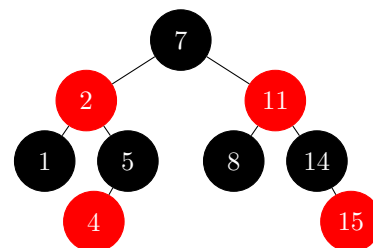


Figura 7.10: Le varianti simmetriche, con lo zio y figlio *sinistro*. **In alto**: zio nero e z figlio destro — caso 3: $p[z]$ nero, $p[p[z]]$ rosso e rotazione *sinistra* del nonno (specchio del caso 2). **In basso**: zio nero e z figlio sinistro — caso 2: la rotazione *destra* di $p[z]$ riconduce al caso qui sopra, e l'ex $p[z]$ (D) è il nuovo z . [slide RB pp. 34–35]

$\text{RB-INSERT}(T, z)$ è $O(\lg n)$: l'inserimento in un albero rosso-nero costa $O(\lg n)$ nel caso peggiore. [slide RB p. 36; Cormen cap. 13.3]

Il libro illustra tutto questo su un esempio che vale la pena rifare a mano: si parte dall'albero 11, 2, 1, 7, 5, 8, 14, 15 e si inserisce 4: il nuovo nodo finisce sotto il 5, rosso su rosso, e la riparazione mette in fila *tutti e tre* i casi.

(a) $z = 4$: il 5 (padre) è rosso(b) caso 1: 5 e 8 neri, 7 rosso, $z = 7$ (c) caso 3: rotazione sinistra su 2, $z = 2$ 

(d) caso 2: rotazione destra su 11, fine

Figura 7.11: L'esempio della figura 13.4 del libro: inseriamo 4 nell'albero 11, 2, 1, 7, 5, 8, 14, 15. (a) il 4 finisce sotto il 5, rosso su rosso, e lo zio 8 è rosso $\Rightarrow$ caso 1, che ricolore e risale: $z = 7$. (b) $z = 7$ è un figlio destro e lo zio 14 è nero $\Rightarrow$ caso 3: rotazione sinistra su 2, e il 2 è il nuovo z . (c) ora il 2 è figlio sinistro con lo zio nero $\Rightarrow$ caso 2: 7 nero, 11 rosso, rotazione destra su 11, e l'albero (d) è valido. [Cormen cap. 13.3, fig. 13.4; casi numerati come nelle slide RB pp. 30–35]

7.4 Un esercizio svolto: inserire 41, 38, 31, 12, 19, 8

È l'esercizio classico (Esercizio 13.3-2 del libro, e le slide pp. 37–40): da un albero vuoto, inserire in quest'ordine 41, 38, 31, 12, 19, 8, disegnando l'albero *dopo ogni inserimento*. La figura mostra la sequenza per intero, con lo stato appena inserito (il nodo nuovo, rosso), l'eventuale passaggio intermedio della riparazione e l'albero finale. Dopo 31 scatta il caso 2 (una rotazione), dopo 12 il caso 1 (il rosso risale fino alla radice, senza rotazioni), dopo 19 il caso 3 e poi il caso 2 (due rotazioni), dopo 8 il caso 1 (che si ferma subito, perché il padre del 19 è nero): in tutta la sequenza si fanno **tre** rotazioni. [slide RB pp. 37–40]

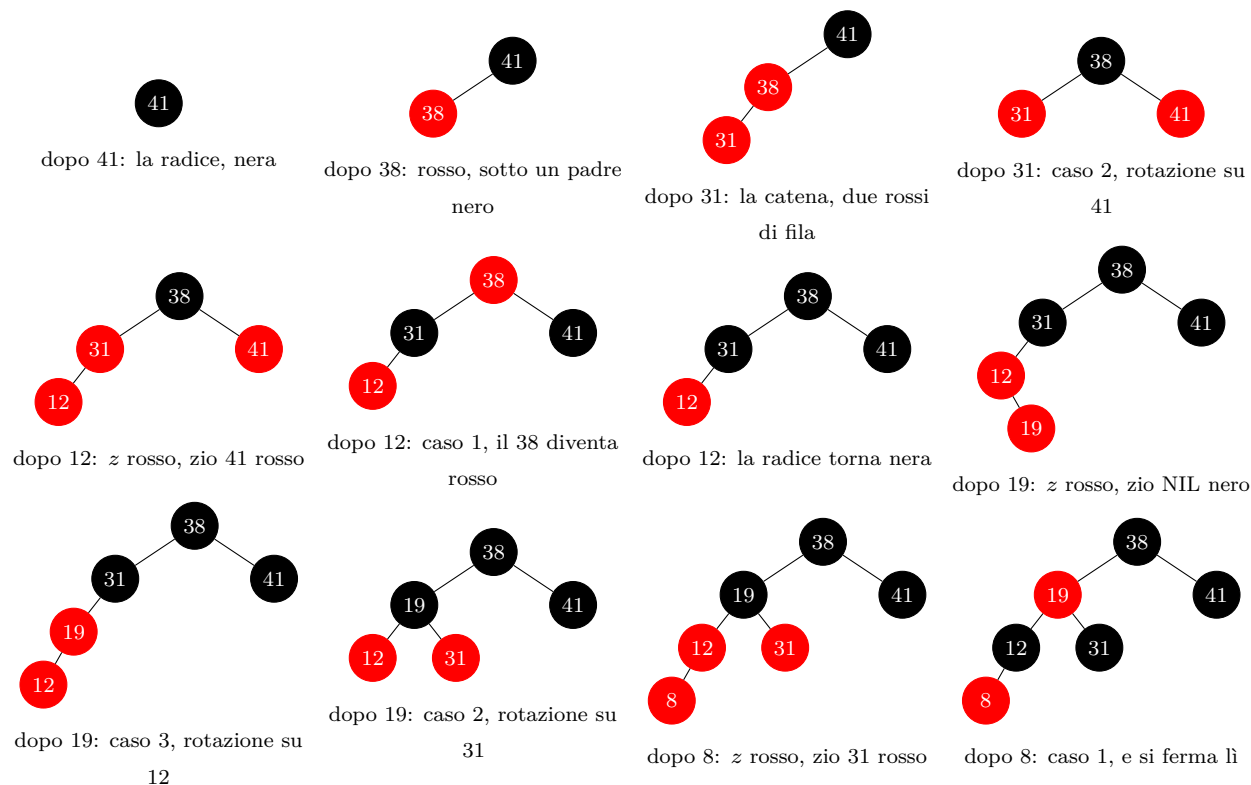


Figura 7.12: L'Esercizio 13.3-2 passo per passo (è anche l'esercizio delle slide): da albero vuoto, 41, 38, 31, 12, 19, 8. Dopo 31 il caso 2 ruota il 41; dopo 12 il caso 1 fa risalire il rosso fino al 38, che era la radice (torna nera); dopo 19 il caso 3 ruota il 12 e il caso 2 ruota il 31; dopo 8 il caso 1 colora 12 e 31 e si ferma subito. In tutto, tre rotazioni. [slide RB pp. 37–40; Cormen es. 13.3-2]

Eseguire una sequenza di inserimenti su un albero rosso-nero

È l'esercizio in cui il disegno è tutto; il metodo è meccanico.

1. **Inserisci z come in un BST** (scendi fino alla foglia $T.nil$) e coloralo di *rosso*. Se il padre è nero hai finito: nessuna violazione. Se il padre è rosso, la proprietà 4 è violata e si ripara con il fixup.
2. **Guarda lo zio y di z** (l'altro figlio del nonno), e il lato di z . Con lo zio a *destra*:
 - zio *rosso* $\Rightarrow$ **caso 1**: padre e zio neri, nonno rosso; il nonno è il nuovo z , torna al punto precedente (nessuna rotazione);
 - zio nero e z figlio *sinistro* $\Rightarrow$ **caso 2**: padre nero, nonno rosso, rotazione *destra* del nonno. Fine;
 - zio nero e z figlio *destro* $\Rightarrow$ **caso 3**: rotazione *sinistra* di $p[z]$; l'ex padre è il nuovo z e ora si ricade nel caso 2.

3. **Zio a sinistra? Tutto specchiato** (*attenzione alle simmetrie*): è il caso 2 (zio nero, z figlio sinistro) a ridurre con la rotazione *destra* di $p[z]$, ed è il caso 3 a chiudere, con $p[z]$ nero, il nonno rosso e la rotazione *sinistra* del nonno.
4. **Ridipingi nera la radice**: è l'ultima riga di RB-INSERT-FIXUP, sempre eseguita.
5. **Disegna ogni stato**, non solo l'ultimo: è anche il modo di controllare i conti. In un inserimento si fanno al più **due** rotazioni e il caso 1 non ne fa nessuna: se stai ruotando di più, c'è un errore. Il caso 1 può invece ripetersi più volte, salendo di due livelli per volta.

7.5 La cancellazione

Il nero che avanza

Cancellare un nodo è più delicato dell'inserire: togliere una chiave può cambiare il *numero* di nodi neri lungo un cammino e far saltare la proprietà 5. L'idea della riparazione è contabile. Se il nodo rimosso era *rosso* non è successo niente: i conti dei neri non cambiano. Se era *nero*, il nodo x che ne prende il posto si porta dietro un «**extra credito**»: un nero in più, come se il colore del nodo tolto gli fosse stato aggiunto. Così i conti tornano, ma x ora è «**doppiamente nero**» (o «rosso e nero») e la proprietà 1 non è più soddisfatta. Il ciclo di RB-DELETE-FIXUP spinge allora l'extra credito verso l'alto con ricolorazioni e rotazioni, finché non finisce su un nodo rosso (che diventa nero: credito assorbito) o sulla radice (dove viene ignorato). [slide RB pp. 44–46]

7.5.1 Prima il successore: nodi con due figli e con al più un figlio

Possiamo supporre di dover eliminare sempre un nodo z che ha *al più un figlio*: non è una restrizione. Se z ha due figli, come nella cancellazione di un BST si sostituisce la chiave di z con quella del suo successore $y = \text{TREE-SUCCESSOR}(T, z)$ e poi si rimuove y . Poiché z ha due figli, y è il nodo più a sinistra del sottoalbero destro di z : non ha figlio sinistro, quindi y ha al più il figlio destro, e la rimozione ricade nel caso facile. [slide RB p. 41]

Sia allora z il nodo da cancellare, x il suo unico figlio e p il padre (se z è una foglia, x è $T.nil$). Per eliminare z :

1. si rimuove z **collegando** p con x : p diventa il padre di x e x il figlio di p (al posto di z);
2. se z era **rosso** abbiamo finito: l'eliminazione di z non causa violazioni delle RB-properties;
3. se z era **nero** potremmo aver causato una violazione della proprietà 5.

[slide RB p. 42]

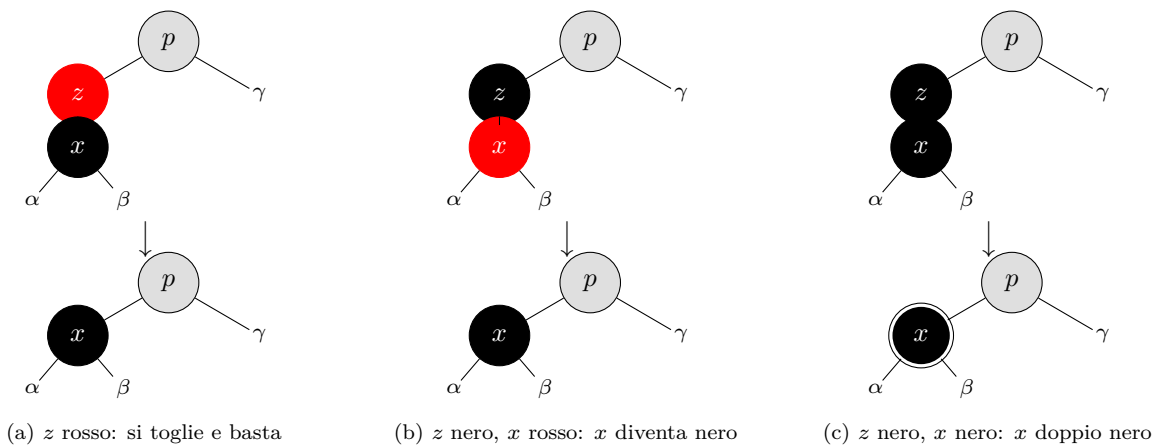


Figura 7.13: L'eliminazione di z nei tre casi delle slide. Il padre p è grigio perché il suo colore è qualunque (z nero può avere il padre rosso o nero); α, β, γ sono sottoalberi arbitrari. In (a) e (b) le RB-properties restano valide; in (c) x eredita il nero in più — il doppio anello — e serve RB-DELETE-FIXUP. [slide RB pp. 43–45]

7.6 La riparazione: RB-DELETE-FIXUP

RB-DELETE ha la struttura di TREE-DELETE: se z ha meno di due figli lo si sostituisce con l'unico figlio (o con la sentinella); se ne ha due, y è il successore di z (il minimo del sottoalbero destro) e viene spostato al posto di z , prendendone il colore. Le variabili in più ricordano cosa potrebbe rompersi: y è il nodo rimosso o spostato, y -original-color è il suo colore *prima* della modifica, e x è il nodo che si sposta nella posizione originale di y (l'unico figlio di y , oppure $T.nil$). Se il colore originale di y era rosso le proprietà restano valide; se era nero, RB-DELETE-FIXUP lavora su x , il nodo che ha ereditato il nero in più. [Cormen cap. 13.4]

RB-TRANSPLANT(T, u, v)

```

1  if  $u.p == T.nil$ 
2       $T.root := v$ 
3  elseif  $u == u.p.left$ 
4       $u.p.left := v$ 
5  else  $u.p.right := v$ 
6   $v.p := u.p$ 
```

Rispetto a TRANSPLANT (capitolo 12) cambiano i NIL e l'ultima riga, che assegna $v.p$ *sempre*, anche quando v è la sentinella: serve al fixup.

RB-DELETE(T, z)

```

1   $y := z$ 
2   $y$ -original-color :=  $y.color$ 
3  if  $z.left == T.nil$ 
4       $x := z.right$ 
5      RB-TRANSPLANT( $T, z, z.right$ )
6  elseif  $z.right == T.nil$ 
7       $x := z.left$ 
8      RB-TRANSPLANT( $T, z, z.left$ )
9  else  $y := TREE-MINIMUM(z.right)$ 
10      $y$ -original-color :=  $y.color$ 
11      $x := y.right$ 
12     if  $y.p == z$ 
13          $x.p := y$ 
14     else RB-TRANSPLANT( $T, y, y.right$ )
15          $y.right := z.right$ 
16          $y.right.p := y$ 
17     RB-TRANSPLANT( $T, z, y$ )
18      $y.left := z.left$ 
19      $y.left.p := y$ 
20      $y.color := z.color$ 
21  if  $y$ -original-color == BLACK
22      RB-DELETE-FIXUP( $T, x$ )
```

Il nodo «doppio nero» [slide RB p. 46; Cormen p. 289]

Per ristabilire la proprietà 5 quando il nodo eliminato z era nero, al nodo x che ne prende il posto si attribuisce un **extra credito**:

- se x è *rosso*, lo si colora di nero: il credito è assorbito e non serve nient'altro;
- se x è già *nero* assume un colore fittizio detto **doppio nero**, che ricorda che due neri sono stati collassati in uno: nel calcolo dell'altezza nera un nodo doppio nero conta *due* volte. Il campo $x.color$ resta BLACK: il nero in più non è un colore, è il fatto che x sia puntato dal fixup.

RB-DELETE-FIXUP(T, x) spinge poi l'extra credito verso la radice con rotazioni e ricolorazioni; se lungo il cammino incontra un nodo rosso, questo viene semplicemente colorato di nero, e alla radice il credito viene ignorato.

```

RB-DELETE-FIXUP( $T, x$ )
1  while  $x \neq T.root$  &  $x.color == BLACK$ 
2      if  $x == x.p.left$ 
3           $w := x.p.right$  // il fratello di  $x$ 
4          if  $w.color == RED$ 
5               $w.color := BLACK$  // Caso 3
6               $x.p.color := RED$  // Caso 3
7              LEFT-ROTATE( $T, x.p$ ) // Caso 3
8               $w := x.p.right$  // Caso 3
9          if  $w.left.color == BLACK$  &  $w.right.color == BLACK$ 
10              $w.color := RED$  // Caso 2
11              $x := x.p$  // Caso 2
12          elseif  $w.right.color == BLACK$ 
13              $w.left.color := BLACK$  // Caso 1.2
14              $w.color := RED$  // Caso 1.2
15             RIGHT-ROTATE( $T, w$ ) // Caso 1.2
16              $w := x.p.right$  // Caso 1.2
17              $w.color := x.p.color$  // Caso 1.1
18              $x.p.color := BLACK$  // Caso 1.1
19              $w.right.color := BLACK$  // Caso 1.1
20             LEFT-ROTATE( $T, x.p$ ) // Caso 1.1
21              $x := T.root$  // Caso 1.1
22          else // come sopra, con left/right scambiati
23       $x.color := BLACK$ 

```

All'inizio di ogni iterazione x punta a un nodo doppiamente nero che non è la radice, e w è suo fratello; w non può essere $T.nil$, altrimenti i neri verso w sarebbero meno di quelli verso x . Il conto che rende lecite le trasformazioni: in ogni caso il numero di neri dalla radice del sottoalbero illustrato fino a ciascun sottoalbero $\alpha, \beta, \dots, \zeta$ non cambia (contando il nero in più di x). Se la proprietà 5 valeva prima, continua a valere. [Cormen cap. 13.4]

I casi si scelgono guardando il colore di w e quello dei suoi figli, come nelle slide (che ne fanno cinque più i cinque simmetrici):

- **Caso 3 (w rosso).** I figli di w sono neri (proprietà 4). Si scambiano i colori di w e di $x.p$ e si ruota a sinistra su $x.p$: il nuovo fratello di x è nero, quindi siamo ricaduti nei casi 2.x o 1.x. È l'unico caso che non risolve da solo: prepara gli altri.
- **Caso 2.1 e 2.2 (w nero, entrambi i figli di w neri).** Si toglie un credito nero sia a x sia a w (w diventa rosso) e un nero in più passa a $x.p$. Se $x.p$ era *rosso* (2.1) è finita: $x.p$ diventa nero, il credito è assorbito e il ciclo si ferma. Se $x.p$ era *nero* (2.2), $x.p$ diventa doppio nero: è lui il nuovo x e il ciclo ripete, un livello più su. È l'unico caso che itera, e l'unico che non fa rotazioni; alla radice il nero in più viene buttato via.
- **Caso 1.2 (w nero, figlio destro nero e figlio sinistro rosso).** Si colora di nero il figlio sinistro di w e di rosso w , e si ruota a destra su w : il nuovo w è nero con il figlio destro rosso, cioè il caso 1.1.
- **Caso 1.1 (w nero, figlio destro rosso).** w prende il colore di $x.p$, $x.p$ diventa nero, il figlio destro di w diventa nero, e si ruota a sinistra su $x.p$: x perde il nero in più (torna un nero solo) e il ciclo termina.

I casi 3, 1.2 e 1.1 fanno una rotazione ciascuno, il caso 2 nessuna: in tutto il fixup si fanno al più **tre rotazioni**. Il ciclo ripete solo per il caso 2, che sale di un livello per volta, quindi RB-DELETE-FIXUP richiede un tempo $O(\lg n)$; di conseguenza anche RB-DELETE costa $O(\lg n)$. [slide RB p. 54; Cormen cap. 13.4]

I commenti del pseudocodice seguono la numerazione delle slide; il libro (fig. 13.7) elenca gli stessi casi con un'altra numerazione: il caso 3 delle slide è il caso 1 del libro, il caso 2 è il caso 2, il caso 1.2 è il caso 3 e il caso 1.1 è il caso 4. [Cormen cap. 13.4, fig. 13.7]

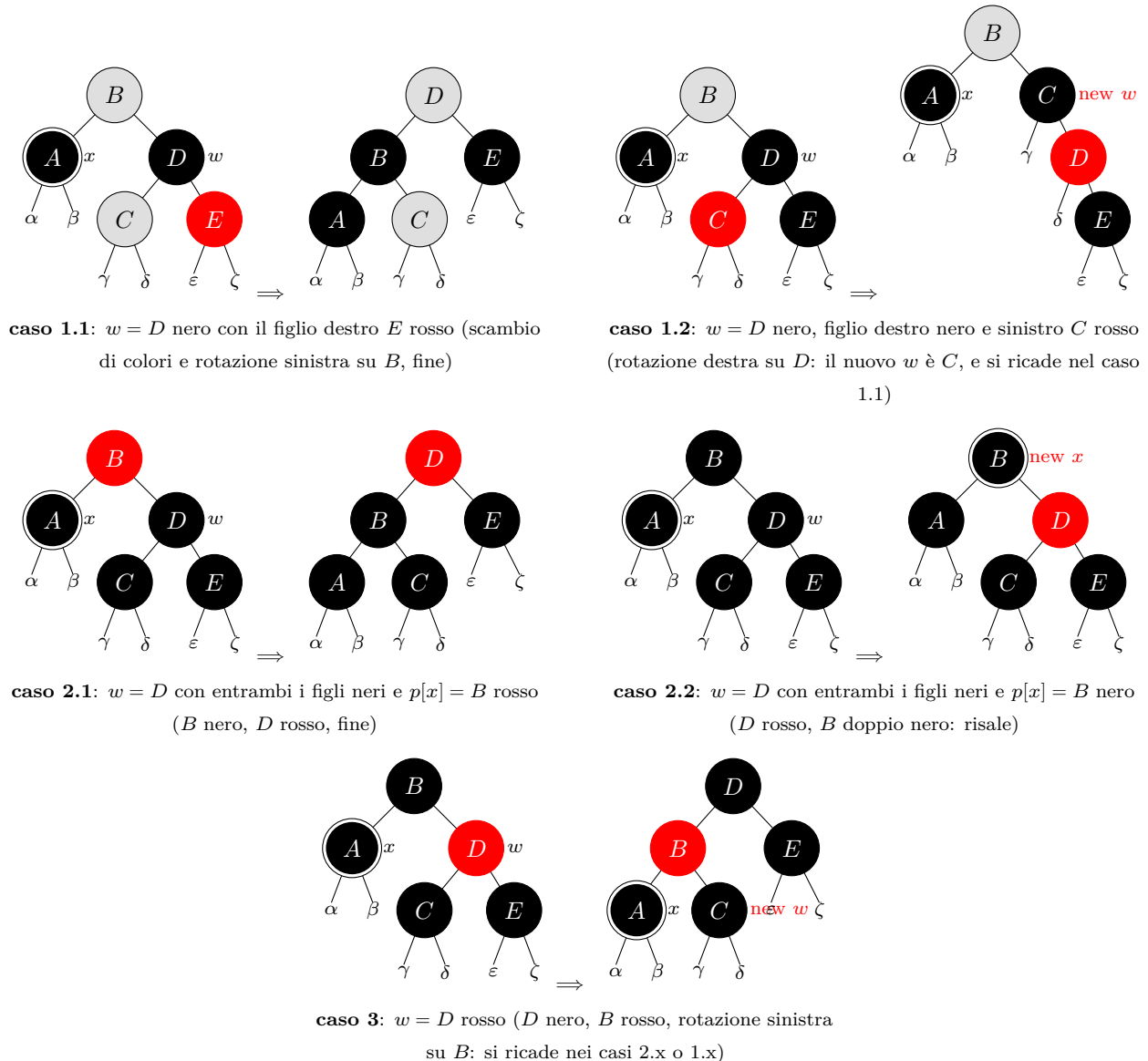


Figura 7.14: I cinque casi di RB-DELETE-FIXUP (più i cinque simmetrici), nella numerazione delle slide: il nodo grigio B è $p[x]$, chi porta il nero in più è puntato da x (ha il doppio anello), w è il fratello, i nodi grigi hanno colore qualunque. In ogni caso la configurazione a sinistra si trasforma in quella a destra; solo il caso 2.2 fa ripetere il ciclo (il doppio nero sale). In fondo al capitolo la figura 7.15 mostra i casi 2.1 e 2.2 su chiavi vere, e la figura 7.16 i casi 3, 1.2 e 1.1. [slide RB pp. 47–53]

7.7 Un esempio svolto: cancellare 8, 12, 19, 31, 38

Le slide chiudono con un esempio da rifare a mano. L'albero di partenza è quello dell'Esercizio 13.3-2 (figura 7.12), costruito inserendo 41, 38, 31, 12, 19, 8. Scritto in forma compatta è $38B(19R(12B(8R, -), 31B), 41B)$: radice 38 nera con figlio sinistro $19R(12B(8R, -), 31B)$ e destro 41B; R è rosso, B nero e «-» un figlio assente. La figura 7.15 ridisegna l'albero dopo ogni cancellazione: è quello che si deve saper fare all'esame. [slide RB pp. 55–58]

- **Si cancella 8**, una foglia *rossa*: basta toglierla, l'albero diventa $38B(19R(12B, 31B), 41B)$ e non c'è niente da riparare.
- **Si cancella 12**, una foglia *nera*: al suo posto va la sentinella $x = T.nil$, che eredita il nero in più (il doppio anello in figura). Il fratello è $w = 31$, nero con entrambi i figli neri (le due sentinelle), quindi caso 2; il padre 19 è rosso, quindi caso **2.1**: 31 diventa rosso, 19 da rosso diventa nero, e il nero in più è finito su un nodo rosso: il ciclo si ferma. L'albero è $38B(19B(-, 31R), 41B)$.
- **Si cancella 19**: ha un solo figlio, 31, rosso. $x = 31$ prende il posto di 19 e, poiché 19 era nero, il figlio rosso incassa l'extra credito diventando nero: $38B(31B, 41B)$.
- **Si cancella 31**, una foglia *nera*: di nuovo $x = T.nil$ doppiamente nero; il fratello $w = 41$ è nero con entrambi i figli neri: caso 2, e stavolta il padre 38 è nero: caso **2.2**. 41 diventa rosso e 38 eredita il nero in più; ma il nuovo $x = 38$ è la radice, quindi il ciclo si ferma e il nero in più viene ignorato (l'ultima riga del fixup ridipinga x di nero). L'altezza nera dell'albero scende da 2 a 1: $38B(-, 41R)$.
- **Si cancella 38**: ha il solo figlio 41, che ne prende il posto; $x = 41$ è ormai la radice e il fixup non ha niente da fare: resta 41B.

La sequenza è quella dell'Esercizio 13.4-3 del libro, che chiede anche la cancellazione successiva: togliere 41 svuota l'albero. A ogni passo le cinque proprietà restano valide. [Cormen cap. 13.4, es. 13.4-3]

Come complemento, un esempio che fa scattare proprio i casi con le rotazioni. L'albero (a) della figura 7.16 si costruisce inserendo 4, 1, 2, 6, 5, 3: è $2B(1B, 5R(4B(3R, -), 6B))$. Cancelliamo 1, una foglia nera: x è la sentinella doppiamente nera. Il fratello $w = 5$ è rosso (caso 3): 5 diventa nero, 2 rosso, rotazione sinistra su 2. Il nuovo $w = 4$ è nero con il figlio sinistro 3 rosso (caso 1.2): 3 nero, 4 rosso, rotazione destra su 4. Il nuovo $w = 3$ ha il figlio destro 4 rosso (caso 1.1): ricolorazione e rotazione sinistra su 2, e il nero in più è assorbito. Tre rotazioni, quante ne può fare RB-DELETE-FIXUP al massimo. [non nelle fonti del corso]

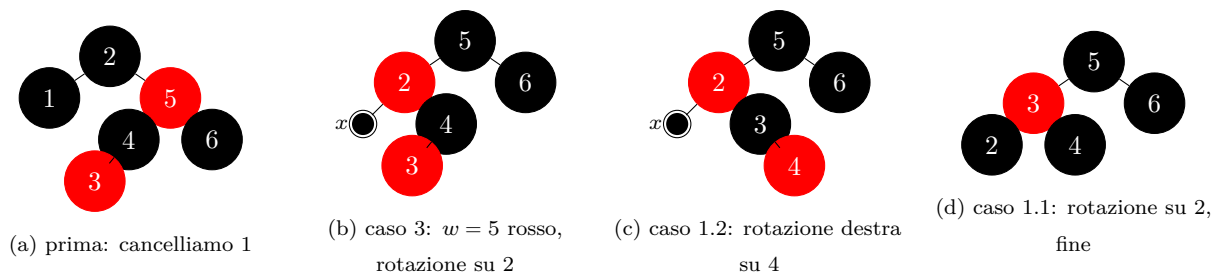


Figura 7.16: Un esempio di cancellazione svolto [non nelle fonti del corso]: da (a) togliamo la chiave 1, una foglia nera; il fixup fa scattare il caso 3 (b), il caso 1.2 (c) e il caso 1.1 (d). In (b) e (c) la sentinella al posto di 1 è il nodo x , doppiamente nero. In tutto tre rotazioni, il massimo per RB-DELETE-FIXUP.

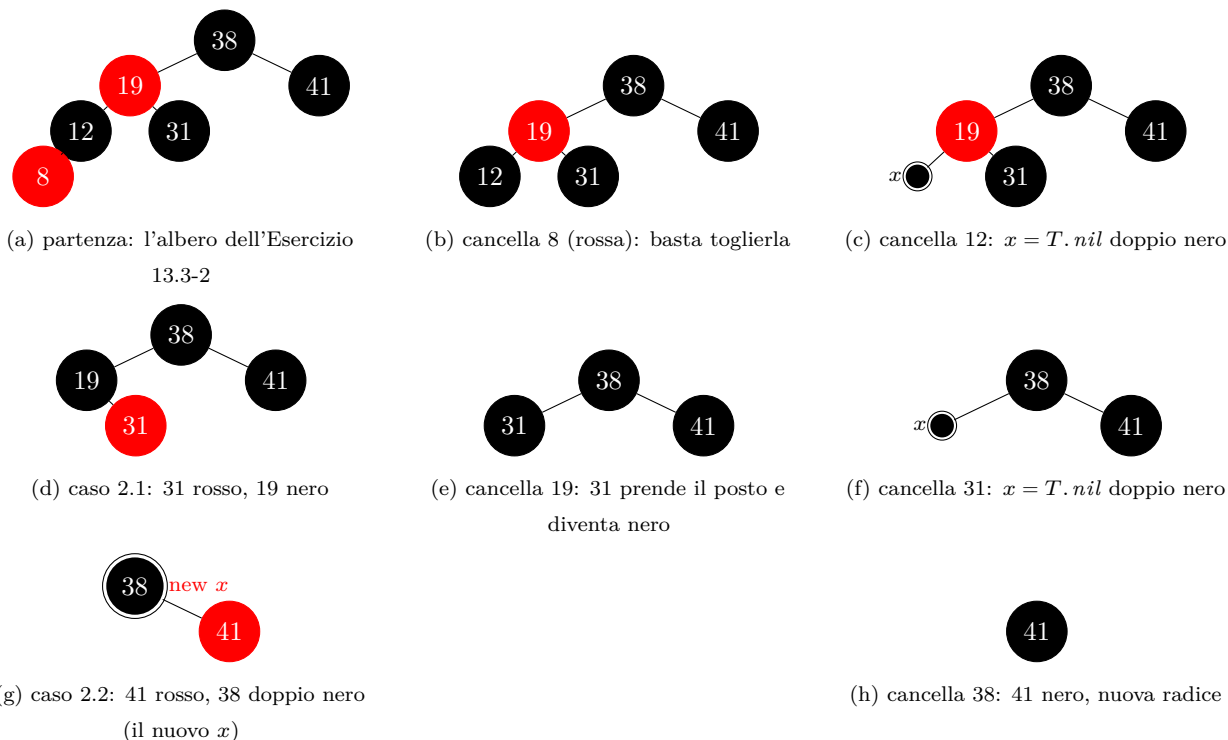


Figura 7.15: Cancellare 8, 12, 19, 31, 38 dall'albero dell'Esercizio 13.3-2, con lo stato ridisegnato dopo ogni passo. x è il nodo che porta il nero in più: in (c) e (f) è la sentinella con il doppio anello; in (d) il caso 2.1 chiude subito (31 rosso, 19 nero), in (g) il caso 2.2 porta il doppio nero fino alla radice, dove viene ignorato. [slide RB pp. 55–58]

7.8 Come si risolve all'esame

Disegnare l'esito di una sequenza di cancellazioni

È l'esercizio in cui il disegno è tutto; il metodo è meccanico. (Per gli inserimenti la disciplina è la stessa: vedi il metodobox del paragrafo sull'inserimento.)

1. **Togli il nodo:** meno di due figli $\Rightarrow$ lo togli e colleghi il padre direttamente al figlio (o alla sentinella); due figli $\Rightarrow$ metti al suo posto il successore (il minimo del sottoalbero destro), che ne prende il colore. Il nodo che sparisce davvero è sempre uno con al più un figlio.
2. **Il colore del nodo tolto decide:** era *rosso* $\Rightarrow$ hai finito. Era *nero* $\Rightarrow$ guarda il fratello w del nodo x che ha preso il suo posto (la sentinella, se il nodo era una foglia): w *rosso* $\Rightarrow$ caso 3 (w nero, $p[x]$ rosso, rotazione sinistra su $p[x]$, poi uno dei casi sotto); w nero con *entrambi i figli neri* $\Rightarrow$ caso 2 (w rosso e l'extra credito sale al padre: **2.1** se il padre era rosso — fine; **2.2** se era nero — il nuovo x è il padre, ripeti); w nero con figlio destro nero e sinistro rosso $\Rightarrow$ caso 1.2 e poi 1.1; w nero con figlio destro rosso $\Rightarrow$ caso 1.1 e fine. Radice nera.
3. **Disegna ogni stato intermedio**, non solo quello finale: lo stato subito dopo la rimozione (con x doppiamente nero, il cerchietto col doppio anello), poi ogni trasformazione del fixup. Rifare il disegno è anche il modo di controllare i conti: l'altezza nera cambia solo quando l'extra credito arriva alla radice.
4. **Controlla alla fine:** radice nera, nessun rosso con figli rossi, e lo stesso numero di neri su tutti i cammini.

Allenamento: gli esempi delle figure 7.15 e 7.16 e l'Esercizio 13.4-3; l'esercizio 6 della prova del 27 ottobre 2025 chiede un albero di 12 chiavi la cui altezza nera *scende* con una cancellazione e uno in cui *sale* con un inserimento. Gli svolgimenti sono nei capitoli 14 e 13.

«Come si cancella in un albero rosso-nero?» — la risposta in sei righe

1. Cancellare è TREE-DELETE più una riparazione: si toglie un nodo con al più un figlio (se ne ha due si sposta il successore, che ne prende il colore) e si chiama RB-DELETE-FIXUP solo se il nodo tolto era nero; entrambe costano $O(\lg n)$.
2. Se il nodo tolto era rosso non cambia niente; se era nero, il figlio x che ne prende il posto eredita un nero in più: è «rosso e nero» se x è rosso, «doppiamente nero» se x è già nero.
3. x rosso $\Rightarrow$ lo si colora di nero e basta. x doppio nero $\Rightarrow$ il fixup spinge l'extra credito verso l'alto con ricolorazioni e al più tre rotazioni.
4. I casi si scelgono guardando il fratello w di x : w rosso $\Rightarrow$ caso 3; w nero con due figli neri $\Rightarrow$ caso 2 (2.1 col padre rosso — fine; 2.2 col padre nero — il doppio nero sale); w nero con figlio destro rosso $\Rightarrow$ caso 1.1 (fine); w nero con figlio sinistro rosso $\Rightarrow$ caso 1.2 e poi 1.1.
5. Il caso 2 è l'unico che ripete il ciclo e non fa rotazioni: x sale di un livello per volta; se arriva alla radice il nero in più viene ignorato (l'ultima riga ridipinga x di nero).
6. Esempio da citare: cancellare 8, 12, 19, 31, 38 (figura 7.15) fa scattare i casi 2.1 e 2.2 e l'altezza nera scende da 2 a 1; l'esempio della figura 7.16 fa scattare i casi 3, 1.2 e 1.1 con tre rotazioni.

Errori tipici nella cancellazione

- **Dimenticare che NIL è nero** (con altezza nera 0): quando x è la sentinella, il doppio nero sta lì e i conti del fixup partono dalla foglia, non dall'ultimo nodo interno.
- **Confondere $y.color$ con $y-original-color$** : la riga 20 di RB-DELETE copia su y il colore di z ; per sapere se riparare serve il colore *originale* di y .
- **Ricolorare x di nero subito**: il nero in più non è un colore, è una contabilità, e il caso si sceglie guardando il fratello w e i *suoi* figli, non i figli di x (il nodo doppiamente nero ha ancora $x.color = \text{BLACK}$).
- **Fermarsi quando w è rosso**: il caso 3 non risolve, prepara — dopo la rotazione su $p[x]$ si ricade in uno dei casi 2.x o 1.x.
- **Ruotare nel caso 2**: il caso 2.2 ripete il ciclo e sale di un livello senza rotazioni; le rotazioni stanno nei casi 3, 1.2 e 1.1, al più tre in tutto il fixup.

Capitolo 8

Programmazione dinamica

La programmazione dinamica è la tecnica per i problemi di ottimizzazione in cui la soluzione si costruisce *combinando soluzioni di sottoproblemi* (come nel divide et impera), ma i sottoproblemi non sono indipendenti: gli stessi sottoproblemi tornano più volte. È il cuore del terzo esercizio dello scritto.

Due precisazioni del libro (p. 297): *programmazione*, qui, significa *metodo tabulare* — i sottoproblemi risolti finiscono in una tabella, non in un programma; e si cerca *una* soluzione ottima, non *la* soluzione ottima, perché di soluzioni di valore ottimo ce ne possono essere più d’una. [Cormen p. 297]

L’idea: non pagare due volte lo stesso conto

Per calcolare $F(n) = F(n-1) + F(n-2)$ la ricorsione pura ricalcola lo stesso valore un numero esponenziale di volte, ma i valori *distinti* sono soltanto n . La programmazione dinamica risolve ogni sottoproblema **una volta sola** e ne scrive la soluzione in una **tabella**: le celle “grandi” si riempiono con quelle “piccole” già calcolate, dal basso verso l’alto (*bottom-up*). Per sapere anche *quale* scelta ha prodotto il valore ottimo si tiene una seconda tabella con le scelte, e da lì si ricostruisce la soluzione.

Le due proprietà e le quattro fasi [Cormen pp. 297 e 313]

Serve una **sottostruttura ottima** (una soluzione ottima contiene soluzioni ottime dei sottoproblemi) e servono **sottoproblemi ripetuti** (i sottoproblemi distinti sono pochi — un polinomio nella dimensione dell’input — ma la ricorsione ne incontra moltissime copie: la tabella risolve ogni sottoproblema una volta, poi lo cerca). Il metodo si svolge in quattro fasi:

1. caratterizzare la struttura di una soluzione ottima;
2. definire in modo ricorsivo il valore di una soluzione ottima;
3. calcolarlo bottom-up;
4. costruire una soluzione ottima dalle informazioni calcolate (questa fase si omette se serve solo il valore).

Il libro apre il capitolo 15 con il *taglio delle aste*; il programma segue invece le **catene di montaggio** (che il libro trattava nella 2^a edizione), poi matrici, LCS, zaino e resto. [Cormen, cap. 15.1, 2^a ed.]

8.1 Le catene di montaggio

Uno stabilimento ha **due catene di montaggio** con n stazioni in fila: $S_{i,j}$ è la stazione j della linea i ($i = 1, 2$). Un telaio entra in linea i pagando e_i e deve attraversare *una stazione per ogni posizione j* , impiegando $a_{i,j}$ alla stazione $S_{i,j}$; dopo la stazione può restare in linea, o trasferirsi sull'altra pagando $t_{i,j}$. Alla fine esce dalla linea i pagando x_i . Si vuole minimizzare il tempo totale. I percorsi possibili sono 2^n , ma basta conoscere i tempi minimi per arrivare alle stazioni precedenti: è la sottostruttura ottima.

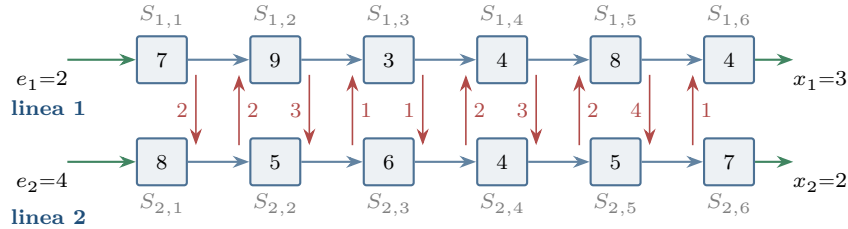


Figura 8.1: Le due catene di montaggio dell'esempio: in ogni stazione il tempo $a_{i,j}$, accanto alle frecce rosse i trasferimenti $t_{i,j}$; a sinistra gli ingressi e_i , a destra le uscite x_i . [Cormen, cap. 15.1, 2^a ed.]

La ricorrenza delle catene di montaggio

Sia $f_i[j]$ il tempo minimo per arrivare e passare per $S_{i,j}$ (compreso $a_{i,j}$); $l_i[j]$ sarà la linea della stazione $j-1$ da cui conviene arrivare. Alla prima stazione si arriva dall'ingresso e per $j \geq 2$ ci sono solo due possibilità: restare in linea o trasferirsi.

$$f_1[1] = e_1 + a_{1,1}, \quad f_2[1] = e_2 + a_{2,1},$$

$$f_1[j] = \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j}),$$

$$f_2[j] = \min(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j}).$$

Alla fine $f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$ (uscita dalla linea l^*). L'algoritmo riempie f e l colonna per colonna in tempo $\Theta(n)$ e spazio $\Theta(n)$. [Cormen cap. 15.1, 2^a ed.]

FASTEST-WAY(a, t, e, x, n)

```

1   $f_1[1] := e_1 + a_{1,1}$ 
2   $f_2[1] := e_2 + a_{2,1}$ 
3  for  $j := 2$  to  $n$ 
4      if  $f_1[j-1] + a_{1,j} \leq f_2[j-1] + t_{2,j-1} + a_{1,j}$ 
5           $f_1[j] := f_1[j-1] + a_{1,j}$ 
6           $l_1[j] := 1$ 
7      else  $f_1[j] := f_2[j-1] + t_{2,j-1} + a_{1,j}$ 
8           $l_1[j] := 2$ 
9          if  $f_2[j-1] + a_{2,j} \leq f_1[j-1] + t_{1,j-1} + a_{2,j}$ 
10              $f_2[j] := f_2[j-1] + a_{2,j}$ 
11              $l_2[j] := 2$ 
12         else  $f_2[j] := f_1[j-1] + t_{1,j-1} + a_{2,j}$ 
13              $l_2[j] := 1$ 
14 if  $f_1[n] + x_1 \leq f_2[n] + x_2$ 
15      $f^* := f_1[n] + x_1, \quad l^* := 1$ 
16 else  $f^* := f_2[n] + x_2, \quad l^* := 2$ 
```

L'esempio svolto

Con l'istanza della Figura 8.1 ($n = 6$, $e = (2, 4)$, $x = (3, 2)$, $a_1 = (7, 9, 3, 4, 8, 4)$, $a_2 = (8, 5, 6, 4, 5, 7)$, $t_1 = (2, 3, 1, 3, 4)$, $t_2 = (2, 1, 2, 2, 1)$): $f_1[1] = 9$, $f_2[1] = 12$, e per esempio $f_2[2] = \min(12 + 5, 9 + t_{1,1} + 5) = 16$: conviene trasferirsi dalla linea 1, quindi $l_2[2] = 1$. Proseguendo fino a $j = 6$:

j	1	2	3	4	5	6
$f_1[j]$	9	18	20	24	32	35
$f_2[j]$	12	16	22	25	30	37
$l_1[j]$	—	1	2	1	1	2
$l_2[j]$	—	1	2	1	2	2

Alla fine $f^* = \min(f_1[6] + x_1, f_2[6] + x_2) = \min(38, 39) = 38$, con uscita dalla linea 1 ($l^* = 1$). La ricostruzione parte da l^* e risale le scelte, stazione per stazione: $l_1[6] = 2$ (stazione 5 in linea 2), $l_2[5] = 2$ (stazione 4 in linea 2), $l_2[4] = 1$ (stazione 3 in linea 1), $l_1[3] = 2$ (stazione 2 in linea 2), $l_2[2] = 1$ (stazione 1 in linea 1). Il percorso ottimo è $S_{1,1} \rightarrow S_{2,2} \rightarrow S_{1,3} \rightarrow S_{2,4} \rightarrow S_{2,5} \rightarrow S_{1,6}$, di costo $2 + 7 + 2 + 5 + 1 + 3 + 1 + 4 + 5 + 1 + 4 + 3 = 38$ (ingresso, stazione e trasferimento a ogni passo, uscita). [Cormen cap. 15.1, 2^a ed.]

8.2 La moltiplicazione di una sequenza di matrici

L'idea: spostare le parentesi

Moltiplicare una matrice $p \times q$ per una $q \times r$ costa pqr prodotti scalari, e il prodotto è associativo: il risultato non cambia, ma il *costo* sì. Con $A_1 = 10 \times 100$, $A_2 = 100 \times 5$, $A_3 = 5 \times 50$: $(A_1 A_2) A_3$ costa 7500 prodotti scalari, $A_1 (A_2 A_3)$ ne costa 75000, dieci volte tanto. Il problema è decidere *dove mettere le parentesi*: non si moltiplica davvero nulla, si cerca solo l'ordine più economico.

Data $\langle A_1, \dots, A_n \rangle$ con A_i di dimensioni $p_{i-1} \times p_i$, si cerca la **parentesizzazione completa** (una singola matrice, oppure il prodotto tra parentesi di due parentesizzazioni complete) che minimizza i prodotti scalari. Per $\langle A_1, A_2, A_3, A_4 \rangle$ i modi distinti sono 5, e in generale crescono come $\Omega(2^n)$ (numeri di Catalan): la forza bruta non basta. [Cormen pp. 306 e 308]

Fase 1 (struttura). Sia $A_{i..j} = A_i A_{i+1} \dots A_j$: se $i < j$, ogni sua parentesizzazione suddivide il prodotto fra A_k e A_{k+1} per un certo k , con $i \leq k < j$. *Sottostruttura ottima*: le parentesizzazioni di $A_{i..k}$ e $A_{k+1..j}$ contenute in una ottima di $A_{i..j}$ sono ottime (altrimenti, taglia e incolla, se ne otterrebbe una di costo minore).

Fase 2 (ricorrenza). Sia $m[i, j]$ il costo minimo per $A_{i..j}$: [Cormen eq. (15.7)]

$$m[i, j] = \begin{cases} 0 & \text{se } i = j, \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & \text{se } i < j, \end{cases} \quad (8.1)$$

dove l'ultimo termine è il costo della moltiplicazione dei due pezzi. Insieme a m si tiene la **tabella delle scelte** $s[i, j]$: il k che realizza il minimo.

Fase 3 (bottom-up). I sottoproblemi distinti sono $\Theta(n^2)$ e $m[i, j]$ dipende solo da sottoproblemi più corti: si riempie per lunghezze $l = j - i + 1$ crescenti. [Cormen p. 310]

MATRIX-CHAIN-ORDER(p)

```

1   $n := p.length - 1$ 
2  Siano  $m[1 \dots n, 1 \dots n]$  e  $s[1 \dots n, 1 \dots n]$  due nuove tabelle
3  for  $i := 1$  to  $n$ 
4       $m[i, i] := 0$ 
5  for  $l := 2$  to  $n$  //  $l$  è la lunghezza della sequenza
6  for  $i := 1$  to  $n - l + 1$ 
7       $j := i + l - 1$ 
8       $m[i, j] := \infty$ 
9      for  $k := i$  to  $j - 1$ 
10          $q := m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
11         if  $q < m[i, j]$ 
12              $m[i, j] := q$ 
13              $s[i, j] := k$ 
14 return  $m$  e  $s$ 

```

Tre cicli annidati danno tempo $\Theta(n^3)$ e spazio $\Theta(n^2)$. [Cormen p. 311]

L'esempio svolto

Dimensioni del libro: $p = \langle 30, 35, 15, 5, 10, 20, 25 \rangle$, cioè A_1 è 30×35 , ..., A_6 è 20×25 . Per $m[2, 5]$, usando $m[3, 5] = 2500$, $m[2, 3] = 2625$, $m[4, 5] = 1000$, $m[2, 4] = 4375$:

$$m[2, 5] = \min \begin{cases} m[2, 2] + m[3, 5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 = 13000 \\ m[2, 3] + m[4, 5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 = 7125 \\ m[2, 4] + m[5, 5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 = 11375 \end{cases}$$

quindi $m[2, 5] = 7125$ e $s[2, 5] = 3$. Ogni cella usa le due celle “a sud ovest” e “a sud est” (i sottoprodotti più corti); le due tabelle complete sono nella Figura 8.2.

$i \backslash j$	1	2	3	4	5	6	$i \backslash j$	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125	1	1	1	3	3	3	
2		0	2625	4375	7125	10500	2		2	3	3	3	
3			0	750	2500	5375	3			3	3	3	
4				0	1000	3500	4				4	5	
5					0	5000	5					5	
6						0	6						5

Figura 8.2: Le tabelle m e s per $p = \langle 30, 35, 15, 5, 10, 20, 25 \rangle$; si usano solo la diagonale e il triangolo superiore ($i \leq j$). Lo sfondo segna la lunghezza $l = j - i + 1$ (dalla più chiara alla più scura). Il minimo è $m[1, 6] = 15125$. [Cormen fig. 15.5]

Fase 4 (ricostruzione). Ogni $s[i, j]$ dice dove spezzare $A_{i..j}$; PRINT-OPTIMAL-PARENS($s, 1, n$) stampa la parentesizzazione ottima. [Cormen p. 312]

PRINT-OPTIMAL-PARENS(s, i, j)

```

1  if  $i == j$ 
2      stampa " $A_i$ "
3  else stampa "("
4      PRINT-OPTIMAL-PARENS( $s, i, s[i, j]$ )
5      PRINT-OPTIMAL-PARENS( $s, s[i, j] + 1, j$ )
6      stampa ")"

```

Nell'esempio stampa $((A_1(A_2A_3))((A_4A_5)A_6))$, di costo $m[1, 6] = 15125$. [Cormen pp. 312–313]

8.3 Sottostruttura ottima e sottoproblemi ripetuti

Sottostruttura ottima: come si dimostra [Cormen cap. 15.3]

Per riconoscerla si segue sempre lo stesso schema: (1) mostrare che una soluzione consiste in una *scelta* (il taglio, l'indice k , se prendere l'oggetto), che lascia uno o più sottoproblemi; (2) supporre *nota* la scelta di una soluzione ottima, senza chiedersi come sia stata trovata; (3) dimostrare che le soluzioni usate sono *ottime* con la tecnica **taglia e incolla**: se un sottoproblema avesse una soluzione migliore, la si “incollerebbe” al posto della sua, ottenendo una soluzione migliore del problema originale — contraddizione. Non va data per scontata: il *cammino minimo* ce l'ha, il **cammino semplice massimo** no (da q a t il massimo è $q \rightarrow r \rightarrow t$, ma $q \rightarrow r$ non è massimo da q a r : i sottoproblemi non sono indipendenti). [Cormen pp. 316–317]

Sottoproblemi ripetuti, e l'annotazione [Cormen pp. 318–321]

La ricorsione diretta sulla ricorrenza (8.1) delle matrici costa $T(n) \geq 2^{n-1}$ — esponenziale — mentre i sottoproblemi distinti sono solo $\Theta(n^2)$: la tabella ne risolve uno per uno. In alternativa al bottom-up, la tecnica dell'**annotazione** (*memoization*) lascia la ricorsione top-down, ma prima di calcolare $m[i, j]$ controlla se la tabella lo contiene già: MEMOIZED-MATRIX-CHAIN gira in $O(n^3)$ risolvendo solo i sottoproblemi che servono davvero.

8.4 La più lunga sottosequenza comune (LCS)

Una **sottosequenza** di $X = \langle x_1, \dots, x_m \rangle$ è la sequenza che si ottiene togliendo zero o più elementi (non per forza consecutivi); Z è una **sottosequenza comune** di X e Y se è sottosequenza di entrambe. La forza bruta (2^m sottosequenze di X) è esponenziale; la programmazione dinamica risolve in $\Theta(mn)$. [Cormen cap. 15.4]

Teorema 15.1: sottostruttura ottima di una LCS

da saper dimostrare

Siano $X = \langle x_1, \dots, x_m \rangle$, $Y = \langle y_1, \dots, y_n \rangle$ e $Z = \langle z_1, \dots, z_k \rangle$ una LCS di X e Y . Allora:

1. se $x_m = y_n$, allora $z_k = x_m = y_n$ e Z_{k-1} è una LCS di X_{m-1} e Y_{n-1} ;
2. se $x_m \neq y_n$, allora $z_k \neq x_m$ implica che Z è una LCS di X_{m-1} e Y ;
3. se $x_m \neq y_n$, allora $z_k \neq y_n$ implica che Z è una LCS di X e Y_{n-1} .

Dimostrazione. (1) Se $z_k \neq x_m$, accodando $x_m = y_n$ a Z si otterrebbe una sottosequenza comune di lunghezza $k+1$: contraddizione. Quindi $z_k = x_m = y_n$ e Z_{k-1} è una sottosequenza comune di X_{m-1} e Y_{n-1} ; se non fosse una LCS ci sarebbe una W più lunga, e accodandole $x_m = y_n$ si otterrebbe una sottosequenza comune di X e Y più lunga di Z : contraddizione. (2) Z è sottosequenza di X_{m-1} e Y ; se una W comune a X_{m-1} e Y fosse più lunga di Z , sarebbe comune anche a X e Y : contraddizione. (3) Simmetrica alla (2). [Cormen p. 325]

Sia $c[i, j]$ la lunghezza di una LCS dei prefissi X_i e Y_j : se $i = 0$ o $j = 0$ una sequenza è vuota; se $x_i = y_j$ si usa la soluzione di X_{i-1} , Y_{j-1} accodando x_i ; altrimenti si prende il migliore dei due casi (togliere x_i , togliere y_j):

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0, \\ c[i-1, j-1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j, \\ \max(c[i-1, j], c[i, j-1]) & \text{se } i, j > 0 \text{ e } x_i \neq y_j. \end{cases} \quad (8.2)$$

Sono $\Theta(mn)$ sottoproblemi, ciascuno con al più due scelte; la tabella delle scelte $b[i, j]$ dice quale caso ha prodotto $c[i, j]$. [Cormen eq. (15.9)]

LCS-LENGTH(X, Y)

```

1   $m := X.length$ 
2   $n := Y.length$ 
3  Siano  $b[1 \dots m, 1 \dots n]$  e  $c[0 \dots m, 0 \dots n]$  due nuove tabelle
4  for  $i := 1$  to  $m$ 
5       $c[i, 0] := 0$ 
6  for  $j := 0$  to  $n$ 
7       $c[0, j] := 0$ 
8  for  $i := 1$  to  $m$ 
9      for  $j := 1$  to  $n$ 
10         if  $x_i == y_j$ 
11              $c[i, j] := c[i - 1, j - 1] + 1$ 
12              $b[i, j] := \nwarrow$ 
13         else if  $c[i - 1, j] \geq c[i, j - 1]$ 
14              $c[i, j] := c[i - 1, j]$ 
15              $b[i, j] := \uparrow$ 
16         else  $c[i, j] := c[i, j - 1]$ 
17              $b[i, j] := \leftarrow$ 
18 return  $c$  e  $b$ 
```

j		B	D	C	A	B	A
i	0	0	0	0	0	0	0
A	0	0	$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\nwarrow$
B	0	$\nwarrow$	$\nwarrow$	$\nwarrow$	$\uparrow$	$\nwarrow$	$\nwarrow$
C	0	$\uparrow$	$\uparrow$	$\nwarrow$	$\nwarrow$	$\uparrow$	$\uparrow$
B	0	$\nwarrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\nwarrow$
D	0	$\uparrow$	$\nwarrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$
A	0	$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\uparrow$	$\nwarrow$
B	0	$\nwarrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\nwarrow$	$\uparrow$

L'esempio del libro: $X = \langle A, B, C, B, D, A, B \rangle$ (righe), $Y = \langle B, D, C, A, B, A \rangle$ (colonne). In ogni cella ci sono il valore $c[i, j]$ e la freccia $b[i, j]$; lo sfondo segna le posizioni con $x_i = y_j$ del cammino di ricostruzione. Partendo da $b[m, n]$ si seguono le frecce: ogni freccia diagonale aggiunge un elemento alla LCS (in ordine inverso), ogni passo toglie almeno 1 a $i + j$, quindi il costo è $O(m + n)$. Nell'esempio si ottiene **BCBA**, lunghezza $c[7, 6] = 4$.

Figura 8.3: Le tabelle c e b per $X = ABCBDAB$ e $Y = BDCABA$: in ogni cella il valore $c[i, j]$ e la freccia $b[i, j]$; lo sfondo segna il cammino di ricostruzione, che parte da $c[7, 6] = 4$ e risale le frecce. [Cormen fig. 15.8]

8.5 Zaino 0/1 e problema del resto

Zaino: quando il goloso sbaglia

Un ladro ha uno zaino che sopporta W libbre e trova n oggetti; l'oggetto i pesa w_i e vale v_i . Nello zaino **0/1** ogni oggetto si prende o si lascia. La tentazione è il goloso ("prendi prima ciò che vale di più per libbra"), ma la scelta giusta dipende da *quanto spazio resta*: serve una tabella su *due* parametri, gli oggetti considerati e la capacità.

Tolto l'oggetto j dal carico ottimo, ciò che resta è il carico ottimo di peso al più $W - w_j$ sugli altri oggetti: c'è la sottostruttura ottima. Sia $C[i, j]$ il valore massimo con i primi i oggetti e capacità j ($C[0, j] = 0$): [Cormen cap. 16.2]

$$C[i, j] = \begin{cases} C[i - 1, j] & \text{se } w_i > j, \\ \max(C[i - 1, j], C[i - 1, j - w_i] + v_i) & \text{se } w_i \leq j. \end{cases} \quad (8.3)$$

Si riempie per i crescente e, dentro ogni riga, per j crescente: ogni cella usa solo la riga precedente. Tempo e spazio $\Theta(nW)$; nei temi la tabella si chiama $C[i, v]$, con v la capacità residua: è la stessa tabella. [Cormen esercizio 16.2-2]

L'esempio del libro: oggetti $(w_i, v_i) = (10, 60), (20, 100), (30, 120)$, $W = 50$. Il goloso prende l'oggetto 1 (massimo valore specifico) e poi il 2: \$160. L'ottimo è prendere **gli oggetti 2 e 3**: pesano 50 e valgono **\$220**. Nel problema *frazionario* (anche pezzi di oggetto) il goloso arriva a \$240 ed è ottimo. [Cormen pp. 354–355]

$i \backslash v$	0	10	20	30	40	50
nessuno	0	0	0	0	0	0
oggetto 1: $(w_1, v_1) = (10, 60)$	0	60	60	60	60	60
oggetto 2: $(w_2, v_2) = (20, 100)$	0	60	100	160	160	160
oggetto 3: $(w_3, v_3) = (30, 120)$	0	60	100	160	180	220

Figura 8.4: La tabella $C[i, v]$ dell'esempio, per le capacità $v = 0, 10, \dots, 50$. In verde le celle della ricostruzione: $C[2, 20] = 100$ e $C[3, 50] = 220$.

La ricostruzione cammina all'indietro: se $C[i, j] = C[i - 1, j]$ l'oggetto i è stato lasciato, altrimenti è stato preso e si passa a $(i - 1, j - w_i)$. Nell'esempio: $220 \neq 160$ (preso l'oggetto 3, si va a $(2, 20)$), $100 \neq 60$ (preso l'oggetto 2, si va a $(1, 0)$), $0 = 0$ (oggetto 1 lasciato). Il tempo $O(nW)$ è **pseudopolinomiale**: dipende dal valore di W , non dal numero di bit che lo scrivono (lo zaino 0/1 è NP-completo).

Il problema del resto. Dati n tagli $d_1 < \dots < d_n$ e un importo V , minimizzare il numero di monete che formano V . Il libro lo cita fra i problemi "golosi" (prendi la moneta più grande che non supera il resto, [Cormen p. 295]), ma il goloso non è sempre ottimo. Con $R[v] =$ minimo numero di monete per l'importo v : [non nelle fonti del corso]

$$R[0] = 0, \quad R[v] = \min_{d_i \leq v} \{R[v - d_i] + 1\} \quad (v = 1, \dots, V),$$

tempo $O(nV)$, di nuovo pseudopolinomiale. Con i tagli $\langle 1, 3, 4 \rangle$ e $V = 6$ il goloso fa $4 + 1 + 1$ (tre monete), l'ottimo è $3 + 3$ (due): $R[6] = \min\{R[5] + 1, R[3] + 1, R[2] + 1\} = 2$.

8.6 Come si risolve all'esame

L'esercizio 3 dello scritto è un problema di progettazione da svolgere per intero: formalizzazione, ricorrenza, pseudocodice bottom-up, istanza, ricostruzione, complessità.

Come si imposta un problema di programmazione dinamica

1. **Formalizza gli oggetti decisionali:** che cos'è una soluzione (un percorso, una parentesizzazione, un sottoinsieme) e qual è la scelta che la determina (l'indice k , prendere o no l'oggetto). Definisci la funzione costo.
2. **Scegli la tabella** in base a ciò che caratterizza il sottoproblema: un *intervallo* $C[i, j]$, una *risorsa* $C[i, v]$, oppure due prefissi $c[i, j]$ (primi i di X , primi j di Y).
3. **Scrivi la ricorrenza con i casi base:** se la scelta è ..., quali sottoproblemi restano? Prendi il minimo (o il massimo) su tutti i casi possibili; i casi base sono la prima riga e la prima colonna della tabella.
4. **Bottom-up: l'ordine di riempimento.** Una cella si calcola solo quando le celle da cui dipende sono pronte: lunghezze crescenti $l = j - i + 1$ (intervalli), per i e poi per j (risorsa), riga per riga (due sequenze).

5. **Ricostruzione:** o tieni una *tabella delle scelte* mentre riempi, o *cammini all'indietro* confrontando la cella finale con le vicine.
6. **Complessità** in funzione di tutti i parametri dell'input ($O(n^3)$, $\Theta(mn)$, $\Theta(nW)$); se compare W al posto del numero di bit, dillo: **pseudopolinomiale**.

Esempio dai temi (montaggio dei filmati, 22 giugno 2026, E3; lo svolgimento è nel capitolo 13): un filmato ha segmenti $S_1, \dots, S_n$ con f_i fotogrammi; unire due blocchi *consecutivi* di a e b fotogrammi costa $a \cdot b$. Tabella $C[i, j]$ = costo minimo per unire $i..j$; la scelta è l'ultima fusione, fra $i..k$ e $k+1..j$:

$$C[i, j] = \min_{i \leq k < j} \{C[i, k] + C[k+1, j] + F(i, k) \cdot F(k+1, j)\}, \quad C[i, i] = 0,$$

con $F(i, j) = f_i + \dots + f_j$ precalcolabile. Riempimento per $l = j - i + 1$ crescente, tempo $O(n^3)$: è la moltiplicazione di matrici “mascherata”, con i fotogrammi al posto delle dimensioni.

Se ti chiedono: “la programmazione dinamica”

1. **Definizione:** tecnica per problemi di ottimizzazione che combina soluzioni di sottoproblemi risolvendo ogni sottoproblema *una volta sola* e memorizzandolo in una tabella.
2. **Quando si applica:** sottostruttura ottima e sottoproblemi ripetuti (distinti in numero polinomiale).
3. **Le 4 fasi:** struttura, ricorrenza, calcolo bottom-up, ricostruzione.
4. **Esempi del corso:** catene di montaggio, matrici, LCS; zaino e resto.
5. **Complessità:** sottoproblemi $\times$ scelte ($\Theta(n^3)$, $\Theta(mn)$, $\Theta(nW)$ pseudopolinomiale).
6. **Differenze:** dal divide et impera perché i sottoproblemi si ripetono; dal greedy perché non si sceglie una volta per tutte ma si confrontano *tutte* le scelte (zaino: il greedy si ferma a 160, l'ottimo è 220).

Se ti chiedono: “la sottostruttura ottima”

1. **Definizione:** una soluzione ottima contiene al suo interno soluzioni ottime dei sottoproblemi che usa.
2. **Come si dimostra:** taglia e incolla — se un sottoproblema avesse una soluzione migliore, la si incolla al posto della sua e si ottiene una soluzione migliore del problema di partenza: contraddizione.
3. **Esempi:** LCS (Teorema 15.1) e matrici (la parentesizzazione ottima contiene quelle ottime dei due sottoprodotti).
4. **Controesempio:** il cammino semplice massimo non ce l'ha ($q \rightarrow r \rightarrow t$ è massimo da q a t , ma $q \rightarrow r$ non è massimo da q a r : i sottoproblemi non sono indipendenti).
5. **Attenzione:** ce l'hanno anche i problemi golosi; da sola non basta, serve anche la ripetizione dei sottoproblemi.

Errori tipici con la programmazione dinamica

- **Confondere il valore con la soluzione:** $m[1, n]$, $c[m, n]$, $C[n, W]$ danno il valore; per la *soluzione* serve la tabella delle scelte (o il camminamento all'indietro).
- **Dimenticare i casi base** (prima riga, prima colonna, o la diagonale $C[i, i] = 0$), o **riempire la tabella nell'ordine sbagliato**: una cella si calcola solo quando le celle da cui dipende sono pronte.
- **Usare il goloso sul problema sbagliato**: nello zaino 0/1 il massimo valore specifico dà 160 contro 220; funziona solo per lo zaino frazionario. E **fermarsi a “ $O(nW)$ ”**: se W è un valore e non la dimensione dell'input, la complessità è *pseudopolinomiale*, va detto.
- **Sbagliare la ricostruzione dello zaino**: si prende solo se $C[i, j] \neq C[i-1, j]$, e allora j va ridotto di w_i .

Capitolo 9

Algoritmi golosi

Un algoritmo **goloso** (*greedy*) risolve un problema di ottimizzazione scegliendo, a ogni passo, quella che *sembra* la soluzione migliore in quel momento — una scelta localmente ottima — e senza mai tornare indietro. È la più semplice delle tecniche del corso: niente tabelle, spesso solo un ordinamento e una scansione, e tempi che scendono da $\Theta(nW)$ o $\Theta(n^3)$ a $\Theta(n)$ o $O(n \lg n)$. Il prezzo: la scelta golosa è sicura solo per certi problemi, gli altri vogliono la programmazione dinamica del capitolo 8. Qui: la selezione di attività (16.1), le due proprietà della strategia (16.2), i codici di Huffman (16.3), con le dimostrazioni del Laboratorio. [Cormen cap. 16.1–16.3]

L'idea: la scelta che non si rimpiange

Un algoritmo goloso decide come un giocatore che non può tornare sui suoi passi: a ogni bivio prende la strada che gli sembra migliore *adesso* — l'attività che finisce prima, l'oggetto che vale di più per libbra, le due lettere meno frequenti. Funziona quando esiste un argomento di **scambio**: se una soluzione ottima fa un'altra scelta, si sostituisce la scelta golosa al posto della sua senza peggiorare la soluzione. Allora esiste un ottimo che *comincia* come vuole il goloso, e dopo la scelta resta un sottoproblema dello stesso tipo: l'argomento si ripete. Quando lo scambio non si costruisce, il goloso è solo un'euristica.

Gli ingredienti: scelta golosa e sottostruttura ottima [Cormen cap. 16.2]

Un problema di ottimizzazione si risolve in modo goloso quando valgono due proprietà.

- **Proprietà della scelta golosa:** esiste una soluzione ottima del problema che si ottiene facendo la scelta golosa, cioè la scelta che sembra migliore per il problema *corrente*, senza guardare le soluzioni dei sottoproblemi. Si dice che la scelta è *sicura*.
- **Sottostruttura ottima:** fatta la scelta golosa resta *un solo* sottoproblema dello stesso tipo, e una sua soluzione ottima, combinata con la scelta, è una soluzione ottima del problema.

Sono le stesse proprietà della programmazione dinamica, con due differenze decisive: la scelta si fa *prima* di risolvere i sottoproblemi (schema dall'alto verso il basso, non bottom-up) e non dipende mai dalle loro soluzioni. La scaletta [Cormen p. 352]: esprimere il problema in modo che, fatta una scelta, resti un solo sottoproblema; dimostrare che la scelta è sicura, di solito con uno *scambio* (si prende un ottimo qualsiasi, si sostituisce la sua scelta con quella golosa e si mostra che non peggiora); dimostrare la sottostruttura ottima. Se lo scambio non si chiude, il goloso è un'euristica: serve la programmazione dinamica.

9.1 Un esempio: la selezione di attività

Un insieme di n attività $a_1, \dots, a_n$ deve usare la stessa risorsa (per esempio un'aula), una alla volta. L'attività a_i occupa l'intervallo semiaperto $[s_i, f_i)$: inizia in s_i e finisce in f_i . Due attività sono **compatibili** se i loro intervalli non si sovrappongono, cioè se $s_i \geq f_j$ oppure $s_j \geq f_i$. Il **problema della selezione di attività** chiede il più grande insieme di attività mutuamente compatibili, e le attività arrivano ordinate per tempo di fine crescente:

$$f_1 \leq f_2 \leq \dots \leq f_n. \quad (16.1)$$

L'esempio che accompagna il capitolo è quello del libro [Cormen p. 345]: le undici attività della Figura 9.1, per le quali il massimo è 4 attività.

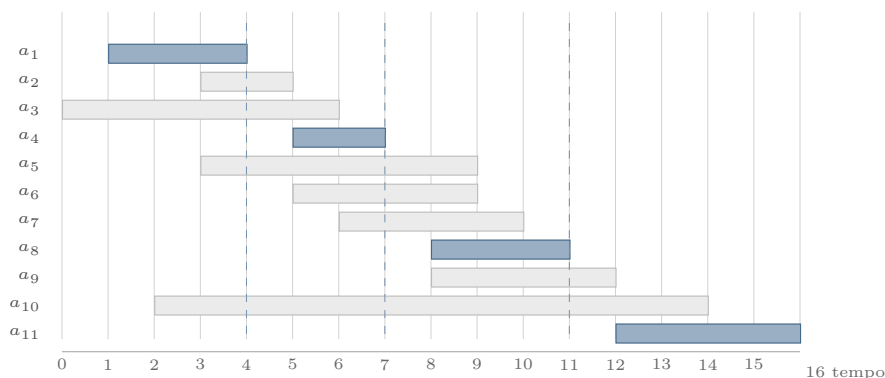


Figura 9.1: La timeline dell'istanza di pagina 345: in grigio le attività scartate, in blu quelle scelte, $\{a_1, a_4, a_8, a_{11}\}$; le tratteggiate scendono dai tempi di fine delle scelte (4, 7, 11). Nessun insieme di 5 attività è compatibile, quindi 4 è il massimo. [Cormen p. 345]

Prima la programmazione dinamica. Il problema ha la sottostruttura ottima. Per $i < j$ sia S_{ij} l'insieme delle attività che iniziano dopo la fine di a_i e finiscono prima dell'inizio di a_j : se una soluzione ottima A_{ij} di S_{ij} contiene a_k , le porzioni $A_{ik} = A_{ij} \cap S_{ik}$ e $A_{kj} = A_{ij} \cap S_{kj}$ sono ottime per i due sottoproblemi (altrimenti, taglia e incolla, un insieme più grande darebbe $|A_{ik}| + |A'_{kj}| + 1 > |A_{ij}|$ attività in S_{ij}). Con $c[i, j]$ dimensione di una soluzione ottima di S_{ij} :

$$c[i, j] = \begin{cases} 0 & \text{se } S_{ij} = \emptyset, \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{se } S_{ij} \neq \emptyset, \end{cases} \quad (16.2)$$

cioè la programmazione dinamica *prova ogni* attività a_k di S_{ij} . La scelta giusta è una sola: l'attività che **finisce prima**, così la risorsa resta libera il più presto possibile. Ordinate le attività per tempo di fine è a_1 ; poiché $s_1 < f_1 \leq f_j$ per ogni j , ogni attività compatibile con a_1 inizia dopo f_1 e resta *un solo* sottoproblema, $S_1 = \{a_i : s_i \geq f_1\}$. [Cormen pp. 345–347]

La scelta è sicura: è la dimostrazione di scambio della selezione di attività.

Teorema 16.1: la scelta golosa della selezione di attività

da saper dimostrare

Sia S_k un sottoproblema non vuoto e sia a_m l'attività di S_k che ha il primo tempo di fine. Allora a_m appartiene a qualche sottoinsieme massimo di attività mutuamente compatibili di S_k .

Dimostrazione. Sia A_k un sottoinsieme massimo di attività mutuamente compatibili di S_k e sia a_j l'attività di A_k che finisce per prima. Se $a_j = a_m$ abbiamo finito: A_k è un insieme massimo che contiene proprio a_m . Altrimenti sia $A'_k = A_k - \{a_j\} \cup \{a_m\}$, cioè A_k con a_m al posto di a_j . Le attività di A'_k sono ancora mutuamente compatibili: ogni attività di A_k diversa da a_j inizia dopo f_j (perché a_j è quella che finisce per prima in A_k), e $f_m \leq f_j$ perché a_m ha il primo tempo di fine di S_k , quindi iniziano anche dopo f_m ; e a_m stessa inizia dopo f_k , perché appartiene a S_k . Infine $|A'_k| = |A_k|$, quindi A'_k è un sottoinsieme massimo di S_k che contiene a_m . $\square$

[Cormen p. 347]

Il teorema è la proprietà della scelta golosa; l'ordinamento per tempo di fine e la discussione di sopra danno la sottostruttura ottima. Ripetendo la scelta si ottiene l'algoritmo iterativo, che scandisce le attività una volta sola tenendo traccia dell'ultima scelta k : [Cormen p. 350]

GREEDY-ACTIVITY-SELECTOR(s, f)

```

1   $n := s.length$ 
2   $A := \{a_1\}$ 
3   $k := 1$ 
4  for  $m := 2$  to  $n$ 
5      if  $s[m] \geq f[k]$ 
6           $A := A \cup \{a_m\}$ 
7           $k := m$ 
8  return  $A$ 
```

Se le attività sono già ordinate, il tempo è $\Theta(n)$: ogni attività è esaminata una sola volta. Se l'input non è ordinato si paga prima l'ordinamento, $O(n \lg n)$. [Cormen p. 350]

L'esempio svolto

Nell'istanza della Figura 9.1 l'algoritmo parte con $A = \{a_1\}$, $k = 1$ e scandisce $m = 2, \dots, 11$ confrontando s_m con f_k :

m	2	3	4	5	6	7	8	9	10	11
s_m	3	0	5	3	5	6	8	8	2	12
$s_m \geq f_k?$	no	no	sì	no	no	no	sì	no	no	sì

Dopo a_4 diventa $f_k = 7$; dopo a_8 diventa $f_k = 11$. Le attività scelte sono $\{a_1, a_4, a_8, a_{11}\}$; un altro insieme massimo è $\{a_2, a_4, a_9, a_{11}\}$. La forza bruta su tutti i 2^{11} sottoinsiemi conferma che quattro è il massimo. [Cormen p. 345]

9.2 Gli elementi della strategia golosa

Come si indovina la scelta golosa? Si prova, e poi si dimostra. Il punto delicato è la differenza con la programmazione dinamica, che lo zaino mostra al lavoro.

Lo zaino. Un ladro trova n oggetti; l'oggetto i pesa w_i libbre e vale v_i dollari, e lo zaino sopporta W libbre. Nello **zaino 0/1** ogni oggetto si prende o si lascia (un lingotto d'oro: niente frazioni); nello **zaino frazionario** si prendono anche frazioni (polvere d'oro). Entrambi hanno la sottostruttura ottima; per il frazionario la scelta golosa è il **valore specifico** v_i/w_i : prende più che può dell'oggetto col massimo v_i/w_i , poi quello col massimo successivo, e via così ($O(n \lg n)$ per l'ordinamento); funziona perché le frazioni aggiustano sempre il riempimento. Nello zaino 0/1 la *stessa* scelta **non è sicura**: nell'esempio del libro (Figura 9.2) il goloso prende gli oggetti 1 e 2, si ferma a \$160 lasciando 20 libbre vuote, mentre l'ottimo è prendere gli oggetti 2 e 3, \$220 con lo zaino pieno.

[Cormen cap. 16.2]

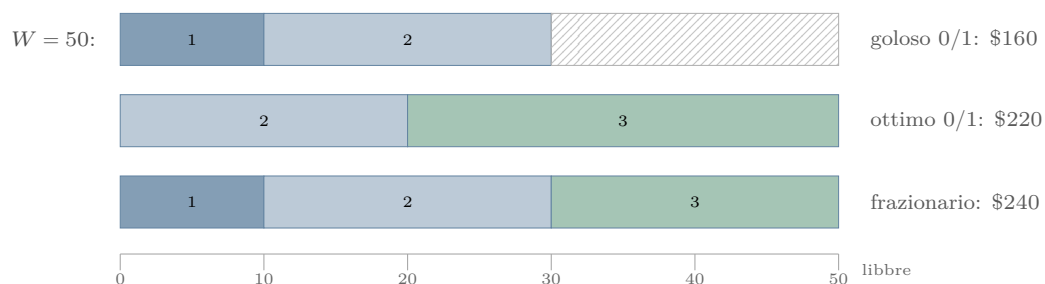


Figura 9.2: Il confronto della Figura 16.2: oggetti (10 libbre, \$60), (20, \$100), (30, \$120), zaino da 50. In alto il goloso 0/1 (oggetti 1 e 2, 20 libbre sprecate); al centro l'ottimo 0/1 (oggetti 2 e 3); in basso il frazionario, con 20/30 dell'oggetto 3: qui è anche l'ottimo. [Cormen fig. 16.2]

Esercizio 16.2-1: la scelta golosa dello zaino frazionario*da saper dimostrare*

Ordinati gli oggetti per valore specifico v_i/w_i decrescente, il primo oggetto è *sicuro*: esiste una soluzione ottima che ne prende $m = \min(w_1, W)$ libbre, tutto quello che ne sta nello zaino.

Dimostrazione. Sia O una soluzione ottima e sia a_1 il peso dell'oggetto 1 in O . Se $a_1 < m$, avanza oggetto 1 e O non lascia spazio vuoto, quindi contiene $W - a_1 \geq m - a_1$ libbre di altri oggetti. *Scambio:* tolgo dagli altri oggetti $\delta = m - a_1$ libbre e ci metto δ libbre dell'oggetto 1 (ci stanno: $m \leq w_1$). Il peso non cambia; ogni libbra sostituita era pagata al più v_1/w_1 , quindi il valore cambia di $\delta(v_1/w_1 - v_j/w_j) \geq 0$: in un ottimo resta uguale, e O' è ancora ottima con m libbre dell'oggetto 1. Se $m = W$ basta così; se $m = w_1$, resta lo stesso problema con capacità $W - w_1$ sugli altri oggetti, e la parte di O' che li usa ne è ottima (altrimenti, con l'oggetto 1, batterebbe O'): sottostruttura ottima, e l'induzione chiude. Con δ frazionario (l'ultimo oggetto preso in parte) il conto è identico. $\square$

[Cormen esercizio 16.2-1, pp. 354–355]

Il conto dello scambio. Sull'istanza della Figura 9.2 (valori specifici 6, 5, 4): dalla soluzione con gli oggetti 2 e 3 interi (\$220) tolgo 10 delle 30 libbre dell'oggetto 3 e prendo 10 libbre dell'oggetto 1; il peso resta 50 e il valore diventa $220 - 10 \cdot 4 + 10 \cdot 6 = 220 + 10 \cdot (6 - 4) = 240$: è l'ottimo, nella forma del goloso (20/30 dell'oggetto 3). [Cormen fig. 16.2]

Goloso che funziona, goloso che sbaglia. Nello zaino 0/1 la scelta buona dipende da *quanto spazio resta*, cioè dai sottoproblemi, e il goloso decide senza guardarli; nel frazionario no. Falsi golosi anche per le attività: la durata minima e l'inizio minimo *non* danno il massimo (Esercizio 16.1-3). [Cormen pp. 351–355]

Scelta golosa contro programmazione dinamica

È la scelta di progetto che l'Esercizio 3 dello scritto mette alla prova: goloso o tabella? Si procede così.

1. **Formula la scelta** e chiediti da cosa dipende: se serve la soluzione di un sottoproblema (quanto spazio resta), programmazione dinamica; se basta un dato locale (tempo di fine, valore specifico), il goloso è candidato.
2. **Prova lo scambio** (scaletta fissa, vale punti): in un ottimo A sostituisci la prima scelta con quella golosa g ($A' = A - \{x\} \cup \{g\}$), dimostra che A' è ancora ammissibile — qui si usa la proprietà di g : finisce prima, pesa meno — e che non peggiora A : la scelta è sicura.
3. **Prova la sottostruttura:** dopo g resta un sottoproblema dello stesso tipo, e il suo ottimo più g dà l'ottimo. Con 2 e 3 l'induzione chiude: l'algoritmo è corretto.
4. **Struttura dati:** ordinamento (attività per tempo di fine, oggetti per valore specifico) o **coda di priorità** (Huffman, fusioni); complessità $O(n \lg n)$.
5. **Se lo scambio non si chiude, cambia metodo:** il controesempio più richiesto è lo zaino 0/1 (\$160 contro \$220); poi il resto $\langle 1, 3, 4 \rangle$.

Goloso che funziona: selezione di attività, zaino frazionario, Huffman, resto con 1, 5, 10, 25 (o con le potenze di una base). Goloso che sbaglia: zaino 0/1, resto con 1, 3, 4.

9.3 I codici di Huffman

I **codici di Huffman** comprimono un file: a ogni carattere c di un alfabeto C si assegna una **parola in codice** binaria, corta per i frequenti e lunga per i rari. Con 6 caratteri la lunghezza fissa costa 3 bit per carattere; il codice di Huffman, goloso e ottimo, risparmia tipicamente dal 20% al 90%. [Cormen pp. 356–357]

Codici prefissi, alberi, costo [Cormen cap. 16.3]

Un codice è **prefisso** se nessuna parola in codice è prefisso di un'altra: la decodifica è unica, perché la prima parola è inequivocabile. Ogni codice prefisso si rappresenta con un **albero binario** le cui foglie sono i caratteri; la parola in codice è il cammino dalla radice alla foglia (0 = figlio sinistro, 1 = destro). Non è un albero di ricerca: le foglie non sono ordinate e i nodi interni non hanno chiavi. Un codice ottimo ha un albero binario *pieno* (ogni nodo interno ha due figli): con $|C|$ foglie ci sono esattamente $|C| - 1$ nodi interni. Il **costo** dell'albero, cioè i bit per codificare il file, è [Cormen eq. (16.4)]:

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c), \quad (16.4)$$

dove $d_T(c)$ è la profondità della foglia di c , cioè la lunghezza della sua parola in codice.

L'esempio del libro è un file di 100 000 caratteri su sei lettere, con le frequenze (in migliaia) $a.freq = 45$, $b = 13$, $c = 12$, $d = 16$, $e = 9$, $f = 5$; a lunghezza fissa servono 300 000 bit. L'algoritmo costruisce l'albero dal basso: parte da $|C|$ foglie e fa $|C| - 1$ "fusioni", prendendo ogni volta i *due alberi con la frequenza minima*, con una coda di min-priorità Q con chiave $freq$. [Cormen p. 359]

HUFFMAN(C)

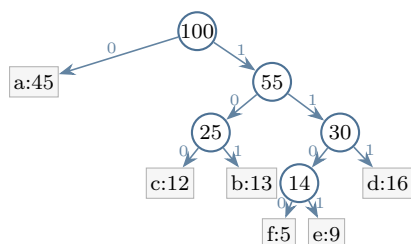
```

1   $n := |C|$ 
2   $Q := C$  // coda di min-priorità, chiave  $c.freq$ 
3  for  $i := 1$  to  $n - 1$ 
4      alloca un nuovo nodo  $z$ 
5       $z.left := x := \text{EXTRACT-MIN}(Q)$ 
6       $z.right := y := \text{EXTRACT-MIN}(Q)$ 
7       $z.freq := x.freq + y.freq$ 
8       $\text{INSERT}(Q, z)$ 
9  return  $\text{EXTRACT-MIN}(Q)$  // restituisce la radice dell'albero
```

Con Q su min-heap binario l'inizializzazione costa $O(n)$ e ogni operazione $O(\lg n)$: HUFFMAN gira in $O(n \lg n)$. [Cormen p. 359]

L'esempio svolto

Le cinque fusioni: $f:5 + e:9 = 14$, $c:12 + b:13 = 25$, $14 + d:16 = 30$, $25 + 30 = 55$, $a:45 + 55 = 100$.



le 5 fusioni:
 $5+9=14$, $12+13=25$, $14+16=30$,
 $25+30=55$, $45+55=100$

codici:					
a	b	c	d	e	f
0	101	100	111	1101	1100

Figura 9.3: L'albero di Huffman dell'esempio: i bit sui bordi (0 sinistro, 1 destro), e a ogni nodo interno la somma del suo sottoalbero. [Cormen figg. 16.3–16.5]

Dall'albero di Figura 9.3: $B(T) = 45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4 = 224$ migliaia di bit, contro 300; il risparmio è $1 - 224/300 \approx 25\%$, e 224 è anche la somma dei costi delle fusioni (Esercizio 16.3-4). Resta da dimostrare che l'albero di Huffman è ottimo: servono la proprietà della scelta golosa (la fusione delle due lettere meno frequenti è sicura) e la sottostruttura ottima. [Cormen p. 360]

Lemma 16.2: la scelta golosa dei codici (le due meno frequenti)

da saper dimostrare

Sia C un alfabeto dove ogni carattere c ha frequenza $c.freq$. Siano x e y due caratteri di C con le frequenze più piccole. Allora esiste un codice prefisso ottimo per C in cui le parole in codice di x e y hanno la stessa lunghezza e differiscono soltanto nell'ultimo bit (cioè x e y sono foglie sorelle).

Dimostrazione. Si prende un albero T di un codice prefisso ottimo arbitrario e lo si modifica, con due scambi, in un albero ottimo dove x e y sono foglie sorelle a profondità massima. Siano a e b due foglie sorelle di T a profondità massima, con $a.freq \leq b.freq$; poiché x e y hanno le due frequenze minime, $x.freq \leq a.freq$ e $y.freq \leq b.freq$. (Se $x.freq = b.freq$ le quattro frequenze sono uguali — Esercizio 16.3-1 — e il lemma è banale; quindi $x.freq \neq b.freq$, in particolare $x \neq b$.) Si scambiano le posizioni di a e x in T , ottenendo T' , e poi quelle di b e y in T' , ottenendo T'' : qui x e y sono foglie sorelle a profondità massima. Nello scambio $a \leftrightarrow x$ cambiano solo i termini di a e x nella somma del costo (16.4):

$$\begin{aligned} B(T) - B(T') &= \sum_{c \in C} c.freq d_T(c) - \sum_{c \in C} c.freq d_{T'}(c) \\ &= x.freq d_T(x) + a.freq d_T(a) - x.freq d_T(a) - a.freq d_T(x) \\ &= (a.freq - x.freq)(d_T(a) - d_T(x)) \geq 0, \end{aligned}$$

perché $a.freq - x.freq \geq 0$ (x ha frequenza minima) e $d_T(a) - d_T(x) \geq 0$ (a è a profondità massima). Dunque $B(T') \leq B(T)$; con lo stesso conto $B(T'') \leq B(T')$. Poiché T è ottimo, $B(T) \leq B(T'')$, quindi $B(T'') = B(T)$: T'' è un albero ottimo in cui x e y sono foglie sorelle a profondità massima, e le loro parole in codice differiscono solo nell'ultimo bit. $\square$

[Cormen pp. 360–361]

Il senso goloso: fondere x e y è sicuro, al costo minimo $x.freq + y.freq$.

Lemma 16.3 e Teorema 16.4: sottostruttura ottima e correttezza

da saper dimostrare

(Lemma 16.3) Siano x e y i due caratteri di C con frequenza minima e sia C' il problema che si ottiene togliendo x e y e aggiungendo un carattere z con $z.freq = x.freq + y.freq$.

Se T' è un albero ottimo per C' , l'albero T ottenuto sostituendo la foglia z con un nodo interno con figli x e y è ottimo per C .

Dimostrazione. Nel passaggio da T' a T ogni carattere $c \neq x, y$ mantiene la profondità, mentre $d_T(x) = d_T(y) = d_{T'}(z) + 1$; per la (16.4),

$$x.freq d_T(x) + y.freq d_T(y) = (x.freq + y.freq)(d_{T'}(z) + 1) = z.freq d_{T'}(z) + x.freq + y.freq,$$

quindi $B(T) = B(T') + x.freq + y.freq$. Per assurdo, se T non fosse ottimo per C esisterebbe T'' con $B(T'') < B(T)$; per il Lemma 16.2 possiamo supporre che T'' abbia x e y come foglie sorelle. Sostituendo il padre di x e y con una foglia z si ottiene T''' per C' con $B(T''') = B(T'') - x.freq - y.freq < B(T) - x.freq - y.freq = B(T')$, contro l'ottimalità di T' . Quindi T è ottimo per C .

(Teorema 16.4) La procedura HUFFMAN produce un codice prefisso ottimo. *Dimostrazione.* Conseguenza immediata dei Lemmi 16.2 e 16.3: la prima fusione è sicura (16.2) e ciò che resta è un problema della stessa forma su C' (16.3); per induzione, tutte le fusioni lo sono. $\square$

[Cormen p. 362]

Se ti chiedono: «i codici di Huffman»

La domanda tipo: enunciare il problema, fornire la funzione costo, dimostrare la proprietà della scelta golosa. La scaletta:

1. **Il problema:** dato l'alfabeto C con le frequenze $c.freq$, trovare un codice *prefisso* di costo minimo; non si perde nulla, perché un codice ottimo è un albero binario pieno.
2. **La funzione costo:** $B(T) = \sum_{c \in C} c.freq \cdot d_T(c)$, con $d_T(c)$ profondità della foglia; vale anche come somma dei costi delle fusioni (Esercizio 16.3-4).
3. **La scelta golosa:** fondere ogni volta i due alberi di frequenza minima, con una coda di min-priorità; $n - 1$ fusioni, $O(n \lg n)$.
4. **Scelta golosa (Lemma 16.2):** le due lettere meno frequenti sono foglie sorelle a profondità massima in un albero ottimo. Scambio $a \leftrightarrow x$, $b \leftrightarrow y$: $B(T) - B(T') = (a.freq - x.freq)(d_T(a) - d_T(x)) \geq 0$.
5. **Sottostruttura (Lemma 16.3):** fusa la coppia in z , $B(T) = B(T') + x.freq + y.freq$; un albero migliore di T ne darebbe uno migliore di T' , contro l'ottimalità. Con il Lemma 16.2 si chiude il Teorema 16.4.
6. **L'esempio:** frequenze 45, 13, 12, 16, 9, 5; codici $a=0$, $b=101$, $c=100$, $d=111$, $e=1101$, $f=1100$; costo 224 000 bit contro 300 000.

9.4 Il problema del resto, e le fusioni ottimali

Il problema del resto. Formare un importo n con il minor numero di monete. Il goloso naturale: prendi ogni volta la moneta più grande che non supera il resto. Con i tagli 1, 5, 10, 25, e in generale con le potenze di una base $(1, c, c^2, \dots)$, funziona sempre [Cormen problema 16-1]. In generale però non è sicuro: con i tagli $\langle 1, 3, 4 \rangle$ e resto 6 il goloso prende $4 + 1 + 1$, tre monete, mentre l'ottimo è $3 + 3$, due monete [non nelle fonti del corso]. Per un insieme qualsiasi di tagli serve la programmazione dinamica (capitolo 8).

Quando il goloso funziona: il criterio di verifica

La scelta golosa è ottima per i sistemi di monete *canonici*: le monete reali (1, 5, 10, 25) e le potenze di una base (1, c , c^2 , ...); fuori, un controesempio basta a farla cadere. Il criterio per un sistema qualsiasi è il confronto esaustivo goloso contro tabella: il minimo $R[v]$ della programmazione dinamica per ogni somma $v = 1, \dots, V$ (capitolo 8) e il goloso devono dare sempre lo stesso numero di monete [non nelle fonti del corso]. Con 1, 5, 10, 25 il controllo fino a 99 non trova nessun fallimento; con $\langle 1, 3, 4 \rangle$ il primo importo sbagliato è 6 (l'esempio qui sopra). Il controllo vale fino al limite scelto: non è una dimostrazione.

Problemi di fusione iterativa a costo minimo

Il tipo di esercizio (prove dell'1/09/2026 e del 21/09/2026, svolte nel capitolo 13): n parti con pesi $p_1, \dots, p_n$; ogni operazione fonde due parti, al costo $p_a + p_b$; si vuole il **costo totale minimo**. L'istanza dell'1/09/2026 è $\langle 8, 3, 14, 6, 11, 5 \rangle$.

Il modello e il metodo. Una sequenza di fusioni è un **albero binario pieno** con le n parti alle foglie; ogni parte paga il proprio peso a ogni fusione in cui è coinvolta, e le fusioni che la coinvolgono sono gli antenati della sua foglia, quindi costo = $\sum p_i \cdot d_i$ con d_i profondità della foglia i : è *esattamente* la funzione costo (16.4) di Huffman, con i pesi al posto delle frequenze. Quindi: (1) metti le parti in una **coda di min-priorità** con chiave il peso; (2) estrai le due più piccole, fondile e somma $p_a + p_b$ al costo; (3) ripeti $n - 1$ volte. Tempo $O(n \lg n)$, spazio $O(n)$.

Perché è ottimo: è la scelta golosa di Huffman. La parte più leggera e la seconda più leggera sono foglie sorelle a profondità massima in qualche albero ottimo (Lemma 16.2, con lo stesso scambio), e dopo la fusione resta lo stesso problema sui pesi rimasti (Lemma 16.3).

L'istanza $\langle 8, 3, 14, 6, 11, 5 \rangle$: ordinata è 3, 5, 6, 8, 11, 14; le fusioni sono $3+5=8$ (costo 8), $6+8=14$ (14), $8+11=19$ (19), $14+14=28$ (28), $19+28=47$ (47); costo totale $8 + 14 + 19 + 28 + 47 = 116$. La forza bruta su tutte le sequenze di fusioni conferma che 116 è il minimo. Il 21/09/2026, sull'istanza $\langle 4, 7, 9, 12, 15, 18 \rangle$, si ottiene $11 + 20 + 27 + 38 + 65 = 161$.

Errori tipici con gli algoritmi golosi

- **Usare il goloso senza dimostrare la scelta**: “sembra migliore” non è una dimostrazione. Zaino 0/1: \$160 contro \$220; resto $\langle 1, 3, 4 \rangle$: tre monete contro due.
- **Confondere l'ottimo locale con l'ottimo globale**: la scelta è sicura solo se esiste un ottimo che la contiene; la prova è lo scambio.
- **Dimenticare la pre-elaborazione nei tempi**: attività ordinate per tempo di fine (o $O(n \lg n)$ dichiarato), frequenze di Huffman in una coda di min-priorità.
- **Nel metodo delle fusioni, prendere coppie qualsiasi** (“i primi due della lista”): si fondono i *due più piccoli*, come in Huffman.

Capitolo 10

Visite dei grafi e ordinamento topologico

Un **grafo** $G = (V, E)$ è un insieme di **vertici** V e di **archi** E : *orientato* se ogni arco è una coppia ordinata (u, v) , *non orientato* se è una coppia non ordinata $\{u, v\}$; un **cammino** da u a v è una sequenza di vertici in cui ogni passo è un arco, *semplice* se non ripete vertici; un **ciclo** è un cammino semplice che torna al punto di partenza, e un grafo orientato aciclico è un **dag**. La **visita in ampiezza** (BFS) procede a strati e dà le distanze in numero di archi; la **visita in profondità** (DFS) scava il più possibile prima di tornare indietro e rivela la struttura del grafo: foresta, tempi, tipi di arco. [Cormen cap. 22]

10.1 Rappresentare un grafo: liste e matrice di adiacenza

L'idea: due modi di memorizzare gli archi

Per ogni vertice si annota con chi è collegato; oppure si scrive una tabella $|V| \times |V|$ con una casella per arco, che spreca spazio ma risponde subito a « u è collegato a v ?».

Liste di adiacenza [Cormen cap. 22.1]

La rappresentazione con **liste di adiacenza** di G è un array Adj di $|V|$ liste: $Adj[u]$ contiene tutti i v con $(u, v) \in E$ (nel libro è l'attributo $G.Adj[u]$). Se G è orientato la somma delle lunghezze delle liste è $|E|$ (ogni arco è in una lista sola); se non è orientato è $2|E|$, perché $\{u, v\}$ compare in $Adj[u]$ e in $Adj[v]$. In entrambi i casi la memoria è $\Theta(V + E)$. Nei **grafi pesati** il peso $w(u, v)$ si memorizza accanto a v nella lista di u . Nel grafo della figura 22.1 del libro (cinque vertici, sette archi) le liste sono $Adj[1] = (2, 5)$, $Adj[2] = (1, 5, 3, 4)$, ..., $Adj[5] = (4, 1, 2)$: quattordici elementi, $2|E|$.

Matrice di adiacenza [Cormen cap. 22.1]

Si numerano i vertici $1, \dots, |V|$ e si usa una matrice $A = (a_{ij})$ di dimensioni $|V| \times |V|$: $a_{ij} = 1$ se $(i, j) \in E$, altrimenti $a_{ij} = 0$. La memoria è $\Theta(V^2)$, indipendente dagli archi; in un grafo non orientato $A = A^T$ (simmetrica); nei grafi pesati al posto di 1 si scrive $w(i, j)$, e 0 o NIL se l'arco non c'è. Conviene per i grafi *densi* o per sapere *subito* se un arco esiste: un test da $O(1)$ invece della scansione di $Adj[u]$. Per il grafo della figura 22.1 la matrice (colonne 1 ... 5) ha quattordici uni, quattro nella sola riga 2: il vertice 2 ha grado 4.

Per il **grafo orientato** della figura 22.2 del libro (sei vertici e otto archi, con il cappio $6 \rightarrow 6$) le liste hanno in tutto $|E| = 8$ elementi e la matrice non è simmetrica. Il **trasposto** G^T ha gli archi con il verso invertito, si costruisce in $O(V + E)$ e serve per le componenti fortemente connesse (Paragrafo 10.5). [Cormen cap. 22.1, esercizio 22.1-3]

10.2 La visita in ampiezza (BFS)

Ripasso: la visita in ampiezza (BFS) [Cormen, cap. 22.2]

La BFS scopre i vertici *a strati*: una coda tiene la *frontiera* fra i vertici GRAY (scoperti, in coda) e quelli WHITE (da scoprire); i BLACK hanno la lista già esaminata. Ogni vertice è scoperto una volta sola, dal primo che lo incontra, e riceve la distanza $u.d$ dalla sorgente e il predecessore $u.\pi$; gli archi $(u.\pi, u)$ formano l'**albero BF**. Se $\delta(s, v)$ è il minimo numero di

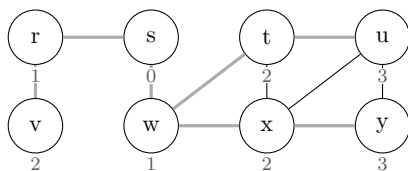


Figura 10.1: La figura 22.3 del libro alla fine della $\text{BFS}(G, s)$: sotto ogni vertice la distanza $d[u]$, in grigio gli archi dell'albero BF. [Cormen, fig. 22.3]

archi di un cammino da s a v (∞ se non è raggiungibile), al termine $v.d = \delta(s, v)$ (Teorema 22.5); la BFS costa $\Theta(V + E)$: ogni vertice entra ed esce dalla coda una volta e ogni lista è scandita una volta. Nel grafo della figura 22.3 i predecessori sono $r.\pi = w.\pi = s$, $v.\pi = r$, $t.\pi = x.\pi = w$, $u.\pi = t$, $y.\pi = x$, e la coda vale $\langle s \rangle$, $\langle w, r \rangle$, $\langle r, t, x \rangle$, $\langle t, x, v \rangle$, $\langle x, v, u \rangle$, $\langle v, u, y \rangle$, $\langle u, y \rangle$, $\langle y \rangle$, $\langle \rangle$ a ogni iterazione.

La correttezza si appoggia su tre fatti: per ogni arco $\delta(s, v) \leq \delta(s, u) + 1$ (Lemma 22.1); $v.d \geq \delta(s, v)$ per induzione sugli accodamenti (Lemma 22.2); in coda i valori d non decrescono e differiscono al più di 1 (Lemma 22.3) — è questo che rende la visita «in ampiezza». [Cormen, cap. 22.2]

10.3 La visita in profondità (DFS)

L'idea: scava, poi torna indietro

Dall'ultimo vertice scoperto che ha ancora archi da esaminare si prosegue in profondità; quando non resta nulla si torna indietro (lo schema delle chiamate ricorsive). Finché ci sono vertici bianchi la DFS ne sceglie uno e riparte. Ogni vertice riceve due tempi, *scoperta* e *completamento*, che letti come parentesi raccontano l'annidamento della visita.

$\text{DFS}(G)$

```

1  for ogni  $u \in G.V$ 
2       $u.\text{color} := \text{WHITE}$ 
3       $u.\pi := \text{NIL}$ 
4   $\text{time} := 0$ 
5  for ogni  $u \in G.V$ 
6      if  $u.\text{color} == \text{WHITE}$ 
7           $\text{DFS-VISIT}(G, u)$ 
```

$\text{DFS-VISIT}(G, u)$

```

1   $\text{time} := \text{time} + 1$ 
2   $u.d := \text{time}$ 
3   $u.\text{color} := \text{GRAY}$ 
4  for ogni  $v \in G.\text{Adj}[u]$ 
5      if  $v.\text{color} == \text{WHITE}$ 
6           $v.\pi := u$ 
7           $\text{DFS-VISIT}(G, v)$ 
8   $u.\text{color} := \text{BLACK}$ 
9   $\text{time} := \text{time} + 1$ 
10  $u.f := \text{time}$ 
```

I tempi $u.d$ e $u.f$ sono interi fra 1 e $2|V|$ con $u.d < u.f$; u è WHITE prima di $u.d$, GRAY fra $u.d$ e $u.f$ (la lista è in esame), BLACK dopo. Il sottografo dei predecessori è la **foresta DF** (un albero per sorgente) e i suoi archi sono gli **archi d'albero**. Ogni lista è scandita una volta: la DFS costa $\Theta(V + E)$. [Cormen cap. 22.3]

Teorema 22.7 (teorema delle parentesi)

da saper dimostrare

In una visita in profondità di un grafo $G = (V, E)$, per ogni coppia di vertici u e v vale una sola delle tre condizioni:

1. $[u.d, u.f]$ e $[v.d, v.f]$ sono disgiunti, e u e v non sono discendenti l'uno dell'altro nella

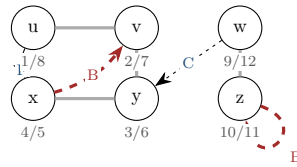


Figura 10.2: La figura 22.4 del libro: la DFS del grafo orientato u, v, w, x, y, z (ordine alfabetico) con i tempi d/f sotto ogni vertice. Archi d'albero in grigio spesso, gli altri tratteggiati: B all'indietro (rossi), F in avanti, C trasversale; la foresta ha due alberi (radici u e w). [Cormen fig. 22.4]

foresta DF;

2. $[u.d, u.f]$ è contenuto in $[v.d, v.f]$, e u è un discendente di v ;
3. $[v.d, v.f]$ è contenuto in $[u.d, u.f]$, e v è un discendente di u .

Dimostrazione. Supponiamo $u.d < v.d$. Se $v.d < u.f$, v è stato scoperto mentre u era grigio (quindi è un discendente di u) e, scoperto più recentemente, si completa prima di u : $v.f < u.f$, cioè $[v.d, v.f]$ sta dentro $[u.d, u.f]$. Se invece $v.d > u.f$, da $u.d < u.f < v.d < v.f$ segue che gli intervalli sono disgiunti e nessuno dei due è discendente dell'altro (né è stato scoperto mentre l'altro era grigio). Il caso $v.d < u.d$ è simmetrico. Di conseguenza (Corollario 22.8) v è un discendente proprio di u se e soltanto se $u.d < v.d < v.f < u.f$. $\square$

Teorema 22.9 (teorema del cammino bianco)

da saper dimostrare

Nella foresta DF di un grafo $G = (V, E)$, v è un discendente di u se e soltanto se, al tempo $u.d$ in cui u viene scoperto, v è raggiungibile da u lungo un cammino formato solo da vertici bianchi.

Dimostrazione. ($\Rightarrow$) Se $v = u$ il cammino è il solo u , ancora bianco quando si fissa $u.d$; se v è un discendente proprio, per il Corollario 22.8 $u.d < v.d$ (v è bianco) e i vertici del cammino da u a v nella foresta DF sono antenati di v , scoperti non prima di u : bianchi anche loro.

($\Leftarrow$) Supponiamo che al tempo $u.d$ ci sia un cammino di vertici bianchi da u a v ma che v non diventi discendente di u . Prendiamo il v più vicino a u con questa proprietà e sia w il suo predecessore sul cammino: w è un discendente di u (eventualmente $w = u$), quindi $w.f \leq u.f$. Il vertice v , adiacente a w , è ancora bianco quando w lo esamina: è scoperto durante la visita di w , cioè $u.d < v.d < w.f \leq u.f$. Per il Teorema 22.7 $[v.d, v.f]$ è dentro $[u.d, u.f]$ e, per il Corollario 22.8, v è un discendente di u : contraddizione. $\square$

10.3.1 Classificare gli archi

I quattro tipi di arco, letti dal colore [Cormen cap. 22.3]

In una visita in profondità di un grafo *orientato*, quando l'arco (u, v) è ispezionato: se v è WHITE è un **arco d'albero** (è l'arco con cui v viene scoperto); se v è GRAY è un **arco all'indietro** (va a un antenato; i cappi, come $z \rightarrow z$, sono all'indietro); se v è BLACK è **in avanti** quando $u.d < v.d$ (verso un discendente) e **trasversale** quando $u.d > v.d$ (verso un vertice già completato). I tempi dicono lo stesso fatto (Esercizio 22.3-5): (u, v) è d'albero o in avanti se e solo se $u.d < v.d < v.f < u.f$; è all'indietro se e solo se $v.d \leq u.d < u.f \leq v.f$; è trasversale se e solo se $v.d < v.f < u.d < u.f$.

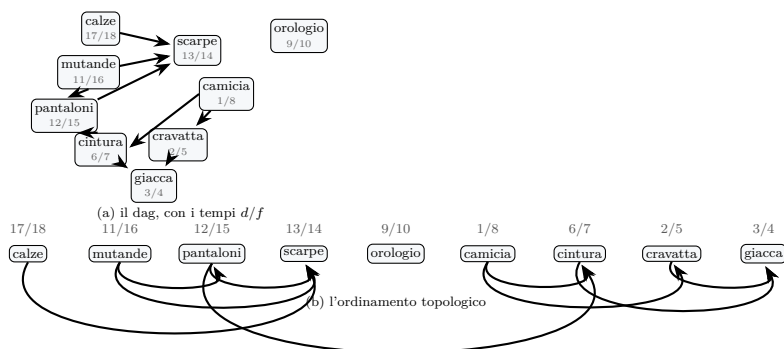


Figura 10.3: La figura 22.7 del libro: il dag del professor Bumstead, che si veste. (a) Con i tempi d/f della DFS; (b) l'ordinamento topologico che ne risulta, in ordine decrescente di completamento: calze, mutande, pantaloni, scarpe, orologio, camicia, cintura, cravatta, giacca (tutti gli archi vanno da sinistra a destra). L'ordine non è unico (qui ne esistono 1728). [Cormen fig. 22.7]

Teorema 22.10

da saper dimostrare

In una visita in profondità di un grafo *non orientato* gli archi possono essere solo d'albero o all'indietro.

Dimostrazione. Sia $\{u, v\}$ un arco, con $u.d < v.d$: v è scoperto e completato mentre u è grigio. Se l'arco è ispezionato prima da u verso v , v è ancora bianco — altrimenti sarebbe già stato ispezionato nell'altro verso — e l'arco è d'albero; se prima da v verso u , u è ancora grigio ed è all'indietro. Per un arco non orientato si guarda la prima volta che lo si incontra, in un verso o nell'altro. $\square$

10.4 L'ordinamento topologico

L'idea: mettere i vertici in fila indiana

Un dag descrive precedenze: l'arco $u \rightarrow v$ significa « u prima di v », come le calze prima delle scarpe. Un *ordinamento topologico* elenca i vertici in modo che *tutti* gli archi vadano da sinistra a destra: la DFS lo produce gratis, con i tempi di completamento decrescenti. Se il grafo ha un ciclo, una fila così non esiste.

Ordinamento topologico [Cormen cap. 22.4]

Un **ordinamento topologico** di un dag $G = (V, E)$ è un ordinamento lineare di tutti i vertici tale che, se $(u, v) \in E$, allora u precede v ; esiste se e solo se il grafo è aciclico (un ciclo chiederebbe a un vertice di precedere se stesso).

TOPOLOGICAL-SORT(G)

- 1 chiama DFS(G) per calcolare i tempi di completamento $v.f$ di ogni vertice v
- 2 quando la visita di un vertice è completata, inseriscilo in testa a una lista concatenata
- 3 **return** la lista concatenata dei vertici

Lemma 22.11

da saper dimostrare

Un grafo orientato G è aciclico se e soltanto se una visita in profondità di G non genera archi all'indietro.

Dimostrazione. ($\Rightarrow$) Se ci fosse un arco all'indietro (u, v) , v sarebbe un antenato di u : esiste un cammino da v a u e l'arco (u, v) lo chiuderebbe in un ciclo.

($\Leftarrow$) Se G contiene un ciclo c , sia v il primo vertice di c scoperto e (u, v) l'arco che precede v in c : al tempo $v.d$ i vertici di c formano un cammino di vertici bianchi da v a u , quindi per il teorema del cammino bianco u è un discendente di v e (u, v) è all'indietro. $\square$

Teorema 22.12: la correttezza

da saper dimostrare

TOPOLOGICAL-SORT(G) produce un ordinamento topologico di un dag G .

Dimostrazione. Basta dimostrare che per ogni arco (u, v) si ha $v.f < u.f$. Quando la DFS ispeziona (u, v) , v non può essere grigio (sarebbe un antenato di u e l'arco sarebbe all'indietro, contro il Lemma 22.11): è bianco o nero. Se è bianco, diventa un discendente di u e $v.f < u.f$ per il Corollario 22.8. Se è nero, la sua visita è già completata e $v.f$ è fissato, mentre quella di u non è ancora finita: quando le verrà assegnato $u.f$ sarà ancora $v.f < u.f$. $\square$

Il costo è quello della DFS più un inserimento in testa $O(1)$ per vertice: $\Theta(V + E)$. [Cormen p. 513]

10.5 Le componenti fortemente connesse

Due vertici di un grafo orientato sono *fortemente connessi* se sono raggiungibili l'uno dall'altro; le classi sono le **componenti fortemente connesse**, che si trovano con due visite: DFS(G), poi DFS(G^T) sul trasposto, in ordine decrescente di tempo di completamento. [Cormen, cap. 22.5]

10.6 Come si risolve all'esame

Scrivere una procedura ricorsiva su un dag, senza parametri aggiuntivi

La procedura ha una firma fissata — per esempio SAME-TERMINAL(M, x, n) — e deve restituire *un intero* da cui si capisca se la proprietà vale. Il metodo:

1. **Che cosa ritorna:** la risposta *per il sotto-dag di radice x* , non per tutto il grafo: in SAME-TERMINAL il nodo terminale comune ai cammini da x , in SAME-COST il loro costo.
2. **Il caso base:** se x non ha archi uscenti (la riga x di M è tutta zero) è esso stesso terminale: SAME-TERMINAL ritorna x , SAME-COST ritorna 0 (cammino vuoto).
3. **Il valore per «non vale»:** un intero che non può mai essere una risposta vera, per esempio -1 (i nodi sono $1..n$, i costi positivi): appena un figlio ritorna -1 , tutto ritorna -1 .
4. **Come si combinano i figli:** richiama la ricorsione su *ogni* figlio y con $M[x][y] \neq 0$; il primo «propone» la risposta e gli altri devono *confermarla*; per il costo il peso si aggiunge *dopo* la chiamata ($M[x][y] + \text{SAME-COST}(M, y, n)$). È corretto perché ogni cammino da x è un arco più un cammino da un figlio, e il dag non ha cicli: termina.

Esempio (istanze della guida, [non nelle fonti del corso]; gli svolgimenti dei temi sono nel capitolo 13). Con gli archi $1 \rightarrow 2$ (peso 2), $1 \rightarrow 3$ (1), $2 \rightarrow 4$ (1), $3 \rightarrow 4$ (2) vale SAME-TERMINAL($M, 1, 4$) = 4 e SAME-COST($M, 1, 4$) = 3; con un arco $4 \rightarrow 5$ (1) e il peso di $3 \rightarrow 4$ portato a 3, il terminale resta lo stesso (5) ma i costi no (4 contro 5): SAME-COST = -1 .

Eseguire BFS, DFS e ordinamento topologico su un grafo

1. **BFS:** fissa la sorgente e l'ordine delle liste (se non è detto, l'alfabetico), poi scrivi la tabella delle iterazioni: vertice estratto, vertici scoperti con la loro d , coda. Controllo: lungo la coda le d non decrescono e crescono al più di 1.
2. **DFS:** $u.d$ si assegna quando colori u di grigio, $u.f$ quando la sua lista è finita (e u diventa nero): scrivi d/f accanto a ogni vertice e il tipo di ogni arco (bianco T , grigio B , nero F se $u.d < v.d$ altrimenti C).

3. **Ordinamento topologico:** controlla che sia un dag (una DFS senza archi all'indietro), elenca i vertici in ordine decrescente di f (o toglie ripetutamente un vertice di grado entrante 0: Esercizio 22.4-5), poi verifica che ogni arco (u, v) abbia u prima di v (l'ordine non è unico: la figura 10.3 ne ammette 1728).

Come rispondere: «BFS, DFS e ordinamento topologico»

1. **BFS:** scopre i vertici a strati con una coda; $u.d = \delta(s, u)$ (minimo numero di archi), perché $\delta(s, v) \leq \delta(s, u) + 1$ su ogni arco e i valori d nella coda non decrescono; l'albero BF è dato dai predecessori; costo $\Theta(V + E)$.
2. **DFS:** ricorsiva (o con una pila); $1 \leq u.d < u.f \leq 2|V|$; i predecessori formano la foresta DF, costo $\Theta(V + E)$. Proprietà da dimostrare: parentesi, cammino bianco, classificazione degli archi.
3. **Orientato o no:** in un non orientato solo archi d'albero e all'indietro, ognuno classificato una volta sola; in un orientato un arco all'indietro segnala un ciclo (Lemma 22.11).
4. **Ordinamento topologico:** esiste se e solo se il grafo è aciclico; i vertici in testa alla lista quando la DFS li completa (ordine decrescente di f), costo $\Theta(V + E)$; correttezza: Teorema 22.12.

Tempi della DFS, grafi orientati e non

- **$u.d$ vuol dire due cose diverse:** nella BFS è la *distanza* dalla sorgente (numero di archi); nella DFS è il *tempo di scoperta*, e $u.f$ non è una distanza ma il momento in cui la lista di u è finita. I tempi sono interi fra 1 e $2|V|$, con $u.d < u.f$, e dipendono dall'ordine di visita.
- **Grafo non orientato:** niente archi in avanti né trasversali (Teorema 22.10), e ogni arco si classifica *una volta sola*, alla prima ispezione; il Lemma 22.11 vale solo per gli *orientati*.
- **Niente ordinamento topologico con un ciclo:** se la DFS trova un arco all'indietro l'ordinamento non esiste; quando esiste non è unico (figura 10.3). E la BFS dà cammini minimi solo *in numero di archi* (capitoli 11 e 12 per i pesi).

Capitolo 11

Cammini minimi da sorgente unica

Un grafo orientato *pesato* ha un peso reale $w(u, v)$ su ogni arco (distanza, tempo, costo, ricavo). Il problema dei **cammini minimi da sorgente unica** chiede, da una sorgente s a *ogni* altro vertice, il cammino di peso totale minore — come chi va da Phoenix a Indianapolis sulla carta stradale: elencare tutte le strade è impensabile, servono le stime.

L'idea: non elencare i cammini, migliorare le stime

Per ogni vertice v teniamo *una sola* stima $v.d$ del cammino minimo dalla sorgente, e la *miglioriamo* un arco per volta: se il cammino migliore per u costa $u.d$ e c'è l'arco (u, v) , passare per u costa $u.d + w(u, v)$; se è meno di $v.d$, si aggiorna. I tre algoritmi del capitolo differiscono solo per *quante volte* e in quale *ordine* guardano gli archi; i campi $v.\pi$ ricordano da dove arriva ogni stima.

Peso di un cammino e δ [Cormen cap. 24.1]

Sia $G = (V, E)$ un grafo orientato pesato con funzione peso $w : E \rightarrow \mathbb{R}$. Il **peso di un cammino** $p = \langle v_0, v_1, \dots, v_k \rangle$ è $w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$. Il **peso di un cammino minimo** da u a v è $\delta(u, v) = \min\{w(p) : u \xrightarrow{p} v\}$ se esiste un cammino da u a v , altrimenti $\delta(u, v) = \infty$; un **cammino minimo** è un p con $w(p) = \delta(u, v)$.

Con le frecce invertite si risolve il problema «verso una destinazione unica», il cammino fra due vertici è il caso u sorgente, e il problema fra *tutte le coppie* è del capitolo 12; sui grafi non pesati δ è la distanza della visita in ampiezza [Cormen, cap. 22.2].

11.1 Le proprietà dei cammini minimi

Un cammino minimo è fatto di cammini minimi: percorrerlo a pezzi, ogni pezzo è il modo migliore di andare dal suo inizio alla sua fine. È la *sottostruttura ottima*, la stessa proprietà che regge la programmazione dinamica (capitolo 8).

Lemma 24.1 — i sottocammini di cammini minimi sono cammini minimi *da saper dimostrare*

Dato un grafo orientato pesato $G = (V, E)$ con funzione peso $w : E \rightarrow \mathbb{R}$, sia $p = \langle v_0, v_1, \dots, v_k \rangle$ un cammino minimo da v_0 a v_k e, per $0 \leq i \leq j \leq k$, sia $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ il sottocammino di p da v_i a v_j . Allora p_{ij} è un cammino minimo da v_i a v_j . [Cormen cap. 24.1]

Dimostrazione. Scomponendo p in $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p_{ij}} v_j \xrightarrow{p_{jk}} v_k$ si ha $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk})$. Se esistesse un cammino p'_{ij} con $w(p'_{ij}) < w(p_{ij})$, incollandolo al posto di p_{ij} si otterrebbe un cammino da v_0 a v_k di peso $w(p_{0i}) + w(p'_{ij}) + w(p_{jk}) < w(p)$, contro l'ipotesi che p sia minimo: è il solito taglia-e-incolla. $\square$

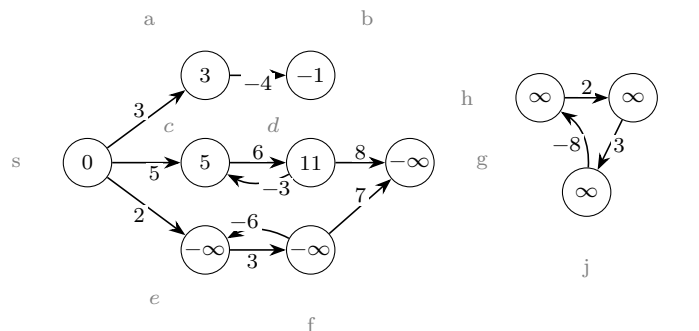


Figura 11.1: La Figura 24.1 del libro: dentro ogni vertice $\delta(s, \cdot)$. $\langle c, d \rangle$ pesa $3 > 0$ ($\delta(s, c) = 5$, $\delta(s, d) = 11$); $\langle e, f \rangle$ pesa $-3 < 0$, quindi $\delta(s, e) = \delta(s, f) = \delta(s, g) = -\infty$; h, i, j hanno un ciclo negativo ma non sono raggiungibili da s . [Cormen fig. 24.1]

Archi di peso negativo e cicli negativi

Se il grafo non contiene cicli di peso negativo raggiungibili da s , $\delta(s, v)$ resta definito *anche con archi negativi*; se un ciclo negativo è raggiungibile da s , invece, i pesi dei cammini minimi non sono più definiti e si scrive $\delta(s, v) = -\infty$ (percorso il ciclo quante volte si vuole, il peso cala senza limite).

Un cammino minimo è sempre **semplice**: un ciclo positivo all'interno si toglie e il peso cala; un ciclo negativo non può starci (il minimo non esisterebbe); i cicli di peso 0 si eliminano senza cambiare il peso. Quindi un cammino minimo ha al più $|V| - 1$ archi: è il numero dei passaggi di Bellman-Ford.

Predecessori e albero di cammini minimi [Cormen cap. 24.1]

Come nella visita in ampiezza, ogni vertice ha un **predecessore** $v.\pi$ (un altro vertice oppure NIL); il **sottografo dei predecessori** $G_\pi = (V_\pi, E_\pi)$ ha $V_\pi = \{v : v.\pi \neq \text{NIL}\} \cup \{s\}$ e $E_\pi = \{(v.\pi, v) \in E : v \in V_\pi - \{s\}\}$. Un **albero di cammini minimi** radicato in s è un sottografo $G' = (V', E')$ con: (1) $V' =$ i vertici raggiungibili da s ; (2) G' è un albero con radice s ; (3) per ogni $v \in V'$, l'unico cammino semplice $s \rightsquigarrow v$ in G' è un cammino minimo in G . Gli algoritmi impostano i π in modo che alla fine G_π sia un tale albero: la catena dei predecessori da v risale un cammino minimo, che PRINT-PATH stampa [Cormen, cap. 22.2].

11.2 Il rilassamento

Rilassare un arco: provare a migliorare una stima

$v.d$ è un *limite superiore* per $\delta(s, v)$: parte da ∞ (o 0 per la sorgente) e scende man mano. Rilassare l'arco (u, v) significa chiedersi se $v.d > u.d + w(u, v)$: se sì, la stima migliora e si annota da dove è arrivata ($v.\pi := u$). Il nome è storico: è il "rilassamento" del vincolo $v.d \leq u.d + w(u, v)$, che per la disuguaglianza triangolare *deve* valere quando le stime sono definitive.

INITIALIZE-SINGLE-SOURCE(G, s)

```

1  for ogni vertice  $v \in G.V$ 
2       $v.d := \infty$ 
3       $v.\pi := \text{NIL}$ 
4   $s.d := 0$ 
```

RELAX(u, v, w)

```

1  if  $v.d > u.d + w(u, v)$ 
2       $v.d := u.d + w(u, v)$ 
3       $v.\pi := u$ 
```

Dopo l'inizializzazione $s.d = 0$, $v.d = \infty$ e $v.\pi = \text{NIL}$ per $v \neq s$. Da lì in poi l'*unico* modo di cambiare stime e predecessori è un passo di rilassamento; i tre algoritmi differiscono solo per quante

volte e in che ordine chiamano RELAX: una volta per arco (grafi aciclici, Dijkstra) oppure $|V| - 1$ volte (Bellman-Ford).

Le proprietà di cammini minimi e rilassamento [Cormen cap. 24.1]

Dopo INITIALIZE-SINGLE-SOURCE, con cambiamenti solo per rilassamenti (i lemmi sono nel 24.1, dimostrati nel 24.6 [Cormen, cap. 24.6]):

- **Disuguaglianza triangolare** (Lemma 24.10): $\delta(s, v) \leq \delta(s, u) + w(u, v)$ per ogni arco $(u, v) \in E$.
- **Limite superiore** (Lemma 24.11): $v.d \geq \delta(s, v)$ sempre e, raggiunto $\delta(s, v)$, non cala più; chi non è raggiungibile resta a ∞ (Cor. 24.12).
- **Convergenza** (Lemma 24.14): se $s \xrightarrow{p} u \rightarrow v$ è un cammino minimo e $u.d = \delta(s, u)$ *prima* del rilassamento di (u, v) , da lì in poi $v.d = \delta(s, v)$.
- **Rilassamento del cammino** (Lemma 24.15): se gli archi di un cammino minimo $\langle v_0, \dots, v_k \rangle$ vengono rilassati nell'ordine, allora $v_k.d = \delta(s, v_k)$ — anche con altri rilassamenti in mezzo.
- **Sottografo dei predecessori** (Lemma 24.17): se $v.d = \delta(s, v)$ per tutti i v , G_π è un albero di cammini minimi radicato in s .

11.3 L'algoritmo di Bellman-Ford

Rilassare tutti gli archi, $|V| - 1$ volte

Se un passaggio rilassa *tutti* gli archi, il k -esimo sistema i vertici di cammino minimo con k archi (rilassamento del cammino): dopo $|V| - 1$ passaggi le stime sono definitive, e se c'è un ciclo negativo raggiungibile il controllo finale se ne accorge.

BELLMAN-FORD(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i := 1$  to  $|G.V| - 1$ 
3      for ogni arco  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for ogni arco  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```

Restituisce TRUE se e soltanto se non ci sono cicli di peso negativo raggiungibili dalla sorgente; in quel caso le stime sono i $\delta(s, v)$ e G_π è un albero di cammini minimi. Costo: $O(VE)$; con il grafo in **matrice di adiacenza** ogni passaggio scandisce le $|V|^2$ celle (anche le vuote), e il costo diventa $\Theta(V^3)$ [simulazione 22/01/2026, E6].

Il grafo della Figura 24.4, sorgente s , archi rilassati nell'ordine (t, x) , (t, y) , (t, z) , (x, t) , (y, x) , (y, z) , (z, x) , (z, s) , (s, t) , (s, y) : le stime dopo ogni passaggio sono nella tabella (il controllo finale, un giro in più, non cambia nulla).

passo	s	t	x	y	z
inizio	0	∞	∞	∞	∞
1	0	6	∞	7	∞
2	0	6	4	7	2
3	0	2	4	7	2
4	0	2	4	7	-2

Passaggi chiave: al secondo (t, z) porta z a $6 - 4 = 2$ e (y, x) migliora x a $7 - 3 = 4$; al terzo (x, t) porta t a 2; al quarto (t, z) porta z a $2 - 4 = -2$.

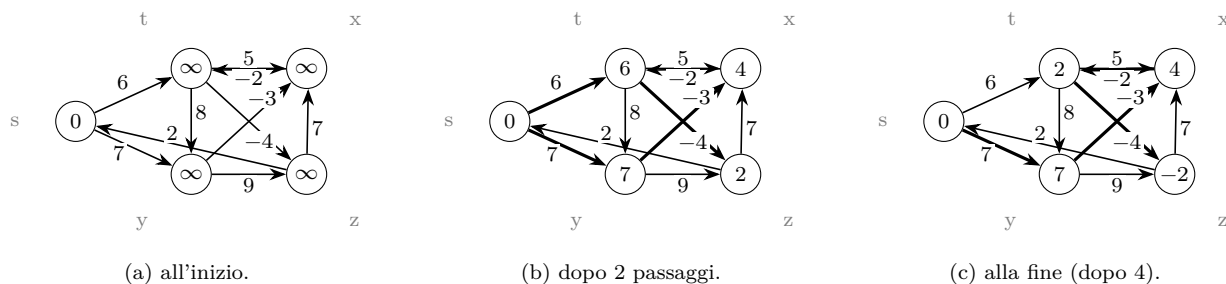


Figura 11.2: La Figura 24.4 del libro (pannelli (a), (c) ed (e)), sorgente s : (a) all'inizio, (b) dopo 2 passaggi, (c) alla fine (archi spessi = $v.\pi$). [Cormen fig. 24.4]

Lemma 24.2 — la convergenza

da saper dimostrare

Se il grafo pesato $G = (V, E)$ con sorgente s non contiene cicli di peso negativo raggiungibili da s , allora dopo $|V| - 1$ iterazioni del ciclo for delle righe 2–4 di BELLMAN-FORD si ha $v.d = \delta(s, v)$ per tutti i vertici v raggiungibili da s . [Cormen cap. 24.2]

Dimostrazione. Applichiamo la proprietà del rilassamento del cammino. Sia v raggiungibile e sia $p = \langle v_0, \dots, v_k \rangle$, con $v_0 = s$ e $v_k = v$, un cammino minimo aciclico; p ha al più $|V| - 1$ archi, quindi $k \leq |V| - 1$. Ciascuna delle $|V| - 1$ iterazioni rilassa tutti gli $|E|$ archi: nell' i -esima iterazione, per $i = 1, \dots, k$, c'è l'arco (v_{i-1}, v_i) . Per la proprietà del rilassamento del cammino, $v.d = v_k.d = \delta(s, v_k) = \delta(s, v)$. $\square$

Teorema 24.4 — correttezza dell'algoritmo di Bellman-Ford

da saper dimostrare

Eseguito su un grafo orientato pesato $G = (V, E)$ con sorgente s e funzione peso $w : E \rightarrow \mathbb{R}$, BELLMAN-FORD restituisce TRUE, con $v.d = \delta(s, v)$ per tutti i v e G_π albero di cammini minimi, se e soltanto se G non contiene cicli di peso negativo raggiungibili da s (se ce n'è uno, restituisce FALSE). [Cormen cap. 24.2]

Dimostrazione. (a) *Nessun ciclo negativo.* Prima asserzione: $v.d = \delta(s, v)$ per tutti i v (Lemma 24.2 per i raggiungibili, assenza di un cammino per gli altri), e allora G_π è un albero di cammini minimi. Per ogni arco $(u, v) \in E$: $v.d = \delta(s, v) \leq \delta(s, u) + w(u, v) = u.d + w(u, v)$ (triangolare): nessun controllo della riga 6 restituisce FALSE.

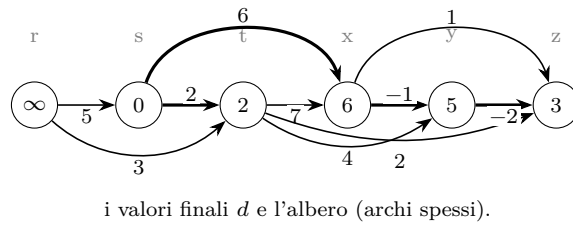
(b) *Un ciclo negativo raggiungibile.* Sia $c = \langle v_0, \dots, v_k \rangle$, $v_0 = v_k$, un tale ciclo, di peso $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$: i suoi vertici sono raggiungibili, quindi dopo $|V| - 1$ passaggi hanno stime finite (Corollario 24.3). Se l'algoritmo restituisse TRUE, sommando $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$ lungo il ciclo e usando $v_0 = v_k$ (le due sommatorie di stime hanno gli stessi addendi) resterebbe $0 \leq \sum_{i=1}^k w(v_{i-1}, v_i) < 0$: contraddizione. $\square$

Eeguire Bellman-Ford a mano, con la tabella

1. **Fissa l'ordine degli archi** (lo dice il libro o il tema; se no, scegline uno e dichiaralo): ogni passaggio li rilassa tutti.
2. **Tabella:** una riga per l'inizializzazione ($s.d = 0$, $v.d = \infty$, $\pi = \text{NIL}$) e una per ognuno dei $|V| - 1$ passaggi; colonne = vertici. In ogni RELAX, se $v.d > u.d + w(u, v)$ aggiorna *sia* $v.d$ *sia* $v.\pi$; ricorda che $\infty + w = \infty$: da ∞ non si migliora.
3. **Controllo finale:** rifai un giro; se un arco si rilassa ancora c'è un ciclo negativo raggiungibile (FALSE), altrimenti TRUE. Le frecce $v.\pi \rightarrow v$ sono l'albero dei cammini minimi.

Se ti chiedono: “dimostra che Bellman-Ford è corretto”

1. **Cosa fa:** rilassa tutti gli archi $|V| - 1$ volte, poi un giro di controllo; se un arco è ancora rilassabile, FALSE.



	r	s	t	x	y	z
inizio	∞	0	∞	∞	∞	∞
dopo s	∞	0	2	6	∞	∞
dopo t	∞	0	2	6	6	4
dopo x	∞	0	2	6	5	4
dopo y	∞	0	2	6	5	3

(dopo r e dopo z non cambia nulla).

Figura 11.3: La Figura 24.5 del libro, sorgente s , ordine topologico $\langle r, s, t, x, y, z \rangle$; x migliora y ($6 - 1 = 5$) e y migliora z ($5 - 2 = 3$). [Cormen fig. 24.5]

2. **Convergenza:** se il cammino minimo verso v ha k archi, il k -esimo passaggio ne rilassa l'ultimo (rilassamento del cammino): da lì $v.d = \delta(s, v)$. I cammini minimi hanno al più $|V| - 1$ archi, e dopo $|V| - 1$ passaggi tutto è definitivo.
3. **Controllo finale:** se nessun arco si rilassa, $v.d \leq u.d + w(u, v)$ per ogni arco e G_π è un albero di cammini minimi (Lemma 24.17); se invece un arco si rilassa, sommando le disuguaglianze lungo il ciclo si otterrebbe $0 \leq \text{peso del ciclo} < 0$: il FALSE segnala un vero ciclo negativo ($\delta = -\infty$). Complessità: $O(VE)$, oppure $\Theta(V^3)$ con la matrice.

11.4 Cammini minimi nei grafi orientati aciclici

Su un DAG basta un passaggio, in ordine topologico

In un grafo aciclico non ci sono cicli negativi: i cammini minimi esistono sempre, anche con pesi negativi. In *ordine topologico* (capitolo 10) ogni cammino minimo viene incontrato arco dopo arco in un solo passaggio: quando tocca a u , $u.d$ è già definitiva.

DAG-SHORTEST-PATHS(G, w, s)

- 1 dispone in ordine topologico i vertici di G
- 2 INITIALIZE-SINGLE-SOURCE(G, s)
- 3 **for** ogni vertice u , preso in ordine topologico
- 4 **for** ogni vertice $v \in G.Adj[u]$
- 5 RELAX(u, v, w)

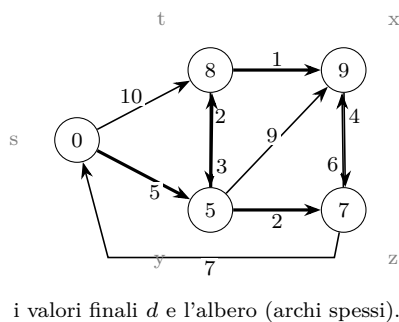
Tempo $\Theta(V + E)$: l'ordinamento topologico costa $\Theta(V + E)$, l'inizializzazione $\Theta(V)$, e ogni arco è rilassato una volta sola.

Teorema 24.5 — correttezza per i DAG

da saper dimostrare

Se il grafo orientato pesato $G = (V, E)$ ha sorgente s e nessun ciclo, al termine di DAG-SHORTEST-PATHS si ha $v.d = \delta(s, v)$ per tutti i v e G_π è un albero di cammini minimi. [Cormen cap. 24.3]

Dimostrazione. Se v non è raggiungibile da s , $v.d = \delta(s, v) = \infty$ (assenza di un cammino). Altrimenti esiste un cammino minimo $p = \langle v_0, \dots, v_k \rangle$, $v_0 = s$, $v_k = v$: in ordine topologico ogni arco di p ha la sorgente prima della destinazione, quindi gli archi di p vengono rilassati nell'ordine giusto $(v_0, v_1), \dots, (v_{k-1}, v_k)$, e per la proprietà del rilassamento del cammino $v_i.d = \delta(s, v_i)$ per ogni i ; G_π è un albero di cammini minimi per la proprietà del sottografo dei predecessori. $\square$



	init	dopo s	dopo y	dopo z	dopo t	dopo x
estratto	—	s	y	z	t	x
s	0	0	0	0	0	0
t	∞	10 (s)	8 (y)	8 (y)	8 (y)	8 (y)
x	∞	∞	14 (y)	13 (z)	9 (t)	9 (t)
y	∞	5 (s)	5 (s)	5 (s)	5 (s)	5 (s)
z	∞	∞	7 (y)	7 (y)	7 (y)	7 (y)

In ogni cella $d(\pi)$: la stima, e da dove arriva.

Figura 11.4: La Figura 24.6 del libro, sorgente s : grafo con i valori finali e tabella d/π dopo le 5 estrazioni ($s: t = 10, y = 5; y: t = 8, x = 14, z = 7; z: x = 13; t: x = 9$). [Cormen fig. 24.6]

11.5 L'algoritmo di Dijkstra

L'idea golosa: il più vicino è già sistemato

Con pesi *non negativi*, il vertice di stima minima fuori dall'insieme S dei sistemati ha la stima *definitiva*: ogni altro cammino passerebbe prima per un vertice con stima maggiore, e da lì i pesi non negativi non fanno scendere il totale. Dijkstra estrae il minimo, lo mette in S e rilassa i suoi archi uscenti.

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S := \emptyset$ 
3   $Q := G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u := \text{EXTRACT-MIN}(Q)$ 
6       $S := S \cup \{u\}$ 
7      for ogni vertice  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )

```

Q è una coda di min-priorità con chiavi $v.d$, e l'invariante è $Q = V - S$: ogni vertice entra in Q una volta, viene estratto una volta e non ci rientra mai. Il ciclo while fa $|V|$ iterazioni, e il vertice estratto ha la stima minima di $V - S$.

Teorema 24.6 — correttezza dell'algoritmo di Dijkstra

da saper dimostrare

Dijkstra, eseguito su un grafo orientato pesato $G = (V, E)$ con funzione peso non negativa w e sorgente s , termina con $u.d = \delta(s, u)$ per tutti i vertici $u \in V$. [Cormen cap. 24.4]

Dimostrazione. Invariante di ciclo: *all'inizio di ogni iterazione del while*, $v.d = \delta(s, v)$ per ogni $v \in S$. Basta dimostrare che $u.d = \delta(s, u)$ quando u entra in S : poi il limite superiore mantiene l'uguaglianza. *Inizializzazione:* $S = \emptyset$, banalmente vera. *Conservazione:* supponiamo per assurdo che u sia il *primo* vertice per cui $u.d \neq \delta(s, u)$ quando entra in S . $u \neq s$ (s è il primo, con $s.d = \delta(s, s) = 0$), quindi $S \neq \emptyset$; e c'è un cammino minimo p da s a u (altrimenti $u.d = \infty = \delta(s, u)$). Siano y il *primo* vertice di p fuori da S e $x \in S$ quello che lo precede: p si scompone in $s \xrightarrow{p_1} x \rightarrow y \xrightarrow{p_2} u$. Quando x entrò in S valeva $x.d = \delta(s, x)$ (x è il *primo* "sbagliato") e l'arco (x, y) fu rilassato: per la proprietà della convergenza $y.d = \delta(s, y)$. E y precede u su p con pesi di p_2 non negativi, quindi $\delta(s, y) \leq \delta(s, u)$; con il limite superiore:

$$y.d = \delta(s, y) \leq \delta(s, u) \leq u.d.$$

Ma quando u è stato scelto nella riga 5, sia u sia y erano in $V - S$ e u aveva la stima minima: $u.d \leq y.d$. Tutto è uguaglianza, $u.d = \delta(s, u)$: contraddizione. Alla fine $Q = \emptyset$, quindi $S = V$, e ogni vertice è estratto con la stima definitiva. $\square$

Corollario 24.7: alla fine G_π è un albero di cammini minimi (Teorema 24.6 più la proprietà del sottografo dei predecessori).

Perché non si possono avere archi negativi

La dimostrazione si rompe su $\delta(s, y) \leq \delta(s, u)$: serve che gli archi da y a u non siano negativi. E con un arco negativo Dijkstra sbaglia davvero: è l'esercizio 24.4-2, che chiede “un semplice esempio”. Nel grafo $s \rightarrow t$ (1), $s \rightarrow x$ (2), $x \rightarrow t$ (-2) Dijkstra “certifica” t con $t.d = 1$, ma il cammino vero pesa $2 - 2 = 0$ [Cormen esercizio 24.4-2].

Quanto costa: array, heap, Fibonacci

Dijkstra chiama INSERT e EXTRACT-MIN una volta per vertice, DECREASE-KEY una per arco al più:

coda di min-priorità	costo di Dijkstra
array indicizzato da 1 a $ V $	$O(V^2 + E) = O(V^2)$ (EXTRACT-MIN costa $O(V)$)
min-heap binario	$O((V + E) \lg V) = O(E \lg V)$ (puntatori vertice $\leftrightarrow$ heap; se tutto è raggiungibile)
heap di Fibonacci	$O(V \lg V + E)$ (DECREASE-KEY ammortizzato $O(1)$)

Eeguire Dijkstra a mano, con la tabella d/π

1. **Prima colonna (init):** $s.d = 0$, gli altri ∞ , i π a NIL; tieni una riga “estratto” per segnare chi entra in S .
2. **Estrai il minimo** fuori da S (a parità, uno qualsiasi): aggiungilo a S , la sua d è definitiva; **rilassa i suoi archi uscenti** aggiornando d e π nella nuova colonna.
3. **Chiusura:** gli archi $v.\pi \rightarrow v$ sono l'albero dei cammini minimi; se compare un arco negativo, Dijkstra non è applicabile.

Se ti chiedono: “dimostra che Dijkstra è corretto”

1. **Invariante:** all'inizio di ogni iterazione, $v.d = \delta(s, v)$ per ogni $v \in S$; basta il momento in cui u entra in S , poi il limite superiore lo mantiene.
2. **Per assurdo:** u = primo vertice aggiunto a S con $u.d \neq \delta(s, u)$; $u \neq s$, c'è un cammino minimo p da s a u . Siano y il primo vertice di p fuori da S e $x \in S$ il suo predecessore: quando x entrò in S , l'arco (x, y) fu rilassato con $x.d = \delta(s, x)$, quindi $y.d = \delta(s, y)$.
3. **Pesi non negativi:** $\delta(s, y) \leq \delta(s, u) \leq u.d$ (il tratto da y a u ha pesi ≥ 0); ma sia u sia y erano in $V - S$, quindi $u.d \leq y.d$: tutto è uguaglianza, $u.d = \delta(s, u)$, contraddizione; alla fine $S = V$ e G_π è un albero di cammini minimi (Cor. 24.7). Con un peso negativo la catena si rompe: controesempio $s \rightarrow t$ (1), $s \rightarrow x$ (2), $x \rightarrow t$ (-2).

Errori tipici

- **Non aggiornare il π insieme alla d in RELAX:** la stima scende, ma l'albero dei cammini minimi resta quello vecchio.
- **Contare male i passaggi di Bellman-Ford:** sono $|V| - 1$ più uno di controllo, non uno solo.
- **Dijkstra con un arco negativo,** o confondere ∞ con $-\infty$: la stima non è definitiva.

Capitolo 12

Cammini minimi fra tutte le coppie

Il capitolo 11 risolve i cammini minimi da *una* sorgente; qui il problema è la tabella completa: per *ogni* coppia di vertici i, j di un grafo orientato e pesato $G = (V, E)$, il peso $\delta(i, j)$ di un cammino minimo da i a j (o ∞ , se j non è raggiungibile da i). L'input si dà come **matrice di adiacenza** $W = (w_{ij})$, $n \times n$ con $n = |V|$: [Cormen eq. (25.1)]

$$w_{ij} = \begin{cases} 0 & \text{se } i = j, \\ \text{il peso dell'arco } (i, j) & \text{se } i \neq j \text{ e } (i, j) \in E, \\ \infty & \text{se } i \neq j \text{ e } (i, j) \notin E. \end{cases} \quad (25.1)$$

Sono ammessi archi di peso **negativo** — quindi Dijkstra non si può usare — ma non cicli di peso negativo. L'output è la matrice delle distanze $D = (d_{ij})$ con $d_{ij} = \delta(i, j)$ e la matrice dei predecessori $\Pi = (\pi_{ij})$, con π_{ij} NIL se $i = j$ o se j non è raggiungibile da i , altrimenti il predecessore di j in un cammino minimo da i : la riga i -esima di Π descrive l'albero dei cammini minimi di radice i . [Cormen p. 574]

Senza inventare nulla si può risolvere il problema con n chiamate a un algoritmo da sorgente unica (capitolo 11): n volte DIJKSTRA se i pesi non sono negativi, n volte BELLMAN-FORD altrimenti — e quest'ultimo, in un grafo denso, costa $O(V^4)$. I costi di tutte le strade sono nella tabella della sezione 12.3; gli algoritmi di questo capitolo arrivano a $O(V^3 \lg V)$ e poi a $\Theta(V^3)$. [Cormen pp. 573–574] Qui si cita l'edizione italiana del libro: il prodotto di matrici è il paragrafo **25.2**, Floyd-Warshall il **25.3**; il 25.1 è la breve introduzione del capitolo, da cui viene la tabella dei costi in fondo.

L'idea: due modi di far crescere i cammini

Un cammino minimo ha la struttura del capitolo 11 (sottostruttura ottima, Lemma 24.1): *ogni sottocammino di un cammino minimo è minimo*. Per riempire una tabella i cammini si possono allora far crescere in due modi. *Un arco in più alla volta*: da $l^{(m-1)}$ a $l^{(m)}$ con una somma e un minimo, le stesse operazioni del prodotto di matrici — con l'elevazione al quadrato ripetuta, $\Theta(n^3 \lg n)$. *Un vertice intermedio in più alla volta*: aggiungere il vertice k costa un solo confronto per cella, e in $\Theta(n^3)$ la matrice è completa — è **Floyd-Warshall**.

12.1 Cammini con al più m archi, e il «prodotto» di matrici

Sia $l_{ij}^{(m)}$ il peso minimo di un cammino da i a j che contiene al più m archi. Con zero archi si va solo da i a i , quindi $l_{ij}^{(0)}$ vale 0 se $i = j$ e ∞ altrimenti. Per $m \geq 1$ un cammino con al più m archi o ne usa al più $m - 1$, oppure è un cammino con al più $m - 1$ archi fino a un vertice k seguito dall'arco (k, j) — e il sottocammino è minimo:

$$l_{ij}^{(m)} = \min \left(l_{ij}^{(m-1)}, \min_{1 \leq k \leq n} \{ l_{ik}^{(m-1)} + w_{kj} \} \right) = \min_{1 \leq k \leq n} \{ l_{ik}^{(m-1)} + w_{kj} \} \quad (25.2)$$

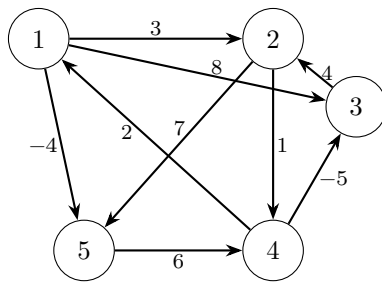
(l'ultima uguaglianza perché $w_{jj} = 0$). Se il grafo non ha cicli di peso negativo, un cammino minimo è semplice e usa al più $n - 1$ archi, quindi $\delta(i, j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = \dots$ [Cormen eq. (25.2)–(25.3)]. La ricorrenza (25.2) si calcola cella per cella (l'_{ij} è il minimo su k di $l_{ik} + w_{kj}$): tre cicli annidati, $\Theta(n^3)$. [Cormen pp. 576–577]

EXTEND-SHORTEST-PATHS(L, W)

```

1   $n := L.rows$ 
2  Sia  $L' = (l'_{ij})$  una nuova matrice  $n \times n$ 
3  for  $i := 1$  to  $n$ 
4      for  $j := 1$  to  $n$ 
5           $l'_{ij} := \infty$ 
6          for  $k := 1$  to  $n$ 
7               $l'_{ij} := \min(l'_{ij}, l_{ik} + w_{kj})$ 
8  return  $L'$ 
```

È lo stesso schema del *prodotto di matrici* $C = A \cdot B$, $c_{ij} = \sum_k a_{ik} b_{kj}$: scambiando $\min \leftrightarrow +$, $+$ $\leftrightarrow \cdot$ e le identità $\infty \leftrightarrow 0$ si passa dall'uno all'altro. Con il «prodotto» $\otimes$ di EXTEND-SHORTEST-PATHS: $L^{(m)} = L^{(m-1)} \otimes W = W^m$, e $\otimes$ è associativo (esercizio 25.2-4). La procedura SLOW-ALL-PAIRS-SHORTEST-PATHS fa i prodotti uno per uno ($L^{(m)} := \text{EXTEND-SHORTEST-PATHS}(L^{(m-1)}, W)$ per $m = 2, \dots, n-1$, $\Theta(n^4)$); con l'**elevazione al quadrato ripetuta**, $L^{(2m)} = L^{(m)} \otimes L^{(m)}$, ne bastano $\lceil \lg(n-1) \rceil$ — con k quadrature si arriva a $L^{(2^k)}$, e oltre $n - 1$ archi le distanze non cambiano più: è FASTER-ALL-PAIRS-SHORTEST-PATHS, $\Theta(n^3 \lg n)$. [Cormen pp. 577–579]



il grafo di W

$$L^{(1)}$$

	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

$$L^{(2)}$$

	1	2	3	4	5
1	0	3	8	2	-4
2	3	0	-4	1	7
3	∞	4	0	5	11
4	2	-1	-5	0	-2
5	8	∞	1	6	0

$$L^{(3)}$$

	1	2	3	4	5
1	0	3	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	11
4	2	-1	-5	0	-2
5	8	5	1	6	0

$$L^{(4)}$$

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

Figura 12.1: Il grafo di esempio della figura 25.1 del libro (a sinistra) e le matrici $L^{(m)}$, con gli indici di riga e colonna in alto e a sinistra; in verde le celle cambiate rispetto alla matrice precedente. $L^{(1)} = W$, e già $L^{(4)} = L^{(5)} = \dots$ dà le distanze finali.

L'esempio svolto

Per andare da 1 a 4 con al più due archi: $1 \rightarrow 2 \rightarrow 4$ costa $3 + 1 = 4$, $1 \rightarrow 5 \rightarrow 4$ costa $-4 + 6 = 2$, quindi $L_{14}^{(2)} = \min(\infty, 4, 2) = 2$. Il libro verifica che $L^{(5)} = L^{(4)} \otimes W = L^{(4)}$: dalla quarta matrice in poi la sequenza si stabilizza.

12.2 Floyd-Warshall

L'idea: far entrare un vertice alla volta

Qui il contatore non è il numero di archi ma il numero di vertici che al cammino è concesso di usare come tappe intermedie. La domanda diventa: quali coppie migliorano se fra i e j autorizzo anche il vertice k ? Solo quelle per cui conviene *passare da k* , cioè $i \rightarrow \dots \rightarrow k \rightarrow \dots \rightarrow j$, con $d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$ minore di quanto costava andare da i a j senza k . Un confronto per cella, e dopo n passi ogni vertice ha avuto il suo turno.

Sia $d_{ij}^{(k)}$ il peso minimo di un cammino da i a j i cui *vertici intermedi* (i vertici del cammino diversi dagli estremi) stanno tutti in $\{1, \dots, k\}$. Per $k = 0$ non ci sono intermedi: resta solo l'eventuale arco (i, j) , quindi $d_{ij}^{(0)} = w_{ij}$. Per $k \geq 1$, un cammino minimo con intermedi in $\{1, \dots, k\}$ o non passa da k — ed è il migliore con intermedi in $\{1, \dots, k-1\}$ — oppure passa da k : si spezza in $i \rightsquigarrow k \rightsquigarrow j$, e per la sottostruttura ottima i due pezzi sono minimi con intermedi in $\{1, \dots, k-1\}$ (il vertice k non è un intermedio di sé stesso). Quindi

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{se } k = 0, \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{se } k \geq 1, \end{cases} \quad (25.5)$$

e siccome per un cammino qualsiasi tutti gli intermedi stanno in $\{1, \dots, n\}$, la soluzione è $D^{(n)}$, con $d_{ij}^{(n)} = \delta(i, j)$. [Cormen eq. (25.5)]

FLOYD-WARSHALL(W)

```

1   $n := W.rows$ 
2   $D^{(0)} := W$ 
3  for  $k := 1$  to  $n$ 
4      Sia  $D^{(k)} = (d_{ij}^{(k)})$  una nuova matrice  $n \times n$ 
5      for  $i := 1$  to  $n$ 
6          for  $j := 1$  to  $n$ 
7               $d_{ij}^{(k)} := \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
```

Il tempo è $\Theta(n^3)$ (tre cicli, un confronto per cella) e la costante nascosta è piccola; lo spazio è $\Theta(n^3)$ con le matrici $D^{(k)}$, ma eliminando gli apici — scrivendo tutto nella stessa D — scende a $\Theta(n^2)$ (esercizio 25.3-4). [Cormen p. 582]

I predecessori. $\pi_{ij}^{(k)}$ è il predecessore di j in un cammino minimo da i con intermedi in $\{1, \dots, k\}$: $\pi_{ij}^{(0)}$ è NIL se $i = j$ o $w_{ij} = \infty$, altrimenti è i [Cormen eq. (25.6)]; per $k \geq 1$ il predecessore cambia solo se il cammino *passa* da k , e allora diventa quello di un cammino minimo da k :

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{se } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}, \\ \pi_{kj}^{(k-1)} & \text{se } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)}. \end{cases} \quad (25.7)$$

[Cormen eq. (25.7)] Nel codice è una riga in più dentro i tre cicli (esercizio 25.3-3). Per stampare un cammino minimo da i a j si risale la catena dei predecessori, restando nella riga i della matrice: da j si passa a π_{ij} , poi a $\pi_{i, \pi_{ij}}$, finché non si arriva a i ; la catena si stampa al contrario [Cormen p. 574].

$D^{(0)}$					
1	2	3	4	5	
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

$D^{(1)}$					
1	2	3	4	5	
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

$D^{(2)}$					
1	2	3	4	5	
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

$D^{(3)}$					
1	2	3	4	5	
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	-1	-5	0	-2
5	∞	∞	∞	6	0

$D^{(4)}$					
1	2	3	4	5	
1	0	3	-1	4	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

$D^{(5)}$					
1	2	3	4	5	
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

Figura 12.2: Le matrici $D^{(k)}$ di Floyd-Warshall per il grafo della Figura 12.1 (figura 25.4 del libro). In azzurro la riga e la colonna k dell'iterazione: si copiano senza cambiare, perché $d_{kk}^{(k-1)} = 0$. In verde le celle che il vertice k migliora rispetto alla matrice precedente, cioè quelle in cui $d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$ batte il valore che c'era. $D^{(5)}$ è la matrice finale delle distanze.

$\Pi^{(0)}$					
1	2	3	4	5	
1	NIL	1	1	NIL	1
2	NIL	NIL	NIL	2	2
3	NIL	3	NIL	NIL	NIL
4	4	NIL	4	NIL	NIL
5	NIL	NIL	NIL	5	NIL

$\Pi^{(5)}$					
1	2	3	4	5	
1	NIL	3	4	5	1
2	4	NIL	4	2	1
3	4	3	NIL	2	1
4	4	3	4	NIL	1
5	4	3	4	5	NIL

Figura 12.3: La matrice dei predecessori: $\Pi^{(0)}$ con la regola (25.6) e $\Pi^{(5)}$ finale, da cui si ricostruiscono i cammini (Figura 12.1).

L'esempio svolto

Al passo $k = 1$ cambiano due sole celle, in riga 4: $d_{42}^{(1)} = \min(\infty, 2 + 3) = 5$ (conviene $4 \rightarrow 1 \rightarrow 2$) e $d_{45}^{(1)} = \min(\infty, 2 - 4) = -2$ ($4 \rightarrow 1 \rightarrow 5$). Il passo più ricco è $k = 4$: $d_{13}^{(4)} = \min(8, 4 + (-5)) = -1$ ($1 \rightarrow 2 \rightarrow 4 \rightarrow 3$), $d_{21}^{(4)} = 3$, $d_{23}^{(4)} = -4$. Alla fine $D^{(5)}$ coincide con $L^{(4)}$: i due modi di far crescere i cammini danno le stesse distanze. Per la *ricostruzione* si usa $\Pi^{(5)}$: da 1 a 2 la catena dei predecessori è $2 \leftarrow 3 \leftarrow 4 \leftarrow 5 \leftarrow 1$, che al contrario dà $1 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2$, di peso $-4 + 6 - 5 + 4 = 1$, contro il 3 dell'arco diretto.

Chiusura transitiva. Se interessa solo se un cammino esiste, si dà peso 1 a ogni arco e allora $d_{ij}^{(n)} < n$ vale se e solo se j è raggiungibile da i ; oppure si sostituiscono $\min \rightarrow \vee$ e $+$ $\rightarrow \wedge$ nella (25.5): $t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)})$, con $t_{ij}^{(0)} = 1$ se $i = j$ o $(i, j) \in E$. È la procedura TRANSITIVE-CLOSURE: sempre $\Theta(n^3)$, ma su valori booleani. [Cormen cap. 25.3]

12.3 Quale algoritmo, per quale input

algoritmo	tempo	pesi neg.?
DIJKSTRA $\times n$, coda ad array	$O(V^3 + VE) = O(V^3)$	no
DIJKSTRA $\times n$, min-heap binario	$O(VE \lg V)$	no
DIJKSTRA $\times n$, heap di Fibonacci	$O(V^2 \lg V + VE)$	no
BELLMAN-FORD $\times n$	$O(V^2 E)$ (denso $O(V^4)$)	sì
SLOW-ALL-PAIRS-SHORTEST-PATHS	$\Theta(V^4)$	sì
FASTER-ALL-PAIRS-SHORTEST-PATHS	$\Theta(V^3 \lg V)$	sì
FLOYD-WARSHALL	$\Theta(V^3)$	sì

Con i pesi negativi Dijkstra non si può usare (capitolo 11); Floyd-Warshall sì, purché non ci siano cicli di peso negativo — e sa rilevarli da solo: se qualche $d_{ii}^{(n)} < 0$, un ciclo negativo c'è (esercizio 25.3-6). [Cormen pp. 573–574] Per i grafi *sparsi* con pesi negativi c'è di meglio, l'algoritmo di Johnson, ma è fuori dal programma. [Cormen, cap. 25.4]

12.4 Come si risolve all'esame

Eseguire Floyd-Warshall e riempire le matrici passo per passo

Ti danno la matrice W (o il disegno del grafo) e chiedono di eseguire Floyd-Warshall mostrando le matrici. La scaletta:

1. $D^{(0)} = W$: diagonale di zeri, w_{ij} se l'arco c'è, ∞ se non c'è; ricopia bene i pesi negativi.
2. **Per** $k = 1, \dots, n$: riga k e colonna k si *copiano* senza cambiarle, perché $d_{kk}^{(k-1)} = 0$ (per la riga: $d_{kj}^{(k)} = \min(d_{kj}^{(k-1)}, 0 + d_{kj}^{(k-1)}) = d_{kj}^{(k-1)}$; per la colonna è lo stesso). Per ogni altra cella scrivi $\min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$: le celle che cambiano sono il segno del vertice k sui cammini, e il nuovo cammino *passa* da k .
3. **Controllo a ogni passo**: la nuova d_{ij} è $\leq d_{ik} + d_{kj}$; se non torna, hai sbagliato a copiare.
4. **Predecessori (se chiesti)**: $\Pi^{(0)}$ con la regola (25.6) (NIL su diagonale e celle ∞ , i sulle celle con arco) e a ogni miglioramento $\pi_{ij} := \pi_{kj}$.
5. **Ricostruzione**: da j si risale π_{ij} , poi $\pi_{i,\pi_{ij}}$, ... fino a i ; il cammino si legge al contrario. Verifica che il peso sommi al valore in D .
6. **Chiusura**: $D^{(n)}$ sono le distanze $\delta(i, j)$; un $d_{ii} < 0$ segnala un ciclo di peso negativo.

Esempio minimo: $d_{42} = \infty \rightarrow 2 + 3 = 5$ con $k = 1$; $d_{13} = 8 \rightarrow 4 + (-5) = -1$ con $k = 4$.

Se ti chiedono: «i cammini minimi fra tutte le coppie»

1. **Il problema**: per tutte le coppie di $G = (V, E)$ pesato (pesi anche negativi, ma senza cicli negativi) la matrice $D = (\delta(i, j))$; l'input è la matrice di adiacenza W .
2. **I modi**: n chiamate a un algoritmo da sorgente unica (DIJKSTRA con pesi non negativi, altrimenti BELLMAN-FORD, $O(V^2E)$), o la programmazione dinamica.
3. **Prodotto di matrici**: $l_{ij}^{(m)}$ = cammini con al più m archi; EXTEND-SHORTEST-PATHS è il prodotto con min al posto di +; associativo, quindi elevazione al quadrato, $\Theta(n^3 \lg n)$.
4. **Floyd-Warshall**: $d_{ij}^{(k)}$ = intermedi in $\{1, \dots, k\}$; $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$; $\Theta(V^3)$ con costante piccola, spazio $\Theta(n^2)$ senza gli apici.
5. **Predecessori e chiusura**: Π con le regole (25.6)–(25.7) (la riga i è l'albero di radice i); $\vee$ e $\wedge$ al posto di min e + danno la chiusura transitiva.

Errori tipici con i cammini fra tutte le coppie

- **Confondere i due contatori**: $l^{(m)}$ conta gli *archi*, $d^{(k)}$ i *vertici intermedi*: le due sequenze di matrici non sono la stessa cosa.
- **Aggiornare Π quando il valore non migliora**: con $\geq$ al posto di $>$ nella (25.7) (esercizio 25.3-5) sul grafo dell'esempio 19 cammini su 25 non si ricostruiscono più: il predecessore cambia solo se il cammino passa davvero da k .
- **Usare Dijkstra con pesi negativi** (l'esempio ha -4 e -5) o rovesciare la ricostruzione: la catena dei predecessori si legge da j verso i e il cammino si scrive al contrario.

Capitolo 13

Le prove scritte del 2026, svolte

La prova generale: le sette prove del 2026 svolte per intero, in ordine **cronologico**, coi testi dei temi come sono (refusi compresi); ogni esercizio: prima **Quale metodo**, poi lo svolgimento, poi il risultato. HalfLogProcess, RefineCombine e le fusioni tornano in più temi: svolte alla *prima* comparsa, altrove richiamo. Tutti i numeri sono prodotti da verifiche/prove_2026.py.

I tre esercizi dello scritto, e il metodo di ciascuno

1. **Procedure da analizzare:** contare le operazioni (chiamate e dimensioni; costo dei cicli), scrivere $T(n)$ e risolverla *col metodo chiesto* (2.3, 2.2, 2.4; scaletta in 2.5), controllando con un secondo metodo.
2. **Procedura ricorsiva** su alberi, dag o stringhe, spesso *senza parametri aggiuntivi*: ritorna la risposta per il sottoalbero di radice x ; caso base in testa o in fondo, “non vale” = -1 ; schema completo a fine capitolo.
3. **Problema di progetto** (DP o goloso) per intero: formalizzazione, ricorrenza, pseudocodice bottom-up, istanza *con la tabella*, ricostruzione, complessità (8.6, 9.4, 9.3).

13.1 16 febbraio 2026

Tema del 16 febbraio 2026

ESAME DI ALGORITMI — Università degli Studi di Catania — Corso di Laurea Triennale in Informatica — **16 febbraio 2026**
Si risolvano i seguenti esercizi in un tempo non superiore a 3 ore.

1. Si considerino le due procedure $\text{SHUFFLEANDSORT}(A, i, j)$ e $\text{HEAPYCRRAFT}(A)$ descritte di seguito. La prima opera su un array, A , di n ($j - i + 1 = n$) elementi mentre la seconda su di un heap, H di n elementi. Per ciascuna di esse, scrivere l'equazione di ricorrenza che descrive il tempo di esecuzione $T(n)$. Risolvere l'equazione di ricorrenza utilizzando uno dei metodi dell'albero di ricorsione.
2. Si consideri un albero Rosso-Nero, cioè un albero binario di ricerca in cui ogni nodo è colorato rosso o nero e vale il vincolo per cui un nodo rosso non può avere figli rossi, mentre un nodo nero può avere figli sia rossi sia neri. Come di consueto, si assumano presenti le foglie esterne NIL, considerate nere. Dato un nodo x dell'albero, si vuole definire e calcolare la seguente quantità: $\text{BLACK-RUN}(x)$ = il massimo numero di nodi neri consecutivi che compaiono lungo un qualsiasi cammino discendente che parte da x e termina in una foglia discendente di x . In altre parole, tra tutti i cammini ($x \rightarrow \dots \rightarrow \text{NIL}$) nel sottoalbero radicato in x , si considerano le sequenze massimali di nodi neri consecutivi presenti sul cammino e si chiede di determinare la lunghezza massima ottenibile. Si richiede di progettare una soluzione ricorsiva che, esplorando i sottoalberi sinistro e destro, calcoli $\text{BLACK-RUN}(x)$ sfruttando il fatto che la risposta per un nodo può essere espressa in funzione delle risposte dei suoi figli.
3. In un laboratorio di genomica si dispone di un insieme di n frammenti di DNA candidati al sequenziamento, $F = \{f_1, f_2, \dots, f_n\}$. Per ciascun frammento f_i si conoscono due valori: il costo c_i di sequenziamento e la quantità v_i di informazione genetica rilevante che esso copre. Per poter effettuare un'analisi biologica significativa, è necessario raggiungere almeno una soglia minima di copertura complessiva V , ma il budget è limitato e si desidera spendere il meno possibile. Si chiede di formalizzare il problema introducendo opportune variabili decisionali e definendo la funzione costo da minimizzare, insieme al vincolo che impone il raggiungimento della soglia minima di copertura. Successivamente, si richiede di definire formalmente la funzione matematica $C[i, v]$ che esprime il costo ottimo in termini dei primi i frammenti considerati e della copertura residua v da raggiungere, e di descrivere l'algoritmo di programmazione dinamica che consente di calcolare tale valore ottimo.

$\text{SHUFFLEANDSORT}(A, i, j)$

```
1 if  $i \geq j$  return
2 for  $k := i$  to  $j$ 
3   SWAP( $A$ ,  $\text{RANDINT}(1, j)$ ,  $\text{RANDINT}(1, j)$ )
4  $m := \lfloor (i + j) / 2 \rfloor$ 
5 BUBBLESORT( $A$ ,  $i, j$ )
6 SHUFFLEANDSORT( $A$ ,  $i, m$ )
7 SHUFFLEANDSORT( $A$ ,  $m + 1, j$ )
```

$\text{HEAPYCRRAFT}(H)$

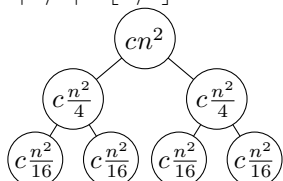
```
1  $n := H.\text{heapsize}$ 
2 if  $n \leq 1$  return  $H[1]$ 
3  $r := \text{RANDOMINT}()$ 
4  $H[1] := r$ 
5 HEAPIFY( $H$ , 1)
6  $H.\text{heapsize} := \lfloor n/2 \rfloor$ 
7 return  $\text{HEAPYCRRAFT}(H)$ 
```

I due pseudocodici del tema: nella riga 2 di A il tema riusa la i (qui la variabile di ciclo si chiama k , il costo non cambia); il testo chiama $\text{HeapyCraft}(A)$ la procedura B , che è $\text{HeapyCraft}(H)$.

Esercizio 1 — le due ricorrenze, con l'albero

Quale metodo. Il tema chiede l'albero di ricorsione (2.3); in coda un controllo col master (2.4).

SHUFFLEANDSORT. Con $n = j - i + 1$: riga 2, un ciclo di n scambi ($\Theta(n)$); riga 5, BUBBLESORT ($\Theta(n^2)$); righe 6–7, due chiamate su $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$ elementi. Dunque $T(n) = 2T(n/2) + \Theta(n^2)$, $T(1) = \Theta(1)$. L'albero:



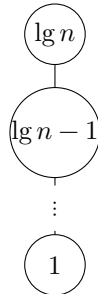
I nodi di dimensione 1 sono il caso base (non pagano): domina la radice.

livello i : 2^i nodi da $n/2^i$, costo $\frac{cn^2}{2^i}$ (in alto i livelli costano di più). Somma:

$$cn^2 \left(1 + \frac{1}{2} + \frac{1}{4} + \dots \right) \leq 2cn^2.$$

Risultato. $T(n) = 2T(n/2) + \Theta(n^2) = \Theta(n^2)$: il ciclo di scambi e le chiamate sono ordini inferiori. Il master, caso 3 ($n^{\log_2 2} = n$, $f = n^2$ regolare), concorda. Contato senza il mescolamento: $T(512) = 519\,167$, fra n^2 e $2n^2$. [tema 16/02/2026, E1]

HEAPY-CRAFT. Le righe costanti spariscono; $\text{HEAPIFY}(H, 1)$ è *Max-Heapify* in radice su n elementi: scende un livello per volta, $\Theta(\lg n)$ (3.2); e la chiamata ricorsiva è *una sola* (riga 7), su $\lfloor n/2 \rfloor$ elementi: nell'array-heap i figli del nodo t stanno in $2t$ e $2t+1$, quindi dimezzando l'heapsize restano i soli nodi interni e il problema si ripete identico su un heap dimezzato (3.2). Dunque $T(n) = T(\lfloor n/2 \rfloor) + \Theta(\lg n)$: una chiamata per livello, e il costo del livello *cala* di 1 a ogni discesa.



una chiamata per livello $(n, \frac{n}{2}, \dots)$, e il livello i costa $\lg \frac{n}{2^i} = \lg n - i$ fino a $\lg n$ livelli; somma:

$$\lg n + (\lg n - 1) + \dots + 1 = \frac{\lg n (\lg n + 1)}{2}.$$

Risultato. $T(n) = T(\lfloor n/2 \rfloor) + \Theta(\lg n) = \Theta(\lg^2 n)$: la somma dei livelli è $1 + 2 + \dots + \lg n$ (su $n = 2^{10}$: il totale è $10 + 9 + \dots + 1 = 55$). Il master non si applica (lavoro non polinomiale): la strada è l'albero. [tema 16/02/2026, E1]

Esercizio 2 — BLACK-RUN su un albero rosso-nero

Quale metodo. Procedura ricorsiva su un albero binario (capitolo 7 per le definizioni; lo stesso tipo di ragionamento del metodobox sui dag, 10.6).

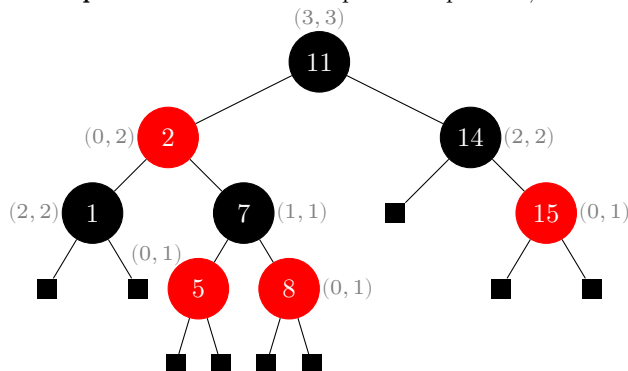
Il contratto. $\text{BLACK-RUN}(x)$ è la più lunga sequenza di nodi neri consecutivi su un cammino da x a una foglia del suo sottoalbero, e il tema scrive i cammini ($x \rightarrow \dots \rightarrow \text{NIL}$): la NIL è “considerata nera” e entra nella sequenza (se il padre è nero, allunga di 1 l'ultima). Le risposte dei figli *non* bastano: servono $S(x)$ = la più lunga sequenza di neri che *parte da* x (0 se rosso) e $R(x)$ = il massimo nel sottoalbero. Foglia NIL: $S = R = 1$ (la sequenza $\{\text{NIL}\}$). Nodo rosso: $S = 0$ e $R(x) = \max\{R(\text{sin}), R(\text{des})\}$, perché una sequenza di neri non attraversa un rosso. Nodo nero: $S(x) = 1 + \max\{S(c) : c \text{ figlio nero}\}$ e $R(x) = \max\{S(x), R(\text{sin}), R(\text{des})\}$.

$\text{BLACK-RUN}(x)$

```

1  if  $x == \text{NIL}$ 
2      return (1, 1) // la NIL è una foglia nera
3  ( $sl, rl$ ) :=  $\text{BLACK-RUN}(x.\text{left})$ 
4  ( $sr, rr$ ) :=  $\text{BLACK-RUN}(x.\text{right})$ 
5  if  $x$  è rosso
6      return (0,  $\max\{rl, rr\}$ ) // la run non passa
7   $run := 1$  //  $x$  è nero: la run parte da qui
8  if  $x.\text{left}$  è nero
9       $run := \max\{run, 1 + sl\}$  // una NIL è nera
10 if  $x.\text{right}$  è nero
11      $run := \max\{run, 1 + sr\}$ 
12 return ( $run, \max\{run, rl, rr\}$ )
```

L'esempio. Sull'albero di esempio del capitolo 7, accanto a ogni nodo la coppia (S, R) :



Nodo 1 (nero, due NIL): $S = 1 + 1 = 2$, $R = 2$: la sequenza $1 \rightarrow \text{NIL}$.

Nodo 7 (nero, figli rossi): $S = 1$, $R = 1$.

Nodo 2 (rosso): $S = 0$, $R = \max\{2, 1\} = 2$: la migliore sta sotto 1.

Nodo 14 (nero, una NIL e il figlio 15 rosso): $S = 1 + 1 = 2$ ($14 \rightarrow \text{NIL}$), $R = 2$.

Nodo 11 (nero, figli 2 rosso e 14 nero): $S = 1 + S(14) = 3$, $R = 3$.

Risultato. $\text{BLACK-RUN}(11) = 3$ (la run $11 \rightarrow 14 \rightarrow \text{NIL}$, tre neri di fila). Ogni nodo una volta sola: tempo $\Theta(n)$, spazio $\Theta(h)$ di pila. Il conteggio segue il tema alla lettera — il cammino ($x \rightarrow \dots \rightarrow \text{NIL}$) con la NIL “considerata nera”; fermandosi prima della NIL ogni risposta calerebbe di 1 sui cammini che finiscono in nero. Verificato contro la forza bruta su tutti i cammini (300 alberi casuali). [tema 16/02/2026, E2]

Esercizio 3 — la DP del DNA ($C[i, v]$)

Quale metodo. Programmazione dinamica su due parametri, i e la copertura residua v : la stessa tabella dello zaino 0/1 (8.5, che nei temi si chiama appunto $C[i, v]$), con “almeno” al posto di “al più”.

L’idea: lo zaino 0/1 “al contrario”

Si prende o si lascia ogni frammento (scelta binaria): è lo zaino 0/1 (8.5) col vincolo e l’obiettivo rovesciati — lì “non superare il peso” e massimizzare il valore, qui “copertura almeno V ” e minimizzare il costo. La tabella $C[i, v]$ risolve “primi i frammenti, residuo v da coprire”, decidendo ogni cella con l’ultimo frammento: se lo prendo pago c_i e il residuo scende a $v - v_i$.

(a) **Formalizzazione.** Variabili decisionali $x_i \in \{0, 1\}$ ($x_i = 1$: sequenziare f_i):

$$\min \sum_{i=1}^n c_i x_i \quad \text{soggetto a} \quad \sum_{i=1}^n v_i x_i \geq V, \quad x_i \in \{0, 1\}.$$

È uno zaino “al contrario”: il vincolo è in $\geq$, non in $\leq$.

(b) **La funzione** $C[i, v]$. $C[i, v]$ = costo minimo per raggiungere una copertura *residua* v coi primi i frammenti: $C[i, 0] = 0$, $C[0, v] = +\infty$ per $v > 0$. Sull’ultimo frammento:

$$C[i, v] = \min(C[i-1, v], \quad c_i + C[i-1, \max(0, v - v_i)]) \quad (13.1)$$

cioè: o f_i si lascia, o si prende e si paga c_i più il costo del residuo $v - v_i$ (se v_i copre più del residuo, il residuo si azzerà: il $\max(0, \cdot)$ è il “tutto qui”). La risposta è $C[n, V]$.

(c) **L’algoritmo.** Righe per i crescente, colonne per v crescente: ogni cella usa solo la riga precedente.

COPERTURA-MINIMA(c, v, n, V)

```

1  for  $t := 1$  to  $V$ 
2       $C[0, t] := +\infty$ 
3  for  $i := 1$  to  $n$ 
4       $C[i, 0] := 0$ 
5      for  $t := 1$  to  $V$ 
6           $C[i, t] := C[i-1, t]$ 
7          if  $c_i + C[i-1, \max(0, t - v_i)] < C[i, t]$ 
8               $C[i, t] := c_i + C[i-1, \max(0, t - v_i)]$ 
9  return  $C[n, V]$ 
```

Tempo $\Theta(nV)$, spazio $\Theta(nV)$ (due righe per il valore, più le scelte booleane per ricostruire). La ricostruzione cammina all’indietro da (n, V) : se $C[i, v] = C[i-1, v]$ il frammento i è stato lasciato, altrimenti si passa a $(i-1, \max(0, v - v_i))$.

L’istanza (il tema non ne dà una; frammenti $\langle (c_i, v_i) \rangle = \langle (2, 3), (3, 2), (4, 5), (1, 3) \rangle$, $V = 6$) [non nelle fonti del corso]:

$i \backslash v$	0	1	2	3	4	5	6
0 (nessuno)	0	∞	∞	∞	∞	∞	∞
1: $(c_1, v_1) = (2, 3)$	0	2	2	2	∞	∞	∞
2: $(c_2, v_2) = (3, 2)$	0	2	2	2	5	5	∞
3: $(c_3, v_3) = (4, 5)$	0	2	2	2	4	4	6
4: $(c_4, v_4) = (1, 3)$	0	1	1	1	3	3	3

Lettura di due celle: $C[4, 6]$: lasciare f_4 costerebbe $C[3, 6] = 6$; prenderlo costa $c_4 + C[3, 3] = 1 + 2 = 3$. $C[3, 4]$: lasciare f_3 costa $C[2, 4] = 5$; prenderlo costa $4 + C[2, \max(0, 4 - 5)] = 4 + 0 = 4$. La ricostruzione (celle in grassetto, si cammina all’indietro da $(4, 6)$): $C[4, 6] = 3 \neq C[3, 6] = 6$, preso f_4 , residuo 3; $C[3, 3] = C[2, 3] = 2$, f_3 lasciato; $C[2, 3] = C[1, 3] = 2$, f_2 lasciato; $C[1, 3] = 2 \neq C[0, 3] = \infty$, preso f_1 , residuo 0: ferma. Totale $\{f_1, f_4\}$: costo $2 + 1 = 3$, copertura $3 + 3 = 6 \geq V$.

Risultato. $C[n, V]$ con $C[i, v] = \min\{c_i + C[i-1, \max(0, v - v_i)], C[i-1, v]\}$, tempo e spazio $\Theta(nV)$ (pseudopolinomiale in V); sull’istanza $C[4, 6] = 3$, con i frammenti f_1, f_4 (verificato: è anche il minimo su tutti i 2^4 sottoinsiemi). [tema 16/02/2026, E3]

13.2 31 marzo 2026

Tema del 31 marzo 2026

ESAME DI ALGORITMI — Università degli Studi di Catania — Corso di Laurea Triennale in Informatica — **31 marzo 2026**

Si risolvano i seguenti esercizi in un tempo non superiore a 3 ore.

1. Si considerino le due procedure $\text{HalfLogProcess}(A, i, j)$ e $\text{RefineCombine}(A, i, j)$ descritte di seguito. La prima opera su un array A di n elementi ($j - i + 1 = n$), mentre la seconda opera anch'essa su un array A di n elementi. Per ciascuna di esse, scrivere l'equazione di ricorrenza che descrive il tempo di esecuzione $T(n)$. Risolvere l'equazione di ricorrenza utilizzando il metodo dell'albero di ricorsione.
2. Si considerino due alberi binari T_1 e T_2 , i cui nodi contengono valori interi. Ciascun albero è definito mediante nodi che possiedono un valore e due riferimenti, rispettivamente al figlio sinistro e al figlio destro (eventualmente nulli). Due alberi si dicono *isomorfi* se è possibile renderli identici scambiando, in uno o più nodi, il figlio sinistro con il figlio destro. In altre parole, gli alberi devono avere la stessa struttura e gli stessi valori nei nodi corrispondenti, a meno di eventuali scambi tra sottoalberi sinistro e destro. Si richiede di progettare e descrivere una procedura ricorsiva che, dati in input due alberi binari T_1 e T_2 , restituisca in output un valore booleano che indichi se i due alberi sono isomorfi secondo la definizione fornita. La soluzione deve essere espressa in forma di pseudocodice. Facoltativamente, si discuta la complessità della procedura proposta.
3. Il commissario Montalbano deve organizzare una serie di operazioni per fermare i componenti di una banda criminale prima che possano completare un piano particolarmente pericoloso. Per ciascuna operazione sono noti: il tempo necessario (espresso in ore) per eseguire l'operazione sul campo; il tempo necessario (espresso in ore) per prepararla e coordinarla con la squadra; la riduzione del danno atteso che si ottiene portando a termine con successo l'operazione. Per impedire che la banda arrechi un danno troppo elevato, il commissario deve ottenere una riduzione complessiva del danno atteso almeno pari a un valore prefissato D . Si richiede di determinare un sottoinsieme di operazioni tale che la riduzione complessiva del danno atteso sia almeno D , minimizzando il tempo totale impiegato. Progettare un algoritmo efficiente per risolvere il problema, descrivendone l'idea, fornendone la funzione ricorsiva, lo pseudocodice e la relativa complessità computazionale.

$\text{HALFLOGPROCESS}(A, i, j)$

```

1  if  $i \geq j$  return
2   $m := \lfloor (i + j) / 2 \rfloor$ 
3   $k := 1$ 
4  while  $k < (j - i + 1)$ 
5       $A[k] := A[k] + 1$ 
6       $k := 2k$ 
7   $\text{HALFLOGPROCESS}(A, i, m)$ 
```

$\text{REFINECOMBINE}(A, i, j)$

```

1  if  $i \geq j$  return  $A[i]$ 
2   $m := \lfloor (i + j) / 2 \rfloor$ 
3   $L := \text{REFINECOMBINE}(A, i, m)$ 
4   $R := \text{REFINECOMBINE}(A, m + 1, j)$ 
5  for  $k := i$  to  $j$ 
6       $A[k] := A[k] + L + R$ 
7   $L_2 := \text{REFINECOMBINE}(A, i, m)$ 
8   $R_2 := \text{REFINECOMBINE}(A, m + 1, j)$ 
9  return  $L_2 + R_2$ 
```

Esercizio 1 — HALFLOGPROCESS e REFINECOMBINE

Quale metodo. Albero di ricorsione (2.3).

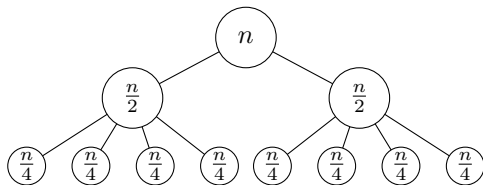
HALFLOGPROCESS . Con $n = j - i + 1$: il **while** raddoppia k , quindi gira $\lceil \lg n \rceil$ volte (1, 2, 4, ...): $\Theta(\lg n)$; una sola chiamata (riga 7), su $\lceil n/2 \rceil$ elementi. Dunque

$$T(n) = T(\lceil n/2 \rceil) + \Theta(\lg n),$$

che è la stessa ricorrenza di HEAPYCRAFT del 16 febbraio, esercizio 1: albero risolto lì, livello i costa $\lg(n/2^i) = \lg n - i$, la somma è $1 + 2 + \dots + \lg n$.

Risultato. $T(n) = T(\lceil n/2 \rceil) + \Theta(\lg n) = \Theta(\lg^2 n)$ (per $n = 2^k$: $k(k+1)/2$; su $n = 2^{10}$ il totale è 55). Richiamo: albero alla pagina del 16/02, E1. [tema 31/03/2026, E1]

REFINECOMBINE . Il lavoro fuori è un solo ciclo da n giri (righe 5–6): $\Theta(n)$; le chiamate ricorsive sono *quattro* (righe 3, 4, 7, 8), due su $\lceil n/2 \rceil$ e due su $\lfloor n/2 \rfloor$. Dunque $T(n) = 4T(n/2) + \Theta(n)$, $T(1) = \Theta(1)$, e l'albero ha 4^i nodi al livello i , ciascuno con costo $\Theta(n/2^i)$:



livello 0: 4^0 nodi, costo n ;
 livello 1: 4 nodi, costo $4 \cdot \frac{n}{2} = 2n$;
 livello 2: 16 nodi, costo $4^2 \cdot \frac{n}{4} = 4n$;
 ... livello i : $4^i \cdot \frac{n}{2^i} = n \cdot 2^i$.

La somma *cresce* di livello in livello, l'ultimo livello interno domina:

$$n(1 + 2 + 4 + \dots + 2^{\lg n - 1}) = n(n - 1) \approx n^2.$$

Risultato. $T(n) = 4T(n/2) + \Theta(n) = \Theta(n^2)$: l'ultimo livello interno ha $n^2/4$ nodi da $\Theta(2)$, $n^2/2$ in tutto. Master caso 1 ($n^{\log_2 4} = n^2$): concorda. Contato su $n = 2^8$: $65\,280 = 256 \cdot 255$. [tema 31/03/2026, E1]

Esercizio 2 — alberi isomorfi

Quale metodo. Procedura ricorsiva su alberi binari: contratto fissato dalla firma, caso base alle foglie (lo stesso schema di BLACK-RUN, E2 del 16 febbraio).

Il contratto. $\text{ISOMORPHIC}(x, y)$ è vero quando x e y hanno lo stesso “scheletro a meno di scambi”: o accoppiati sinistro-sinistro e destro-destro, o incrociati; basta che *una* delle due possibilità sopravviva in tutto il sottoalbero (scambio deciso nodo per nodo), con foglie con foglie e valori uguali nei nodi corrispondenti.

$\text{ISOMORPHIC}(x, y)$

```

1  if x == NIL and y == NIL
2      return TRUE // due foglie: uguali
3  if x == NIL or y == NIL
4      return FALSE // una sola foglia
5  if x.val ≠ y.val
6      return FALSE // valori diversi
7  dritti := ISOMORPHIC(x.left, y.left) and ISOMORPHIC(x.right, y.right)
8  incrociati := ISOMORPHIC(x.left, y.right) and ISOMORPHIC(x.right, y.left)
9  return dritti or incrociati

```

L'esempio (foglia = -). $T_1 = (1, (2, (4)), (3))$ e $T_2 = (1, (3), (2, -, (4)))$ sono isomorfi: alla radice “dritti” fallisce ($2 \neq 3$), “incrociati” chiama ISOMORPHIC su $(2, (4))$ contro $(2, -, (4))$ e su (3) contro (3) ; il secondo è vero subito, il primo di nuovo per incrociati. $T_3 = (1, (2), (3, (4)))$ no: il 4 pende dalla parte sbagliata e 2 e 3 sono figli diretti della radice, non si scambiano.

Risultato. Col cortocircuito di **and/or**, “dritti” appena vero chiude la coppia, e una differenza di valori chiude in $O(1)$: in pratica una visita sola, $\Theta(\min(n_1, n_2))$ (ogni coppia di nodi corrispondenti una volta: 511 chiamate su due copie identiche di un perfetto da 255 nodi). Se invece si valutassero sempre tutte e quattro le chiamate, ogni coppia proseguirebbe: $\approx \frac{4}{3} \min(n_1, n_2)^2$ (87 381 sugli stessi alberi). Spazio $\Theta(\min(h_1, h_2))$. Verificato contro la forma canonica su 500 coppie casuali. [tema 31/03/2026, E2]

Esercizio 3 — Montalbano: tempo minimo, riduzione almeno D

Quale metodo. La DP di copertura del 16/02, E3 (la tabella $C[i, v]$), letta da un'altra storia: v è la *riduzione residua*, il costo è il tempo.

L'idea: la storia di Montalbano è la DP di copertura

Ogni operazione i ha un tempo t_i (esecuzione più preparazione) e una riduzione r_i : è lo zaino-di-copertura di (13.1) con (t_i, r_i) al posto di (c_i, v_i) . Il goloso “prima le operazioni che riducono di più” non è affidabile: come nello zaino 0/1 (8.5) guarda il caso peggiore, non la combinazione.

La funzione ricorsiva. $C[i, v]$ = tempo minimo per una riduzione residua v con le prime i operazioni ($C[i, 0] = 0$, $C[0, v] = +\infty$ per $v > 0$):

$$C[i, v] = \min(C[i-1, v], t_i + C[i-1, \max(0, v-r_i)]), \quad C[n, D] \text{ è la risposta.}$$

Lo pseudocodice è COPERTURA-MINIMA (16/02, E3) con $(c, v) \rightarrow (t, r)$, $V \rightarrow D$ e la stessa ricostruzione.

$i \backslash v$	0	1	2	3	4	5
0 (nessuna)	0	∞	∞	∞	∞	∞
1: $(t_1, r_1) = (3, 2)$	0	3	3	∞	∞	∞
2: $(t_2, r_2) = (2, 3)$	0	2	2	2	5	5
3: $(t_3, r_3) = (5, 4)$	0	2	2	2	5	5
4: $(t_4, r_4) = (1, 2)$	0	1	1	2	3	3

Istanza illustrativa (il tema non ne dà una; operazioni $\langle (t_i, r_i) \rangle = \langle (3, 2), (2, 3), (5, 4), (1, 2) \rangle$, $D = 5$) [non nelle fonti del corso]: la ricostruzione (celle in grassetto) dice preso o_4 (residuo 3), o_3 lasciato ($C[3, 3] = C[2, 3]$), preso o_2 (residuo 0): $C[4, 5] = 3$ con $\{o_2, o_4\}$ — tempo $2 + 1 = 3$, riduzione $3 + 2 = 5$ (la forza bruta sui 2^4 sottoinsiemi conferma). Cella tipica: $C[2, 5] = 5$: il residuo 2 lo copre o_1 .

Risultato. DP con $C[i, v] = \min(C[i-1, v], t_i + C[i-1, \max(0, v-r_i)])$, tempo e spazio $\Theta(nD)$ (pseudopolinomiale in D); sull'istanza $C[4, 5] = 3$ con $\{o_2, o_4\}$. [tema 31/03/2026, E3]

13.3 27 aprile 2026

Tema del 27 aprile 2026

ESAME DI ALGORITMI — Università degli Studi di Catania — Corso di Laurea Triennale in Informatica — **27 aprile 2026**
Si risolvano i seguenti esercizi in un tempo non superiore a 3 ore.

1. Si considerino le due procedure `HalfLogProcess(A, i, j)` e `RefineCombine(A, i, j)` descritte di seguito. La prima opera su un array A di n elementi ($j - i + 1 = n$), mentre la seconda opera anch'essa su un array A di n elementi. Per ciascuna di esse, scrivere l'equazione di ricorrenza che descrive il tempo di esecuzione $T(n)$. Risolvere l'equazione di ricorrenza utilizzando il metodo di sostituzione.

2. Siano date due stringhe x e y , di lunghezze rispettivamente n e m , con $n, m \geq 0$. Si richiede di progettare una procedura ricorsiva che stabilisca se la stringa x è una sottosequenza della stringa y , ossia se x può essere ottenuta a partire da y mediante la rimozione di zero o più caratteri, senza alterare l'ordine relativo dei caratteri rimanenti. La procedura deve essere descritta in pseudocodice e deve avere come parametri di ingresso le due stringhe x e y , insieme alle rispettive lunghezze n e m . In particolare, si richiede di definire una procedura della forma `isSubsequence(x, y, n, m)` che restituisca un valore booleano indicante se x è o meno una sottosequenza di y . Non è consentito l'uso di costrutti iterativi; la soluzione deve essere formulata esclusivamente mediante l'uso della ricorsione.

3. Sia dato un array $A = \{A[1], A[2], \dots, A[n]\}$ di numeri interi. Una sottosequenza crescente di A è una sequenza di elementi $A[i_1], A[i_2], \dots, A[i_k]$ tale che $1 \leq i_1 < i_2 < \dots < i_k \leq n$ e $A[i_1] < A[i_2] < \dots < A[i_k]$. Si richiede di determinare la lunghezza della sottosequenza crescente più lunga di A . Si richiede di risolvere il problema mediante la tecnica della programmazione dinamica. In particolare, si richiede di: definire formalmente una funzione di costo che rappresenti la lunghezza della migliore sottosequenza crescente e fornire una corrispondente definizione ricorsiva basata su sottoproblemi più piccoli; progettare e descrivere in pseudocodice una procedura iterativa di tipo bottom-up che calcoli il valore della soluzione ottima; analizzare la complessità temporale e spaziale della procedura proposta; progettare una procedura ricorsiva che, utilizzando le informazioni calcolate, ricostruisca e restituisca una sottosequenza crescente di lunghezza massima.

`HALFLOGPROCESS(A, i, j)`

```
1  if  $i \geq j$  return
2   $m := \lfloor (i + j) / 4 \rfloor$ 
3   $k := 1$ 
4  while  $k < (j - i + 1)$ 
5       $A[k] := A[k] + 1$ 
6       $k := k + 1$ 
7  HALFLOGPROCESS(A, i, m)
8  HALFLOGPROCESS(A, j - m, j)
```

`REFINECOMBINE(A, i, j)`

```
1  if  $i \geq j$  return  $A[i]$ 
2   $m := \lfloor (i + j) / 2 \rfloor$ 
3   $L := \text{REFINECOMBINE}(A, i, m)$ 
4   $R := \text{REFINECOMBINE}(A, m + 1, j)$ 
5  for  $k := i$  to  $j$ 
6       $A[k] := A[k] + L + R$ 
7   $L_2 := \text{REFINECOMBINE}(A, i, m)$ 
8   $R_2 := \text{REFINECOMBINE}(A, m + 1, j)$ 
9  return  $L_2 + R_2$ 
```

Esercizio 1 — `HALFLOGPROCESS` (variante) e `REFINECOMBINE` (richiamo)

Quale metodo. Sostituzione (2.2).

`HALFLOGPROCESS` (la variante del 27/04). Con $n = j - i + 1$: il **while** incrementa k di 1, quindi gira $n - 1$ volte: $\Theta(n)$. Le due chiamate dividono in quarti: i tratti $[i, m]$ e $[j - m, j]$ hanno circa $n/4$ elementi (come i blocchi di `QUARTERMIX` e `SPECIALCOMPUTE`). Dunque

$$T(n) = 2T(\lfloor n/4 \rfloor) + \Theta(n).$$

Sostituzione. Intuisco $T(n) = O(n)$ e provo $T(n) \leq dn$ per induzione: $T(n) \leq 2d \frac{n}{4} + cn = \frac{dn}{2} + cn \leq dn$ per $d \geq 2c$ (caso base: c grande). E $T(n) \geq cn$ (il ciclo in radice): $\Theta(n)$.

Risultato. $T(n) = 2T(\lfloor n/4 \rfloor) + \Theta(n) = \Theta(n)$: l'albero somma $n + \frac{n}{2} + \frac{n}{4} + \dots = 2n$ (contato: $T(16) = 21$, $T(256) = 465$). *Attenzione:* il tema chiede la sostituzione; e il master concorda (caso 3: $n^{\log_4 2} = \sqrt{n}$ e $f(n) = n$ è polinomialmente più grande). La riga 8 stampata ($A, j - m, j$), presa alla lettera con m indice assoluto, non è ben fondata (il secondo tratto si ripete o contiene il primo: su $n = 64$, $(39, 64) \rightarrow (39, 64)$ — verificato): è il quarto finale, di $\approx n/4$ elementi. [tema 27/04/2026, E1]

`REFINECOMBINE`. È identica a quella del 31 marzo: $T(n) = 4T(n/2) + \Theta(n) = \Theta(n^2)$ (albero lì). La sostituzione del tema: con $T(n) \leq dn^2$ il passo non torna ($dn^2 + cn$); si rafforza a $T(n) \leq dn^2 - \alpha n$: $4(d\frac{n^2}{4} - \alpha\frac{n}{2}) + cn = dn^2 - 2\alpha n + cn \leq dn^2 - \alpha n$ per $\alpha \geq c$: torna. [tema 27/04/2026, E1]

Esercizio 2 — `ISSUBSEQUENCE`

Quale metodo. Procedura ricorsiva su stringhe (l'ambiente LCS in 8.4), senza cicli: si consuma un carattere per volta.

Il contratto. `ISSUBSEQUENCE(x, y, n, m)` è vero quando i primi n caratteri di x stanno, nell'ordine, dentro i primi m di y : se $x[n] = y[m]$ passo a $(n - 1, m - 1)$, altrimenti butto $y[m]$ e passo a $(n, m - 1)$. Casi base in testa: $n = 0$ vero (la vuota c'è sempre); $m = 0$ con $n > 0$ falso.

`ISSUBSEQUENCE(x, y, n, m)`

```
1  if  $n == 0$  return TRUE // la stringa vuota c'è sempre
2  if  $m == 0$  return FALSE //  $x$  non vuota,  $y$  finita
3  if  $x[n] == y[m]$ 
4      return ISSUBSEQUENCE(x, y, n - 1, m - 1)
5  else return ISSUBSEQUENCE(x, y, n, m - 1)
```

L'esempio ($x = ab$, $y = aab$): $(2, 3)$: $b = b \rightarrow (1, 2)$; $(1, 2)$: $a = a \rightarrow (0, 1)$; $n = 0$: vero. (Con $y = acb$: $(2, 3) \rightarrow (1, 2) \rightarrow (1, 1) \rightarrow (0, 0)$: il c viene saltato, ancora vero.)

Risultato. Ogni chiamata toglie un indice: al più $n + m$ chiamate, tempo $\Theta(n + m)$, spazio $\Theta(n + m)$. Verificato: ab in aab sì, acb in abc no, $abcd$ in $dcba$ no. [tema 27/04/2026, E2]

Esercizio 3 — la più lunga sottosequenza crescente (LIS)

Quale metodo. Programmazione dinamica su *un* indice, 8.3: la sottostruttura è “la migliore che finisce qui”.

(a) **La funzione di costo.** $L[j]$ = lunghezza della più lunga sottosequenza *crescente* di $A[1..j]$ che *finisce in* $A[j]$. Una qualsiasi sottosequenza crescente che finisce in $A[j]$, tolto $A[j]$, è una sottosequenza crescente che finisce in un qualche $A[i]$ con $i < j$ e $A[i] < A[j]$; e per la lunghezza conviene la più lunga.

(b) **La ricorrenza.** $L[j] = 1$ se nessun elemento a sinistra è più piccolo, altrimenti

$$L[j] = 1 + \max\{L[i] : i < j, A[i] < A[j]\}.$$

La risposta è $\max_j L[j]$, non $L[n]$: la sottosequenza può finire ovunque.

(c) **L’algoritmo bottom-up.** $L[1..n]$ per j crescente; il vettore $pred[j]$ ricorda *quale* i ha dato il massimo, per la ricostruzione.

LIS-LUNGHEZZE(A, n)

```

1  for  $j := 1$  to  $n$ 
2       $L[j] := 1$ ;  $pred[j] := \text{NIL}$ 
3      for  $i := 1$  to  $j - 1$ 
4          if  $A[i] < A[j]$  and  $L[i] + 1 > L[j]$ 
5               $L[j] := L[i] + 1$ ;  $pred[j] := i$ 
6   $j^* := \arg \max_j L[j]$ ; return  $L[j^*], L, pred$ 
```

(d) **Complessità.** $\Theta(n^2)$ tempo ($n(n-1)/2$ coppie), $\Theta(n)$ spazio (la ricorsione ingenua ne esaminerebbe 2^n).

(e) **La ricostruzione ricorsiva.** Le informazioni ci sono tutte in L e $pred$: parto da j^* e risalgo i predecessori.

LIS-RICOSTRUISCI($A, pred, j$)

```

1  if  $j == \text{NIL}$  return
2  LIS-RICOSTRUISCI( $A, pred, pred[j]$ )
3  stampa  $A[j]$  // in ordine crescente di indice
```

L’istanza (il tema non ne dà una): $A = \langle 3, 1, 4, 1, 5, 9, 2, 6 \rangle$ [non nelle fonti del corso], $L = \langle 1, 1, 2, 1, 3, 4, 2, 4 \rangle$: lunghezza 4 ($L[6] = 4$). Ricostruzione: $pred[6] = 5$, $pred[5] = 3$, $pred[3] = 1$: stampa **3, 4, 5, 9**.

Risultato. $L[j] = 1 + \max\{L[i] : i < j, A[i] < A[j]\}$ (o 1), risposta $\max_j L[j]$; $\Theta(n^2)$ tempo, $\Theta(n)$ spazio; su A esce 3, 4, 5, 9 (concorda la forza bruta su tutte le 2^8 sottosequenze). Soluzione $O(n \lg n)$ [Cormen, esercizio 15.4-6], ma il tema chiede la DP. [tema 27/04/2026, E3]

13.4 22 giugno 2026

Tema del 22 giugno 2026

ESAME DI ALGORITMI — Università degli Studi di Catania — Corso di Laurea Triennale in Informatica — **22 giugno 2026**

Si risolvano i seguenti esercizi in un tempo non superiore a 3 ore.

1. Si considerino le due procedure `HalfLogProcess(A,i,j)` e `RefineCombine(A,i,j)` descritte di seguito. Entrambe le procedure operano su un array A di n elementi ($j - i + 1 = n$). Per ciascuna di esse, scrivere l'equazione di ricorrenza che descrive il tempo di esecuzione $T(n)$. Risolvere l'equazione di ricorrenza utilizzando il metodo di sostituzione. [nota della guida: il pseudocodice A del tema si chiama *MidnightCompute*, non *HalfLogProcess*.]

2. Si consideri un albero binario. Si richiede di scrivere una procedura *ricorsiva* che, ricevuta come unico parametro la radice di un albero (o di un suo sottoalbero), verifichi se tutte le foglie sono poste alla stessa profondità rispetto alla radice considerata. In altre parole, la procedura deve stabilire se tutti i cammini che collegano la radice alle foglie hanno la medesima lunghezza. La procedura deve restituire un valore booleano che indichi se la proprietà è soddisfatta oppure no e *non deve utilizzare parametri aggiuntivi* oltre alla radice del sottoalbero da analizzare.

3. Una casa di produzione cinematografica deve assemblare un filmato composto da una sequenza ordinata di segmenti video $S_1, S_2, \dots, S_n$. Il segmento S_i contiene f_i fotogrammi. Per ottenere il filmato finale, i segmenti devono essere combinati rispettando il loro ordine nella sequenza. È tuttavia possibile scegliere liberamente l'ordine con cui effettuare le operazioni di montaggio intermedie. Quando due blocchi video consecutivi contenenti rispettivamente a e b fotogrammi vengono combinati, il costo dell'operazione è pari a $a \cdot b$. Il blocco risultante contiene $a + b$ fotogrammi. Si richiede di progettare una soluzione basata sulla *programmazione dinamica* che determini la strategia di montaggio a costo minimo. In particolare: (a) definire una funzione costo per il problema e individuare i sottoproblemi rilevanti; (b) formulare una relazione ricorsiva che esprima il costo di una soluzione ottima; (c) progettare un algoritmo *bottom-up* per il calcolo del costo minimo complessivo; (d) progettare una procedura che ricostruisca e stampi la sequenza ottima delle operazioni di montaggio; (e) determinare la complessità temporale e spaziale degli algoritmi proposti.

MIDNIGHTCOMPUTE(A, i, j)

```

1  if  $i \geq j$  return
2   $m := \lfloor (i + j)/4 \rfloor$ 
3  for  $k := 1$  to  $(j - i + 1)$ 
4       $A[i] := A[i] + 1$ 
5  MIDNIGHTCOMPUTE( $A, i, i + m$ )
6  for  $k := 1$  to  $(j - i + 1)$ 
7       $A[i] := A[i] + 1$ 
8  MIDNIGHTCOMPUTE( $A, i + m, i + 2m$ )
```

REFINECOMBINE(A, i, j)

```

1  if  $i \geq j$  return  $A[i]$ 
2   $m := \lfloor (i + j)/2 \rfloor$ 
3   $L := \text{REFINECOMBINE}(A, i, m)$ 
4   $R := \text{REFINECOMBINE}(A, m + 1, j)$ 
5  for  $c := 1$  to 10
6      for  $k := i$  to  $j$ 
7           $A[k] := A[k] + L + R$ 
8   $L_2 := \text{REFINECOMBINE}(A, i, m)$ 
9   $R_2 := \text{REFINECOMBINE}(A, m + 1, j)$ 
10 for  $b := 1$  to 5
11     for  $k := i$  to  $j$ 
12          $L_2 := L_2 + A[k]$ 
13 return  $L_2 + R_2$ 
```

Esercizio 1 — MIDNIGHTCOMPUTE e REFINECOMBINE (richiami)

Quale metodo. Sostituzione (2.2).

MIDNIGHTCOMPUTE. Due cicli da n giri ($\Theta(n)$ l'uno) e due chiamate, sui blocchi $[i, i + m]$ e $[i + m, i + 2m]$: per $i = 1$, due blocchi di circa $n/4$ elementi (come QUARTERMIX dell'1/09). Quindi $T(n) = 2T(\lfloor n/4 \rfloor) + \Theta(n)$, che è la ricorrenza di HALFLOGPROCESS del 27 aprile: sostituzione lì, $T(n) \leq dn$ con $d \geq 2c$ (e $T(n) \geq cn$).

Risultato. $T(n) = 2T(\lfloor n/4 \rfloor) + \Theta(n) = \Theta(n)$: l'albero somma $2n(1 + \frac{1}{2} + \frac{1}{4} + \dots) = 4n$ (contato davvero: $T(256) = 960$). [tema 22/06/2026, E1]

REFINECOMBINE. Stessa procedura del 31 marzo, coi cicli esterni 10 e 5 (righe 5–7 e 10–12): fuori costa $10n + 5n = 15n$, sempre $\Theta(n)$. Quattro chiamate su metà: $T(n) = 4T(n/2) + 15n = \Theta(n^2)$ (stesso albero del 31/03, coi $15n$ al posto di n : somma $15n(n - 1)$; contato: $T(256) = 979\,200$). La sostituzione $T(n) \leq dn^2 - \alpha n$ torna per $\alpha \geq c$.

Risultato. $\Theta(n^2)$. [tema 22/06/2026, E1]

Esercizio 2 — foglie tutte alla stessa profondità

Quale metodo. Procedura ricorsiva su albero binario, *senza parametri aggiuntivi*: la ricorsione restituisce un intero che è la risposta per il sottoalbero di radice x (come BLACK-RUN, 16/02, E2).

Il contratto. SAME-DEPTH(x): se le foglie del sottoalbero sono tutte alla stessa distanza da x restituisce *quella distanza* (una foglia: 0), altrimenti -1 , che non può mai essere una distanza vera. Il booleano chiesto è l'esito ($\neq -1$): un bool “puro” non saprebbe dire *quanto* distano i due lati. Combino i figli *dopo* le chiamate: un figlio assente si ignora; due presenti devono portare la stessa distanza, e la distanza di x è quella $+1$.

SAME-DEPTH(x)

```

1  if  $x == \text{NIL}$  return 0
2  if  $x.\text{left} == \text{NIL}$  and  $x.\text{right} == \text{NIL}$ 
3      return 0 // foglia: distanza 0
4  if  $x.\text{left} = \text{NIL}$ 
5       $d := \text{SAME-DEPTH}(x.\text{right})$ 
6  elseif  $x.\text{right} = \text{NIL}$ 
7       $d := \text{SAME-DEPTH}(x.\text{left})$ 
8  else  $d_1 := \text{SAME-DEPTH}(x.\text{left}); d_2 := \text{SAME-DEPTH}(x.\text{right})$ 
9      if  $d_1 < 0$  or  $d_2 < 0$  or  $d_1 \neq d_2$ 
10         return  $-1$ 
11     return  $d_1 + 1$ 
12 if  $d == -1$  return  $-1$ 
13 return  $d + 1$ 
```

L'esempio. Su $((a, b), (c, d))$ (albero perfetto): le due foglie sotto la radice distano 1, quindi $1 + 1 = 2$ da entrambi i lati: 2. Su $((a, (b, -)), (c, d))$ il primo figlio porta distanze 1 e 2: -1 , e la radice risponde -1 .

Risultato. Un nodo per volta: tempo $\Theta(n)$, spazio $\Theta(h)$ di pila. Verificato su 400 alberi casuali. [tema 22/06/2026, E2]

Esercizio 3 — il montaggio dei filmati

Quale metodo. Programmazione dinamica su intervalli, di tipo moltiplicazione di matrici (8.2): la scelta è l'*ultima* fusione, fra un blocco $i..k$ e un blocco $k + 1..j$.

(a) **La funzione costo.** Una strategia è un albero binario pieno sulle foglie $S_1, \dots, S_n$ che rispetta l'ordine; il costo è la somma, su ogni nodo interno, del prodotto dei fotogrammi dei figli. Con $F(i, j) = f_i + \dots + f_j$, fondere $i..k$ e $k + 1..j$ costa $F(i, k) \cdot F(k + 1, j)$. Sottoproblema: $C[i, j]$ = costo minimo per il solo intervallo $i..j$.

(b) **La relazione ricorsiva.** $C[i, i] = 0$, e per $i < j$:

$$C[i, j] = \min_{i \leq k < j} \{ C[i, k] + C[k + 1, j] + F(i, k) \cdot F(k + 1, j) \}, \quad C[1, n] \text{ è la risposta.}$$

(c) **L'algoritmo bottom-up.** Come MATRIX-CHAIN-ORDER (8.2), per lunghezze crescenti $l = j - i + 1$: quando serve $C[i, j]$, le due parti sono più corte e sono già state calcolate.

MONTAGGIO-MINIMO(f, n)

```

1  for  $i := 1$  to  $n$ 
2       $C[i, i] := 0; F[i, i] := f_i$ 
3  for  $l := 2$  to  $n$ 
4      for  $i := 1$  to  $n - l + 1$ 
5           $j := i + l - 1; C[i, j] := +\infty; F[i, j] := F[i, j - 1] + f_j$ 
6          for  $k := i$  to  $j - 1$ 
7               $cand := C[i, k] + C[k + 1, j] + F[i, k] \cdot F[k + 1, j]$ 
8              if  $cand < C[i, j]$ 
9                   $C[i, j] := cand; s[i, j] := k$ 
10 return  $C[1, n], s$ 
```

(d) **La ricostruzione:** $s[i, j]$ = dove è tagliata l'ultima fusione; si stampano prima i due lati, poi la fusione.

STAMPA-MONTAGGIO(s, i, j)

```

1  if  $i == j$  return  $S_i$ 
2   $k := s[i, j]$ 
3  return “fondi(STAMPA-MONTAGGIO( $s, i, k$ ), STAMPA-MONTAGGIO( $s, k + 1, j$ ))”
```

L'istanza. $f = \langle 5, 2, 3, 4 \rangle$ [non nelle fonti del corso] (il tema non ne dà una); $F(1, 2) = 7, F(2, 3) = 5, F(3, 4) = 7, F(1, 3) = 10, F(2, 4) = 9, F(1, 4) = 14$:

$C[i, j]$	$j=1$	$j=2$	$j=3$	$j=4$
$i=1$	0	$5 \cdot 2 = 10$	$\min\{6 + 5 \cdot 5, 10 + 7 \cdot 3\} = 31$	71
$i=2$		0	$2 \cdot 3 = 6$	$\min\{12 + 2 \cdot 7, 6 + 5 \cdot 4\} = 26$
$i=3$			0	$3 \cdot 4 = 12$
$i=4$				0

Per $C[1, 4]$ le tre scelte $k = 1, 2, 3$ danno $26 + 5 \cdot 9 = 71$, $10 + 12 + 7 \cdot 7 = 71$, $31 + 10 \cdot 4 = 71$: *tutte uguali*. Non è un caso: ogni coppia di fotogrammi di segmenti diversi viene confrontata esattamente una volta, alla fusione in cui i loro blocchi si incontrano, quindi *ogni* strategia costa $\sum_{i < j} f_i f_j = 71$: la DP chiesta dal tema trova comunque il minimo (tutte le k lo realizzano), e la ricostruzione dà $(S_1(S_2(S_3S_4)))$ (verificato su tutti gli alberi di montaggio di istanze casuali).

(e) **Complessità.** $O(n^2)$ intervalli, $O(n)$ scelte a testa (il ciclo su k): tempo $O(n^3)$, spazio $\Theta(n^2)$ per le due tabelle C e s .

Risultato. Costo minimo $C[1, n]$ con $C[i, j] = \min_k \{C[i, k] + C[k + 1, j] + F(i, k) \cdot F(k + 1, j)\}$; sull'istanza $\langle 5, 2, 3, 4 \rangle$: $C[1, 4] = 71$ (e ogni strategia costa la stessa somma $\sum_{i < j} f_i f_j$); $O(n^3)$ tempo, $\Theta(n^2)$ spazio. [tema 22/06/2026, E3]

13.5 13 luglio 2026

Tema del 13 luglio 2026

ESAME DI ALGORITMI — Università degli Studi di Catania — Corso di Laurea Triennale in Informatica — **13 luglio 2026**

Si risolvano i seguenti esercizi in un tempo non superiore a 3 ore.

1. Si considerino le due procedure $\text{MadChain}(A, i, j)$ e $\text{SpecialCompute}(A, i, j)$ descritte di seguito. Entrambe le procedure operano su un array A di n elementi ($j - i + 1 = n$). Per ciascuna di esse, scrivere l'equazione di ricorrenza che descrive il tempo di esecuzione $T(n)$. Risolvere l'equazione di ricorrenza utilizzando il metodo dell'albero di ricorsione. [nota della guida: i due pseudocodici sono, riga per riga, quelli del 22/06: MadChain è la RefineCombine , SpecialCompute è il MidnightCompute .]

2. Si consideri un albero binario. Definire lo pseudo-codice della procedura ricorsiva $\text{min-max-check}(x)$ che, ricevendo come unico argomento la radice x di un albero (o di un suo sottoalbero), verifichi se la distanza minima e la distanza massima tra la radice e una foglia sua discendente coincidono. La procedura deve restituire un valore intero o booleano che indichi se tale proprietà è soddisfatta e *non deve utilizzare parametri aggiuntivi* oltre alla radice.

3. Si consideri una iscrizione archeologica rinvenuta in frammenti, costituita da una sequenza ordinata di porzioni $F_1, F_2, \dots, F_n$. La porzione F_i contiene c_i segni ancora leggibili. Per ricostruire l'iscrizione completa, le porzioni devono essere assemblate rispettando il loro ordine nella sequenza. È tuttavia possibile scegliere liberamente l'ordine con cui effettuare le operazioni di ricostruzione intermedie. Quando due porzioni consecutive contenenti rispettivamente a e b segni vengono unite, gli archeologi devono confrontare ogni segno della prima porzione con ogni segno della seconda per verificarne la compatibilità. Il costo dell'operazione è pertanto pari a $a \cdot b$. La nuova porzione ricostruita contiene complessivamente $a + b$ segni. Si richiede di progettare una soluzione basata sulla *programmazione dinamica* che determini la strategia di ricostruzione a costo minimo. In particolare: (a) definire una funzione costo per il problema; (b) formulare una relazione ricorsiva che esprima il costo di una soluzione ottima; (c) progettare un algoritmo *bottom-up* per il calcolo del costo minimo complessivo; (d) progettare una procedura che ricostruisca e stampi la sequenza ottima delle operazioni di ricostruzione; (e) determinare la complessità temporale e spaziale degli algoritmi proposti.

$\text{MADCHAIN}(A, i, j)$

```

1  if  $i \geq j$  return  $A[i]$ 
2   $m := \lfloor (i + j)/2 \rfloor$ 
3   $L := \text{MADCHAIN}(A, i, m)$ 
4   $R := \text{MADCHAIN}(A, m + 1, j)$ 
5  for  $c := 1$  to 10
6    for  $k := i$  to  $j$ 
7       $A[k] := A[k] + L + R$ 
8   $L_2 := \text{MADCHAIN}(A, i, m)$ 
9   $R_2 := \text{MADCHAIN}(A, m + 1, j)$ 
10 for  $b := 1$  to 5
11   for  $k := i$  to  $j$ 
12      $L_2 := L_2 + A[k]$ 
13 return  $L_2 + R_2$ 
```

$\text{SPECIALCOMPUTE}(A, i, j)$

```

1  if  $i \geq j$  return
2   $m := \lfloor (i + j)/4 \rfloor$ 
3  for  $k := 1$  to  $(j - i + 1)$ 
4     $A[i] := A[i] + 1$ 
5   $\text{SPECIALCOMPUTE}(A, i, i + m)$ 
6  for  $k := 1$  to  $(j - i + 1)$ 
7     $A[i] := A[i] + 1$ 
8   $\text{SPECIALCOMPUTE}(A, i + m, i + 2m)$ 
```

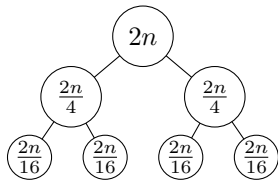
Esercizio 1 — MADCHAIN e SPECIALCOMPUTE (richiami, con l'albero)

Quale metodo. Albero di ricorsione (2.3); qui si disegna quello della ricorrenza coi quarti, che era stata risolta per sostituzione ad aprile.

MADCHAIN. È riga per riga la REFINECOMBINE del 22 giugno: quattro chiamate su metà, $15n$ fuori, quindi $T(n) = 4T(n/2) + \Theta(n) = \Theta(n^2)$ (richiamo: albero e somma al 31/03, E1b; coi $15n$: $15n(n - 1)$, contato 979 200 su $n = 2^8$).

Risultato. $\Theta(n^2)$. [tema 13/07/2026, E1]

SPECIALCOMPUTE. È il MIDNIGHTCOMPUTE del 22 giugno: due cicli da n , due chiamate su blocchi di circa $n/4$: $T(n) = 2T(\lfloor n/4 \rfloor) + \Theta(n)$. Questa volta l'albero:



livello 0: un nodo, $2n$;
 livello 1: 2 nodi, $2 \cdot \frac{2n}{4} = \frac{2n}{2}$;
 livello 2: 4 nodi, $4 \cdot \frac{2n}{16} = \frac{2n}{4}$;
 livello i : 2^i nodi da $n/4^i$, costo $\frac{2n}{2^i}$: serie geometrica, somma $4n$.
 Sui quarti la discesa finisce subito: $\log_4 n$ livelli.

Risultato. $T(n) = 2T(\lfloor n/4 \rfloor) + \Theta(n) = \Theta(n)$: stesso risultato della sostituzione di aprile (e le due strade concordano). [tema 13/07/2026, E1]

Esercizio 2 — MIN-MAX-CHECK

Quale metodo. Ricorsione senza parametri aggiuntivi, come SAME-DEPTH del 22/06: è la stessa proprietà, letta come “minima = massima distanza”. Infatti min e max delle distanze radice-foglia coincidono se e solo se *tutte* le foglie stanno alla stessa distanza: valgono contratto, pseudocodice (righe 1–13 di SAME-DEPTH) e complessità $\Theta(n)$, $\Theta(h)$ di quell'esercizio, col nuovo nome.

L'esempio. $((-, -), (-, -))$: le foglie stanno a 2 da entrambi i lati: 2 (min = max = 2, proprietà soddisfatta). $(-, (-, -))$: a sinistra una foglia dista 1, a destra 2: min = 1, max = 2: -1. Verificato che coincide con la proprietà “tutte le foglie uguali” su 300 alberi casuali.

Risultato. $\Theta(n)$ tempo, $\Theta(h)$ spazio; il valore di ritorno è la distanza comune o -1. [tema 13/07/2026, E2]

Esercizio 3 — l'iscrizione archeologica

Quale metodo. È *fisicamente lo stesso problema* del montaggio del 22 giugno (E3): fusione di blocchi consecutivi, costo $a \cdot b$, programmazione dinamica su intervalli. Le parti (a)–(e) sono quelle: qui bastano i nomi nuovi (*porzioni*, c_i segni) e il rimando.

(a)–(b). $C[i, j]$ = costo minimo per ricostruire la sola porzione di iscrizione $i..j$; $G(i, j) = c_i + \dots + c_j$; l'ultima fusione è fra $i..k$ e $k+1..j$: $C[i, j] = \min_{i \leq k < j} \{C[i, k] + C[k+1, j] + G(i, k)G(k+1, j)\}$, $C[i, i] = 0$. (c) MONTAGGIO-MINIMO con $f \rightarrow c$ (i crescente per lunghezza l ; G precalcolabile). (d) STAMPA-MONTAGGIO con $s[i, j]$ (la k dell'ultimo taglio). (e) $O(n^3)$ tempo, $\Theta(n^2)$ spazio. E come al 22 giugno: con il costo $a \cdot b$ ogni strategia costa la stessa somma $\sum_{i < j} c_i c_j$ — ogni coppia di segni è confrontata una sola volta — quindi il minimo è quello, e la DP lo trova con qualunque k .

Risultato. $\Theta(n^3)$ tempo, $\Theta(n^2)$ spazio; ogni strategia ha costo $\sum_{i < j} c_i c_j$. [tema 13/07/2026, E3]

13.6 1 settembre 2026

Tema dell'1 settembre 2026

ESAME DI ALGORITMI — Università degli Studi di Catania — Corso di Laurea Triennale in Informatica — **1 settembre 2026**
 Si risolvano i seguenti esercizi in un tempo non superiore a 3 ore.

1. Si considerino le due procedure $\text{TwistCompute}(A, i, j)$ e $\text{QuarterMix}(A, i, j)$ descritte di seguito. Entrambe le procedure operano su un array A di n elementi ($j - i + 1 = n$). Per ciascuna di esse, scrivere l'equazione di ricorrenza che descrive il tempo di esecuzione $T(n)$. Risolvere l'equazione di ricorrenza utilizzando il metodo dell'albero di ricorsione. [nota della guida: in B la numerazione delle righe riparte da 12 nel tema; qui è 1–11.]

2. Si consideri un grafo orientato aciclico (DAG) rappresentato mediante una matrice di adiacenza M di dimensione $n \times n$. Definire lo pseudocodice di una procedura ricorsiva $\text{same-terminal}(M, x, n)$ che, ricevendo in ingresso la matrice di adiacenza M , un nodo x e il numero n di nodi del grafo, verifichi se tutti i cammini orientati che partono da x e terminano in un nodo terminale conducono allo stesso nodo terminale. Un nodo si considera terminale se non possiede archi uscenti. La procedura deve utilizzare la ricorsione per esplorare il grafo e deve restituire un valore intero opportunamente scelto che consenta di determinare se la proprietà richiesta è soddisfatta.

3. Un centro logistico deve preparare per la spedizione un insieme di n lotti di merce, inizialmente separati. Il lotto L_i ha peso p_i . Prima della spedizione tutti i lotti devono essere consolidati in un unico carico. Il consolidamento avviene mediante operazioni di consolidamento: a ogni operazione vengono scelti due carichi disponibili e trasferiti insieme su una nuova unità di trasporto. Il costo di ciascuna operazione di consolidamento è pari al peso complessivo della merce movimentata durante quell'operazione. Poiché un carico già consolidato può essere nuovamente coinvolto in operazioni successive, la scelta dell'ordine con cui effettuare i consolidamenti può influire sul costo complessivo dell'intero processo. Si richiede di progettare un algoritmo che determini una sequenza di operazioni di consolidamento avente costo totale minimo. In particolare: (a) formalizzare il costo complessivo di una sequenza di consolidamenti; (b) progettare un algoritmo che determini una sequenza di consolidamenti di costo minimo; (c) fornire lo pseudocodice dell'algoritmo, specificando le strutture dati utilizzate; (d) applicare l'algoritmo all'istanza $\langle 8, 3, 14, 6, 11, 5 \rangle$ mostrando le operazioni effettuate e il costo complessivo ottenuto; (f) determinare la complessità temporale e spaziale dell'algoritmo.

$\text{TWISTCOMPUTE}(A, i, j)$

```

1  if  $i \geq j$  return  $A[i]$ 
2   $m := \lfloor (i + j)/2 \rfloor$ 
3   $X := \text{TWISTCOMPUTE}(A, i, m)$ 
4   $Y := \text{TWISTCOMPUTE}(A, m + 1, j)$ 
5  for  $h := 1$  to 4
6    for  $k := i$  to  $j$ 
7       $A[k] := A[k] + X - Y$ 
8   $Z := \text{TWISTCOMPUTE}(A, i, m)$ 
9  for  $k := i$  to  $j$ 
10    $A[k] := A[k] + Z$ 
11 return  $X + Y + Z$ 
```

$\text{QUARTERMIX}(A, i, j)$

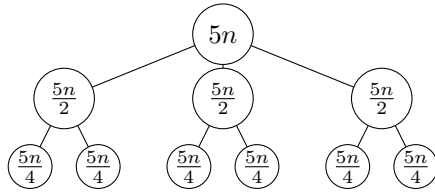
```

1  if  $i \geq j$  return  $A[i]$ 
2   $n := j - i + 1$ 
3   $m := \lfloor n/4 \rfloor$ 
4  for  $k := i$  to  $j$ 
5     $A[k] := A[k] + 1$ 
6   $X := \text{QUARTERMIX}(A, i, i + m - 1)$ 
7   $Y := \text{QUARTERMIX}(A, i + m, i + 2m - 1)$ 
8  for  $h := 1$  to  $n$ 
9    for  $k := 1$  to 3
10      $X := X + k$ 
11 return  $X + Y$ 
```

Esercizio 1 — TWISTCOMPUTE e QUARTERMIX

Quale metodo. Albero di ricorsione (2.3).

TWISTCOMPUTE. Il lavoro fuori: $4n$ (righe 5–7) $+ n$ (righe 9–10) $= 5n = \Theta(n)$; tre chiamate ricorsive (righe 3, 4, 8), su circa $n/2$: due su $\lceil n/2 \rceil$, una su $\lfloor n/2 \rfloor$. Quindi $T(n) = 3T(n/2) + \Theta(n)$: il caso in cui il master è scomodo ($n^{\log_2 3} = n^{1.585}$) e l'albero è la strada naturale.



livello i : 3^i nodi da $n/2^i$, costo $3^i \cdot \frac{5n}{2^i} = 5n(\frac{3}{2})^i$: i livelli *crescono*; l'ultimo domina:

$$5n \left(1 + \frac{3}{2} + \left(\frac{3}{2}\right)^2 + \dots \right) = \Theta(n^{1.585}).$$

Contato davvero: $T(8) = 190$, $T(64) = 6650$, $T(256) = 63050$.

Risultato. $T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.585})$. Qui $f(n) = \Theta(n)$ è *polinomialmente più piccola* di $n^{\log_2 3}$: il caso 1 del master funziona ($\varepsilon = 0.585$) e conferma. [tema 01/09/2026, E1]

QUARTERMIX. Cicli da n e da $3n$: fuori $\Theta(n)$; due chiamate su blocchi *espliciti* di $m = \lfloor n/4 \rfloor$ elementi. Quindi $T(n) = 2T(\lfloor n/4 \rfloor) + \Theta(n)$: la ricorrenza del 27 aprile, E1a, e di SPECIALCOMPUTE — per sostituzione, $T(n) \leq dn$ con $d \geq 2c$.

Risultato. $T(n) = 2T(\lfloor n/4 \rfloor) + \Theta(n) = \Theta(n)$: albero nel 13/07, E1 (contato: $T(64) = 448$, $T(256) = 1920$). [tema 01/09/2026, E1]

Esercizio 2 — SAME-TERMINAL su un dag

Quale metodo. Procedura ricorsiva su un dag senza parametri aggiuntivi: l'intero restituito è la risposta per il sotto-dag di radice x (scaletta in 10.6).

Il contratto. SAME-TERMINAL(M, x, n) restituisce il nodo terminale comune a tutti i cammini che partono da x ; -1 se non c'è. Caso base: se la riga x di M è tutta zero, x è terminale: la risposta è x stesso. Passo: ogni figlio y ($M[x][y] \neq 0$) propone il suo terminale; il primo “fissa” il candidato, gli altri lo confermano, e appena uno risponde -1 o un terminale diverso tutta la chiamata risponde -1 . Il dag non ha cicli: la ricorsione termina.

SAME-TERMINAL(M, x, n)

```

1  t := NIL // il terminale candidato
2  for y := 1 to n
3      if M[x][y] ≠ 0
4          ty := SAME-TERMINAL(M, y, n)
5          if ty == -1 return -1
6          if t == NIL
7              t := ty
8          elseif ty ≠ t
9              return -1
10 if t == NIL return x // x è terminale
11 return t
```

L'esempio (i due dag dell'esempio del capitolo 10.6). Dag (a): $1 \rightarrow 2(2)$, $1 \rightarrow 3(1)$, $2 \rightarrow 4(1)$, $3 \rightarrow 4(2)$. SAME-TERMINAL($M, 1, 4$): figli 2 e 3; da 2 il cammino finisce in 4, da 3 pure: $t = 4$, confermato: **4**. Dag (b), dove un cammino finisce in 4 e un altro prosegue fino a 5: il primo figlio fissa 4, il secondo risponde $5 \neq 4$: **-1**.

Risultato. Un nodo per volta, ma le chiamate si moltiplicano sui cammini (un dag a “diamante” come (a) riesplora il tratto condiviso): una chiamata per ogni prefisso di cammino (al più n per cammino), e ogni chiamata scandisce la riga di M ($\Theta(n)$): il tempo è $O(n \cdot \#\text{prefissi})$, e su un dag a diamanti annidati i prefissi sono esponenziali ($2^{k+2} - 3$ chiamate su k diamanti): la ricorsione pura li riesplora tutti, mentre con una visita con memoria (una chiamata per nodo) si scende a $\Theta(n^2)$. Spazio $\Theta(h)$ di pila. Nessun parametro aggiuntivo: la risposta viaggia nel valore di ritorno. [tema 01/09/2026, E2]

Esercizio 3 — il consolidamento dei lotti (goloso)

Quale metodo. Fusione iterativa a costo minimo = *Huffman*: coda di min-priorità, fondi sempre i due carichi più leggeri (9.4, 9.3).

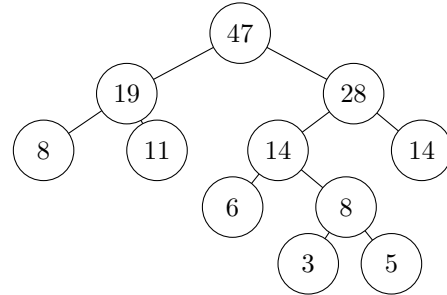
(a) **Il costo.** Una sequenza di consolidamenti è un albero binario pieno coi lotti alle foglie; ogni operazione muove i due carichi scelti, quindi il costo totale è $\sum_i p_i d_i$, con d_i la profondità della foglia del lotto i : una foglia profonda paga il suo peso tante volte.

(b) **L'algoritmo.** Il goloso di Huffman: prendi i due *più leggeri*, fondili, rimetti il carico somma; ripeti $n - 1$ volte. Non è “a caso”: le due parti più leggere stanno in fondo all'albero ottimo (Lemma 16.2, scambio), e dopo la fusione resta lo stesso problema sui pesi rimasti (Lemma 16.3).

(c) **Lo pseudocodice** (coda di min-priorità Q con EXTRACT-MIN/INSERT, come in HUFFMAN): $Q :=$ tutti i p_i ; **mentre** $|Q| > 1$: $a := \text{EXTRACT-MIN}(Q)$; $b := \text{EXTRACT-MIN}(Q)$; $tot := tot + a + b$; INSERT($Q, a + b$). Serve un *heap binario* (o un heap di Fibonacci): estrazione e inserimento in $O(\lg n)$.

(d) L'istanza $\langle 8, 3, 14, 6, 11, 5 \rangle$: ordinata $\langle 3, 5, 6, 8, 11, 14 \rangle$, e le operazioni (la prima colonna è il costo):

operazione	costo
$3 + 5 = 8$	8
$6 + 8 = 14$	14
$8 + 11 = 19$	19
$14 + 14 = 28$	28
$19 + 28 = 47$	47
totale	116



L'albero delle fusioni: ogni nodo interno è il costo di quella fusione (il peso trasferito), e la somma dei nodi interni è il costo totale: $8 + 14 + 19 + 28 + 47 = 116$. La forza bruta su *tutte* le sequenze di fusioni possibili (2700) conferma che 116 è il minimo.

(f) **Complessità.** Una coda di min-priorità e $2n - 1$ operazioni da $O(\lg n)$: tempo $O(n \lg n)$, spazio $O(n)$.

Risultato. Consolidamento ottimo: fondi sempre i due carichi più leggeri (come HUFFMAN); l'istanza costa **116**, con le cinque operazioni $3+5$, $6+8$, $8+11$, $14+14$, $19+28$. Nota di lettura: il tema salta da (d) a (f) (la (e) non c'è). [tema 01/09/2026, E3]

13.7 21 settembre 2026

Tema del 21 settembre 2026

ESAME DI ALGORITMI — Università degli Studi di Catania — Corso di Laurea Triennale in Informatica — **21 settembre 2026**
Si risolvano i seguenti esercizi in un tempo non superiore a 3 ore.

1. Si considerino le due procedure `PairCompute(A, i, j)` e `SkewMix(A, i, j)` descritte di seguito. Entrambe le procedure operano su un array A di n elementi ($j - i + 1 = n$). Per ciascuna di esse, scrivere l'equazione di ricorrenza che descrive il tempo di esecuzione $T(n)$ e risolverla utilizzando il metodo di sostituzione.
2. Si consideri un grafo orientato aciclico (DAG) pesato, rappresentato mediante una matrice M di dimensione $n \times n$. Per ogni coppia di nodi u, v , il valore $M[u, v]$ è uguale al peso positivo dell'arco (u, v) , se tale arco esiste, e vale 0 altrimenti. Definire lo pseudocodice di una procedura ricorsiva `same-cost(M, x, n)` che, ricevendo in ingresso la matrice M , un nodo x e il numero n di nodi del grafo, verifichi se tutti i cammini orientati che partono da x e terminano in un nodo terminale hanno lo stesso costo complessivo, dove il costo di un cammino è la somma dei pesi degli archi che lo compongono. Un nodo si considera terminale se non possiede archi uscenti. La procedura deve utilizzare la ricorsione per esplorare il grafo e deve restituire un valore intero opportunamente scelto che consenta di determinare se la proprietà richiesta è soddisfatta.
3. Un'azienda deve certificare un sistema software composto da n moduli indipendenti $M_1, \dots, M_n$. Per il modulo M_i è disponibile una suite di p_i test. La certificazione del sistema avviene costruendo progressivamente sottosistemi sempre più grandi. Inizialmente ciascun modulo costituisce un sottosistema separato. A ogni passo si scelgono due sottosistemi già certificati e li si integra in un nuovo sottosistema. Per certificare il nuovo sottosistema devono essere rieseguiti tutti i test appartenenti ai due sottosistemi coinvolti nell'integrazione. Di conseguenza, se i due sottosistemi contengono complessivamente rispettivamente a e b test, il costo dell'operazione di integrazione è $a + b$, e il nuovo sottosistema contiene complessivamente $a + b$ test. Il procedimento termina quando tutti i moduli appartengono a un unico sottosistema certificato. Si richiede di progettare un algoritmo che determini una sequenza di integrazioni avente costo totale minimo. In particolare: (a) formalizzare il costo complessivo di una sequenza di integrazioni; (b) progettare un algoritmo che determini una sequenza di integrazioni di costo minimo; (c) fornire lo pseudocodice dell'algoritmo, specificando le strutture dati utilizzate; (d) applicare l'algoritmo all'istanza $\langle 4, 7, 9, 12, 15, 18 \rangle$ mostrando le operazioni effettuate e il costo complessivo ottenuto; (e) giustificare brevemente perché la strategia proposta produce una soluzione ottima; (f) determinare la complessità temporale e spaziale dell'algoritmo.

`PAIRCOMPUTE(A, i, j)`

```

1  if  $i \geq j$  return  $A[i]$ 
2   $m := \lfloor (i + j) / 2 \rfloor$ 
3   $X := \text{PAIRCOMPUTE}(A, i, m)$ 
4   $Y := \text{PAIRCOMPUTE}(A, m + 1, j)$ 
5  for  $h := 1$  to 3
6    for  $k := i$  to  $j$ 
7       $A[k] := A[k] + X - Y$ 
8  return  $X + Y$ 
```

`SKEWMIX(A, i, j)`

```

1   $n := j - i + 1$ 
2  if  $n \leq 2$  return  $A[i]$ 
3   $m := \lfloor n / 3 \rfloor$ 
4   $X := \text{SKEWMIX}(A, i, i + m - 1)$ 
5   $Y := \text{SKEWMIX}(A, j - m + 1, j)$ 
6  for  $k := i$  to  $j$ 
7     $A[k] := A[k] + X + Y$ 
8  return  $X - Y$ 
```

Esercizio 1 — PAIRCOMPUTE e SKEWMIX

Quale metodo. Sostituzione (2.2).

PAIRCOMPUTE. Il lavoro fuori sono $3n$ giri del ciclo interno (righe 5–7: h da 1 a 3): $\Theta(n)$; due chiamate su $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$. Quindi $T(n) = 2T(n/2) + 3n$: il caso 2 del master direbbe $\Theta(n \lg n)$, ma il tema chiede la sostituzione, e si fa così. Ipotesi $T(n) \leq dn \lg n$:

$$T(n) \leq 2 \cdot d \frac{n}{2} \lg \frac{n}{2} + 3n = dn(\lg n - 1) + 3n = dn \lg n - \underbrace{(d-3)n}_{\leq 0 \text{ se } d \geq 3} \leq dn \lg n.$$

Caso base: d grande abbastanza. Quindi $T(n) = O(n \lg n)$; e $T(n) \geq cn \lg n$ (l'albero somma $\Theta(n \lg n)$), quindi è Θ .

Risultato. $T(n) = 2T(n/2) + 3n = \Theta(n \lg n)$; sulle potenze di 2 vale $T(2^k) = 3 \cdot 2^k k$ esatto (contato: $T(1024) = 30720 = 3 \cdot 1024 \cdot 10$). [tema 21/09/2026, E1]

SKEWMIX. Il caso base è $n \leq 2$ (con $n = 2$ si esce subito); il ciclo interno è n giri: $\Theta(n)$; le due chiamate usano $m = \lfloor n/3 \rfloor$: $[i, i + m - 1]$ (la *testa*) e $[j - m + 1, j]$ (la *coda*), lunghe m . Quindi $T(n) = 2T(\lfloor n/3 \rfloor) + n$. Ipotesi $T(n) \leq dn$: $T(n) \leq 2d\frac{n}{3} + n = \frac{2dn}{3} + n \leq dn$ non appena $n \leq \frac{dn}{3}$, cioè $d \geq 3$. Quindi $T(n) = O(n)$; il solo ciclo in radice dà $T(n) \geq n$: $\Theta(n)$.

Risultato. $T(n) = 2T(\lfloor n/3 \rfloor) + n = \Theta(n)$: l'albero somma $n(1 + \frac{2}{3} + \frac{4}{9} + \dots) = 3n$; contato: $T(1000) = 2726 \leq 3000$. [tema 21/09/2026, E1]

Esercizio 2 — SAME-COST su un dag pesato

Quale metodo. Ricorsione senza parametri aggiuntivi sul dag (10.6), come SAME-TERMINAL dell'1/09: il valore di ritorno è il costo comune per il sotto-dag di x , -1 se non c'è.

Il contratto. Caso base: x terminale (riga di M tutta zero) $\Rightarrow 0$ (il cammino è vuoto). Passo: per ogni figlio y , il costo del cammino da y più il peso dell'arco $M[x][y]$; il primo figlio fissa il candidato, gli altri devono coincidere; il peso si aggiunge *dopo* la chiamata.

SAME-COST(M, x, n)

```

1  cd := NIL // il costo candidato
2  for y := 1 to n
3      if M[x][y] ≠ 0
4          cy := SAME-COST(M, y, n)
5          if cy == -1 return -1
6          if cd == NIL
7              cd := M[x][y] + cy
8          elseif M[x][y] + cy ≠ cd
9              return -1
10 if cd == NIL return 0 // x terminale, cammino vuoto
11 return cd
```

L'esempio (dell'esempio del capitolo 10.6). Dag (a): $1 \rightarrow 2(2)$, $1 \rightarrow 3(1)$, $2 \rightarrow 4(1)$, $3 \rightarrow 4(2)$: da 2 il costo è 1, totale $2 + 1 = 3$; da 3 il costo è 2, totale $1 + 2 = 3$: uguali, **3**. Dag (b): $4 \rightarrow 5$ in più, pesi $1 \rightarrow 2(2)$, $2 \rightarrow 4(1)$, $1 \rightarrow 3(1)$, $3 \rightarrow 4(3)$: i due cammini costano $2 + 1 + 1 = 4$ e $1 + 3 + 1 = 5$: **-1**. E se i cammini finiscono in nodi *diversi* ma con lo stesso costo (due archi di peso 1 verso 4 e verso 5), il costo comune è 2: SAME-COST risponde 2, SAME-TERMINAL no.

Risultato. Come SAME-TERMINAL: una chiamata per ogni prefisso di cammino e $\Theta(n)$ per scandire la riga di M ; spazio $\Theta(h)$; -1 come “non vale”. [tema 21/09/2026, E2]

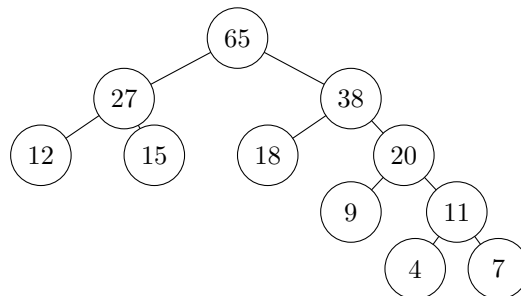
Esercizio 3 — la certificazione dei sottosistemi

Quale metodo. È l'esercizio del consolidamento dell'1/09 (E3): integrazione di sottosistemi, costo $a + b$ = somma dei test rieseguiti: goloso su coda di min-priorità (9.4). Le parti (a)–(d) e (f) sono quelle, coi nomi dei *test*; qui la parte (e), la giustificazione.

(e) Perché è ottimo. L'albero delle integrazioni: ogni sottosistema paga i suoi test a ogni integrazione in cui entra, quindi costo = $\sum_i p_i d_i$ con d_i = numero di integrazioni del modulo i : è esattamente la (16.4) di Huffman con i pesi al posto delle frequenze. Il **Lemma 16.2** (scambio): nel primo passo esiste un albero ottimo in cui i due moduli più leggeri sono *fratelli* alla profondità massima; il **Lemma 16.3** (sottostruttura): fusi i due in uno solo di peso $p_1 + p_2$, i restanti sono ancora un problema di fusioni ottimali. Per induzione: il goloso è ottimo.

L'istanza $\langle 4, 7, 9, 12, 15, 18 \rangle$: già ordinata; le operazioni:

operazione	costo
$4 + 7 = 11$	11
$9 + 11 = 20$	20
$12 + 15 = 27$	27
$18 + 20 = 38$	38
$27 + 38 = 65$	65
totale	161



Risultato. Costo **161** = $11 + 20 + 27 + 38 + 65$; è il minimo (forza bruta su tutte le sequenze di fusioni). Complessità $O(n \lg n)$ tempo, $O(n)$ spazio. [tema 21/09/2026, E3]

Il tipo ricorrente: «una procedura ricorsiva, senza parametri aggiuntivi»

Il secondo esercizio è una procedura ricorsiva su alberi, dag o stringhe, spesso senza parametri aggiuntivi: la risposta in sei righe. (1) **Cosa ritorna:** la risposta per il sottoalbero (o sottodag) di radice x — non per

tutto l'albero; se un valore non basta, una coppia (BLACK-RUN: "la run che parte da me" e "il massimo del sottoalbero") o una sentinella (-1 = "non vale", in SAME-DEPTH, MIN-MAX-CHECK, SAME-TERMINAL, SAME-COST). (2) **Caso base**, da scrivere per primo: foglia (0), NIL (in BLACK-RUN una foglia nera: $S = R = 1$), nodo senza archi uscenti (x o 0). (3) **Il passo**: richiamo sui figli, *poi* combino (max, somma col peso dell'arco, uguaglianza). (4) **Perché termina**: l'albero scende di un livello, il dag non ha cicli (dillo!). (5) **Costo**: un nodo (o un cammino) alla volta: $\Theta(n)$ per alberi e stringhe, $\Theta(n)$ per chiamata sui dag ma tante chiamate quanti i prefissi di cammino (esponenziali sui diamanti: attenzione), $\Theta(n + m)$ per ISUBSEQUENCE. (6) **Verifica**: a mano su un albero piccolo, col caso "che non vale" compreso.

Errori tipici in queste prove

- **Il metodo lo chiede il tema**: svolgerne un altro è un esercizio incompleto.
- **Contare le operazioni giuste**: una chiamata costa n^2 se dentro c'è un ordinamento quadratico (SHUFFLEANDSORT), non $\lg n$; e due cicli da n costano n^2 solo se *annidati* (MIDNIGHTCOMPUTE ne ha due *separati*, $\Theta(n)$ l'uno).
- **Dimenticare il caso base** e scrivere $T(n) = 2T(n/2) + \dots$ senza dire che le taglie piccole costano $\Theta(1)$: i nodi di dimensione 1, nell'albero, *non* pagano.
- **Nei problemi a intervalli, ignorare l'ordine**: nelle fusioni (1/09, 21/09) si fondono i due *più piccoli*; nel montaggio (22/06, 13/07) i blocchi sono *consecutivi*: l'ultima fusione è fra $i..k$ e $k+1..j$ e si sceglie su k .
- **La tabella delle scelte**: senza $s[i, j]$, $pred[j]$ o la scelta booleana non si ricostruisce: salva *mentre* riempi la tabella.

Capitolo 14

La prova del 2025 e la simulazione, svolte

Due temi a sei esercizi: il tema del **27 ottobre 2025** e la **simulazione del 22 gennaio 2026**, svolti per intero e in ordine cronologico: i due temi “brevi”, sei esercizi in tre ore. I testi sono riportati com’è, refusi compresi; tutti i numeri del capitolo sono prodotti da `verifiche/prove_2025.py`.

14.1 27 ottobre 2025

Tema del 27 ottobre 2025

ESAME DI ALGORITMI — Università degli Studi di Catania — Corso di Laurea Triennale in Informatica — **27 ottobre 2025**

Si risolvano i seguenti esercizi in un tempo non superiore a 3 ore.

1. Si considerino le due procedure `Property` e `Value` descritte di seguito. La prima opera su un array *ordinato*, mentre la seconda su un *sotto-array* i cui indici variano da $i > 0$ a $j < n$. Per ciascuna di esse, scrivere l’equazione di ricorrenza che descrive il tempo di esecuzione $T(n)$ ottenuta quando la procedura è chiamata su di un array di n elementi. Risolvere l’equazione utilizzando uno dei metodi noti (albero di ricorsione, sostituzione o Teorema Master).
2. Scrivere una funzione ricorsiva `redMax(x)`, preso in input il nodo radice x di un albero Rosso-Nero, che restituisca la chiave di valore maggiore, limitatamente alle chiavi contenute all’interno di nodi rossi. Nel caso in cui non fossero presenti nodi rossi all’interno dell’albero rosso-nero, la procedura restituisce $-\infty$.
3. Un sistema di scheduling gestisce le richieste di un insieme di n processi $\{p_1, p_2, \dots, p_n\}$ che hanno necessità di accedere ad una risorsa condivisa del sistema. Una volta che un processo ha utilizzato la risorsa, la sua richiesta decade. Ad ogni processo p_i è assegnata una priorità v_i di accesso alla risorsa, ma tale priorità può variare (diminuire o aumentare) ogni secondo. Definisci la struttura dati che il sistema di scheduling utilizzerebbe per gestire l’assegnazione, fornendo lo pseudo-codice delle procedure coinvolte nella gestione.
4. Fornire la funzione ricorsiva su cui si basa l’algoritmo di programmazione dinamica per il calcolo del costo di una parentesizzazione ottimale di una sequenza di n matrici. Si fornisca inoltre anche la procedura iterativa che permette di calcolare, utilizzando una strategia bottom-up, il costo di tale parentesizzazione ottimale.
5. Si enunci il problema di ottimizzazione affrontato e risolto dal ben noto algoritmo di Huffman. Si fornisca la funzione che assegna ad un albero di Huffman T il suo costo. Si dimostri che il problema gode della proprietà della scelta greedy.
6. Fornire un esempio concreto di un albero rosso-nero valido che contiene 12 chiavi distinte in cui un’operazione di cancellazione provoca la diminuzione dell’altezza nera dell’albero. Successivamente, fornire un’altro esempio concreto di un albero rosso-nero valido che contiene 12 chiavi distinte in cui un’operazione di inserimento provoca l’aumento dell’altezza nera dell’albero. Per entrambi gli esempi, disegnare la configurazione dell’albero prima e dopo le operazioni..

Le due procedure dell’esercizio 1 (scritte nei temi in stile C: ecco la trascrizione negli usuali `codebox`, la forma da usare sul foglio):

`PROPERTY(A, n)`

```
1  if n == 1 return A[0]
2  m := 0
3  for i := 0 to n - 1
4      for j := i to n - 1
5          m := m + A[i] · A[j]
6  return m + PROPERTY(A, n/2)
```

`VALUE(A, i, j)`

```
1  if i ≥ j - 2 return A[i]
2  m := max(A[i ... j])
3  d := (j - i + 1) / 4
4  m := m + VALUE(A, i, i + d)
5  m := m + VALUE(A, j - d, j)
6  return m
```

Quale metodo. Due ricorrenze da ricavare contando le operazioni; si risolvono col teorema master (per `PROPERTY`) e con l’albero di ricorsione e la sostituzione (per `VALUE`, dove il “+1” delle dimensioni impedisce di applicare il master così com’è).

`PROPERTY(A, n)`. Il doppio ciclo esegue $\sum_{i=0}^{n-1} (n-i) = n(n+1)/2 = \Theta(n^2)$ somme; poi la procedura richiama sé stessa sui primi $\lfloor n/2 \rfloor$ elementi, $A[0 \dots \lfloor n/2 \rfloor - 1]$ (le posizioni contano, il contenuto no: che l’array sia ordinato non cambia i conti). Dunque, con $T(1) = \Theta(1)$:

$$T(n) = T(\lfloor n/2 \rfloor) + \frac{n(n+1)}{2}, \quad (14.1)$$

ovvero $T(n) = T(\lfloor n/2 \rfloor) + \Theta(n^2)$. Teorema master (2.4), caso 3 ($a = 1$, $b = 2$, $n^{\lg_b a} = 1$; e la regolarità regge, $f(n/2) = \frac{n(n+2)}{8} = \frac{1}{4}f(n)(1 + o(1))$): $T(n) = \Theta(n^2)$. Conti esatti per $n = 2^k$ (sommati livello per

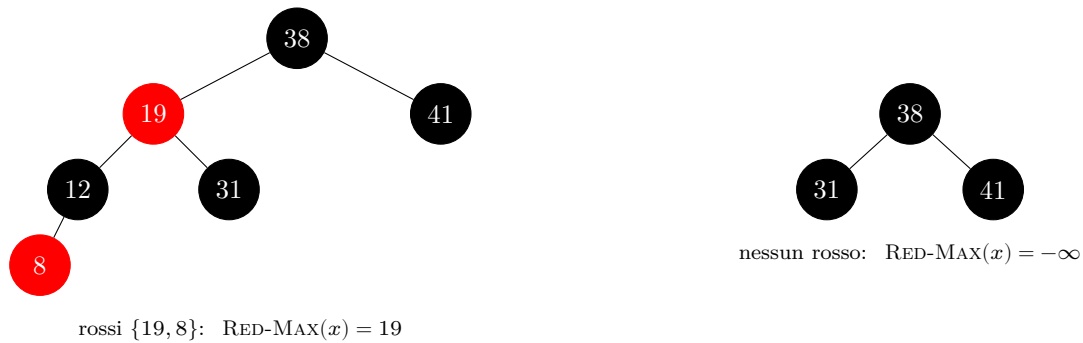


Figura 14.1: RED-MAX sui due alberi del libro: la figura dell'Esercizio 13.3-2 (a sinistra, albero di 7.12) e l'albero tutto nero che si incontra a metà della sequenza di cancellazioni dell'Esercizio 13.4-3. [Cormen esercizi 13.3-2 e 13.4-3]

livello): le somme sono 3, 13, 49, 185, 713, ..., cioè

$$T(n) = \frac{2n^2 + 3n - 5}{3} = \frac{2}{3}n^2 + O(n) :$$

la ricorrenza è dominata dal livello della radice.

VALUE(A, i, j). Qui sta l'attenzione alle *dimensioni*: con $s = j - i + 1$ elementi, il massimo della riga 2 costa $\Theta(s)$, e i due sottoarray delle righe 4–5 hanno $d + 1 = \lfloor s/4 \rfloor + 1$ elementi — *non* $s/4$: c'è un elemento in più. La ricorrenza è

$$T(s) = 2T(\lfloor s/4 \rfloor + 1) + \Theta(s), \quad T(s) = \Theta(1) \quad (s \leq 3), \quad (14.2)$$

perché il caso base $i \geq j - 2$ copre al più tre elementi. Il “+1” sposta solo di poco le dimensioni: al livello k ci sono 2^k sottoarray da circa $s/4^k + \frac{4}{3}$ elementi, ciascuno di costo $\Theta(s/4^k)$; sommando, $\sum_k s/2^k = 2s$ dal massimo e $\sum_k 2^k \cdot \frac{4}{3} = O(\sqrt{s})$ dal “+1”: $T(s) = \Theta(s)$. (Senza il +1, $2T(s/4) + \Theta(s)$ sarebbe $\Theta(s)$ già col master, caso 3; con il +1 chiude la sostituzione $T(s) \leq ds$, $d = 4c$, per $s \geq 16$.) Controllo esatto per $s = 4^k$: le operazioni sono 6, 30, 126, 510, 2046, ... = $2s - 2$, e $T(s)/s \rightarrow 2$: lineare.

Risultato. $T(n) = T(\lfloor n/2 \rfloor) + n(n+1)/2 = \Theta(n^2)$; $T(s) = 2T(\lfloor s/4 \rfloor + 1) + \Theta(s) = \Theta(s)$ (il conteggio esatto è $2s - 2$ per $s = 4^k$). [tema 27/10/2025, E1]

Quale metodo. Secondo esercizio: una **procedura ricorsiva** sull'albero rosso-nero, con il solo nodo come parametro; il valore neutro per il massimo è $-\infty$. La scaletta è quella del capitolo 7 (metodobox sulle procedure senza parametri aggiuntivi).

RED-MAX(x). Alla sentinella non c'è nessun nodo rosso: $-\infty$. Altrimenti si prendono i due massimi dei figli e, se x è rosso, si aggiunge la sua chiave al confronto.

RED-MAX(x)

```

1  if  $x == T.nil$  return  $-\infty$ 
2   $l := \text{RED-MAX}(x.left)$ ;  $r := \text{RED-MAX}(x.right)$ 
3  if  $x.color == \text{RED}$  return  $\max(x.key, l, r)$ 
4  return  $\max(l, r)$ 
```

Sull'albero dell'Esercizio 13.3-2 (figura 14.1, a sinistra) i nodi rossi sono 19 e 8: la chiave massima dell'albero è 41, ma è nera, quindi RED-MAX(x) = 19. A destra l'albero (tutto nero) che compare nella sequenza di cancellazioni dell'Esercizio 13.4-3, dopo 8, 12 e 19: lì RED-MAX(x) = $-\infty$. Non si può “potare” (una chiave rossa grande può stare in qualsiasi punto): ogni nodo si visita una volta sola, $\Theta(n)$ di tempo e $O(h)$ di pila.

Risultato. RED-MAX(38) = 19 sull'albero 13.3-2 (i rossi sono {19, 8}); $-\infty$ sull'albero tutto nero 38(31, 41). Ogni nodo una volta: $\Theta(n)$. [tema 27/10/2025, E2]

Quale metodo. Un problema di progetto di **struttura dati**: le priorità salgono e scendono, e ogni secondo serve sapere chi ha la priorità massima. La struttura è una **coda di priorità** (max-heap) con un indice dei processi: le operazioni sono quelle del capitolo 03 (3.5), più la DECREASE-KEY, che il libro non tratta (si ottiene da MAX-HEAPIFY).

La struttura dati. Un max-heap $A[1 \dots m]$ (i figli di $A[i]$ sono $A[2i]$ e $A[2i + 1]$, come nel capitolo 03) che memorizza coppie (p_i, v_i) — il processo e la sua priorità corrente — con in più un array $A.pos$ indicizzato dai processi: $A.pos[p_i]$ è la posizione di p_i nell'heap. Le operazioni dello scheduling:

- **assegnare la risorsa:** il processo con priorità massima è $\text{HEAP-MAXIMUM}(A) = A[1]$, in $\Theta(1)$; quando la richiesta *decade* si fa HEAP-EXTRACT-MAX , servendo $A[1]$;
- **nuova richiesta:** MAX-HEAP-INSERT ;
- **la priorità cambia:** CAMBIA-PRIORITY , che trova p_i con l'indice $A.pos$ (niente ricerca $O(m)$: le procedure lo aggiornano a ogni scambio).

$\text{CAMBIA-PRIORITY}(A, p, v)$

```
1   $i := A.pos[p]$  // la posizione di  $p$ 
2  if  $v \geq A[i].prio$ 
3       $\text{HEAP-INCREASE-KEY}(A, i, v)$ 
4  else  $\text{HEAP-DECREASE-KEY}(A, i, v)$ 
```

$\text{HEAP-DECREASE-KEY}(A, i, key)$

```
1   $A[i].prio := key$ 
2   $\text{MAX-HEAPIFY}(A, i)$  // da lì può solo scendere
```

HEAP-INCREASE-KEY , MAX-HEAP-INSERT e HEAP-EXTRACT-MAX sono nel capitolo 03 (3.5): la chiave sale risalendo dai padri (righe 4–6 di HEAP-INCREASE-KEY), e i loro scambi aggiornano insieme $A.pos$, quindi ogni operazione resta in $O(\lg n)$ — il cammino radice-foglia è alto $\lg n$. Sull'istanza a cinque processi (stessa coppia processo-priorità dell'esecuzione nello script):

operazione	heap (processo: priorità), dall'alto verso il basso
insert $(p_1, 8), (p_2, 3), (p_3, 12), (p_4, 1), (p_5, 6)$	$p_3:12, p_5:6, p_1:8, p_4:1, p_2:3$
$\text{CAMBIA-PRIORITY}(p_2, 15)$	$p_2:15, p_3:12, p_1:8, p_4:1, p_5:6$
$\text{CAMBIA-PRIORITY}(p_3, 2)$	$p_2:15, p_5:6, p_1:8, p_4:1, p_3:2$
HEAP-EXTRACT-MAX	servito p_2 (priorità 15); resta $p_1:8, p_5:6, p_3:2, p_4:1$

(aumentare p_2 la fa risalire fino alla radice; abbassare p_3 la fa scendere con MAX-HEAPIFY ; la proprietà del max-heap è verificata dopo ogni singola operazione nello script.) [non nelle fonti del corso]

Risultato. Max-heap di coppie (p_i, v_i) con l'indice $A.pos$: $\text{CAMBIA-PRIORITY} = \text{HEAP-INCREASE-KEY}$ o HEAP-DECREASE-KEY ($O(\lg n)$), più HEAP-MAXIMUM ($\Theta(1)$), HEAP-EXTRACT-MAX e MAX-HEAP-INSERT . [tema 27/10/2025, E3]

Quale metodo. Programmazione dinamica su intervalli: la ricorrenza $m[i, j]$ e la procedura bottom-up $\text{MATRIX-CHAIN-ORDER}$ del capitolo 08 (8.1 e 8.2); qui la risposta d'esame, con la verifica sui numeri del libro.

La funzione ricorsiva. Con $p = \langle p_0, p_1, \dots, p_n \rangle$ le dimensioni delle matrici (A_i è $p_{i-1} \times p_i$), il costo minimo $m[i, j]$ della parentesizzazione ottima di $A_{i..j}$ è

$$m[i, j] = \begin{cases} 0 & i = j, \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\} & i < j, \end{cases}$$

e insieme a m si tiene la scelta $s[i, j]$ (il k che realizza il minimo), che serve a ricostruire la parentesizzazione. La sottostruttura ottima: una parentesizzazione ottima di $A_{i..j}$ si spezza tra A_k e A_{k+1} , e i due pezzi devono essere ottimi.

La procedura iterativa. I sottoproblemi distinti sono $\Theta(n^2)$ ($1 \leq i \leq j \leq n$) e $m[i, j]$ dipende solo da lunghezze minori: si riempiono le tabelle per *lunghezze crescenti* $l = j - i + 1 = 2, 3, \dots, n$, e per ogni (i, j) si provano i $j - i$ valori di k . È lo pseudocodice del capitolo 08 (8.2), tre cicli annidati: tempo $\Theta(n^3)$, spazio $\Theta(n^2)$.

Controllo sui numeri del libro: $p = \langle 30, 35, 15, 5, 10, 20, 25 \rangle$: $m[1, 6] = 15125$ con $s[1, 6] = 3$ (la tavola completa, cella per cella, è nel capitolo 08 8.2).

Risultato. $m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\}$ (vale 0 per $i = j$); $\text{MATRIX-CHAIN-ORDER}$ riempie per lunghezze crescenti in $\Theta(n^3)$. Sulle dimensioni del libro $m[1, 6] = 15125$. [Cormen cap. 15.2; tema 27/10/2025, E4]

Quale metodo. Il problema dei codici di Huffman: la scelta golosa (fondere le due frequenze minime) si dimostra con lo *scambio* di due foglie, ed è la dimostrazione del capitolo 09 (9.3); la risposta d'esame completa:

Enunciato. Dato un alfabeto C con una frequenza $c.freq$ per ogni carattere, si cerca un **codice prefisso** ottimo: un albero binario T con i caratteri sulle foglie (codice = cammino radice-foglia, 0 a sinistra e 1 a destra) di costo minimo. Formalmente: minimizzare

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c),$$

dove $d_T(c)$ è la profondità della foglia c (numero di bit del codice). [Cormen eq. (16.4)] Per l'Esercizio 16.3-4, $B(T)$ è anche la somma dei costi delle fusioni: il costo di una fusione è la somma delle frequenze fuse.

La proprietà della scelta golosa (Lemma 16.2). Siano x e y i due caratteri di *frequenza minima* e a , b due foglie sorelle di *profondità massima* in un albero ottimo T . Scambiando le posizioni di a e x (albero T') e poi di b e y (albero T''):

$$B(T) - B(T') = (a.freq - x.freq)(d_T(a) - d_T(x)) \geq 0$$

perché x ha frequenza minima e a profondità massima; analogamente $B(T') - B(T'') \geq 0$. Quindi $B(T'') \leq B(T)$, e poiché T è ottimo, $B(T'') = B(T)$: esiste un albero ottimo in cui x e y sono foglie sorelle alla massima profondità. Fondere x e y per prima è la scelta golosa, ed è sicura. La dimostrazione estesa (i due scambi, la sottostruttura ottima del Lemma 16.3 e il Teorema 16.4) è nel capitolo 09 (9.3). Sull'esempio del libro ($a:45$, $b:13$, $c:12$, $d:16$, $e:9$, $f:5$) le fusioni costano $14 + 25 + 30 + 55 + 100 = 224$ migliaia di bit, contro i 300 a lunghezza fissa.

Risultato. $B(T) = \sum_c c.freq d_T(c)$, minimo; scelta golosa: fondere le due frequenze minime. La dimostrazione è lo scambio $a \leftrightarrow x$, $b \leftrightarrow y$, da cui $B(T'') \leq B(T)$ (dettagli in 9.3). [Cormen cap. 16.3, Lemma 16.2; temi 27/10/2025 e 22/01/2026, E5]

Quale metodo. L'esercizio in cui il disegno è tutto: un albero e un'operazione, *prima* e *dopo*, coi casi del fixup del capitolo 7 (7.14 per la cancellazione). Aumenta l'altezza nera un inserimento la cui catena di *casi* 1 arriva fino alla radice; la abbassa una cancellazione il cui *caso* 2 risale fino alla radice: in entrambi i casi, alla fine la radice resta nera e ogni cammino guadagna (o perde) esattamente un nero.

(a) L'inserimento (12 chiavi). L'albero di sinistra è valido (cinque proprietà e $bh = 2$: verificato) e ha 12 chiavi. Si inserisce 13: scende come in un BST e si appende *rosso* come figlio destro di 12. Il padre 12 è rosso e lo zio 10 è rosso: *caso 1* (12 e 10 diventano neri, il nonno 11 rosso, si risale a 11). Di nuovo: il padre 9 è rosso e lo zio 2 è rosso: *caso 1* (9 e 2 neri, 5 rosso, si risale a 5, che è la radice). La radice torna nera e ogni cammino ha guadagnato un nero: bh sale da 2 a 3.

Il calo con 12 chiavi non esiste: perché, e che cosa rispondere

Con 12 chiavi un esempio di cancellazione che abbassa l'altezza nera non esiste — e nemmeno con 13 o 14. Verificato per esaurizione nello script: i 688 alberi con 12 chiavi, ogni chiave cancellata: zero cali; idem con 13 (1440) e 14 (3008). Il motivo: per far scendere bh il *caso 2* deve risalire fino alla radice, e ogni passo impone un fratello nero con *entrambi* i figli neri; i conteggi di nodi realizzabili sono 3, poi da 7 a 11, poi da 15 in su (verificato fino a 18): 12, 13 e 14 restano fuori. Il più grande esempio fino a 12 chiavi ne ha quindi 11, ed è quello qui sotto: se il tema chiede 12 chiavi per il calo, si risponde *mostrando questo e dicendo che con dodici non esiste*.

(b) La cancellazione (11 chiavi). Si cancella 1 (foglia nera, figlio sinistro di 2) dall'albero con 11 chiavi e $bh = 3$: il posto di 1 resta a un *doppio nero* x (la NIL), il cui fratello 3 è nero con due figli neri: *caso 2* (3 diventa rosso, il nero in più risale a 2). Il fratello di 2 è ora 8, nero con figli neri (6 e 10): *caso 2* di nuovo (8 rosso, il nero in più risale a 4, che è la radice). La radice resta nera, tutti i cammini hanno perso un nero: bh scende da 3 a 2 (i due passi del caso 2 sono su 3 e su 8).

Risultato. (a) con 12 chiavi l'inserimento di 13 porta bh da 2 a 3 (figura 14.2); (b) il calo con 12 chiavi *non esiste*: il più grande esempio ne ha 11, e cancellando 1 porta bh da 3 a 2 (figura 14.3). [tema 27/10/2025, E6; verifica esaustiva in `verifiche/prove_2025.py`]

14.2 22 gennaio 2026 (simulazione)

Simulazione del 22 gennaio 2026

ESAME DI ALGORITMI (SIMULAZIONE) — Università degli Studi di Catania — Corso di Laurea Triennale in Informatica — **22 gennaio 2026**

Si risolvano i seguenti esercizi in un tempo non superiore a 3 ore.

1. Si considerino le due procedure `MaxHeapSnap` e `BTreeFlop` descritte di seguito. La prima opera su un max Heap, A , mentre la seconda su un nodo di un BTree, x . Per ciascuna di esse, scrivere l'equazione di ricorrenza che descrive il tempo di esecuzione $T(n)$ ottenuta quando la procedura è chiamata su di un heap di n elementi e su di un sotto-albero di n nodi, rispettivamente. Risolvere l'equazione utilizzando uno dei metodi noti (albero di ricorsione, sostituzione o Teorema Master).

2. Scrivere una funzione ricorsiva `PathLen( $v, \pi$ )`, che, preso in input il nodo v di un grafo direzionato e l'array π dei

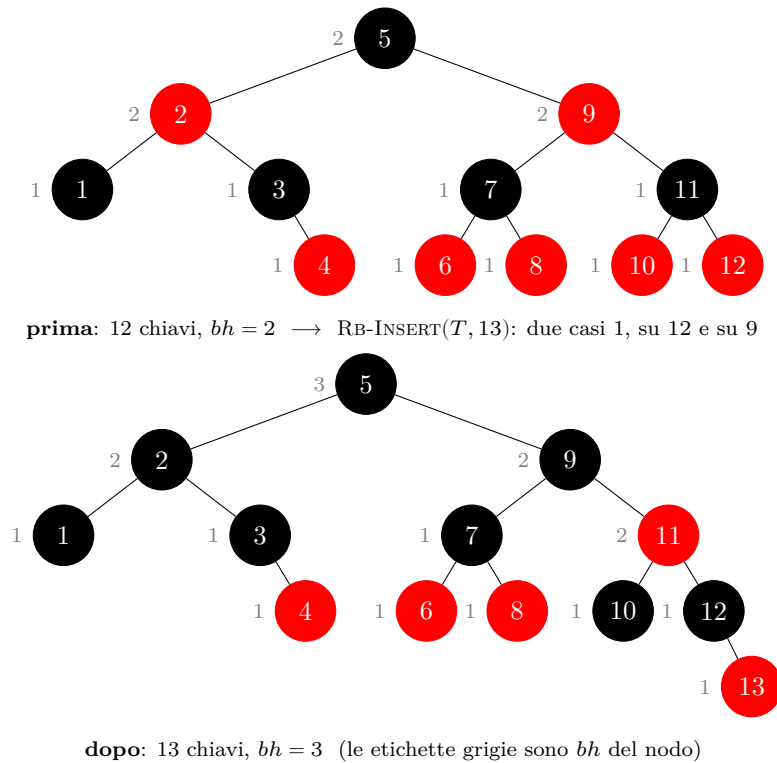


Figura 14.2: Esercizio 6(a): un albero rosso-nero valido con 12 chiavi in cui $\text{RB-INSERT}(T, 13)$ *aumenta* l'altezza nera, da 2 a 3.

- predecessori, calcolato da un qualsiasi algoritmo di SSSP, restituisca la lunghezza del cammino minimo dalla sorgente s al nodo v .
3. Descrivere formalmente il problema del calcolo della distanza di editing tra due stringhe x e y di lunghezza n ed m , rispettivamente. Fornire la funzione ricorsiva utile al calcolo del costo di una soluzione ottima per tale problema.
 4. Fornire la definizione formale di ordinamento topologico di un grafo direzionato. Successivamente, fornire l'esempio grafico di un grafo direzionato con 6 nodi che ammetta uno ed un solo ordinamento topologico.
 5. Si enunci il problema di ottimizzazione affrontato e risolto dal ben noto algoritmo di Huffman. Si fornisca la funzione che assegna ad un albero di Huffman T il suo costo. Si dimostri che il problema gode della proprietà della scelta greedy.
 6. Scrivere lo pseudo-codice dell'algoritmo di Bellman-Ford assumendo l'implementazione del grafo con matrice di adiacenza. Fornire la complessità asintotica della procedura fornita.

Le due procedure dell'esercizio 1, trascritte nei **codebox**:

MAXHEAPSNAP(A)

```

1  if  $n \leq 1$  return
2   $n := \text{HEAPSIZE}(A)$ 
3  for  $i := 1$  to  $n/2$ 
4       $k := \text{EXTRACTMAX}(A)$ 
5  MAXHEAPSNAP( $A$ )
6   $n := \text{HEAPSIZE}(A)$ 
7  for  $i := 1$  to  $n$ 
8       $k := \text{EXTRACTMAX}(A)$ 
9      HEAPINSERT( $A, k$ )
10     HEAPINSERT( $A, k + 1$ )

```

BTREEFLOP(x)

```

1  if  $x == \text{NULL}$  return
2  BTREEFLOP(LEFT( $x$ ))
3  INORDER( $x$ )
4  BTREEFLOP(RIGHT( $x$ ))

```

Quale metodo. Due ricorrenze da contare. **MAXHEAPSNAP** dimezza (estrarne $n/2$ e poi ricorrere su quel che resta) e a ogni livello paga $\Theta(n \log n)$ di operazioni sull'heap; **BTREEFLOP** è una visita ricorsiva che lavora per nodo.

MAXHEAPSNAP(A). Con $n = \text{HEAPSIZE}$: il primo ciclo fa $n/2$ estrazioni e lascia $\lceil n/2 \rceil$ elementi; la chiamata ricorsiva li lascia in numero invariato (k estratto, inseriti k e $k + 1$); il secondo ciclo, altri $n/2$ giri, riporta l'heap da $n/2$ a n elementi, sempre estraendo e reinserendo. Ogni operazione è $O(\log n)$, quindi *per livello* si

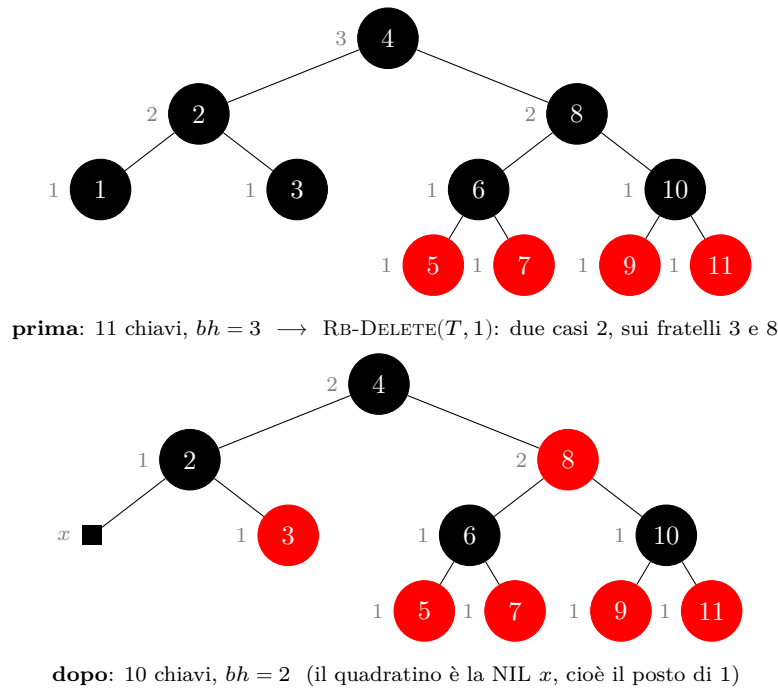


Figura 14.3: Esercizio 6(b): il più grande esempio (11 chiavi, il massimo fino a 12) in cui $\text{RB-DELETE}(T, 1)$ *diminuisce* l'altezza nera, da 3 a 2.

pagano $\Theta(n)$ operazioni da $\Theta(\log n)$, cioè $\Theta(n \log n)$:

$$T(n) = T(n/2) + \Theta(n \log n), \quad T(1) = \Theta(1). \quad (14.3)$$

Teorema master, caso 3 ($a = 1$, $b = 2$, $n^{\lg_b a} = 1 = n^0$ e $f(n) = n \log n = \Omega(n^{0+\epsilon})$, con regolarità $f(n/2) \leq \frac{1}{2}f(n)$): $T(n) = \Theta(n \log n)$; la stessa cosa dice l'albero di ricorsione, $\sum_j \frac{n}{2^j} \log \frac{n}{2^j} = \Theta(n \log n)$. Controprova esatta sull'esecuzione: estrazioni e inserimenti sono $2n - 2$ e $2n - 2$ (per $n = 1024$: 2046 e 2046). Traccia su un heap di otto elementi, i primi otto di quello della figura 6.4(a) del libro ($[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$), cioè $A = [16, 14, 10, 8, 7, 9, 3, 2]$:

$$[16, 14, 10, 8, 7, 9, 3, 2] \xrightarrow{1^{\text{o}} \text{ ciclo}} [8, 7, 3, 2] \xrightarrow{\text{ricorsione}} [5, 4, 3, 2] \xrightarrow{2^{\text{o}} \text{ ciclo}} [9, 8, 7, 5, 4, 3, 6, 2].$$

$\text{BTREEFLOP}(x)$. Ogni chiamata su un nodo non nullo fa due chiamate sui sottoalberi e una visita $\text{INORDER}(x)$ di costo costante. Su un sottoalbero *bilanciato* di n nodi, i due lati hanno $\approx n/2$ nodi:

$$T(n) = 2T(n/2) + \Theta(1), \quad T(1) = \Theta(1). \quad (14.4)$$

Teorema master, caso 1 ($n^{\lg_b a} = n^1 > n^0$): $T(n) = \Theta(n)$; l'albero di ricorsione ha n foglie da $\Theta(1)$, e il totale dei livelli è $c(1 + 2 + 4 + \dots + n) = \Theta(n)$. In realtà il conto è lo stesso per *qualsiasi* albero binario: le chiamate sono una per nodo più una per ogni figlio nullo, $2n + 1$ in tutto (per $n = 1023$: 2047; verificato anche su alberi casuali sbilanciati). Con più rigore, per l'albero generico vale $T(n) = T(k) + T(n - 1 - k) + \Theta(1)$, ancora $\Theta(n)$: ogni nodo si visita una volta.

Risultato. $T(n) = T(n/2) + \Theta(n \log n) = \Theta(n \log n)$ per MAXHEAPSNAP (sull'esempio: estrazioni e inserimenti $2n - 2$); $T(n) = 2T(n/2) + \Theta(1) = \Theta(n)$ per BTREEFLOP ($2n + 1$ chiamate). [simulazione 22/01/2026, E1]

Quale metodo. Procedura ricorsiva sui *predecessori*: la risposta per v si legge da quella di $\pi[v]$ (si risale il cammino fino alla sorgente); niente parametri aggiuntivi.

$\text{PATHLEN}(v, \pi)$. Con i soli predecessori si conta il numero di *archi* (i pesi non ci sono: non si possono sommare). La sorgente ha $\pi[s] = \text{NIL}$: è il caso base.

$\text{PATHLEN}(v, \pi)$

- 1 **if** $\pi[v] == \text{NIL}$ **return** 0 // v è la sorgente
- 2 **return** 1 + $\text{PATHLEN}(\pi[v], \pi)$

Sui predecessori di Bellman-Ford della figura 24.4 (11.2): $\pi[t] = x$, $\pi[x] = y$, $\pi[y] = s$, $\pi[z] = t$, $\pi[s] = \text{NIL}$; la catena delle chiamate dà

$$\text{PATHLEN}(s) = 0, \quad \text{PATHLEN}(y) = 1, \quad \text{PATHLEN}(x) = 2, \quad \text{PATHLEN}(t) = 3, \quad \text{PATHLEN}(z) = 4,$$

coerenti con i cammini minimi ($z: s \rightarrow y \rightarrow x \rightarrow t \rightarrow z$, 4 archi). Il tempo è $\Theta(d)$ e lo spazio $O(d)$ (d = lunghezza del cammino): una chiamata per arco. Limite: con i soli π un vertice non raggiungibile ha $\pi = \text{NIL}$ come la sorgente (distanza letta 0); lo si assume raggiungibile.

Risultato. $\text{PATHLEN}(v, \pi) = 0$ se $\pi[v] = \text{NIL}$, altrimenti $1 + \text{PATHLEN}(\pi[v], \pi)$; sulla figura 24.4: 0, 1, 2, 3, 4 per s, y, x, t, z . [simulazione 22/01/2026, E2]

Quale metodo. Definizione formale + ricorrenza: è il problema della distanza di editing ([Cormen problema 15-5]), nella versione classica a tre operazioni.

Formalizzazione. Date due stringhe $x[1 \dots m]$ e $y[1 \dots n]$, una **trasformazione** di x in y è una sequenza di operazioni elementari su x : *inserimento* di un carattere, *cancellazione* di un carattere, *sostituzione* di un carattere con un altro. Assegnato costo 1 a ciascuna operazione, la **distanza di editing** $d(x, y)$ è il costo minimo di una trasformazione che porta x in y . (Il libro, Problema 15-5, usa più operazioni — copia, scambio, distruggi — con costi per operazione; la ricorrenza si adatta mettendo i costi al posto degli 1.)

La funzione ricorsiva. Sia $d[i, j]$ la distanza tra i prefissi $x[1 \dots i]$ e $y[1 \dots j]$, per $0 \leq i \leq m$, $0 \leq j \leq n$; l'ultima mossa del piano ottimo agisce sull'ultimo carattere di un prefisso:

$$d[i, j] = \min\{d[i-1, j] + 1, \quad d[i, j-1] + 1, \quad d[i-1, j-1] + [x_i \neq y_j]\}, \quad (14.5)$$

dove la parentesi vale 1 se i caratteri differiscono (sostituzione) e 0 se coincidono (copia gratuita); le basi sono $d[i, 0] = i$ (cancellare tutto) e $d[0, j] = j$ (inserire tutto). Le celle sono $\Theta(mn)$, quindi tempo e spazio $\Theta(mn)$ riempiendo per righe (o per colonne), come la tabella di LCS (8.3). Sull'esempio del libro, $d(\text{algorithm}, \text{altruistic}) = 6$: per esempio sostituzioni $g \rightarrow t, o \rightarrow r, r \rightarrow u, h \rightarrow i, m \rightarrow c$ e un inserimento di s (5 sostituzioni più 1 inserimento; le copie sono gratuite); con le sei operazioni del libro a costo 1 quell'esempio costa invece $3 + 1 + 1 + 4 + 1 + 1 = 11$, come nella tabella di pagina 338. Verificato con la forza bruta su coppie corte.

Risultato. $d[i, j] = \min\{d[i-1, j] + 1, d[i, j-1] + 1, d[i-1, j-1] + [x_i \neq y_j]\}$, basi $d[i, 0] = i$, $d[0, j] = j$; $d(\text{algorithm}, \text{altruistic}) = 6$. [simulazione 22/01/2026, E3]

Quale metodo. Una definizione da enunciare e un esempio da costruire: il grafo deve avere *un solo* ordinamento topologico.

Definizione. Un **ordinamento topologico** di un dag $G = (V, E)$ è un ordinamento lineare di tutti i suoi vertici tale che, se G contiene un arco (u, v) , allora u appare prima di v nell'ordinamento; esiste se e solo se il grafo è aciclico (un ciclo chiederebbe a un vertice di precedere se stesso). [Cormen cap. 22.4] Una definizione equivalente: elencare i vertici lungo una retta in modo che tutti gli archi vadano da sinistra a destra. L'algoritmo TOPOLOGICAL-SORT (DFS e tempi di completamento decrescenti, in $\Theta(V + E)$) è nel capitolo 10.4.

L'esempio (un solo ordinamento). Il grafo della figura 14.4 ha la catena $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$ più due archi "in più" ($1 \rightarrow 3$ e $4 \rightarrow 6$). L'unicità si controlla a mano: all'inizio l'unico vertice con grado entrante zero è 1; togliendolo, l'unico è 2; e così via — a ogni passo c'è *una sola* scelta, quindi l'unico ordinamento è 1, 2, 3, 4, 5, 6. Il programma lo verifica su *tutte* le $720 = 6!$ permutazioni di $\{1, \dots, 6\}$: esattamente una è un ordinamento topologico. (Basterebbe la sola catena: i due archi in più non aggiungono libertà.)

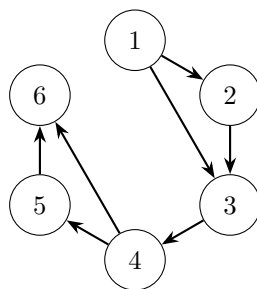
Risultato. Definizione: ordinamento lineare dei vertici in cui ogni arco (u, v) ha u prima di v ; esiste sse il grafo è aciclico. Esempio: la catena $1 \rightarrow 2 \rightarrow \dots \rightarrow 6$ con gli archi $1 \rightarrow 3$ e $4 \rightarrow 6$: unico ordinamento 1, 2, 3, 4, 5, 6. [simulazione 22/01/2026, E4]

Quale metodo. Esercizio *identico* al 5 del tema di ottobre (14.1): stessa risposta — $B(T) = \sum_c c \cdot \text{freq}_T(c)$ minimo e scelta golosa dimostrata con lo scambio delle due foglie sorelle più profonde con le due frequenze minime ($B(T'') \leq B(T)$); i Lemmi 16.2–16.3 e il Teorema 16.4 per intero sono nel capitolo 9.3.

Risultato. Vedi l'esercizio 5 del 27 ottobre 2025: $B(T) = \sum_c c \cdot \text{freq}_T(c)$; fondere le due frequenze minime è la scelta golosa sicura (scambio $a \leftrightarrow x$, $b \leftrightarrow y$). [simulazione 22/01/2026, E5; Cormen cap. 16.3]

Quale metodo. Bellman-Ford del capitolo 11.3 (11.2 per la tabella dei passi) scritto per la **matrice di adiacenza**: un passaggio scorre *tutte le celle*, quindi la complessità è $\Theta(V^3)$.

Sia $W[1 \dots n, 1 \dots n]$ la matrice dei pesi: $W.i, j$ è il peso dell'arco (i, j) , ∞ se l'arco non esiste (gli archi entranti in j sono la colonna j): la struttura di BELLMAN-FORD con "per ogni arco (u, v) " sostituito da un doppio ciclo su tutte le celle.



l'unico ordinamento topologico: 1 2 3 4 5 6 (la catena sul bordo, i due archi 1→3 e 4→6 all'interno)

Figura 14.4: Un grafo orientato con 6 nodi e *un solo* ordinamento topologico: verificato su tutte le 720 permutazioni.

BELLMAN-FORD-MATRICE(W, s)

```

1   $n := W.rows$ 
2  for  $i := 1$  to  $n$ 
3       $d[i] := \infty$ ;  $\pi[i] := \text{NIL}$ 
4   $d[s] := 0$ 
5  for  $k := 1$  to  $n - 1$  //  $n - 1$  passaggi
6      for  $i := 1$  to  $n$ 
7          for  $j := 1$  to  $n$ 
8              if  $W.i, j < \infty$  and  $d[i] + W.i, j < d[j]$ 
9                   $d[j] := d[i] + W.i, j$ 
10                  $\pi[j] := i$ 
11 for  $i := 1$  to  $n$  // ciclo di peso negativo?
12     for  $j := 1$  to  $n$ 
13         if  $W.i, j < \infty$  and  $d[i] + W.i, j < d[j]$ 
14             return FALSE
15 return TRUE

```

La complessità. L'inizializzazione è $\Theta(V)$; i relax sono $(V - 1)$ passaggi da V^2 celle ciascuno; il controllo finale è un altro giro di V^2 celle: in tutto $(V - 1)V^2 + V^2 = V^3$ celle, quindi $\Theta(V^3)$ (anche sui grafi sparsi; con la lista degli archi sarebbe $O(VE)$). Controprova sull'esecuzione: sulla figura 24.4 ($V = 5$, vertici numerati come nel libro) le celle esaminate sono $125 = 5^3$ e i passi sono esattamente quelli della tabella di 11.2 (s, t, x, y, z : dopo il primo $0, 6, \infty, 7, \infty$; dopo il quarto $0, 2, 4, 7, -2$); il controllo finale riconosce un ciclo di peso negativo raggiungibile (nel programma di verifica: l'anello $a \leftrightarrow b$ di peso -6 con $s \rightarrow a$ [non nelle fonti del corso]) e restituisce FALSE.

Risultato. Lo pseudocodice qui sopra; costo $\Theta(V^3)$: ogni passaggio scandisce V^2 celle, il controllo finale un altro giro. [simulazione 22/01/2026, E6]

I sei esercizi dei temi brevi, e il metodo di ciascuno

1. **Due procedure da analizzare:** contare le operazioni, scrivere $T(n)$ e risolverla col metodo noto (2.3, 2.2, 2.4).
2. **Una procedura ricorsiva** su alberi, heap, grafi o stringhe (spesso senza parametri aggiuntivi): la risposta per il nodo ricevuto.
3. **Una struttura dati** da progettare e il suo pseudocodice (coda di priorità, heap con indice).
4. **Programmazione dinamica o goloso:** ricorrenza, pseudocodice, complessità; con Huffman, la scelta golosa dimostrata (9.3).
5. **Definizioni formali** (topologico, distanza di editing) con un esempio costruito a mano.
6. **Esempi di alberi rosso-neri:** prima e dopo l'operazione, con l'altezza nera annotata.

Il tipo ricorrente: l'esempio di albero rosso-nero "prima e dopo"

La scaletta (esercizio 6 del 27 ottobre). (1) Disegna l'albero e **verifica le cinque proprietà** (NIL nere comprese); (2) **annota bh su ogni nodo**, è la grandezza che deve cambiare; (3) esegui l'operazione **passo per passo**, citando i casi del fixup (inserimento 1-3, cancellazione 1-4); (4) **ridisegna il "dopo"**, riannota bh e chiudi dicendo perché: sale se la catena di *casi 1* arriva alla radice, scende se è il *caso 2* ad arrivarci. Se il tema chiede l'impossibile (12 chiavi per il calo), rispondi con la

dimostrazione di impossibilità *più* il più grande esempio esistente (11 chiavi).

Errori tipici in queste prove

- **Trascurare il +1 delle dimensioni** (VALUE): i sottoarray hanno $\lfloor s/4 \rfloor + 1$ elementi, non $s/4$.
- **Cambiare priorità senza l'indice** (esercizio 3): serve $A.pos$, o trovare p_i costa $O(n)$.
- **Parentesizzazione** senza caso base ($m[i, i] = 0$) e senza salvare la scelta $s[i, j]$ mentre si riempie la tabella.
- **Scelta golosa «a parole»** (Huffman): serve lo scambio delle foglie col conto su $B(T) - B(T') \geq 0$ (9.3).
- **Esempi rosso-neri non verificati**: un rosso con figlio rosso o bh diverse ai lati si vedono subito; con 12 chiavi il calo *non* si può disegnare (notabox qui sopra).
- **Bellman-Ford su matrice**: il controllo dei cicli negativi è un altro giro su *tutte* le celle ($\Theta(V^3)$, non $O(VE)$).

Capitolo 15

L'orale

L'idea: allo scritto si fa, all'orale si dice

Lo scritto misura se sai *fare* gli esercizi; l'orale misura se sai *dire* che cosa stai facendo, e perché funziona. Il docente parte da una definizione o da un teorema del programma e ti lascia parlare: vuole sentire la definizione *precisa*, l'idea in una frase, la dimostrazione (o un esempio eseguito a mano), il costo, e un collegamento con il resto del corso. Le prove brevi lo dicono in chiaro: gli esercizi del 27 ottobre 2025 e della simulazione del 22 gennaio 2026 chiedono «si enunci», «si definisca formalmente», «si dimostri», «si fornisca la complessità». Questo capitolo raccoglie quello che c'è da saper dire: come si imposta la risposta (15.1), le dimostrazioni da saper fare con la pagina (15.2), le domande tipiche argomento per argomento (15.3) e le scalette «in due minuti» (15.4).

15.1 Come funziona l'orale, e come si risponde

L'esame è in due parti: la **prova scritta obbligatoria** (tre ore di tempo) e poi la **prova orale**, che può tenersi lo stesso giorno o pochi giorni dopo; i due moduli, Algoritmi e Laboratorio, si esaminano insieme, con un voto unico [programma del corso]. L'orale è *obbligatorio* per gli ammessi con riserva; chi è ammesso senza riserva con voto ≥ 23 può chiedere l'**esonero**, accettando una riduzione di **5 punti** sul voto comunicato.

L'orale «integra il voto dello scritto e non lo incrementa» e valuta «preparazione, capacità di ragionamento e proprietà di linguaggio»; i criteri di voto lo dicono meglio di ogni altra riga: 18–23 è «padronanza minima, esposizione e collegamenti modesti», 24–27 «buona padronanza, buona esposizione, pochi errori», 28–30 e lode «tutti i contenuti, collegamenti critici, esercizi completi e senza errori» [programma del corso]. Il salto di voto non sta nel dire più cose: sta nel dirle *precise* e *collegate*.

Rispondere all'orale, passo per passo

1. **La definizione, con gli oggetti giusti:** le costanti c_1, c_2, n_0 per Θ ; le cinque proprietà degli alberi rosso-neri (con NIL nero di altezza nera 0); il fattore di carico $\alpha = n/m$; il dag e l'ordinamento topologico. Ogni capitolo comincia con la definizione del libro: quella, non una versione approssimata.
2. **L'idea, in una frase:** il *perché* funziona, prima delle formule. «I nodi alti sono pochi, quindi BUILD-MAX-HEAP costa $O(n)$ »; «se la scelta golosa non fosse sicura, uno scambio la renderebbe tale». Gli *ideabox* dei capitoli sono già scritti così.
3. **La dimostrazione**, se la richiesta è «si dimostri»: sono le più chieste e stanno tutte in 15.2, con la pagina. Se invece la richiesta è operativa («si esegua», «si calcoli»), al suo posto viene l'**esempio** eseguito a mano, come nei metodobox dei capitoli.
4. **La complessità:** il tempo della procedura nel caso peggiore — e nel caso medio dove esiste (6.2.2 per le tabelle hash, 4.4 per il quicksort) — con il *perché* in una riga: « $O(\lg n)$ perché l'heap è alto $\lfloor \lg n \rfloor$ ».
5. **Il collegamento:** da dove viene e dove porta. Gli invarianti di ciclo sono induzioni (1.3); il teorema master risolve MAX-HEAPIFY (3.2), merge sort (2.4) e le ricorrenze dei temi; la sottostruttura ottima è la stessa in programmazione dinamica (8.3) e sui cammini

minimi (11.1); goloso e DP si spartiscono gli stessi problemi (9.2). Sono i «collegamenti critici» che valgono il voto alto.

Quando la richiesta è un esercizio, si risponde come allo scritto: la scaletta dei metodobox (2.5 per le ricorrenze, 8.6 per i progetti) e le tracce dei temi svolti nei capitoli 13 e 14.

15.2 Le dimostrazioni da saper fare

Il Laboratorio del programma è esplicito: servono la «dimostrazione della proprietà di sottostruttura ottima», le «dimostrazioni per la selezione di attività e per Huffman», la «correttezza di Bellman-Ford e Topological Sort» e la «correttezza di Dijkstra» [programma del corso]. Qui ci sono tutte le dimostrazioni delle parti assegnate, con la pagina dello svolgimento; la stella ★ segna quelle che una prova scritta ha chiesto per esteso: la scelta golosa di Huffman (esercizio 5 del 27 ottobre 2025 e della simulazione del 22 gennaio 2026).

Dimostrazione	Che cosa chiede il docente	Dove
Ricorrenze — cap. 2		
Teorema master (Teorema 4.1)	L'enunciato coi tre casi: si confronta $f(n)$ con $n^{\log_b a}$; le due avvertenze, l' ε «polinomiale» e la regolarità del caso 3.	par. 2.4 p. 12
La sua dimostrazione (Lemmi 4.2–4.4)	Prima per n potenza esatta di b (tre casi: foglie, livelli, radice), poi ogni n col trucco di <i>floor</i> e <i>ceiling</i> .	par. 2.4.1 pp. 12–13
Ordinamenti — capp. 3–5		
MAX-HEAPIFY: $T(n) \leq T(2n/3) + \Theta(1)$	Il sottoalbero peggiore ha taglia $2n/3$ (l'ultimo livello pieno a metà); caso 2 del master: $\Theta(\lg n)$.	par. 3.2 p. 17
BUILD-MAX-HEAP in $O(n)$	La sommatoria dei costi per livello, $\sum_h \lceil n/2^{h+1} \rceil O(h)$ con $\sum_h h/2^h = 2$: perché $O(n)$ e non $O(n \lg n)$.	par. 3.3 pp. 18–19
Correttezza di PARTITION	L'invariante delle quattro regioni a ogni iterazione, e la verifica finale, quando $q = i + 1$.	par. 4.2 pp. 24–25
Limite inferiore $\Omega(n \lg n)$ (Teorema 8.1)	L'albero di decisione e la catena $n! \leq l \leq 2^h$: vale per heapsort e merge sort, non per counting, radix e bucket.	par. 5.1 p. 30
Hashing — cap. 6		
Teoremi 11.1–11.2 (concatenamento)	Ricerca senza e con successo in $\Theta(1 + \alpha)$ nel caso medio, sotto l'ipotesi di hashing uniforme semplice.	par. 6.2.2 pp. 37–38
Teorema 11.5: $\mathcal{H}_{pm}$ è universale	Il conto delle coppie di chiavi che collidono: ogni coppia ha probabilità $1/m$.	par. 6.3.2 p. 39
Teoremi 11.6–11.8 (indirizzamento aperto)	Le due ricerche, con $\frac{1}{1-\alpha}$ e $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$ sonde attese.	par. 6.4.2 pp. 41–42
Alberi rosso-neri — cap. 7		
Lemma 13.1: $h \leq 2 \lg(n + 1)$	Prima l'asserzione ausiliaria per induzione (un sottoalbero di altezza nera k ha almeno $2^k - 1$ nodi interni), poi la proprietà 4: i neri sono almeno la metà.	par. 7.1 pp. 44–51

Dimostrazione	Che cosa chiede il docente	Dove
Grafi — cap. 10		
Proprietà della DFS (Teoremi 22.7, 22.9, 22.10)	Il teorema delle parentesi, il teorema del cammino bianco e la classificazione degli archi: sono le proprietà che il Laboratorio chiede di enunciare e dimostrare.	par. 10.3 pp. 83–84
Lemma 22.11 + Teorema 22.12	Aciclico $\iff$ nessun arco all'indietro; e perché i tempi di completamento decrescenti danno l'ordinamento topologico.	par. 10.4 pp. 84–85
Cammini minimi — cap. 11		
Lemma 24.1 (i sottocammini)	Taglia e incolla: ogni pezzo di un cammino minimo è un cammino minimo.	par. 11.1 p. 87
Lemma 24.2 + Teorema 24.4 (Bellman-Ford)	La convergenza in $ V - 1$ passaggi (proprietà del rilassamento del cammino) e i due versi: senza cicli negativi TRUE, con un ciclo negativo FALSE.	par. 11.3 pp. 89–91
Teorema 24.5 (grafi aciclici)	Un solo passaggio di rilassamento, in ordine topologico, basta.	par. 11.4 p. 91
Teorema 24.6 (Dijkstra)	L'invariante su S , l'assurdo sul primo vertice y fuori da S , e il punto in cui serve $w \geq 0$.	par. 11.5 p. 92
Strategia golosa — cap. 9		
Teorema 16.1 (selezione di attività)	Lo scambio: l'attività che finisce per prima appartiene a un insieme massimo.	par. 9.1 p. 73
Lemma 16.2 + Lemmi 16.3 e Teorema 16.4 (Huffman) ★	I due scambi (le due frequenze minime diventano foglie sorelle a profondità massima) e la sottostruttura col carattere fuso.	par. 9.3 pp. 77–79
Zaino e resto: dove il goloso funziona	Scelta golosa e sottostruttura del frazionario e del resto, col controesempio dei tagli $\langle 1, 3, 4 \rangle$; e perché lo zaino 0/1 no (\$160 contro \$220).	par. 9.2, 9.4 pp. 75, 79
Programmazione dinamica — cap. 8		
Teorema 15.1 (sottostruttura ottima della LCS)	I tre casi ($x_m = y_n$, $x_m \neq y_n$) e la dimostrazione con taglia e incolla.	par. 8.4 p. 69
Lo schema della sottostruttura ottima	Il metodo generale (scelta, sottoproblemi, taglia e incolla) e il controesempio del cammino semplice massimo.	par. 8.3 p. 69

15.3 Le domande tipiche, argomento per argomento

Non esiste un elenco ufficiale di domande: questo è ricavato dai temi, dalle dispense del docente e dalle rispostabox dei capitoli [non nelle fonti del corso]. La colonna «Dove» porta alla sezione in cui la risposta è già scritta per intero.

Argomento	Le domande, in forma di domanda	Dove
Analisi e notazioni (cap. 1)	«Che cosa significa analizzare un algoritmo? Che cosa si conta?» «Mi dimostri che $3n^2 + 5n = O(n^2)$, con la definizione» «Che cos'è un invariante di ciclo, e come si usa?» «Che differenza c'è fra O e Θ ?»	1.1, 1.3, 1.5
Ricorrenze (cap. 2)	«Enunci e dimostri il teorema master» «Come si risolve una ricorrenza col metodo di sostituzione?» «Che ricorrenza ha merge sort, e perché dall'albero si vede $n \lg n$?»	2.4, 2.2, 2.1
Heap e code di priorità (cap. 3)	«Che cos'è un heap? Quanto costa costruirne uno, e perché $O(n)$?» «Come funziona heapsort su un array?» «Coda di priorità: le operazioni e i tempi»	3.1, 3.3, 3.5
Quicksort (cap. 4)	«Come funziona PARTITION? Qual è l'invariante?» «Caso peggiore e caso migliore? Perché nella pratica è $O(n \lg n)$?»	4.2, 4.3, 4.4
Ordinamento lineare (cap. 5)	«Perché un ordinamento per confronti non può fare meglio di $\Omega(n \lg n)$?» «Counting sort: perché è stabile, e a che cosa serve?» «Come si seleziona la mediana in tempo lineare atteso?»	par. 5.1, 5.2, 5.6
Hashing (cap. 6)	«Quanto costa una ricerca, con e senza successo, nel concatenamento?» «Che cos'è l'hashing universale?» «Indirizzamento aperto: le tre scansioni e i due teoremi»	6.2.2, 6.3.2, 6.4.2

Argomento	Le domande, in forma di domanda	Dove
Alberi rosso-neri (cap. 7)	«Perché gli alberi rosso-neri? Quanto è alto un albero con n nodi?» «Come funziona una rotazione, e che cosa conserva?» «Come e in quanti casi riparano inserimento e cancellazione?»	par. 7.1–7.2, 7.3–7.4
Programmazione dinamica (cap. 8)	«Quando si usa la programmazione dinamica?» «Mi dimostri la sottostruttura ottima della LCS» «Che cosa sono i sottoproblemi ripetuti e l'annotazione?»	8.3, 8.4
Algoritmi golosi (cap. 9)	«Mi dimostri la scelta golosa per la selezione di attività» «Enunci il problema di Huffman, la sua funzione costo, e dimostri la scelta golosa» ★ «Goloso o DP per lo zaino?»	9.1, 9.3, 9.2
Grafi (cap. 10)	«Quali proprietà ha la visita in profondità, e perché valgono?» «Perché l'ordinamento topologico è corretto?» «BFS: perché d è la distanza vera?»	10.3, 10.4, 10.2
Cammini minimi (cap. 11)	«Che proprietà hanno i cammini minimi, e i loro sottocammini?» «Dimostri che Bellman-Ford è corretto» «Dimostri che Dijkstra è corretto»	11.1, 11.3, 11.5
Cammini fra tutte le coppie (cap. 12)	«Floyd-Warshall: la ricorrenza, e perché $\Theta(V^3)$?» «Perché non basta eseguire n volte Dijkstra?»	12.2, 12.3

15.4 In due minuti

Le cinque scalette per le esposizioni più lunghe: se ti chiedono «mi parli degli ordinamenti» o «che cosa sono gli alberi rosso-neri», parti da qui [non nelle fonti del corso]. Ognuna segue i cinque passi del metodo: definizione, idea, proprietà, esempio, costo.

Analisi e notazioni, in due minuti

1. Il problema: due algoritmi per lo stesso problema si confrontano sul numero di operazioni elementari in funzione di n , nel modello RAM (una riga di pseudocodice, una operazione) — 1.1.
2. Un esempio completo: INSERTION-SORT col conteggio per riga, $T(n)$ nel migliore ($an + b$) e nel peggiore ($an^2 + bn + c$) — 1.2.
3. Le notazioni: O , Ω , Θ con le costanti c_1, c_2, n_0 ; per un limite asintotico si esibiscono le costanti o si usa il limite del rapporto — 1.4, 1.5 (esempio da citare: $3n^2 + 5n = O(n^2)$ con $c = 4$ e $n_0 = 5$).
4. La tecnica che torna ovunque: l'invariante di ciclo (tre proprietà, dimostrazione come induzione) — 1.3.

Gli ordinamenti, in due minuti

1. Il limite: per confronti servono $\Omega(n \lg n)$ confronti nel caso peggiore, perché $n! \leq l \leq 2^h$ — par. 5.1.
2. I tre $\Theta(n \lg n)$: merge sort (divide et impera, $T(n) = 2T(n/2) + \Theta(n)$), heapsort (build $O(n)$ e $n - 1$ estrazioni), quicksort (partizione $\Theta(n)$; peggiore $\Theta(n^2)$, atteso $\Theta(n \lg n)$ col pivot casuale).
3. Heap, in fila: heap, MAX-HEAPIFY, BUILD-MAX-HEAP ($O(n)$), HEAPSORT, code di priorità (3.1–3.5); un esempio da saper rifare: la traccia di heapsort (3.4).
4. Fuori dal confronto: counting, radix e bucket, lineari, e non violano il limite perché non confrontano (par. 5.2–5.4).

Gli alberi rosso-neri, in due minuti

1. Su un BST le operazioni costano $O(h)$: senza bilanciamento l'albero può essere una catena e costare $O(n)$ (par. 7.1).
2. Le cinque proprietà; NIL è nero, con altezza nera 0; l'altezza nera $bh.x$ è ben definita.
3. Lemma 13.1: $h \leq 2 \lg(n+1)$ — l'asserzione ausiliaria per induzione ($\geq 2^k - 1$ nodi interni) e la proprietà 4.
4. Le rotazioni: $O(1)$, conservano la proprietà dei BST; nei due fixup al più 2 rotazioni (inserimento) e 3 (cancellazione) (par. 7.2).
5. Un esempio: la sequenza 41, 38, 31, 12, 19, 8 coi casi dell'inserimento (fig. 7.12).

Programmazione dinamica e goloso, in due minuti

1. Quando: sottostruttura ottima e sottoproblemi ripetuti; si riempie una tabella bottom-up e si ricostruisce la soluzione (8.6).
2. Gli esempi DP: catene di montaggio, matrici, LCS, zaino 0/1; il costo passa da esponenziale a $\Theta(n^2)$ o $\Theta(n^3)$.
3. Quando il goloso: scelta golosa *dimostrata per scambio* più sottostruttura — attività, Huffman, frazionario, resto (9.2).
4. Il discriminante da citare: lo zaino 0/1 (\$160 contro \$220) e il resto coi tagli $\langle 1, 3, 4 \rangle$.
5. Un esempio: Huffman su un alfabeto piccolo, costo 224 000 bit contro 300 000 (fig. 9.3).

I grafi, in due minuti

1. Rappresentazioni: liste $\Theta(V + E)$, matrice $\Theta(V^2)$; la scelta dipende dalla densità e dalle operazioni (10.1).
2. BFS: per livelli, $d = \delta(s, v)$ in numero di archi, albero BF (10.2).
3. DFS: tempi d/f , colori, teoremi delle parentesi e del cammino bianco, classificazione degli archi; nel non orientato solo archi T e B (10.3).
4. Ordinamento topologico: dag $\iff$ nessun arco all'indietro, tempi decrescenti, $\Theta(V + E)$ (10.4).
5. Cammini minimi: il rilassamento; Bellman-Ford $O(VE)$, DAG $\Theta(V + E)$, Dijkstra $O(V^2)$ o $O(E \lg V)$, Floyd-Warshall $\Theta(V^3)$ (11.5, 12.2).

Errori tipici all'orale

- Dire « O » quando vale anche il limite inferiore: se lo sai, il caso peggiore di heapsort è $\Theta(n \lg n)$, non solo $O(n \lg n)$.
- Enunciare il teorema master senza le ipotesi ($a \geq 1$, $b > 1$) e senza la regolarità del caso 3.
- Dimostrare Dijkstra senza dire *dove* serve $w \geq 0$: è nell'assurdo sul primo vertice fuori da S .
- «Si vede che è ottimo» al posto della dimostrazione: la scelta golosa si dimostra per scambio, la sottostruttura con taglia e incolla.
- Dimenticare che NIL è nero, con altezza nera 0: senza, il Lemma 13.1 non parte.